

强叔侃墙

Mr. Firewall's Talk Show

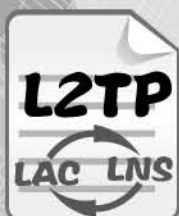


企业网络资料部 防火墙产品部联合出品 2014年第2期总第2期

VPN专题篇
*内功心法 21帖合集



GRE



L2TP VPN的诞生及演进

L2TP Client-Initiated VPN

L2TP LAC-Auto-Initiated VPN



IPSec携手IKE珠联璧合

IPSec模板海纳百川

IPSec遭遇NAT处变不惊



SSL VPN登上历史舞台

URL地址隐藏改写各千秋

端口转发又一村



序

大约从今年4、5月份开始陆续听到一线的声音：“《强叔侃墙》太棒了！”

“强叔是谁？”，合作伙伴的工程师也毫不吝啬地给出好评：“强叔水平真不一般”，“研发专家吧”。一晃半年过去了，“强叔”还在努力耕耘，

《强叔侃墙》已经形成了完整的一个系列：从安全策略、到NAT，现在走到了VPN，并且还要继续跟随产品发展的步子一直讲到下一代防火墙。通过通俗的语言，深入浅出地将产品和原理阐述清晰，形成系列的指导性文档，这在华为防火墙确实是第一次，是我们最期望的，也是最急需向用户传递的。

这一期是VPN专题，GRE、L2TP、IPSec、SSL VPN，从VPN的发展演进，到核心原理、场景应用，防火墙转发流程的深入剖析、数字证书的知识普及，整个专题思路清晰、逻辑严谨，语言通俗幽默、又不失专业深度。

没有技术深度很难搞懂协议细节，没有经验积累很难把握场景变化，不能熟练驾驭文字很难写出如此通俗的技术文档——这些“强叔”都做到了。感谢

“强叔”的持续奉献，希望更多的网络工程师掌握安全技术，和“强叔”一起成为华为防火墙技术专家！

企业网络技术服务部





Contents

01 VPN技术简介 1

02 GRE 8

03 L2TP VPN的诞生及演进 15

04 L2TP Client-Initiated VPN 17

05 L2TP NAS-Initiated VPN 25

06 L2TP LAC-Auto-Initiated VPN 33

07 IPSec闪亮登场 38

08 IPSec携手IKE珠联璧合（上篇） 47

09 IPSec携手IKE珠联璧合（下篇） 58

10 IPSec模板海纳百川 65

11 IPSec遭遇NAT处变不惊 72

12 IPSec引入数字证书 81

13 IPSec兼容GRE和L2TP 89

14 对等体检测和双链路备份 99

15 SSL VPN登上历史舞台 109

16 URL地址隐藏改写各千秋 121

17 端口转发又一村 137

18 网络扩展惊鸿出世一统乾坤（上） 145

19 网络扩展惊鸿出世一统乾坤（下） 153

 防火墙报文处理流程 162

 数字证书浅析 172



VPN 技术简介

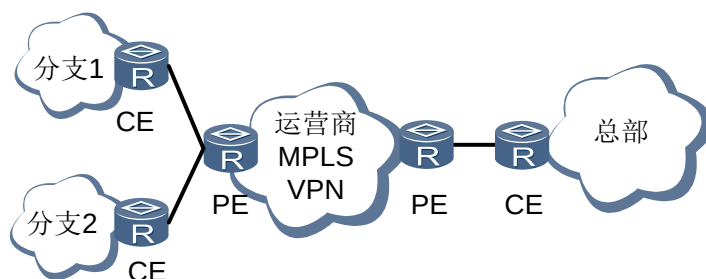
强叔最近正在忙一件比较头疼的事，那就是为分公司、办事处、出差人员、合作单位解决访问公司总部网络资源的问题，同时要求总部对分公司、办事处访问 Internet 进行控制。大家都知道这种情况需要用 VPN（Virtual Private Network，虚拟私有网络）技术解决，但选哪种 VPN 技术还是挺有讲究的，下面强叔就谈谈这方面的心得吧！

VPN 是指在公用网络上建立一个私有的、专用的虚拟通信网络。在企业网络中 VPN 广泛应用于分支机构和出差员工连接公司总部网络。VPN 网络和 VPN 技术通常是如何分类的呢？

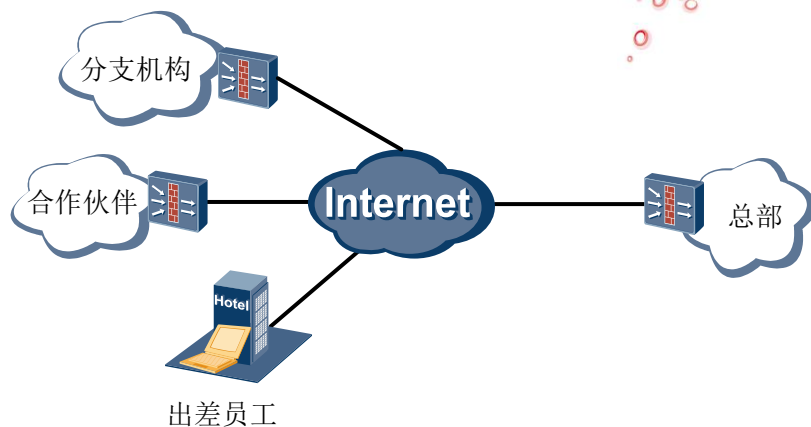
根据建设单位不同分类

这种分类根据 VPN 网络端点设备（**关键设备**）由运营商提供，还是由企业自己提供来划分。

🍁 租用运营商 VPN 专线搭建企业 VPN 网络：目前主要指租用运营商 MPLS VPN 专线，比如联通、电信都提供 MPLS VPN 专线服务。跟传统的租用传输专线（比如租用 E1、SDH 专线）相比，MPLS VPN 专线的优势主要在于线路租用成本低。

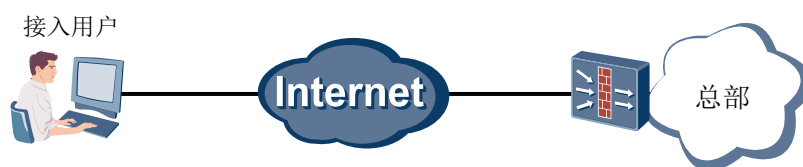


🍁 用户自建企业 VPN 网络：目前最常用的就是基于 Internet 建立企业 VPN 网络，具体技术包括 GRE、L2TP、IPSec、SSL VPN 等。这种方案企业只需要支付设备购买费用和上网费用，没有 VPN 专线租用费用；另外企业在网络控制方面享有更多的主动权、更方便企业进行网络调整。强叔要分享的正是这类 VPN。



根据组网方式不同分类

- 远程访问 VPN (Access VPN): 适合出差员工 VPN 拨号接入的场景。员工可以在任何能够接入公网的地方, 通过远程拨号接入企业内网, 从而访问内网资源。

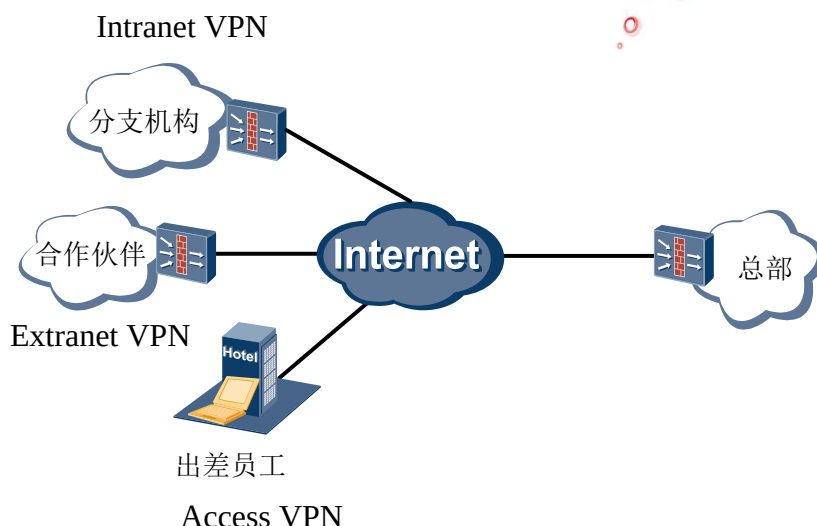


- 局域网到局域网的 VPN (也称为网关到网关的 VPN): 它适用于公司两个异地机构的局域网互连。



根据应用场景不同分类

- 远程访问 VPN: Access VPN 面向出差员工。允许出差员工跨越公用网络远程接入公司内部网络。
- Intranet VPN (企业内部虚拟专网): Intranet VPN 通过公用网络进行企业内部各个网络的互连。
- Extranet VPN (扩展的企业内部虚拟专网): Extranet VPN 是指利用 VPN 将企业网延伸至合作伙伴处, 使不同企业间通过公网来构筑 VPN。Intranet VPN 和 Extranet VPN 的不同点主要在于访问公司总部网络资源的权限有区别。



按照 VPN 技术实现的网络层次进行分类

- 基于数据链路层的 VPN: L2TP、L2F、PPTP。其中 L2F 和 PPTP 已经基本上被 L2TP 替代了，强叔不再关注这两种技术了。
- 基于网络层的 VPN: GRE、IPSec
- 基于应用层的 VPN: SSL

基于 Internet 的 VPN 技术有一个共同点就是必须解决 VPN 网络的安全问题：

- 出差员工的地理接入位置不固定，其所处位置往往不受企业其他信息安全措施的保护，所以需要对出差员工进行严格的接入认证，并且对出差员工可以访问的资源 and 权限进行精确控制。接入认证涉及身份认证技术。
- 合作伙伴需要根据业务开展的情况，灵活进行授权，限制合作伙伴可以访问的网络范围、可以传输的数据类型。此时推荐对合作伙伴进行身份认证，认证通过后进行可以通过策略配置等对合作伙伴的权限进行限制。
- 另外分支机构、合作伙伴和出差用户与公司总部之间的数据传输都必须是安全的。这个涉及数据加密和数据验证技术。

下面简单讲解一下 VPN 用到的几个关键技术点。

隧道技术

隧道技术是 VPN 的基本技术，类似于点到点连接技术。它的基本过程就是在数据进入源 VPN

网关后，将数据“封装”后通过公网传输到目的 VPN 网关后再对数据“解封装”。“封装/解封装”过程本身就可以为原始报文提供安全防护功能，所以被封装的数据包在互联网上传递时所经过的逻辑路径被称为“隧道”。不同的 VPN 技术封装/解封装的过程完全不同，具体封装过程在每个协议中详细介绍。



身份认证技术

主要用于移动办公的用户远程接入的情况。通过对用户的身份进行认证，确保接入内部网络的用户是合法用户，而非恶意用户。

不同的 VPN 技术能提供的用户身份认证方法不同：

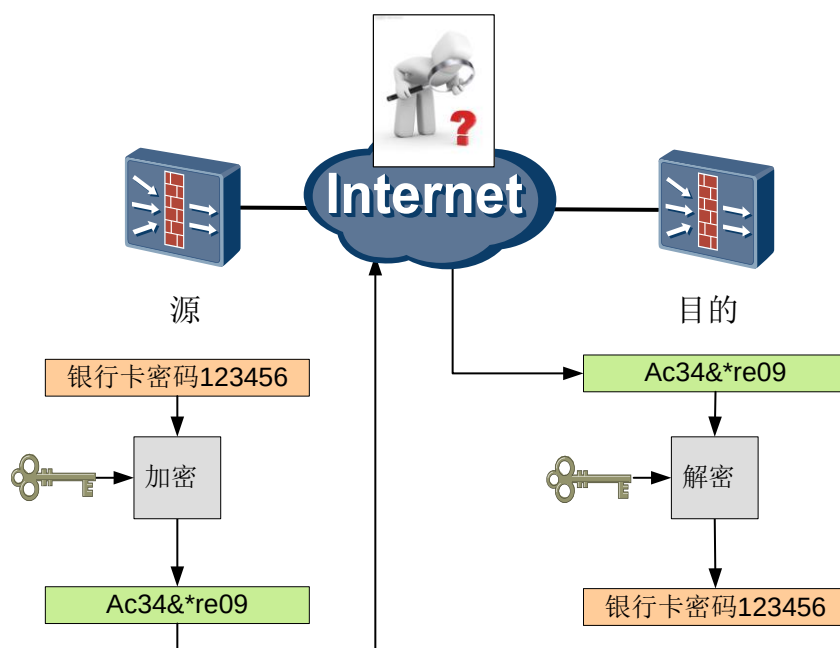
- ❁ GRE 不支持身份认证技术。
- ❁ L2TP：依赖 PPP 提供的认证（比如 CHAP、PAP、EAP）。接入用户的用户名和密码本地认证也可以通过 RADIUS 服务器认证。认证通过以后再给用户分配内部的 IP 地址，通过此 IP 地址对用户进行授权和管理。
- ❁ IPSec：通过 IKEv2 拨号时，支持进行 EAP 认证。接入用户的用户名和密码可以本地认证可以通过 RADIUS 服务器认证。认证通过以后再给用户分配内部的 IP 地址，通过此 IP 地址对用户进行授权和管理。另外 IPSec 还支持数据源认证，在下面的数据验证技术里进行说明。
- ❁ SSL VPN：支持本地认证、证书认证和服务器认证。主要是对服务器进行身份认证，确认 Web 网页的合法性。

加密技术

加密技术就是把能读懂的报文变成无法读懂的报文，也就是把明文变成密文，这样即便是有黑客获取了报文也无法知道其真实含义。加密对象有数据报文和协议报文之分，能够实现协议报文和数据报文都加密的协议安全系数更高。

- GRE 和 L2TP 协议本身不提供加密技术,所以通常结合 IPSec 协议一起使用,使用 IPSec 的加密技术。
- IPSec: 支持数据报文和协议报文加密。IPSec 一般采用对称密钥算法加密数据。对称加密算法采用相同的密钥加密和解密数据。

采用对称密钥加密的示意图:



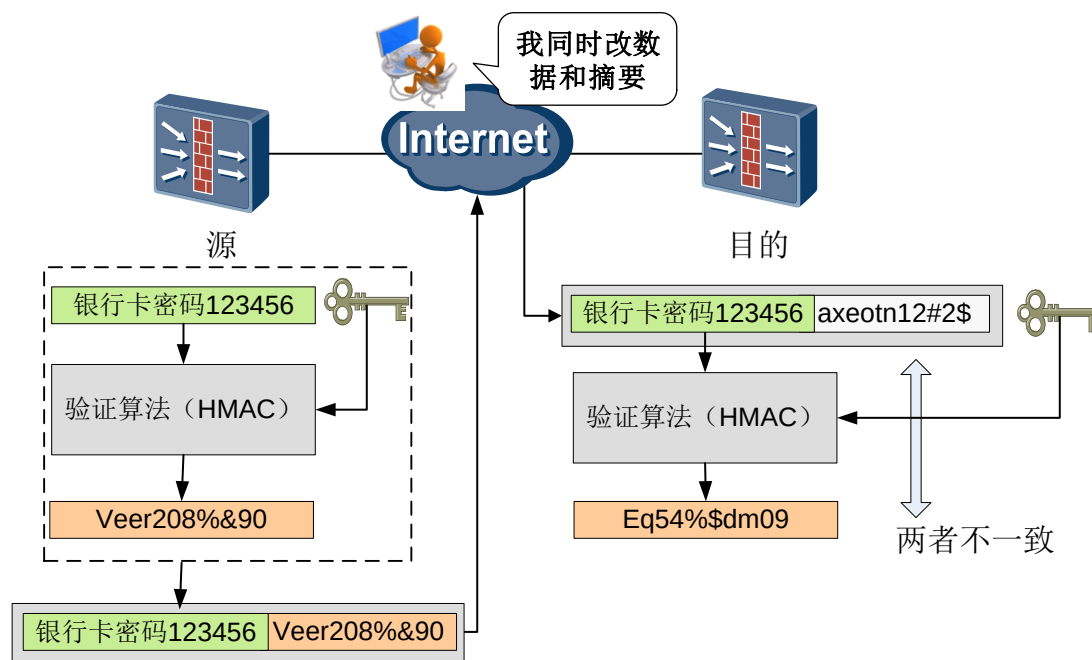
- SSL VPN: 支持数据报文和协议报文加密。SSL VPN 采用公钥体制进行加密。公钥体制加密跟对称密钥加密的差别在于加密和解密的所用的密钥是不同的密钥。采用公钥进行加密, 私钥进行解密。公钥和私钥一一对应。

数据验证技术

数据验证技术就是对收到的报文进行验货。对于伪造的、被篡改的数据进行丢弃。那么验证是如何实现的呢? 它采用一种称为“摘要”的技术。“摘要”技术主要采用 HASH 函数将一段长的报文通过函数变换, 映射为一段短的报文。在收发两端都对报文进行验证, 只有摘要一致的报文才被认可。

- GRE: 本身只提供简单的校验和验证和关键字验证, 但可结合 IPSec 协议一起使用, 使用 IPSec 的数据验证技术。
- L2TP: 本身不提供数据验证技术, 但可结合 IPSec 协议一起使用, 使用 IPSec 的数据验证技术。

- 🍁 **IPSec:** 支持对数据进行完整性验证和数据源验证。在 IPSec 中验证和加密通常一起使用，对加密后的报文 HMAC（Keyed-Hash Message Authentication Code）生成摘要，提供数据的安全性。HMAC 利用 Hash 函数，以一个对称密钥和一个数据包作为输入，生成一个固定长度的输出，这个输出被称为完整性校验值 ICV（Integrity Check Value）。由于在 Hash 运算时包含了密钥，即使用户同时修改了数据和摘要也可以被识别出来。



- 🍁 **SSL VPN:** 支持对数据进行完整性验证和数据源验证。SSL VPN 采用公钥体制，利用 Hash 算法生成摘要，再用私钥加密摘要生成数字签名。利用公钥进行解密。利用公钥和私钥的一一的关系可以对数据源进行认证。

下面强叔总结一下 GRE、L2TP、IPSec 和 SSL VPN 常用的安全技术和使用的场景：

	GRE	L2TP	IPSec	SSL VPN
保护范围	IP层及以上数据	IP层及以上数据	IP层及以上数据	应用层特定数据
适合场景	Intranet VPN	Access VPN, Extranet VPN	Intranet VPN, Access VPN	Access VPN
身份认证	不支持	支持, 基于PPP的Chap、PAP、EAP认证	支持, 采用IP或ID+口令或证书进行数据源认证; IKEv2拨号方式采用EAP认证进行用户身份认证	支持, 用户名+口令+证书对服务器进行认证。也可以进行双向认证
加密技术	不支持	不支持	支持	支持
数据验证	支持 (校验和方式验证、关键字验证)	不支持	支持	支持
如何使用	GRE over IPSec	L2TP over IPSec	单独使用IPSec, 或通过IPSec保护GRE、L2TP	SSL VPN

本篇强叔就简单的介绍了一下 VPN，但是仅了解这些皮毛是不够的。想深入了解每种 VPN 技术的应用、配置和原理，敬请各位关注后面的强叔侃墙。



强叔提问

目前常用的 VPN 协议包括哪些？这些协议的主要优势是什么，分别适合怎样的场景？

GRE

在上一期，强叔为大家拉开了 VPN 技术的帷幕，从本期开始强叔将带领小伙伴们逐一认识 VPN 家族中的各个成员。今天我们先来认识一下 VPN 家族中的第一位成员 GRE。

要说起这个 GRE，我们还得把时间的镜头拉回到 20 年前，回顾一下当年发生的故事。在那个年代，Internet 已经开始快速发展，越来越多的资源被网络所连接，从而使人们的联系更为便捷，这本是一件皆大欢喜的事情。然而看似和谐的网络世界，也充斥着各种不幸。有时就像人生，欢乐大抵相似，痛苦各有不同。被 Internet 互联以后的私有网络就面临着以下几个痛苦：

私有 IP 网络之间无法直接通过 Internet 互通。

这个不用强叔多说，私有网络中使用的都是私有地址，而在 Internet 上传输的报文必须使用公网地址。



异种网络（IPX、AppleTalk）之间无法通过 Internet 直接进行通信。

这个痛苦是先天性造成的，IPX 和 IP 本就不是同一种网络协议，因此 IP 网络不转发 IPX 报文，这倒也情有可原。



私网之间部署的动态路由无法跨越 Internet。

....

或许悲催的故事还有很多，强叔在此就不一一列举了。总之当各种痛苦交织、汇聚在一起之后，形成了一股强大的推动力，迫使网络攻城狮们绞尽脑汁的寻求一种解决之道，终于在 1994

年 GRE 协议问世了，它被 IETF 纳入以后，RFC 标号为 RFC1701 和 RFC1702。

GRE (General Routing Encapsulation) 即通用路由封装协议，它的出现使得上面的那些悲情故事在今天不再重演。好奇的小伙伴肯定就要问了，GRE 到底是用了什么方法竟然能够一一化解上述难题。其实说穿了也很简单，GRE 用的就是当下流行的“马甲”技术。既然私有网络发出的报文由于种种原因不能在 Internet 上进行传输，那何不给这些报文穿上能让 Internet 识别的“马甲”(GRE 封装)，再让它在 Internet 上传输。反正对于 Internet 而言，它是只认“马甲”不认人。这种“马甲”技术在网络中的专用术语就是“封装”。

但凡一种网络封装技术，其基本的构成要素都可以分为 3 个部分：乘客协议、封装协议、运输协议，GRE 也不例外。

🍁 乘客协议

为了便于理解封装技术，我们用邮政系统打个比方。乘客协议就是我们写的信，信的语言可以是汉语、英语、法语等，具体如何解释由写信人、读信人自己负责。

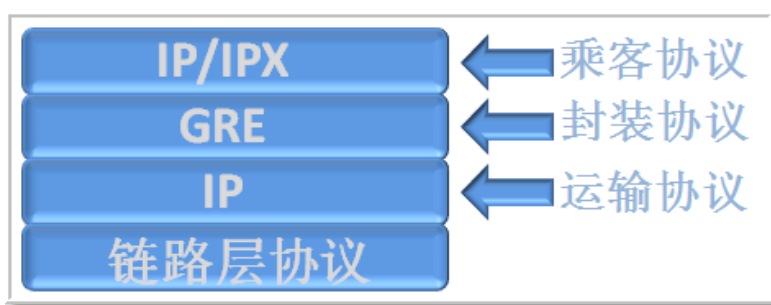
🍁 封装协议

封装协议可以理解为信封，可能是平信、挂号或者是 EMS，这对应于多种封装协议。

🍁 运输协议

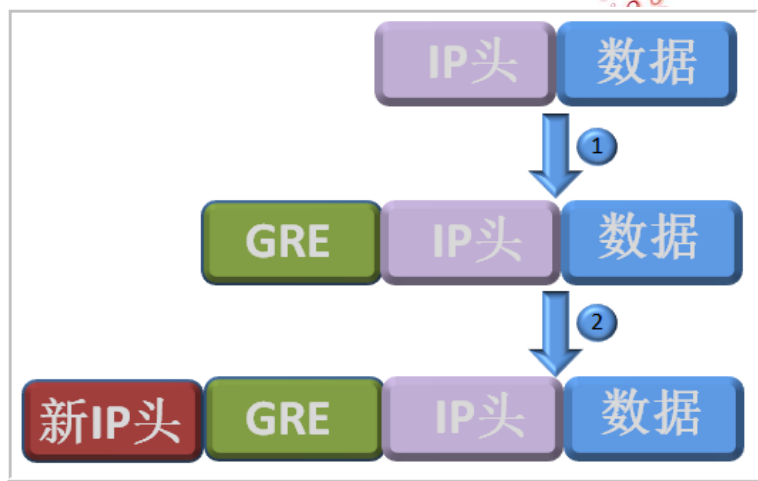
运输协议就是信的运输方式，可以是陆运、海运或者空运，这对应于不同的运输协议。

OK，理解了这个以后，我们再来看看 GRE 中都用到了哪些协议。



从图中我们可以很清晰的看到，GRE 能够承载的协议包括 IP 协议、IPX 协议，GRE 所使用的运输协议是 IP 协议。

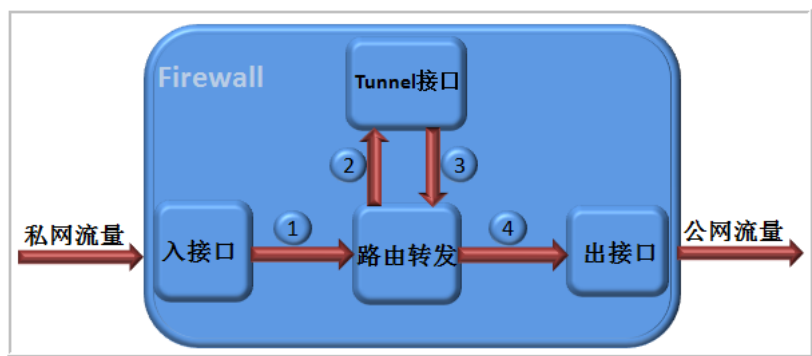
了解了 GRE 的一些基本概念和应用以后，下面强叔将为大家讲解一下 GRE 的封装原理。



GRE 的封装过程可以细分成两步，第一步是在私有数据中添加 GRE 头，第二步是在 GRE 头前面再加上新的 IP 头。加上新的 IP 头以后，就意味着这个私有网络的报文在经过层层封装以后就可以在 Internet 上传输了。

从产品实现的角度来讲，上述的封装操作是通过一个逻辑接口来实现的，这个逻辑接口就是鼎鼎有名的 Tunnel 接口。Tunnel 即隧道，从名字就可以看出来这个逻辑接口就是为隧道而生。由于 Tunnel 接口是一个通用的隧道接口，所以 GRE 协议在使用这个接口的时候，我们会要求将这个接口的封装协议设置为 GRE 协议。

下面强叔将结合 Tunnel 接口介绍一下防火墙对 GRE 流量的转发过程，见下图。



1. 企业的私网流量到达防火墙的入接口，防火墙查询路由表对此流量进行转发。
2. 防火墙根据路由查找结果，将此流量引导到 Tunnel 接口进行 GRE 封装。
3. 封装后的 GRE 报文再次查找路由进行流量转发。
4. 防火墙根据路由查找结果，找到出接口，并将流量发送到 Internet。

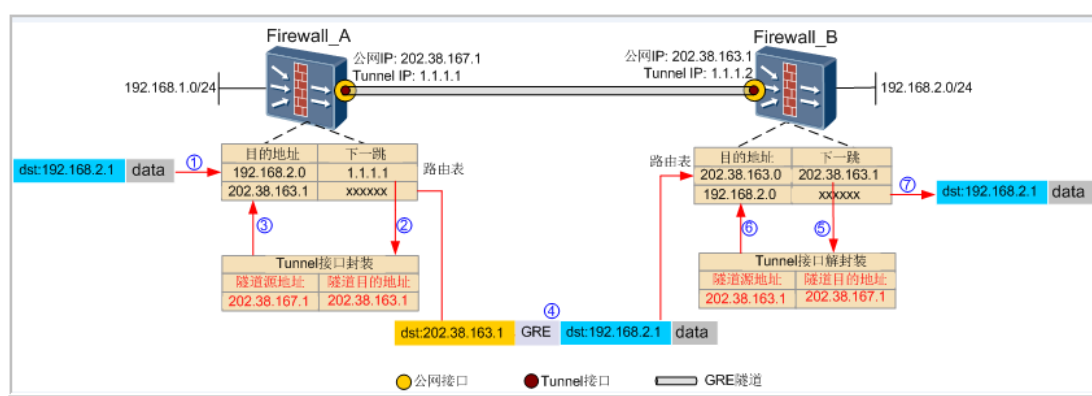
这就是防火墙对私网流量进行 GRE 封装和转发的全过程，很简单吧。

有些小伙伴肯定在想了，强叔这里只介绍了封装过程，那隧道对端是如何解封装的？

其实解封装也很简单。首先隧道对端在接收到报文以后，先根据该报文目的地址判断是不是

发给自己的，在明确报文是发给自己以后，再去判断这个报文是不是 GRE 报文。怎么判断呢？在封装原理那幅图中我们可以看到封装后的 GRE 报文会有个新的 IP 头，这个新的 IP 头中有个 Protocol 字段，字段中标识了内层协议类型，如果这个 Protocol 字段值是 47，就表示这个报文是 GRE 报文。

上面强叔从原理角度为大家讲解了报文进入隧道、加封装、解封装的过程。想必小伙伴更想知道的是，在防火墙中 GRE 隧道应该怎么配置呢？莫要着急，下面强叔就来结合实例讲讲 GRE 隧道的配置方法。



GRE 隧道的配置也很简单，可以分为两个步骤。

1. 配置 Tunnel 接口的封装参数（配置数据以 Firewall_A 为例）。

```
interface Tunnel 1
ip address 1.1.1.1 24
tunnel-protocol gre
source 202.38.167.1
destination 202.38.163.1
```

首先设置 Tunnel 接口的封装类型为 GRE，然后指定 GRE 隧道的源和目的就 OK 了。

GRE 隧道中大家容易产生疑惑的地方在配置 Tunnel 接口的 IP 地址。有很多小伙伴会有这么几个问题。

- 🍁 Tunnel 接口的 IP 地址是否必须配置？
- 🍁 隧道两端的 Tunnel 接口 IP 地址是否有所关联？
- 🍁 Tunnel 接口使用的是公网 IP 地址还是私网 IP 地址？

首先 Tunnel 接口的 IP 地址必须配置，如果不配置 IP 地址，Tunnel 接口就无法 UP 起来。其次，从 GRE 的封装过程来看，Tunnel 接口的 IP 地址并没有参与报文封装，所以隧道两端的 Tunnel 接口没有任何关联，各配各的。再者，既然 Tunnel 接口不参与封装，所以也就没有必要用公网地址了，配置成私网就可以了。

配置路由，将要进行 GRE 封装的流量引流到 Tunnel 接口。

把流量引入到 GRE 隧道有两种方法，一种是静态路由，一种是动态路由。

静态路由配置：

```
ip route-static 192.168.2.0 24 Tunnel 1
```

动态路由配置：如果私网内部使用的是动态路由（例如 OSPF），则只要把私网地址和 Tunnel 接口地址一起通过 OSPF 发布出去即可。

```
ospf 1
```

```
area 0
```

```
network 1.1.1.0 0.0.0.255
```

```
network 192.168.2.0 0.0.0.255
```

此时有一个特殊的地方需要注意，就是如果 GRE 隧道对应的公网接口也使用了 OSPF，那我们就需要用一个新的 OSPF 进程来发布私网地址和 Tunnel 接口地址。

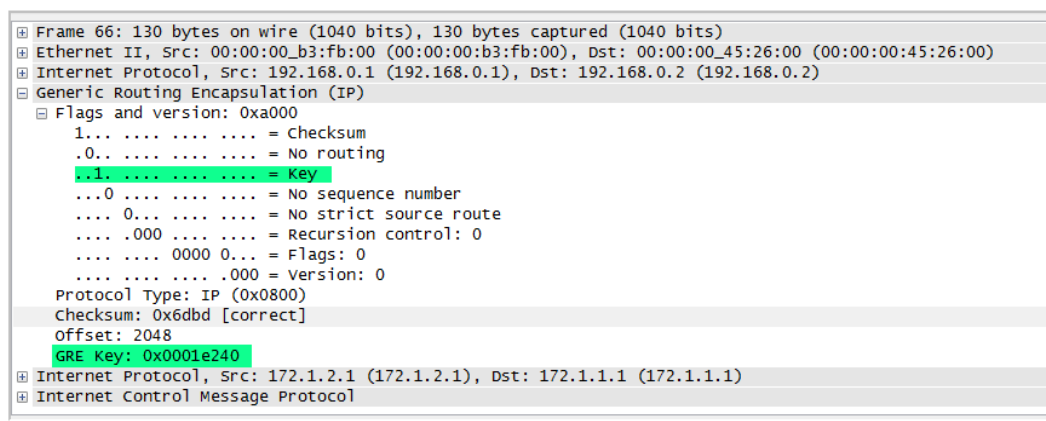
GRE 的配置很简单，强叔就不多说了。结合平时遇到的一些 GRE 问题，强叔再做一些扩展。

有些小伙伴可能会有担忧，说如果 Internet 上的恶意用户伪装成 Firewall_A 向 Firewall_B 发送 GRE 报文，那伪装者不就可以访问 Firewall_B 中的资源了呢？那么 Firewall_A 和 Firewall_B 在建立 GRE 隧道时，是如何做到互信的呢？下面强叔讲一下 GRE 的安全机制。

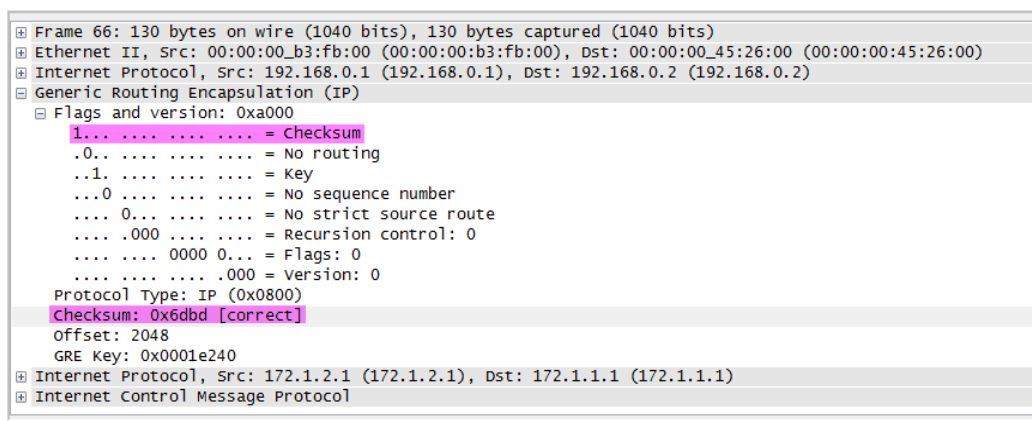
防火墙配置了 GRE 隧道，并不是说防火墙收到所有的 GRE 报文它都会进行处理。它只处理与本防火墙建立 GRE 隧道的对端设备发过来的 GRE 报文。具体做法就是建立 GRE 隧道的两台设备要设置一个“密钥”，这个密钥就是身份凭证，密钥信息被封装在 GRE 报文的头部。

当一端收到一个 GRE 报文以后，都会检查该密钥，只有在密钥一致时，才认为是隧道对端发过来的 GRE 报文。“身份校验”不通过，该 GRE 报文会被丢弃。下图就是一个 GRE 报文，其中 GRE 报文头中的 key 字段为 1 表示用户设置了身份验证功能，下面的 GRE key:

0x0001e240 是用户配置的密钥值。

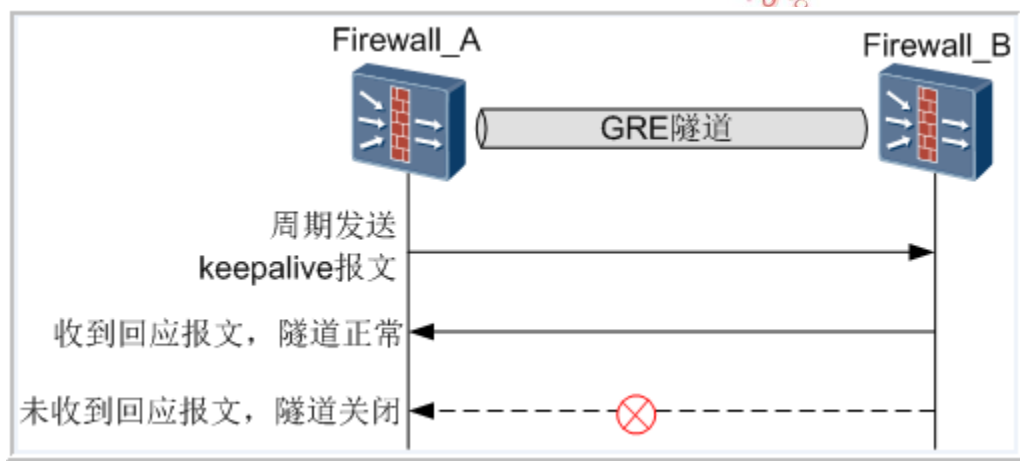


虽然身份校验机制可以保证 GRE 隧道两端可以实现互信,但是如果报文在 Internet 传输途中被其他用户恶意篡改,这又该怎么办? 这里我们还得用到 GRE 头中的另一个字段 checksum。GRE 封装报文时,会根据报文信息计算校验和,并将这个校验和插入到 checksum 字段中。当隧道对端收到该报文时,对端也会根据报文信息计算校验和,并与报文中携带的校验和进行比较,如果校验结果一致,则接受此报文;如果不一致,则丢弃。下图中 checksum 为 1,表示启用了校验和功能,下面 checksum:0x6dbd 是校验和对应的值。



GRE 的安全机制可以实现隧道两端的互信,并保证报文传输的完整性。但是这里还有一个问题,就是如果隧道对端设备出现故障时,隧道本端如何感知的呢?

GRE 隧道是一种无状态类型的隧道,所谓的无状态类型是指隧道本端并不维护与对端的状态。换句话说假如隧道对端出现故障,那隧道本端是感受不到的。为了解决这个问题, GRE 隧道提供了一种保活机制(keepalive)。GRE 隧道源会周期性的向隧道对端发送 keepalive 报文,以检测隧道对端状态。如果 GRE 隧道对端出现问题,则隧道源的 Tunnel 接口状态会被置为 Down,即本端关闭 GRE 隧道。这就避免了因对端不可达而造成的数据黑洞。GRE 的 Keepalive 功能是单向的,对端是否支持或启用 Keepalive 功能不影响本端的 Keepalive 功能。实际配置时,建议隧道两边都配置上保活功能。



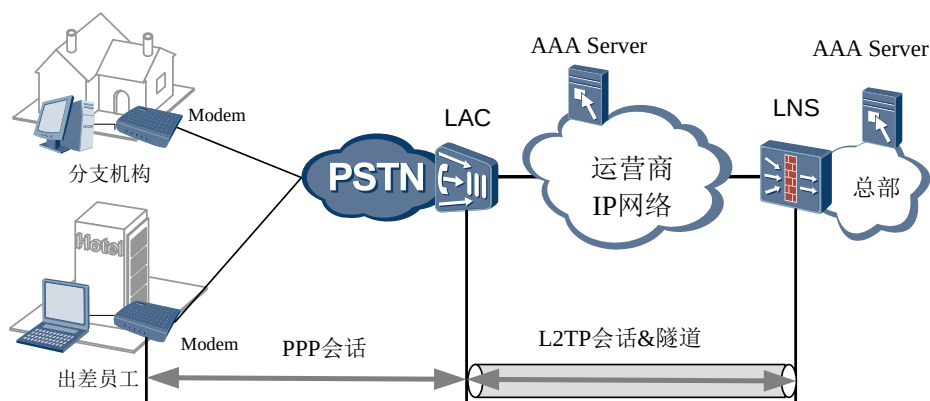
小伙伴们看到这里觉得是不是有了 GRE 隧道就万事大吉了，其实不然。强叔要告诉大家的是 GRE 自身有个缺陷，就是 GRE 技术不带有安全加密功能。没有加密功能的 GRE 报文，只能说是穿了个透明的马甲，它所传输的数据别人可以看得一清二楚。所以我们在实际使用时，很少单纯使用 GRE，而是经常与 IPSec 技术联用。由于 IPSec 技术具备很强的加密功能，就解决了 GRE 的安全性问题。这也就是我们经常听到的 GRE over IPSec 技术。

GRE over IPSec 技术本质并不复杂，就是在 GRE 将报文封装后，再经过 IPSec 封装，就是穿了两层“马甲”而已。GRE over IPSec 的详细内容强叔准备放在 IPSec 技术中再进行详细讲解，到时候大家就会一目了然了。下期强叔将带领大家认识 VPN 的下一位成员：L2TP VPN，敬请期待。

L2TP VPN 的诞生及演进

说到 L2TP VPN 必须先将镜头切到互联网发展初期，我们还通过电话线上网的那个时代（那时候强叔就已经扎入数通领域了，呵呵，能猜出强叔的年纪了吧？）。那个时代个人用户和企业用户大都通过电话线上网，当然企业分支机构和出差用户一般也通过“电话网络（学名叫 PSTN (Public Switched Telephone Network) / ISDN (integrated services digital network)）”来 Access 总部网络，人们将这种基于 PSTN/ISDN 的 VPN 命名为 VPDN (Virtual Private Dial Network)。L2TP VPN 是 VPDN 技术的一种，其它的 VPDN 技术已经逐步退出历史舞台了，这里强叔就不给大家添乱了。

在传统的基于 PSTN/ISDN 的 L2TP VPN 中，运营商在 PSTN/ISDN 和 IP 网络之间部署 LAC（在 VPDN 里称为 NAS，Network Access Server），集中为多个企业用户提供 L2TP VPN 专线服务，配套提供认证和计费功能。当分支机构和出差员工拨打 L2TP VPN 专用接入号时，接入 modem 通过 PPP 协议与 LAC 建立 PPP 会话，同时启动认证和计费。认证通过后 LAC 向 LNS 发起 L2TP 隧道和会话协商，企业总部的 LNS 出于安全考虑，再次认证接入用户身份，认证通过后分支机构和出差员工就可以访问总部网络了。

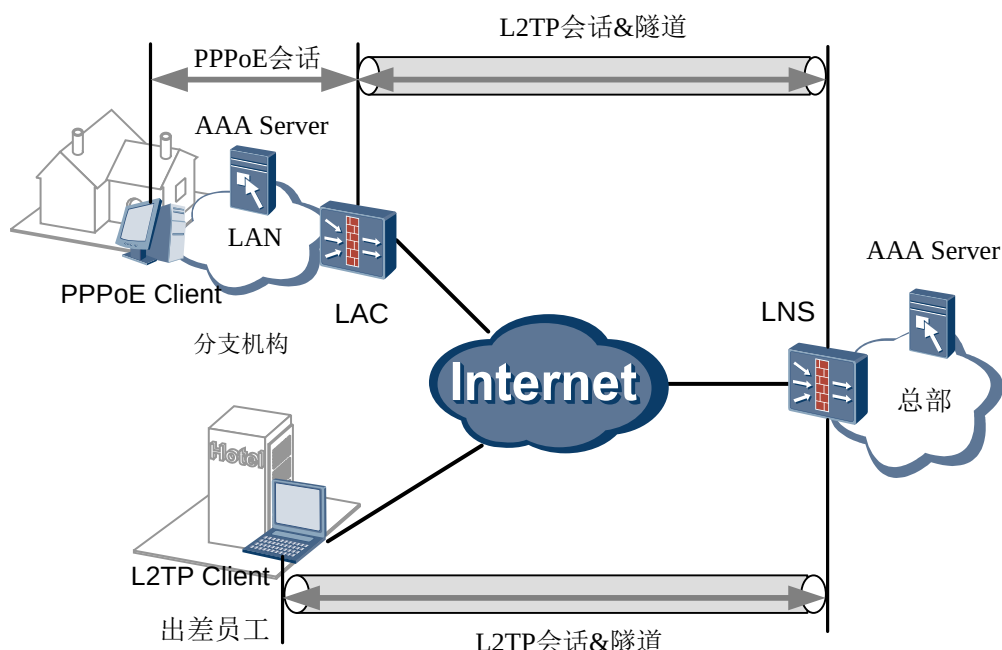


LAC (L2TP Access Concentrator)：接入汇聚点，部署在运营商网络

LNS (L2TP Network Server)：网络服务器，部署在企业总部出口

说明：LAC 和 LNS 是 L2TP 协议里的概念，NAS 是 VPDN 里的概念，在 L2TP VPN 中 LAC 实际上就是 NAS。随着 IP 网络的普及，PSTN/ISDN 网络逐渐退出数据通信领域。企业和个人用户都可以自由通过以太网直接接入 Internet 了，此时 L2TP VPN 也悄悄地向前“迈了两小步”——看似只有两小步，但这两小步却让 L2TP VPN 这个过气明星留在了风云变化的 IP 舞台上。（强叔感叹，在通信领域，不学习、不进步只能被拍死在沙滩上啊！）

现在 L2TP VPN 常用场景如下图所示。



🍁 两小步之一——PPP 屈尊落户以太网：这是拨号网络向以太网演进过程中的必经之路，并非专门为 L2TP VPN 设计，但 L2TP VPN 确实是最大的受益者。分支机构用户安装 PPPoE Client，在以太网上触发 PPPoE 拨号，在 PPPoE Client 和 LAC (PPPoE Server) 之间建立 PPPoE 会话。LAC 和 LNS 之间的 L2TP VPN 建立过程没有变化。

🍁 两小步之二——L2TP 延伸到用户 PC：这种场景下，PC 可以通过系统自带的 L2TP 客户端或第三方 L2TP 客户端软件直接拨号与 LNS 建立 L2TP VPN。L2TP Client 摒弃了 LAC 这位“掮客”跟总部直接建立合作关系，看来这种事情在哪里都会发生啊！

这两种场景跟初始 L2TP VPN 场景相比有一个共同特征，就是企业借用了“运营商管道 (Internet)”自建 VPN，避开了运营商 VPN 专线业务收费。为区分这两种 L2TP VPN，前者（基于 LAC 拨号的 L2TP VPN）被称为 NAS-Initiated VPN，后者（客户端直接拨号的 L2TP VPN）被称为 Client-Initiated VPN。

下期强叔先介绍比较简单一点的 Client-Initiated VPN，由浅入难，如顺水行舟，日行千里呀！

🍁 L2TP Client-Initiated VPN

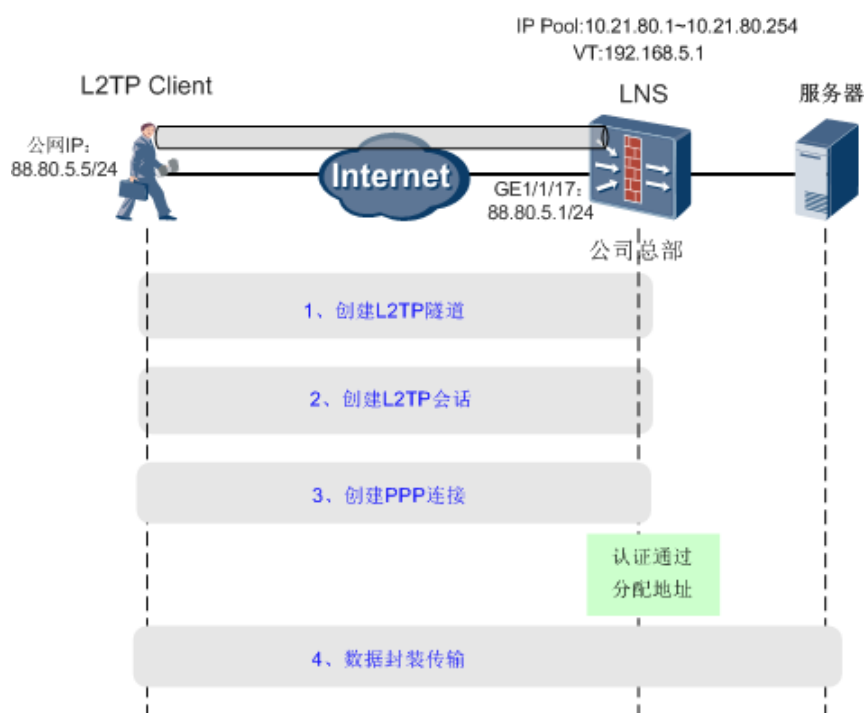
现在大家普遍对 PC、PAD 或手机上的各种客户端不太陌生了，最常见的是 PPPoE 客户端，其次是 VPN 客户端。VPN 客户端的作用就是帮助用户在 PC 或 PAD 或手机上触发建立一条直通公司总部网络的隧道，实现用户自由访问总部网络的意愿。当然用户能轻松进入隧道也必须可以自由离开隧道——切换到直接访问 Internet，这有点像是掌握了通往地球（总部网络）的“虫洞”入口的都教授可以瞬间自由往来于两个遥远的星球，当然也可乘坐星际飞行器在宇宙中漫游（Internet）。无论是在现实社会还是在虚拟网络世界，幸福生活似乎只有在消除了时空距离后方能体会，强叔用真实体验告诉大家 Client-Initiated VPN 可以助您轻松过上都教授的幸福生活。

都教授要借助 L2TP 客户端穿越“虫洞”进入公司网络，必然要先通过“门神”LNS 的身份检查（检查手段毫不含糊，用户名称、密码、主机名称、隧道验证应有尽有）。LNS 为通过验证的用户发放特别通行证（公司内网 IP 地址），对试图混入的人说 bye-bye。Client-Initiated VPN 配置中体现的就这样一个简单的思路（本例仅给出本地认证配置）：

L2TP Client	LNS
对端 IP 地址	l2tp enable
登录用户（PPP 用户）名称	l2tp domain suffix-separator @
登录用户（PPP 用户）密码	interface LoopBack0
主机名称（可选，也叫隧道名称）	ip address 192.168.5.1 255.255.255.0
PPP 认证模式（PAP/CHAP/EAP，有些客户端默认 CHAP）	interface Virtual-Template1
隧道验证（可选，有些客户端不支持）	ppp authentication-mode chap
前三项是必配内容，后三项可能会因客户端不同有所取舍。	ip address unnumbered interface LoopBack0
	remote address pool
	l2tp-group 1
	undo tunnel authentication
	allow l2tp virtual-template 1 remote client1 //指定 VT 接口和对端主机名称
	tunnel name LNS //本端隧道名称
	tunnel timer hello 0
	aaa
	local-user l2tp@radius password
	cipher %\$%\$y>z_1xG mINC.~+P.b()4+"v%\$%\$ //本地用户名、密码
	local-user l2tp@radius service-type ppp
	domain radius

L2TP Client	LNS
	ip pool 0 10.21.80.1 10.21.80.254

VT 接口大家感觉比较陌生吧？它是用于二层协议通信的逻辑接口，比如 PPP 和 L2TP 协议通信、PPP 和 Ethernet 协议通信，都会用到它，所以在 L2TP VPN 中要开启 VT 接口所在安全区域跟 L2TP 物理接口所在安全区域之间的包过滤，保证 PPP 和 L2TP 协议通信畅通无阻。思想是神经元活动的结果，配置最终体现到一系列的消息交互中。配置不太复杂，但消息交互可不太简单。为了让大家一目了然，并方便跟下期的 NAS-Initiated VPN 做对比，强叔画了一张简图，然后对着这张图对 L2TP Client 与 LNS 之间的消息交互进行深入剖析。



结合抓包情况来讲解一下 Client-Initiated VPN 建立的完整过程：

阶段 1 建立 L2TP 隧道（控制连接）：3 条消息协商进入虫洞时机

L2TP Client 和 LNS 通过交互三条消息协商隧道 ID、UDP 端口（LNS 用 1701 端口响应 Client 隧道建立请求）、主机名称、L2TP 的版本、隧道验证（Client 不支持隧道验证时 LNS 的隧道验证要关闭，例如 WIN7 操作系统）等参数。

No.	Time	Source	Destination	Protocol	Info
79	21.958434	88.80.5.5	88.80.5.1	L2TP	Control Message-SCCRQ (tunnel id=0, session id=0)
80	21.960348	88.80.5.1	88.80.5.5	L2TP	Control Message-SCCRP (tunnel id=88, session id=0)
81	21.960444	88.80.5.5	88.80.5.1	L2TP	Control Message-SCCCN (tunnel id=1, session id=0)

仅给出隧道 ID 协商过程：

步骤 1	Client: LNS 兄，用 88 作为 Tunnel ID 跟我通信吧。	Assigned Tunnel ID AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Tunnel ID (9) Tunnel ID: 88
步骤 2	LNS: OK。Client，用 1 作为 Tunnel ID 跟我通信。	Assigned Tunnel ID AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Tunnel ID (9) Tunnel ID: 1
步骤 3	Client: OK。	

阶段 2 建立 L2TP 会话：3 条消息唤醒虫洞门神

L2TP Client 和 LNS 通过交互三条消息协商 Session ID，建立 L2TP 会话。跟“门神”对上话了，才可能提交身份认证材料呀！

No.	Time	Source	Destination	Protocol	Info
83	21.960546	88.80.5.5	88.80.5.1	L2TP	Control Message - ICRQ (tunnel id=1, session id=0)
84	21.969910	88.80.5.1	88.80.5.5	L2TP	Control Message - ZLB (tunnel id=88, session id=0)
85	21.970022	88.80.5.1	88.80.5.5	L2TP	Control Message - ICRP (tunnel id=88, session id=1)
86	21.970086	88.80.5.5	88.80.5.1	L2TP	Control Message - ICCN (tunnel id=1, session id=37)
87	21.970118	88.80.5.5	88.80.5.1	L2TP	Control Message - ZLB (tunnel id=1, session id=0)

注：ZLB 表示目前空闲，没有消息要发送。

Session ID 协商过程：

步骤 1	Client: LNS，采用 1 作为 Session ID 跟我通信吧。	Assigned Session AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Session (14) Assigned Session: 1
步骤 2	LNS: OK。Client，采用 37 作为 Session ID 跟我通信吧。	Assigned Session AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Session (14) Assigned Session: 37
步骤 3	Client: OK。	

阶段 3 创建 PPP 连接：身份认证，发放特别通行证

1. LCP 协商。

LCP 协商是两个方向分开协商的，主要协商 MRU 大小。

说明：MRU 是 PPP 的数据链路层参数，类似以太网中的 MTU。如果 PPP 链路一端设备发送的报文载荷大于对端的 MRU，这个报文在传送时就会被分片。

No.	Time	Source	Destination	Protocol	Info
89	21.980266	88.80.5.1	88.80.5.5	PPP	LCPConfiguration Request
90	21.999769	88.80.5.5	88.80.5.1	PPP	LCPConfiguration Request
91	22.000172	88.80.5.1	88.80.5.5	PPP	LCPConfiguration Reject
92	22.000416	88.80.5.5	88.80.5.1	PPP	LCPConfiguration Request
93	22.009783	88.80.5.1	88.80.5.5	PPP	LCPConfiguration Ack
97	24.980055	88.80.5.1	88.80.5.5	PPP	LCPConfiguration Request
98	25.006786	88.80.5.5	88.80.5.1	PPP	LCPConfiguration Ack

```

❑ PPP Link Control Protocol
  Code: Configuration Request (0x01)
  Identifier: 0x01
  Length: 14
  ❑ Options: (10 bytes)
    Maximum Receive Unit: 1400
    Magic number: 0x56711d79
  
```

2. PPP 验证。

验证方式包括 CHAP、PAP、EAP（高端防火墙不支持），CHAP 或 PAP 可以在本地认证，也可在 AAA 服务器上认证；EAP 只能在 AAA 服务器上认证。EAP 认证比较复杂，我们等到 IPsec 部分再详解，此处仅给出最常用的 CHAP 验证过程：

99	25.008316	88.80.5.1	88.80.5.5	PPP	CHAP	Challenge (NAME='', VALUE=0x258c7d3de12787b76ab267fbc63a9a2c)
100	25.008834	88.80.5.5	88.80.5.1	PPP	CHAP	Response (NAME='l2tp@radius', VALUE=0xe1fc84c891d0db297f722efe33a57d8b)
101	25.046663	88.80.5.1	88.80.5.5	PPP	CHAP	Success (MESSAGE='Welcome to .')

经典的三次握手过程：

步骤 1	LNS: Client, 发给你一个“挑战 (Challenge)”，用它来加密你的密码吧。	<pre> ❑ PPP Challenge Handshake Authentication Protocol Code: Challenge (1) Identifier: 1 Length: 21 ❑ Data value size: 16 value: 258c7d3de12787b76ab267fbc63a9a2c </pre>
步骤 2	Client: OK, 把我的用户名和密码后的密码发给你，请验证。	<pre> ❑ PPP Challenge Handshake Authentication Protocol Code: Response (2) Identifier: 1 Length: 32 ❑ Data (28 bytes) value size: 16 value: e1fc84c891d0db297f722efe33a57d8b Name: l2tp@radius </pre>
步骤 3	LNS: 验证通过，欢迎来到 PPP 的世界。	<pre> ❑ PPP Challenge Handshake Authentication Protocol Code: Success (3) Identifier: 1 Length: 16 Message: welcome to . </pre>

LNS (或 AAA 服务器) 上配置的用户名和密码是用来验证 Client 的，当然要求“本人”和“验证”完全一致，即要求 L2TP Client 和 LNS 上配置的用户名和密码完全一致。这里详解一下

什么叫用户名完全一致：

- 如果在 LNS 上配置的签证为 username (没有 domain)，则 L2TP Client 登录的用户名也要是 username。
- 如果在 LNS 上配置的签证为 fullusername(username@default 或 username@domain)，则 L2TP Client 登录的用户名也要是 username@default 或 username@domain。

这是很多人问过的问题，道理很简单，但却是大家干活时常犯的错误。

讲了半天 domain 的问题，大家会不会问划分 domain 有何意义？

对于大企业来说，往往会按部门来划分多个 domain，LNS(或 AAA 服务器)都支持根据 domain 给不同部门创建不同的地址池，也就是不同部门的网段可以通过地址池规划分开，这样方便后续部署不同的访问控制策略。

3. IPCP 协商，成功后分配 IP 地址。

分配给 Client 的 IP 地址是 10.21.80.2，后续交互的报文都是由 10.21.80.2——Client 的私网地址发出的。

No.	Time	Source	Destination	Protocol	Info
104	25.050847	88.80.5.5	88.80.5.1	PPP	IPCCOnfiguration Ack
105	25.054916	88.80.5.1	88.80.5.5	PPP	IPCCOnfiguration Reject
106	25.055256	88.80.5.5	88.80.5.1	PPP	IPCCOnfiguration Request
107	25.060125	88.80.5.1	88.80.5.5	PPP	IPCCOnfiguration Nak
108	25.060442	88.80.5.5	88.80.5.1	PPP	IPCCOnfiguration Request
109	25.070109	88.80.5.1	88.80.5.5	PPP	IPCCOnfiguration Ack

```

[+] PPP IP Control Protocol
    Code: Configuration Ack (0x02)
    Identifier: 0x04
    Length: 10
    [+] Options: (6 bytes)
        IP address: 10.21.80.2
  
```

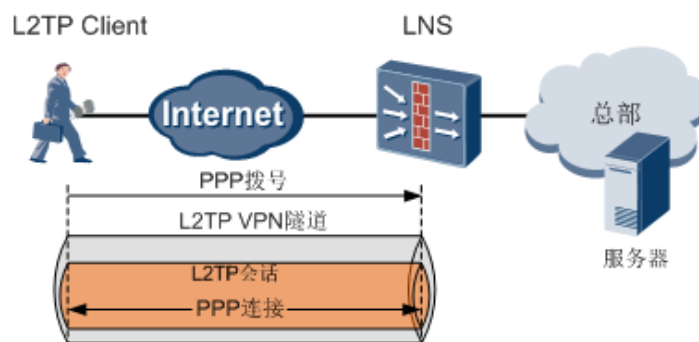
看到这里大家会不会问 VT 接口地址和地址池地址如何规划？注意一点即可：

VT 接口和地址池地址跟其它内网主机地址一样，都应该遵循内网 IP 地址规划原则，建议为 VT 口和地址池规划独立网段，且二者的地址不要重叠。

总结一下 Client-Initiated VPN 的特点：

- L2TP VPN 跟 GRE VPN 有很大不同。GRE VPN 没有隧道协商过程，是没有控制连接的隧道，是没有状态的隧道，所以也无法查看隧道、检查隧道状态。但 L2TP VPN 是有控制连接的隧道，可以查看到隧道和会话。(IPSec VPN 表现的更好，后续再讲)
- 对于 Client-Initiated VPN 来说 Client 和 LNS 之间存在一条 L2TP 隧道，隧道中只有一条

L2TP 会话，PPP 连接就承载在此 L2TP 会话上。（这一点跟 NAS-Initiated VPN 不同，需要关注一下）

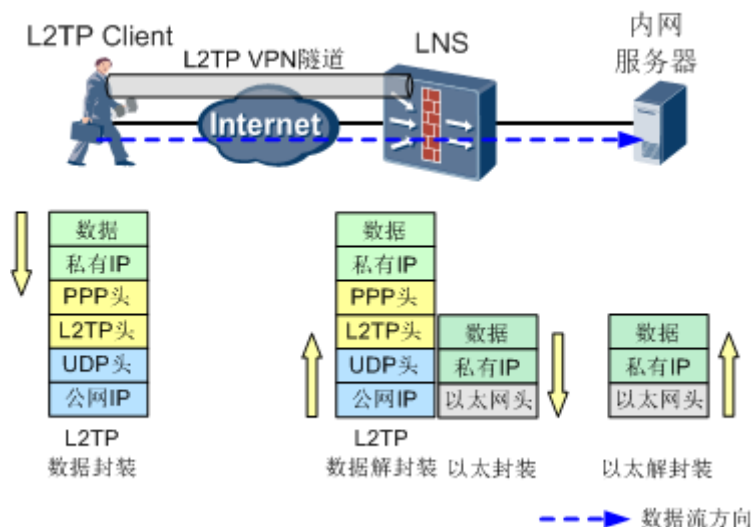


阶段 4 数据封装传输：穿越虫洞，访问地球

想把都教授如何穿越虫洞的过程讲清楚很困难，但是把 L2TP 客户端的数据如何穿越 L2TP 隧道达到总部网络的过程讲清楚不太难，这就涉及到了 L2TP 数据报文的封装过程。这个过程跟 GRE 报文穿“马甲”脱“马甲”的过程很相似，不同的是马甲的样式有点变化：

Frame 1143 (112 bytes on wire, 112 bytes captured)	
Ethernet II, Src: HuaweiTe_57:4e:04 (00:25:9e:57:4e:04), Dst: 54:89:98:73:ef:15 (54:89:98:73:ef:15)	公网 IP
Internet Protocol, Src: 88.80.5.6 (88.80.5.6), Dst: 88.80.5.1 (88.80.5.1)	UDP
User Datagram Protocol, Src Port: 12f (1701), Dst Port: 12f (1701)	L2TP
Layer 2 Tunneling Protocol	PPP
Point-to-Point Protocol	私有 IP
Internet Protocol, Src: 10.21.80.1 (10.21.80.1), Dst: 10.21.80.2 (10.21.80.2)	
Internet Control Message Protocol	

说明：本例中是两个 Client 之间通信，所以私有 IP 地址为两个内网地址。



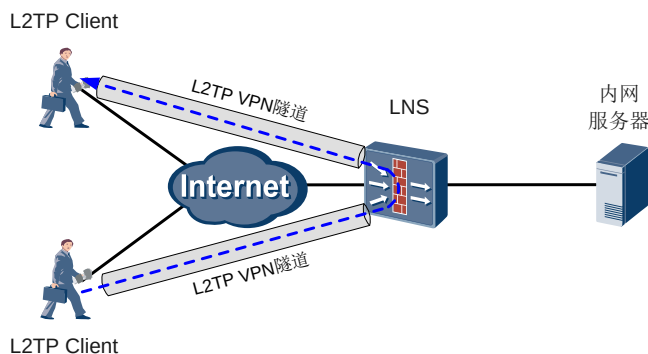
至此，L2TP Client 可以畅通无阻访问总部网络了，就像都教授穿过虫洞回到温暖的家。但有一个疑问，Client 发出的报文进入隧道到达总部服务器没有问题了，但从总部服务器到 Client

的回程报文是如何进入隧道返回 Client 的？我们似乎并没有配置什么路由将回程报文引导到隧道呀？查看 LNS 路由表，发现了一个有趣的现象：**LNS 为获得私网 IP 地址的 Client 自动下发了一条主机路由。**

Destination/Mask	Proto	Pre	Cost	Flags	NextHop	Interface
10.21.3.232/30	Direct	0	0	D	10.21.3.234	GigabitEthernet
1/1/19						
10.21.3.234/32	Direct	0	0	D	127.0.0.1	InLoopBack0
10.21.3.235/32	Direct	0	0	D	127.0.0.1	InLoopBack0
10.21.80.1/32	Unr	0	0	D	10.21.80.1	InLoopBack0
10.21.80.2/32	Unr	0	0	D	10.21.80.2	InLoopBack0
30.0.0.0/8	Direct	0	0	D	30.88.0.1	GigabitEthernet
1/1/15						

这条自动生成的主机路由协议类型为 UNR（User Network Route），下一跳为 Client 本身的地址，出接口是 InLoopBack0。这条路由就是 LNS 上虫洞的入口，指引去往 Client 的报文进入虫洞，回程报文就不愁找不到隧道入口了。看来在虫洞建立时，L2TP 就已经为都教授的来去自如做好了充分准备，那还有什么可担心的呢？

上文我们只使用了一个 Client 来讲解，实际环境中会有多个 Client 同时穿过虫洞访问总部网络。如果 Client 已经不能满足只访问总部网络，还想访问其他的 Client，即 Client 之间实现相互访问，L2TP 能做到吗？别忘了，LNS 是连接多个虫洞的中转站，通过 LNS 来转发，两个 Client 之间也可以自如访问。当然，前提是双方要知道 LNS 为对方分配的 IP 地址。



Client-Initiated VPN 的美妙之处强叔已经展示给各位了，下期强叔要为大家推出 NAS-Initiated VPN，一种更强大的 L2TP VPN。



强叔提问

出差员工上网的环境不固定，可能需要经过 NAT 后才能接入到互联网，那么经过 NAT 后可以正常连接到 LNS 吗？



附录

现在无论 PC、还是 PAD、还是手机都已经考虑到了大家穿越时空的需求，当然强叔也为大家提供了最丰富的参考资料——华为 VPN Client 软件，WINDOWS、苹果、安卓客户端详细配置可以参考《维护宝典》中相关内容。PC 用户通过 L2TP VPN 访问总部同时连接 Internet 的秘诀也在其中，有需要的话可以耐心一步步操作，肯定不会让你失望的。

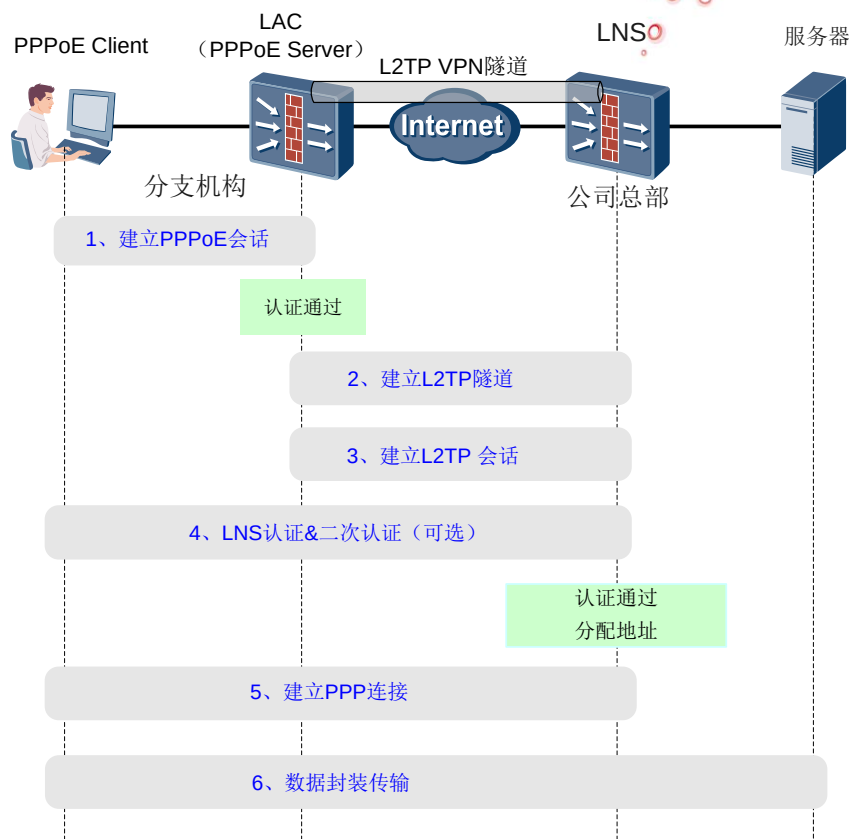
<http://support.huawei.com/enterprise/docinforeader.action?contentId=DOC1000037511&idPath=7919710|9856724|7923140|9859047|5799165>

L2TP NAS-Initiated VPN

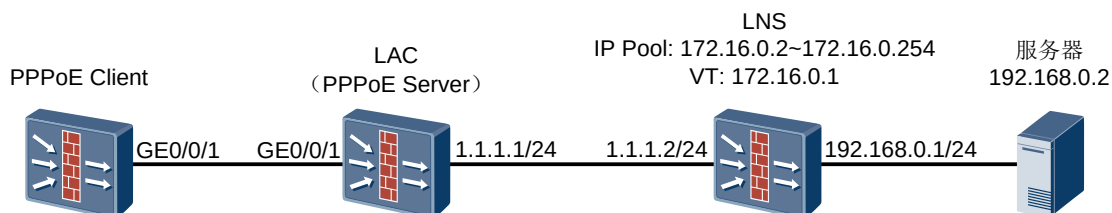
上一篇贴子中我们讲过, Client-Initiated VPN 可以让企业出差员工像都教授一样穿越“虫洞”, 来去自如地访问总部网络。而企业分支机构用户就没有这么幸运, 他们一般通过拨号网络接入 Internet, 面对浩瀚的 Internet 海洋, 没有能力找到“虫洞”的入口, 只能望洋兴叹。即使拨号网络演进到以太网, 也只是解决了本地接入 Internet 的问题, 无法访问总部网络。难道分支机构用户注定与总部网络无缘了吗?

幸好 LAC 横空出世, 帮助分支机构解决了这一难题。一方面, LAC 作为 PPPoE Server, 分支机构用户作为 PPPoE Client 与 LAC 建立 PPPoE 连接, 让 PPP 欢快地跑在以太网上; 另一方面, LAC 作为 LNS 的“中介”, 为分支机构提供“虫洞”的入口, 在分支机构用户看来, 通过 LAC 这扇传送门就可以到达总部网络。

因为在 VPDN 里 LAC 还有一个别名叫做 NAS, 所以这种 L2TP VPN 也称为 NAS-Initiated VPN。可能把 NAS-Initiated VPN 改为 LAC-Initiated VPN 大家更习惯一些, 因为组网图中明明标的是 LAC, 却非得叫一个已经搁置不用的“曾用名”, 这对于后来者而言确实有点不明所以。NAS-Initiated VPN 的建立过程有点小复杂, 为了便于记忆强叔给大家画了一张简图, 方便后续对着这张图一一道来。



为了复习一下 PPPoE 的相关知识,我们用一台防火墙作为 PPPoE Client,模拟 PC 的 PPPoE Client:



阶段 1 拨号口对 VT 口的呼唤：建立 PPPoE 连接

PPP 屈尊落户以太网变为 PPPoE 后,为了在以太网上模拟 PPP 的拨号过程,PPPoE 发明了两个虚拟接口——Dialer 接口和 VT 接口。Dialer 接口用于 PPPoE Client 侧,VT 接口用于 PPPoE Server (即 LAC) 侧,在这两个接口上配置 PPPoE 相关参数。

PPPoE Client	PPPoE Server (LAC)
<pre>interface dialer 1 dialer user user1 dialer-group 1 dialer bundle 1 ip address ppp-negotiate</pre>	<pre>interface Virtual-Template 1 ppp authentication-mode chap interface GigabitEthernet 0/0/1 pppoe-server bind virtual-template 1</pre>
//协商模式下实现 IP 地	//在物理接口上启用 PPPoE Server 并绑定 VT 接口

PPPoE Client

址动态分配

ppp chap user user1 //PPPoE Client 的用户名

ppp chap password cipher Password1 //PPPoE Client 的密码

dialer-rule 1 ip permit

interface GigabitEthernet0/0/1

pppoe-client dial-bundle-number 1 //在物理接口上启用 PPPoE Client 并绑定 dial-bundle

PPPoE Server (LAC)

aaa

local-user user1 password Password1

local-user user1 service-type ppp

注：在 L2TP 中，用户的 IP 地址都是由总部（LNS 或 AAA 服务器）统一进行分配的，所以 LAC 上不需要配置地址池（即使配置了地址池，在 L2TP 隧道已经建立的情况下，也会优先使用总部的地址池进行地址分配），而普通的 PPPoE 拨号则必须在 PPPoE Server 上配置地址池。

通过抓包来分析 PPPoE 连接的建立过程：

No.	Time	Source	Destination	Protocol	Info	
1	0.000000	HuaweiTe_e5:2f:9e	Broadcast	PPPoED	Active Discovery Initiation (PADI)	
2	0.000000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPPoED	Active Discovery Offer (PADO) AC-Name='LAC00000072e700'	PPPoE发现阶段
3	0.047000	HuaweiTe_e5:2f:9e	00:00:00_72:e7:00	PPPoED	Active Discovery Request (PADR) AC-Name='LAC00000072e700'	
4	0.125000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPPoED	Active Discovery Session-confirmation (PADS) AC-Name='LAC00000072e700'	
5	0.141000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP LCP	Configuration Request	
6	0.829000	HuaweiTe_e5:2f:9e	00:00:00_72:e7:00	PPP LCP	Configuration Request	
7	1.032000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP LCP	Configuration Ack	
8	3.079000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP LCP	Configuration Request	PPP LCP协商
9	3.110000	HuaweiTe_e5:2f:9e	00:00:00_72:e7:00	PPP LCP	Configuration Nak	
10	3.141000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP LCP	Configuration Request	
11	3.172000	HuaweiTe_e5:2f:9e	00:00:00_72:e7:00	PPP LCP	Configuration Request	
12	3.204000	HuaweiTe_e5:2f:9e	00:00:00_72:e7:00	PPP LCP	Configuration Ack	
13	3.657000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP LCP	Configuration Ack	
14	3.672000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP CHAP	Challenge (NAME='', VALUE=0x903ec8fe2f32339f611faa450ef646d4)	PPP CHAP认证
15	3.813000	HuaweiTe_e5:2f:9e	00:00:00_72:e7:00	PPP CHAP	Response (NAME='user1', VALUE=0x038f908ad4a5f88ab1bf4a50702e177d)	
16	4.657000	00:00:00_72:e7:00	HuaweiTe_e5:2f:9e	PPP CHAP	Success (MESSAGE='welcome to .')	

这里重点介绍 PPPoE 发现阶段的协商过程，PPPoE Client 和 PPPoE Server 之间通过交互 PADI、PADO、PADR 和 PADS 报文，确定对方以太网地址和 PPPoE 会话 ID：

步骤 1	PPPoE Client: 广播广播，我想接入 PPPoE，谁来帮帮我？	PPP-over-Ethernet Discovery 0001 = Version: 1 0001 = Type: 1 Code: Active Discovery Initiation (PADI) (0x09) Session ID: 0x0000 Payload Length: 10 PPPoE Tags
步骤 2	PPPoE Server: PPPoE Client，找我呀，我可以帮助你！	PPP-over-Ethernet Discovery 0001 = Version: 1 0001 = Type: 1 Code: Active Discovery offer (PADO) (0x07) Session ID: 0x0000 Payload Length: 29 PPPoE Tags
步骤 3	PPPoE Client: 太好了，PPPoE Server，我想跟你建立 PPPoE 会话。	PPP-over-Ethernet Discovery 0001 = Version: 1 0001 = Type: 1 Code: Active Discovery Request (PADR) (0x19) Session ID: 0x0000 Payload Length: 29 PPPoE Tags
步骤 4	PPPoE Server: 没问题，我把会话 ID 发给你，我们就用这个 ID 建立 PPPoE 会话吧。	PPP-over-Ethernet Discovery 0001 = Version: 1 0001 = Type: 1 Code: Active Discovery Session-confirmation (PADS) (0x06) Session ID: 0x0001 Payload Length: 29 PPPoE Tags

阶段2 建立 L2TP 隧道：3 条消息协商进入虫洞时机

LAC 和 LNS 通过交互三条消息协商 L2TP 隧道，这个过程我们在“Client-Initiated VPN”一篇中已经介绍过了，这里再复习一遍。首先来看一下 LAC 和 LNS 的具体配置：

PPPoE Client	PPPoE Server (LAC)
I2tp-group 1 tunnel authentication //避免假冒 LAC 接入 LNS tunnel password cipher Password1 tunnel name lac start I2tp ip 1.1.1.2 fullusername user1 //指定隧道对端地址 I2tp enable	I2tp-group 1 tunnel authentication //避免假冒 LAC 接入 LNS tunnel password cipher Password1 tunnel name lns allow I2tp virtual-template 1 remote lac //允许远端接入 I2tp enable

抓包信息如下：

No.	Time	Source	Destination	Protocol	Info
1	0.000000	1.1.1.1	1.1.1.2	L2TP	Control Message - SCCRQ (tunnel id=0, session id=0)
2	0.000000	1.1.1.2	1.1.1.1	L2TP	Control Message - SCCRP (tunnel id=1, session id=0)
3	0.000000	1.1.1.1	1.1.1.2	L2TP	Control Message - SCCCN (tunnel id=1, session id=0)

隧道 ID 协商过程：

步骤 1	LAC: LNS, 使用 1 作为 Tunnel ID 跟我通信吧。	Assigned Tunnel ID AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Tunnel ID (9) Tunnel ID: 1
步骤 2	LNS: OK。LAC, 你也用 1 作为 Tunnel ID 跟我通信。	Assigned Tunnel ID AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Tunnel ID (9) Tunnel ID: 1
步骤 3	LAC: OK。	-

阶段3 建立 L2TP 会话：3 条消息唤醒虫洞门神

LAC 和 LNS 通过交互三条消息协商 Session ID，建立 L2TP 会话。同样，我们再复习一遍这个过程。

4	0.000000	1.1.1.1	1.1.1.2	L2TP	Control Message - ICRQ (tunnel id=1, session id=0)
5	0.000000	1.1.1.2	1.1.1.1	L2TP	Control Message - ICRP (tunnel id=1, session id=4)
6	0.000000	1.1.1.1	1.1.1.2	L2TP	Control Message - ICCN (tunnel id=1, session id=4)

Session ID 协商过程:

步骤 1	LAC: LNS, 使用 4 作为 Session ID 跟我通信吧。	Assigned Session AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Session (14) Assigned Session: 4
步骤 2	LNS: OK。LAC, 你也使用 4 作为 Session ID 跟我通信吧。	Assigned Session AVP Mandatory: True Hidden: False Length: 8 Vendor ID: Reserved (0) Type: Assigned Session (14) Assigned Session: 4
步骤 3	LAC: OK。	-

阶段 4-5 LNS 冷静接受 LAC: LNS 认证, 分配 IP 地址

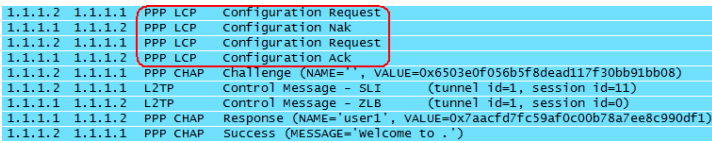
1. LNS 认证&二次认证 (可选)

LAC 将用户信息发给 LNS 进行验证, 但 LNS 清醒认识到 LAC“中介”的本来面目, 对此 LNS 有三种态度:

- 🍁 LAC 代理认证: 相信 LAC 是可靠的, 直接对 LAC 发来的用户信息进行验证。
- 🍁 强制 CHAP 认证: 不相信 LAC, 要求重新对用户进行“资格审查”(强制重新对用户进行 CHAP 验证)。
- 🍁 LCP 重协商: 不仅不相信 LAC, 还对前面签订的业务合同不满, 要求跟用户重新“洽谈业务”(重新发起 LCP 协商, 协商 MRU 参数和认证方式)。

后两种方式统称为 LNS 二次认证, 若 LNS 配置二次认证而 PPPoE Client 不支持二次认证, 将会导致 L2TP VPN 无法建立。两种二次认证的共同特征是 LNS 都绕过了 LAC 直接验证 PPPoE Client 提供的用户信息, 可以为 VPN 业务提供了更高的安全保障。

认证方式	配置	特点
LAC 代理认证 ★	缺省, 不用配置	1.1.1.2 1.1.1.1 PPP CHAP Success (MESSAGE='welcome to .') LNS 直接对 LAC 发来的用户信息进行验证, 验证通过即成功建立 PPP 连接。
强制 CHAP 认证 ★★	l2tp-group 1 mandatory-chap	1.1.1.2 1.1.1.1 PPP CHAP Challenge (NAME='', VALUE=0xeb52bb7eb6aee6aad516f044de419a0) 1.1.1.2 1.1.1.1 L2TP Control Message - SLI (tunnel id=1, session id=8) 1.1.1.1 1.1.1.2 PPP CHAP Response (NAME='user1', VALUE=0x4a045149d6946c7d6abeb79dd0de900f) 1.1.1.2 1.1.1.1 PPP CHAP Success (MESSAGE='welcome to .') LNS 重新对用户进行 CHAP 验证, LNS 发送挑战, PPPoE Client 使用挑战将用户名和加密后的密码发给 LNS, LNS 验证通过成功建立 PPP 连接。

认证方式	配置	特点
LCP 重协商★★★	<pre>interface virtu- al-template 1 ppp authentica- tion-mode chap //重 协商后的验证方式在 VT 下指定 l2tp-group 1 mandatory-lcp</pre>	 LNS 重新发起 LCP 协商，协商 MRU 参数和认证方式，然后进行 CHAP 验证，验证通过即成功建立 PPP 连接。

★代表优先级，三种方式同时配置时 LCP 重协商优先级最高。

2. 分配 IP 地址。

通过 PPP IPCP 协商，LNS 为 PPPoE Client 分配 IP 地址。

13	0.000000	1.1.1.1	1.1.1.2	PPP IPCP	Configuration Request
14	0.000000	1.1.1.2	1.1.1.1	PPP IPCP	Configuration Nak
15	0.000000	1.1.1.1	1.1.1.2	PPP IPCP	Configuration Ack
16	0.000000	1.1.1.1	1.1.1.2	PPP IPCP	Configuration Request
17	0.000000	1.1.1.2	1.1.1.1	PPP IPCP	Configuration Ack

```

PPP IP Control Protocol
  Code: Configuration Ack (0x02)
  Identifier: 0x02
  Length: 10
  Options: (6 bytes)
    IP address: 172.16.0.2

```

关于地址池地址的规划问题，我们在“Client-Initiated VPN”一篇中也讲过。同样，建议把地址池地址和总部网络地址规划为不同的网段。如果地址池地址和总部网络地址配置为同一网段，则必须在 LNS 连接总部网络的接口上开启 ARP 代理功能，并且开启 L2TP 虚拟转发功能，保证 LNS 可以对总部网络服务器发出的 ARP 请求进行应答。

假设 LNS 连接总部网络的接口是 GigabitEthernet0/0/1，开启 ARP 代理功能和 L2TP 虚拟转发功能的配置如下：

```

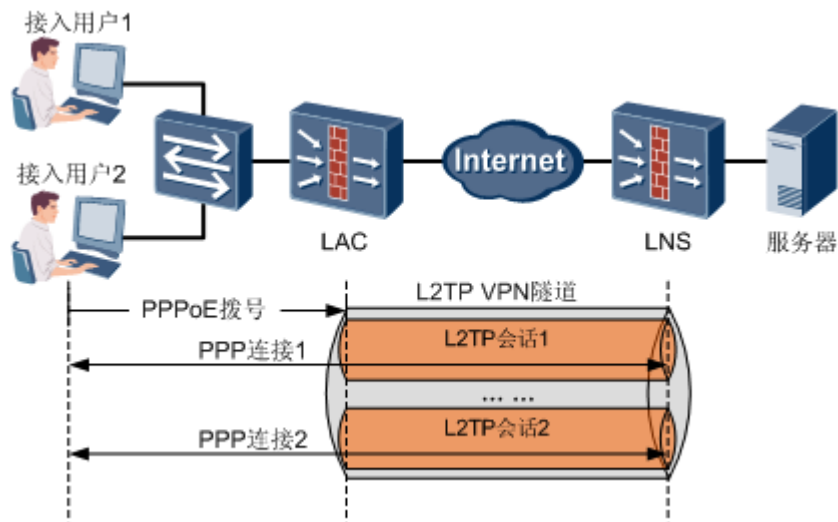
interface GigabitEthernet0/0/1
ip address 192.168.0.1 255.255.255.0
arp-proxy enable //开启 ARP 代理
virtual-l2tpforward enable //开启 L2TP 虚拟转发

```

总结一下 NAS-Initiated VPN 的特点：

NAS-Initiated VPN 场景中，一对 LAC 和 LNS 的连接可存在多条隧道；一条隧道中可承载多条会话。接入用户 1 与 LNS 之间建立 PPP 连接 1 和 L2TP 会话 1，接入用户 2 与 LNS 之间

建立 PPP 连接 2 和 L2TP 会话 2。



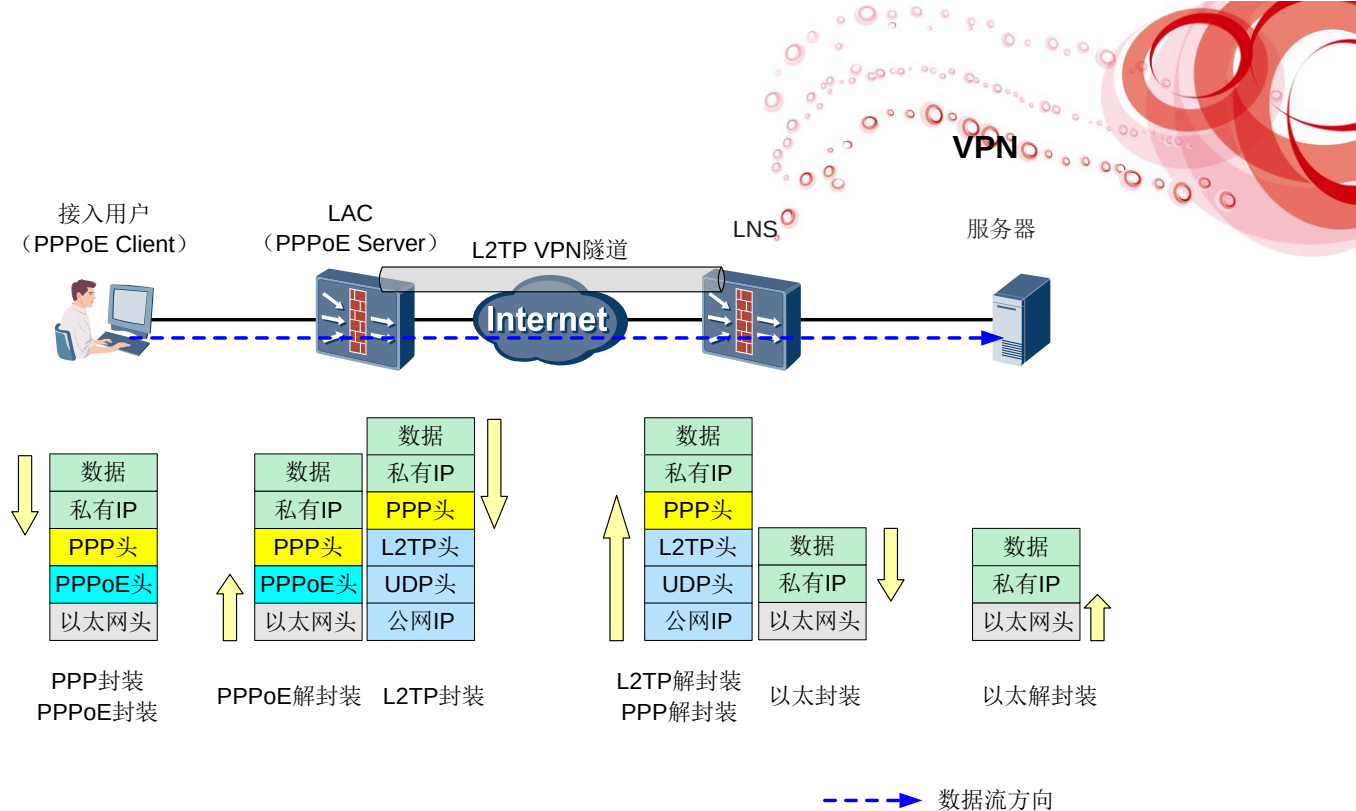
当其中一个用户拨号后，触发 LAC 和 LNS 之间建立隧道。只要此用户尚未下线，则其余用户拨号时，会在已有隧道基础上建立会话，而并非重新触发建立隧道。

阶段 6 一路畅通：数据封装传输

PPPoE Client 访问总部服务器的报文到达 LAC 后，LAC 为报文穿上三层“马甲”，即 L2TP 头、UDP 头和公网 IP 头，然后发送到 LNS。LNS 收到报文后，脱去这三层“马甲”，将报文转发至服务器。

Frame 6: 136 bytes on wire (1088 bits), 136 bytes captured (1088 bits)	
Ethernet II, Src: 00:00:00_72:e7:01 (00:00:00:72:e7:01), Dst: 00:00:00_d6:91:00 (00:00:00:d6:91:00)	
Internet Protocol, Src: 1.1.1.1 (1.1.1.1), Dst: 1.1.1.2 (1.1.1.2)	公网IP
User Datagram Protocol, Src Port: 60416 (60416), Dst Port: 12f (1701)	UDP
Layer 2 Tunneling Protocol	L2TP
Point-to-Point Protocol	PPP
Internet Protocol, Src: 172.16.0.2 (172.16.0.2), Dst: 192.168.0.2 (192.168.0.2)	私网IP
Internet Control Message Protocol	

报文封装和解封装的过程如下图所示：



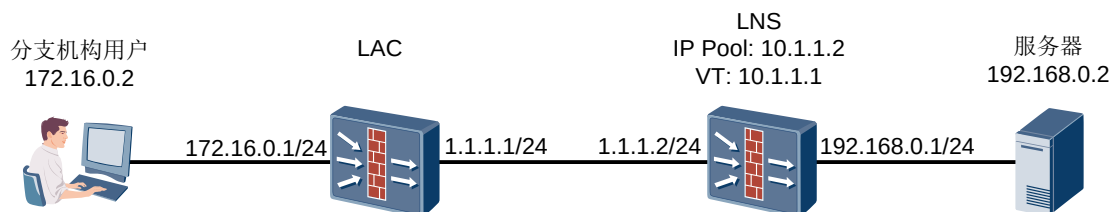
借助于 LAC 的力量，分支机构员工使用 PPPoE Client 就可以畅通无阻访问总部网络。但是从总部服务器到 PPPoE Client 的回程报文是如何进入隧道返回的呢？还记得我们在“Client-Initiated VPN”一篇中介绍过的 UNR 路由吧，NAS-Initiated VPN 也是这样处理的。LNS 会为获得 IP 地址的 PPPoE Client 自动下发了一条 UNR 路由，下一跳为 PPPoE Client 本身的地址，出接口是 InLoopBack0，这条路由将指引回程报文进入隧道。回程报文进入隧道后被封装上公网 IP 地址，然后 LNS 将会以该公网 IP 地址为目的地址再次查找路由，将封装后的回程报文送往 LAC。

NAS-Initiated VPN 场景中，分支机构用户必须拨号才能使用 L2TP VPN，报文还要封装成 PPPoE，太麻烦了。况且拨号网络逐渐消失，以太网一统江湖，分支机构用户就不能直接在以太网上访问总部网络吗？当然可以，人类偷懒的需求才是科技进步的原动力。下期强叔就为大家介绍 LAC-Auto-Initiated VPN，由 LAC 自动拨号到 LNS，省去了分支机构员工拨号的过程，堪称最省事的 L2TP VPN，敬请期待。

L2TP LAC-Auto-Initiated VPN

LAC-Auto-Initiated VPN 也叫做 LAC 自动拨号 VPN，顾名思义，在 LAC 上配置完成后，LAC 会自动向 LNS 发起拨号，建立 L2TP 隧道和会话，不需要分支机构用户拨号来触发。对于分支机构用户来说，访问总部网络就跟访问自己所在的分支机构网络一样，完全感觉不到自己是在远程接入。但是这种方式下 LNS 只对 LAC 进行认证，分支机构用户只要能连接 LAC 即可以使用 L2TP 隧道接入总部，与 NAS-Initiated VPN 相比安全性要差一些。

LAC-Auto-Initiated VPN 的配置也不复杂，以下面这个组网为例：

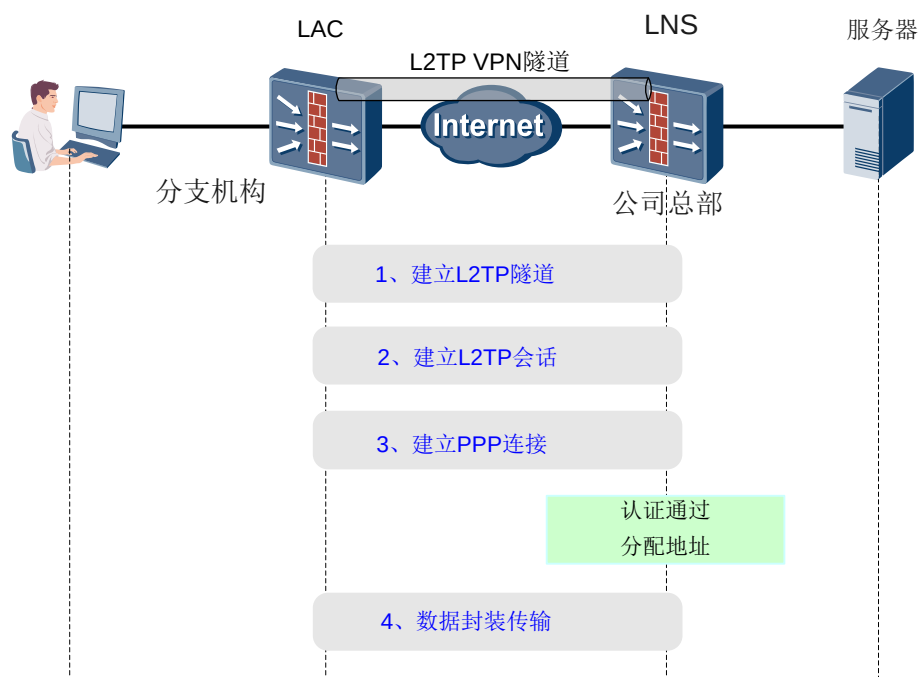


LAC 和 LNS 上的配置如下：

LAC	LNS
<pre> interface Virtual-Template 1 ppp authentication-mode chap ppp chap user lac ppp chap password cipher Password1 ip address ppp-negotiate call-lns local-user lac binding l2tp-group 1 //LAC 向 LNS 发起拨号 l2tp-group 1 tunnel authentication tunnel password cipher Password1 tunnel name lac start l2tp ip 1.1.1.2 fullusername lac //指定隧道对 端地址 l2tp enable ip route-static 192.168.0.0 255.255.255.0 Virtual-Template 1 //配置去往总部网络的静态路由， 此处与 Client-Initiated VPN 以及 NAS-Initiated VPN 不同， LAC 上必须配置该条路由，指引分支机构用户访问总部网 络的报文进入 L2TP 隧道 </pre>	<pre> interface Virtual-Template 1 ppp authentication-mode chap ip address 10.1.1.1 255.255.255.0 remote address pool 1 l2tp-group 1 tunnel authentication tunnel password cipher Password1 tunnel name lns allow l2tp virtual-template 1 remote lac //允许远端接 入 l2tp enable aaa local-user lac password Password1 local-user lac service-type ppp ip pool 1 10.1.1.2 ip route-static 172.16.0.0 255.255.255.0 Virtual-Template 1 //配置去往分支机构网络的静态路由，如果 LAC 上配置了源 NAT，则无需配置该条路由，详细情况后文介绍 </pre>

LAC-Auto-Initiated VPN 的建立过程与 Client-Initiated VPN 类似，只不过在

LAC-Auto-Initiated VPN 中，LAC 取代了 Client-Initiated VPN 中 L2TP Client 的角色。

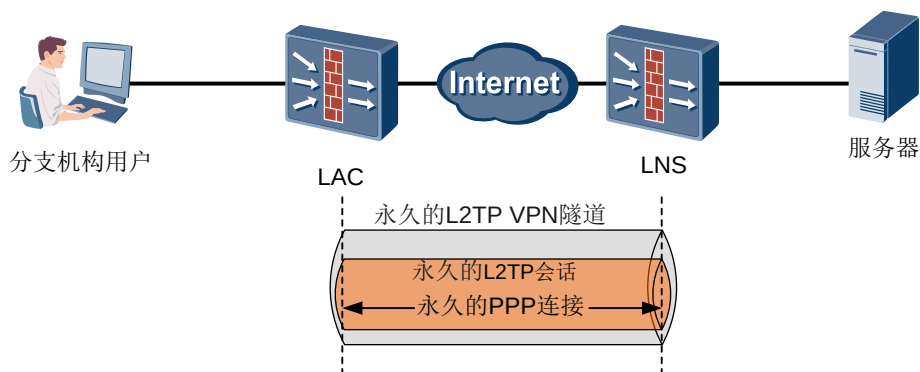


各个阶段的建立过程与 Client-Initiated VPN 的建立过程大同小异，大家可以温习《VPN 篇 L2TP Client-Initiated VPN》，强叔在这里就不多说了。需要注意的一点是，在阶段 3 中，LNS 只对 LAC 进行验证，验证通过后为 LAC 的 VT 接口分配 IP 地址，而不是为分支机构用户分配 IP 地址。虽然 LNS 不为分支机构分配 IP 地址，但是并不代表分支机构的 IP 地址可以随意配置。

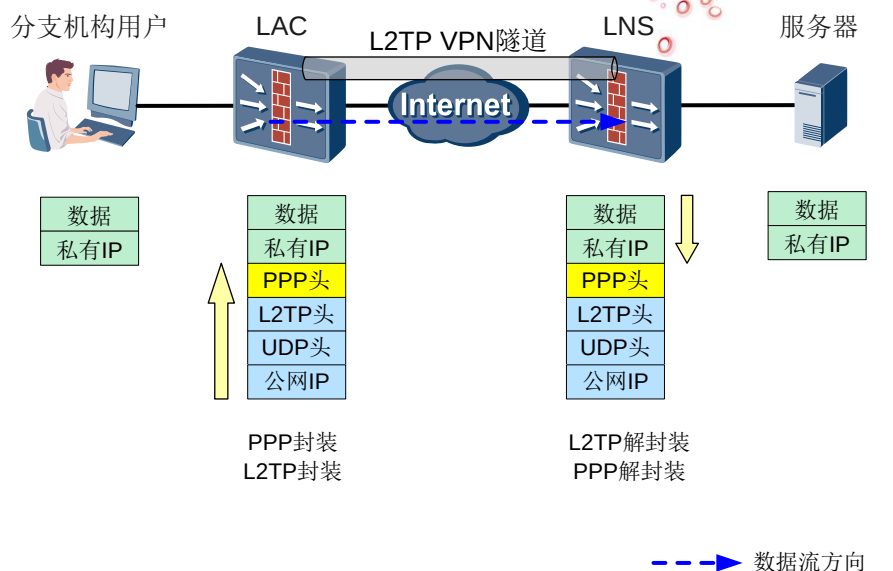
为了保证分支机构网络与总部网路之间正常访问，请为分支机构网络和总部网络规划各自独立的私网网段，要求二者的网段地址不能重叠。

总结一下 LAC-Auto-Initiated VPN 的特点：

LAC-Auto-Initiated VPN 场景中，LAC 和 LNS 之间建立一条永久的隧道，且仅承载一条永久的 L2TP 会话和 PPP 连接。L2TP 会话和 PPP 连接只存在于 LAC 和 LNS 之间。



LAC-Auto-Initiated VPN 的 PPP 封装和 L2TP 封装仅限于 LAC 和 LNS 之间的报文交互：



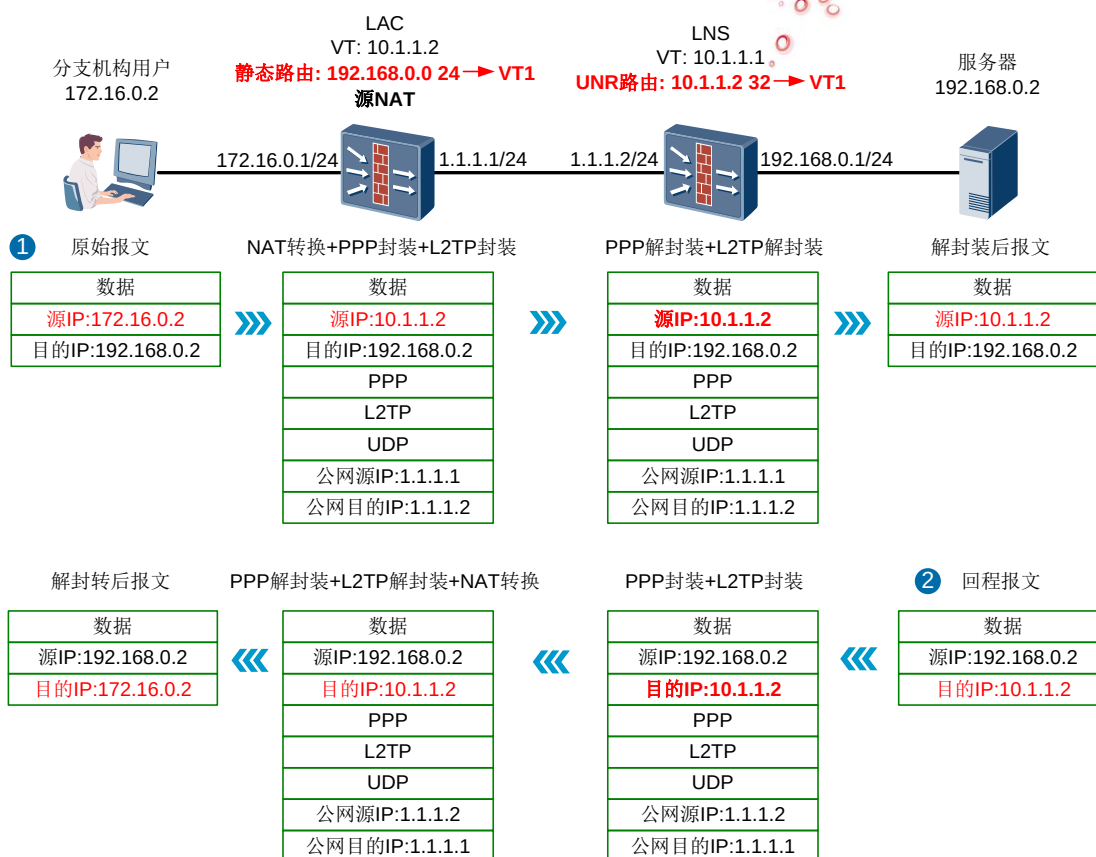
另外，还有一个需要重点关注的问题是回程报文如何进入隧道。与 Client-Initiated VPN 以及 NAS-Initiated VPN 不同，在 LAC-Auto-Initiated VPN 中，LNS 只下发了一条目的地址为 LAC 的 VT 接口地址的 UNR 路由，并没有去往分支机构网络的路由。对此 LNS 振振有词：“我只对我分配出去的 IP 地址负责，所以保证可以到达对端 LAC 的 VT 接口。分支机构网络的地址不是我分配的，我甚至都不知道它们的地址是什么，因此只能说抱歉。”

那么如何解决这个问题呢？最简单方法就是在 LNS 手动配置一条去往分支机构网络的静态路由，指引回程报文进入隧道：

```
ip route-static 172.16.0.0 255.255.255.0 Virtual-Template 1
```

除了配置静态路由之外，还有没有其他方法呢？强叔突然灵光一现，还记得我们之前讲过的 NAT 吧！既然 LNS 只认他分配的 IP 地址，那我们就在 LAC 上配置源 NAT 功能，把分支机构用户访问总部网络报文的源地址都转换成 VT 接口的地址，即 easy-IP 方式的源 NAT。LNS 收到回程报文后，发现目的地址是 LAC 的 VT 接口的 IP 地址，就会按照 UNR 路由进入隧道转发，这样就无需在 LNS 上配置静态路由了。

字不如图，下面强叔就以实际组网为例，介绍一下 LAC 上配置了源 NAT 后，分支机构用户访问总部服务器时的报文封装与解封装全过程：



1. LAC 收到分支机构用户访问总部服务器的原始报文后，根据目的地址查找路由，命中我们手动配置的静态路由，将报文发送至 VT 接口。
2. LAC 在 VT 接口对原始报文进行 NAT 转换，将源地址转换成 VT 接口的地址，然后为报文封装 PPP 头、L2TP 头和公网地址。LAC 根据公网目的地址查找路由，将封装后的报文发送至 LNS。
3. LNS 收到报文后，剥离掉 PPP 头、L2TP 头，根据目的地址查找路由（此处为直连路由），然后将报文发送至总部服务器。
4. LNS 收到总部服务器的回程报文后，根据目的地址查找路由，命中 LNS 自动下发的 UNR 路由，将报文发送至 VT 接口。
5. 报文在 VT 接口封装 PPP 头、L2TP 头和公网地址，LNS 根据公网目的地址查找路由，将封装后的报文发送至 LAC。
6. LAC 收到报文后，剥离掉 PPP 头、L2TP 头，将报文的地址转换成分支机构用户的地址，然后将报文发送至分支机构用户。

在 LAC 上配置 easy-IP 方式源 NAT 的示例如下（假设在 LAC 上连接分支机构网络的接口属于 Trust 区域，VT 接口属于 Untrust 区域）：

```

nat-policy interzone trust untrust outbound
policy 1
action source-nat
policy source 172.16.0.0 0.0.0.255
easy-ip Virtual-Template 1

```

至此，强叔用三期的篇幅介绍了三种 L2TP VPN，下面我们再来对这三种 L2TP VPN 做一下总结：

项目	Client-Initiated VPN	NAS-Initiated VPN	LAC-Auto-Initiated VPN
协商方式	L2TP Client 和 LNS 协商建立 L2TP 隧道和 L2TP 会话、建立 PPP 连接	接入用户使用 PPPoE 拨号触发 LAC 和 LNS 之间协商建立 L2TP 隧道和 L2TP 会话，接入用户和 LNS 协商建立 PPP 连接	LAC 主动拨号，和 LNS 协商建立 L2TP 隧道和 L2TP 会话、建立 PPP 连接
隧道和会话关系	每个 L2TP Client 和 LNS 之间均建立一条 L2TP 隧道，每条隧道中仅承载一条 L2TP 会话和 PPP 连接	LAC 和 LNS 的连接可存在多条 L2TP 隧道，一条 L2TP 隧道中可承载多条 L2TP 会话	LAC 和 LNS 之间建立一条永久的 L2TP 隧道，且仅承载一条永久的 L2TP 会话和 PPP 连接
安全性	LNS 对 L2TP Client 进行 PPP 认证（PAP 或 CHAP），安全性较高	LAC 对接入用户进行认证，LNS 对接入用户进行二次认证（可选），安全性最高	LAC 不对用户进行认证，LNS 对 LAC 配置的用户进行 PPP 认证（PAP 或 CHAP），安全性低
回程路由配置	LNS 上会自动下发 UNR 路由，指导回程报文进入 L2TP 隧道，无需手动配置	LNS 上会自动下发 UNR 路由，指导回程报文进入 L2TP 隧道，无需手动配置	LNS 上需要手动配置目的地址为网段的静态路由，或者在 LAC 上配置 easy-IP 方式的源 NAT

L2TP VPN 就告一段落了，接下来强叔将为大家介绍功能强大配置复杂的 IPsec VPN，希望大家继续关注强叔系列技术贴。

IPsec 闪亮登场

危机四伏

随着 GRE 和 L2TP 的广泛应用，民间组织天地会也与时俱进，在总舵和各个分舵之间部署了 GRE 和 L2TP。总舵和分舵通过 GRE 隧道和 L2TP 隧道传递消息，互通有无，反清复明事业开展得如火如荼。但好景不长，总舵和分舵之间传递的机密信息被官府查获，分舵的堂主帮众悉数被抓，一时间 Internet 上暗潮汹涌，天地会危机四伏。

事关帮派存亡，陈总舵主赶忙召集会议商讨对策，究其原因，无论是 GRE 还是 L2TP，建立的隧道都没有任何安全加密措施，总舵和分舵之间的消息在 GRE 隧道或 L2TP 隧道中都是明文传输，机密信息很容易就被官府获取。如何保证信息安全传输是摆在天地会面前的一道难题，购买专线是个解决方法，但由于四十二章经中的宝藏还未找到，天地会的财力不足以搭建总舵到各个分舵的专线，所以还得利用现有的公共资源---Internet 来想办法。

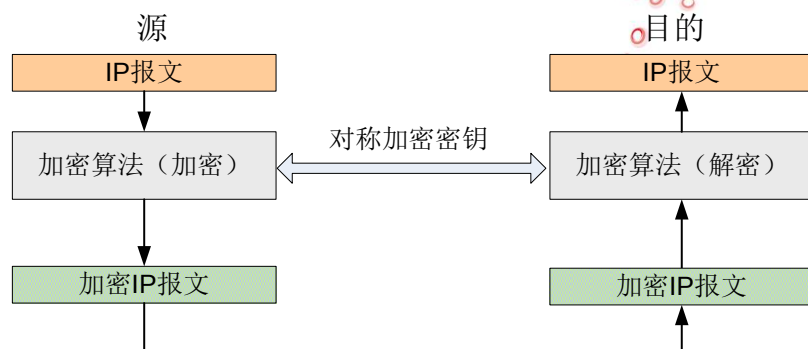
遍访高人之后，天地会终于找到了解决问题之道：IPSec (IP Security)。作为新一代的 VPN 技术，IPSec 可以在 Internet 上建立安全稳定的专用线路。与 GRE 和 L2TP 相比，IPSec 更加安全，能够保证总舵与分舵之间消息的安全传输。

两大绝技

要说这 IPSec 可不简单，它不是一个单一的招式，而是一整套招法。IPSec 巧妙借用了密码学门派所擅长的易容障眼之法，融会到自创的 AH (Authentication Header, 验证头) 和 ESP (Encapsulating Security Payload, 封装安全载荷) 两大绝技中，既可改头换面瞒天过海，也能验明正身完璧归赵。即便消息被截获了也没人能看懂，被篡改了也能及时发现。

改头换面，瞒天过海---加密

IPSec 借用了易容术的巧妙之处，一方传递消息之前，先使用加密算法和加密密钥，将消息改头换面，该过程称为加密；另一方收到消息后，使用相同的加密算法和加密密钥，逆向将消息恢复为真实面貌，该过程称为解密。而消息在传递过程中绝不以真容示人，令窃密者一无所获。



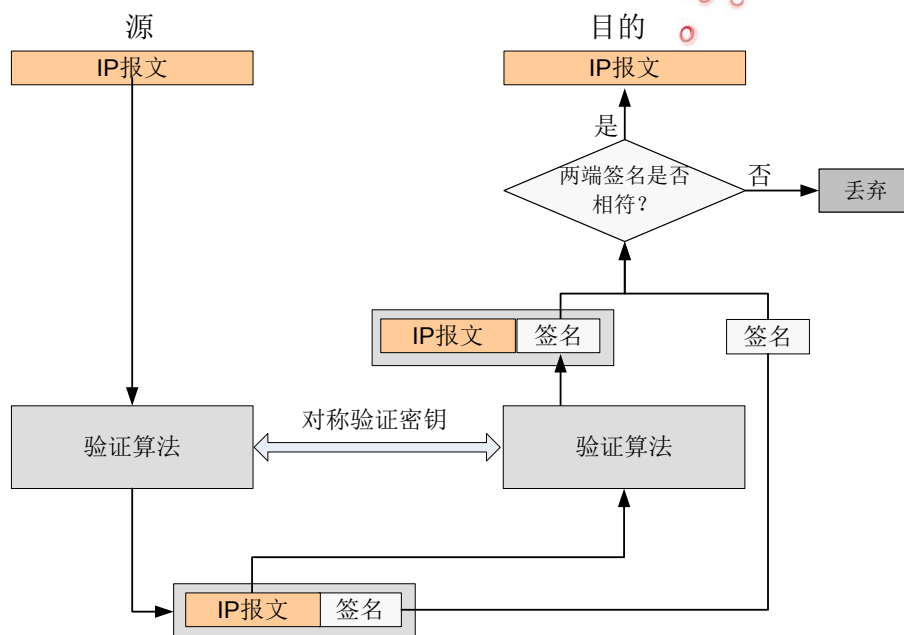
当天地会的总舵和分舵需要互通消息时，双方事先商议好相同的加密算法和加密密钥，假设总舵要向分舵发布命令“八月十五，太湖岸边，共举大事”，总舵先使用颠三倒四加密算法，将文字颠倒位置，然后再将加密密钥“反清复明”插入其中，加密后的命令最终变成“反十五太湖清岸边共举复大事八月明”这样的一段消息，然后传递出去。即使该消息在路上被官府截获，也会让官府看的一头雾水，不会发现任何蛛丝马迹。而分舵收到消息后，使用相同的颠三倒四加密算法和加密密钥“反清复明”，便可将消息恢复成原始的命令“八月十五，太湖岸边，共举大事”。

总舵和分舵使用相同的密钥来加密和解密，这种方式也叫对称加密算法，主要包括 DES、3DES 和 AES。

项目	DES		3DES			AES	
全称	Data	Encryption	Triple	Data	Encryption	Advanced	Encryption
	Standard		Standard			Standard	
密钥长度	56 位		168 位			128 位、192 位、256 位	
安全级别	低		中			高	

🍁 验明正身，完璧归赵---验证

古有蔺相如忠肝义胆，完璧归赵，今有 IPsec 火眼金睛，明辨是非。一方传递消息之前，先使用验证算法和验证密钥对消息进行处理，得到签字画押的文书，即签名。然后将签名随消息一同发出去。另一方收到消息后，也使用相同的验证算法和验证密钥对消息进行处理，同样得到签名，然后比对两端的签名，如果相同则证明该消息没有被篡改。



除了对消息的完整性进行验证，IPSec 还可以对消息的来源进行验证，即验明消息的正身，保证消息来自真实的发送者。

通常情况下，验证和加密配合使用，加密后的报文经过验证算法处理生成签名。常用的验证算法有 MD5 和 SHA 系列。

项目	MD5	SHA1	SHA2
全称	Message Digest 5	Secure Hash Algorithm 1	Secure Hash Algorithm 2
签名长度	128 位	160 位	SHA2-256: 256 位 SHA2-384: 384 位 SHA2-512: 512 位
安全级别	低	中	高

IPSec 的两大绝技中，AH 只能用来验证，没有加密的功能，而 ESP 同时具有加密和验证的功能，AH 和 ESP 可以单独使用也可以配合使用。

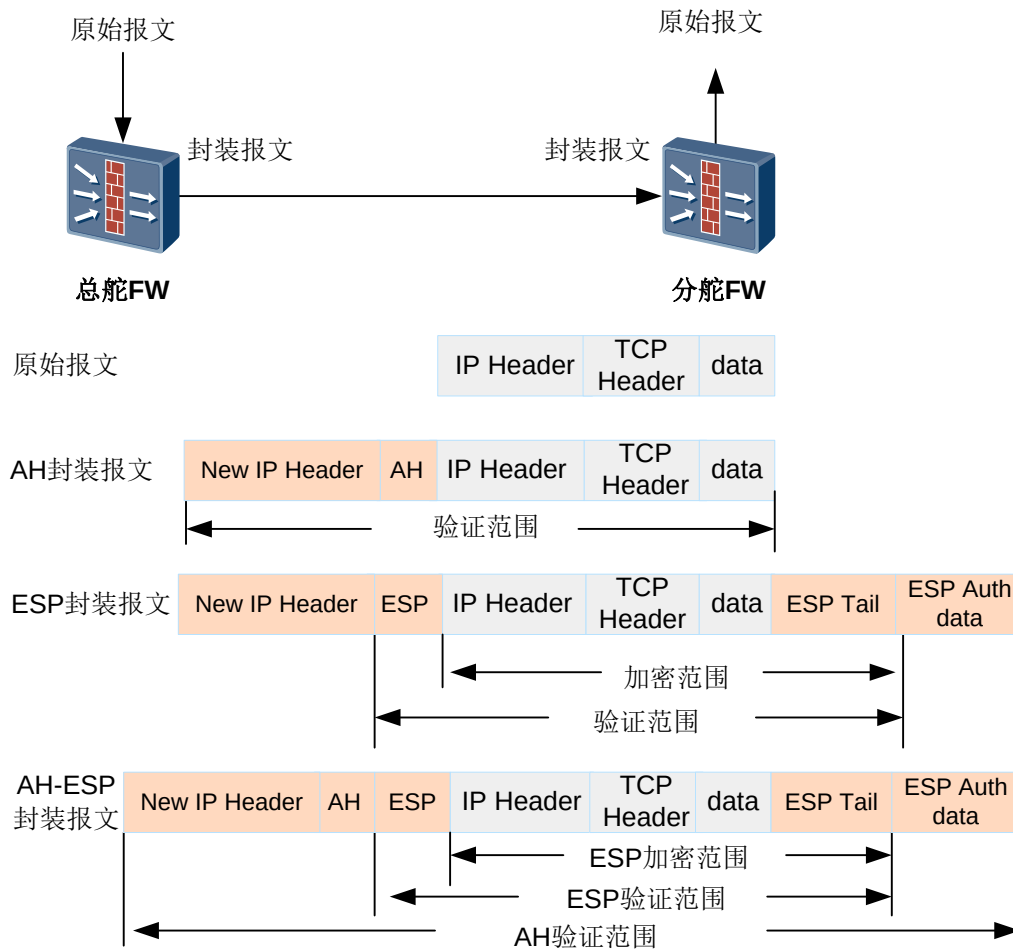
安全封装

天地会不能将反清复明的招牌挂于明处，所以只能用合法生意作为掩护。例如，总舵对外的公开身份是当铺，分舵对外的公开身份是票号，当铺和票号之间正常的生意往来就是最好的保护伞。在为了更好地利用这个保护伞，IPSec 设计了两种封装模式：

🌟 明修栈道暗度陈仓---隧道模式

在隧道模式下，AH 头或 ESP 头被插到原始 IP 头之前，另外生成一个新的报文头放到 AH 头

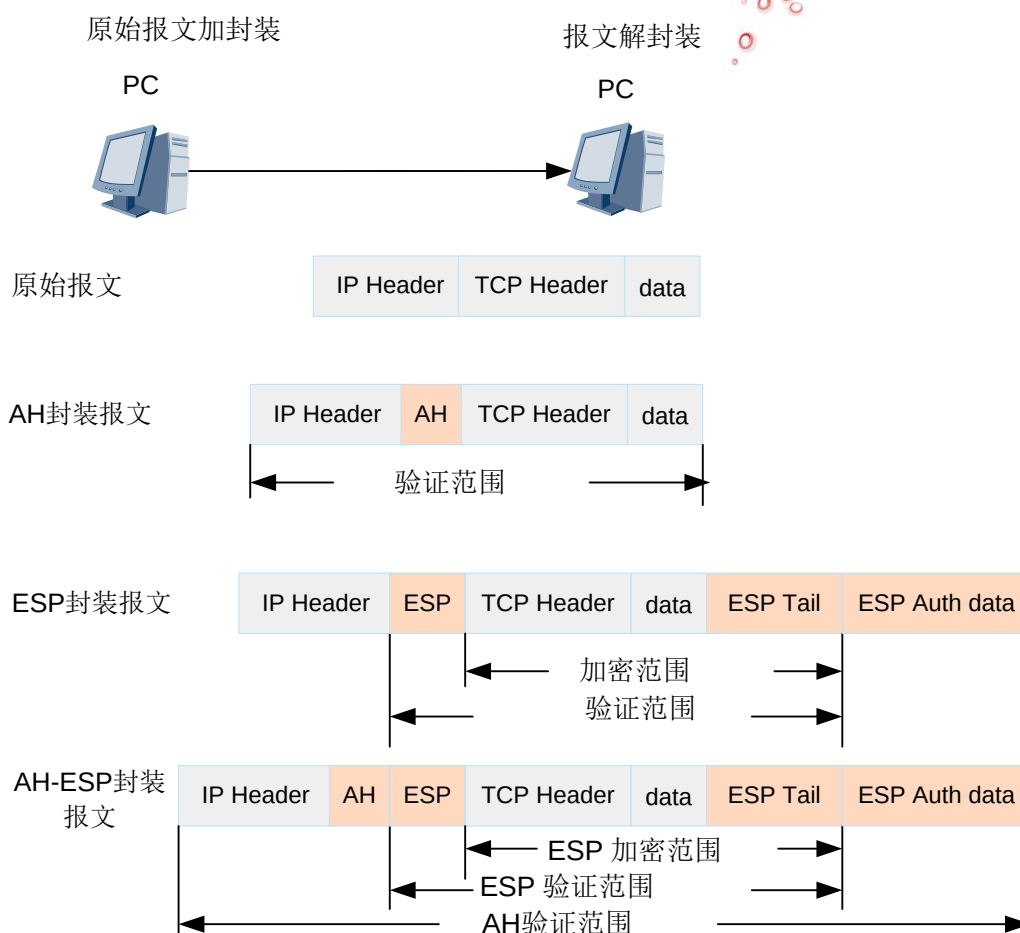
或 ESP 头之前:



隧道模式使用新的报文头来封装消息,可以保护一个网络的消息,适用于两个网关之间通信,是比较常用的封装模式。总舵和分舵内部私网之间的信息经过加密和封装处理后,在外看来只是总舵和分舵的公开身份(公网地址),即当铺和票号之间的通信,不会被人怀疑。

🍁 开门见山直来直往---传输模式

在传输模式中, AH 头或 ESP 头被插入到 IP 头与传输层协议头之间:

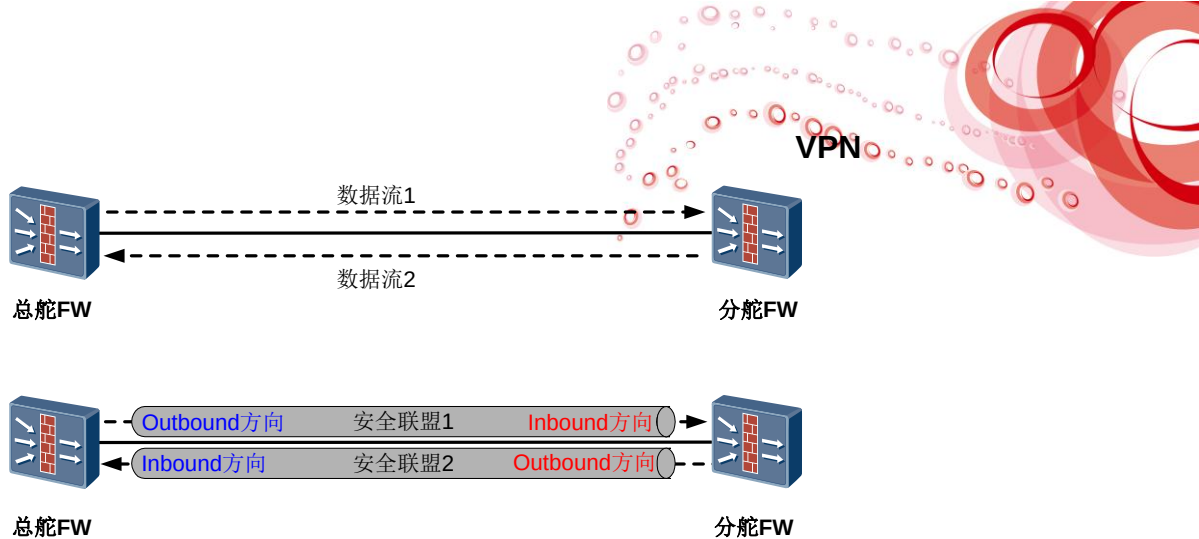


传输模式不改变报文头，隧道的源和目的地址就是最终通信双方的源和目的地址，通信双方只能保护自己发出的消息，不能保护一个网络的消息。所以该模式只适用于两台主机之间通信，不适用于天地会总舵和分舵内部私网之间通信。

互信盟友

IPSec 中通信双方建立的连接叫做安全联盟 SA (Security Association)，顾名思义，通信双方结成盟友，使用相同的封装模式、加密算法、加密密钥、验证算法、验证密钥，相互信任亲密无间。

安全联盟是单向的逻辑连接，为了使每个方向都得到保护，总舵和分舵的每个方向上都要建立安全联盟。总舵入方向上的安全联盟对应分舵出方向上的安全联盟，总舵出方向上的安全联盟对应分舵入方向上的安全联盟。

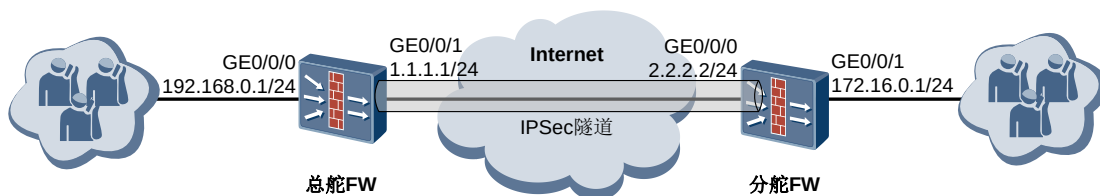


为了区分这些不同方向的安全联盟，IPSec 为每一个安全联盟都打上了唯一的标识符，这个标识符叫做 SPI（Security Parameter Index）。

建立安全联盟最直接的方式就是分别在总舵和分舵上人为设定好封装模式、加密算法、加密密钥、验证算法、验证密钥，即手工方式建立 IPSec 安全联盟。

初显神威

了解到 IPSec 的强大威力后，天地会陈总舵主决定先在总舵和一个分舵之间使用手工方式部署 IPSec 隧道，对总舵和分舵内部网络之间传递的消息进行保护，验证 IPSec 隧道的安全效果。



为了让加密、验证、安全联盟的配置关系更清晰，IPSec 为手工方式定义了四个步骤：

1. 定义需要保护的数据流

只有总舵和分舵内部网络之间交互的消息才被 IPSec 保护，其他消息不受保护。

2. 配置 IPSec 安全提议

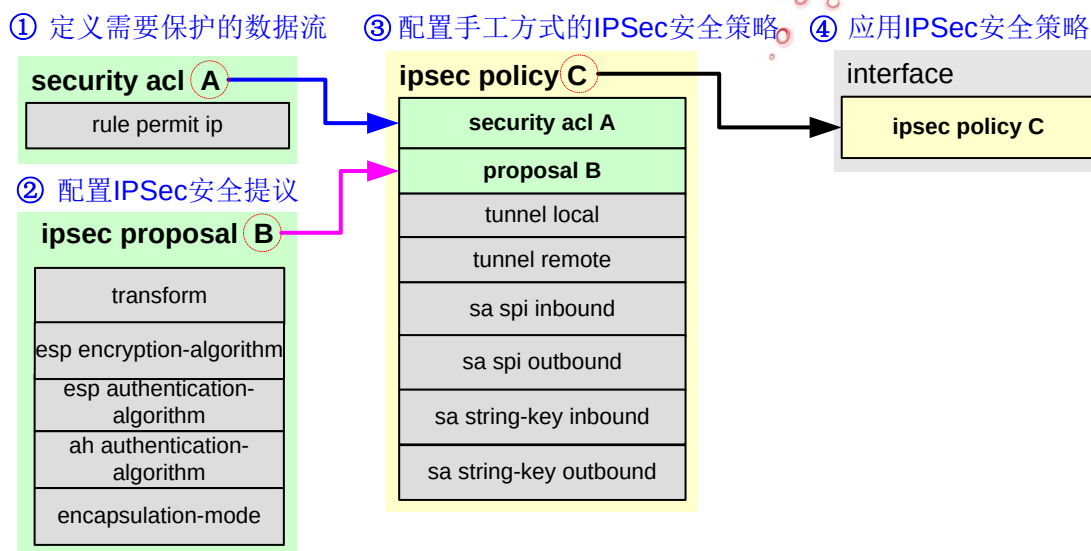
总舵和分舵根据对方的提议，决定能否成为盟友。封装模式、ESP、加密算法和验证算法均在安全提议中设置。

3. 配置手工方式的 IPSec 安全策略

指定总舵和分舵的公网地址、安全联盟标识符 SPI，以及加密密钥和验证密钥。

4. 应用 IPSec 安全策略

手工方式的 IPSec 配置关系如下图所示：



天地会总舵和分舵的关键配置和解释如下：

配置项	天地会总舵	天地会分舵
ACL	acl number 3000 rule 5 permit ip source 192.168.0.0 0.0.0.255 destination 172.16.0.0 0.0.0.255	acl number 3000 rule 5 permit ip source 172.16.0.0 0.0.0.255 destination 192.168.0.0 0.0.0.255
IPSec 安全提议	ipsec proposal pro1 transform esp encapsulation-mode tunnel esp authentication-algorithm sha1 esp encryption-algorithm aes	ipsec proposal pro1 transform esp encapsulation-mode tunnel esp authentication-algorithm sha1 esp encryption-algorithm aes
IPSec 安全策略	ipsec policy policy1 1 manual security acl 3000 proposal pro1 tunnel local 1.1.1.1 tunnel remote 2.2.2.2 sa spi inbound esp 54321 sa spi outbound esp 12345 sa string-key inbound esp huawei@123 sa string-key outbound esp huawei@456	ipsec policy policy1 1 manual security acl 3000 proposal pro1 tunnel local 2.2.2.2 tunnel remote 1.1.1.1 sa spi inbound esp 12345 sa spi outbound esp 54321 sa string-key inbound esp huawei@456 sa string-key outbound esp huawei@123
应用 IPsec 安全策略	interface GigabitEthernet0/0/1 ip address 1.1.1.1 255.255.255.0 ipsec policy policy1	interface GigabitEthernet0/0/0 ip address 2.2.2.2 255.255.255.0 ipsec policy policy1

注：除了上述 IPsec 的配置之外，防火墙安全策略也需要配置，允许总舵和分舵之间建立安全联盟，以及允许总舵私网和分舵私网之间互访。

部署完成后，总舵内部网络向分舵内部网络发出 ping 消息，分舵内部网络回复 ping 消息，天地会在 Internet 上模拟官府设卡检查，发现两个方向的 ping 消息都已经被 IPsec 安全联盟保护。两个方向上 IPsec 安全联盟的标识符 SPI 分别为 0x3039(十进制 12345)以及 0xd431

(十进制 54321)，与上文配置相符。

18	12.406000	1.1.1.1	2.2.2.2	ESP	ESP (SPI=0x00003039)
19	12.453000	2.2.2.2	1.1.1.1	ESP	ESP (SPI=0x0000d431)

Frame 18: 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits)	
Ethernet II, Src: 00:00:00:df:b8:01 (00:00:00:df:b8:01), Dst: HuaweiTe_4a:4c:17 (54:89:98:4a:4c:17)	
Internet Protocol, Src: 1.1.1.1 (1.1.1.1), Dst: 2.2.2.2 (2.2.2.2)	公网报文头
Encapsulating Security Payload	ESP头
ESP SPI: 0x00003039	
ESP Sequence: 10	

0000	54 89 98 4a 4c 17 00 00	00 df b8 01 08 00 45 00	T..JL... ..E.
0010	00 78 00 09 00 00 ff 32	b5 45 01 01 01 02 02	.X.....2..E.....
0020	02 02 00 00 30 39 00 00	00 0a 7b d3 e1 7f e7 f5	...09... ..{..5.
0030	c5 24 a1 7f e5 58 ed f5	54 a1 8b 6b 7d 08 35 e0	\$.X..T..k}.5.
0040	db 58 9f 0b f5 1f 61 6d	fe dd 47 72 c5 79 7e f7	.X...am..Gr.Y~.
0050	c0 3d 0f 20 6b 85 cc 31	97 29 8a c6 22 a5 a8 4a	=.k..1..).J
0060	b0 c8 77 75 d0 3e 7e 8f	d0 83 78 68 c8 f6 1d ea	..wu.>~..xh..
0070	15 de 75 80 28 d6 9e af	b2 db b6 a0 cc 93 3a 86	.u.(.....
0080	db 7b fc 81 b9 07		.f....

加密后消息

分析报文的内容，发现 ping 消息已经被加密，面目全非无法辨识，这样即使该消息真的被官府查获，也不能获取到任何有价值的信息。

为了对比 ESP 和 AH 的作用，天地会又使用 IPSec 中的另一绝技 AH 来建立安全联盟。AH 只有验证功能，没有加密的功能，官府从查获到的消息中看到私网报文头和 ping 消息真容。因此，如果要想实现加密，还是要使用 ESP，或者 AH 和 ESP 配合使用。

1	0.000000	192.168.0.2	172.16.0.2	ICMP	Echo (ping) request	id=0x2041, seq=1/256, ttl=127
2	0.109000	172.16.0.2	192.168.0.2	ICMP	Echo (ping) reply	id=0x2041, seq=1/256, ttl=127
3	1.141000	192.168.0.2	172.16.0.2	ICMP	Echo (ping) request	id=0x2241, seq=2/512, ttl=127
4	1.172000	172.16.0.2	192.168.0.2	ICMP	Echo (ping) reply	id=0x2241, seq=2/512, ttl=127
5	2.172000	192.168.0.2	172.16.0.2	ICMP	Echo (ping) request	id=0x2341, seq=3/768, ttl=127
6	2.219000	172.16.0.2	192.168.0.2	ICMP	Echo (ping) reply	id=0x2341, seq=3/768, ttl=127
7	3.281000	192.168.0.2	172.16.0.2	ICMP	Echo (ping) request	id=0x2441, seq=4/1024, ttl=127
8	3.313000	172.16.0.2	192.168.0.2	ICMP	Echo (ping) reply	id=0x2441, seq=4/1024, ttl=127
9	4.328000	192.168.0.2	172.16.0.2	ICMP	Echo (ping) request	id=0x2541, seq=5/1280, ttl=127
10	4.422000	172.16.0.2	192.168.0.2	ICMP	Echo (ping) reply	id=0x2541, seq=5/1280, ttl=127

Frame 1: 118 bytes on wire (944 bits), 118 bytes captured (944 bits)	
Ethernet II, Src: 00:00:00:df:b8:01 (00:00:00:df:b8:01), Dst: HuaweiTe_4a:4c:17 (54:89:98:4a:4c:17)	
Internet Protocol, Src: 1.1.1.1 (1.1.1.1), Dst: 2.2.2.2 (2.2.2.2)	公网报文头
Authentication Header	AH头
Internet Protocol, Src: 192.168.0.2 (192.168.0.2), Dst: 172.16.0.2 (172.16.0.2)	私网报文头
Internet Control Message Protocol	消息没有被加密
Type: 8 (Echo (ping) request)	
Code: 0	
Checksum: 0x663c [correct]	
Identifier (BE): 8257 (0x2041)	
Identifier (LE): 16672 (0x4120)	
Sequence number (BE): 1 (0x0001)	
Sequence number (LE): 256 (0x0100)	
Data (32 bytes)	

0000	54 89 98 4a 4c 17 00 00	00 df b8 01 08 00 45 00	T..JL... ..E.
0010	00 68 00 1e 00 00 ff 33	b5 3f 01 01 01 02 02	.h.....3?.....
0020	02 02 04 04 00 00 00 00	30 39 00 00 00 06 fb c3	09.....
0030	3b d4 4a 2b a1 23 c7 66	4f f5 45 00 00 3c 41 20	!..J+.#.f 0.E..A
0040	40 00 7f 01 4d e4 c0 a8	00 02 ac 10 00 02 08 00	@...M.....
0050	66 3c 20 41 00 01 08 09	0a 0b 0c 0d 0e 0f 10 11	[< A.....
0060	12 13 14 15 16 17 18 19	1a 1b 1c 1d 1e 1f 20 21	!
0070	22 23 24 25 26 27		#\$%&

自从使用了 IPSec 后，天地会总舵和分舵之间通信顺畅，没过多久，又新发展了好几个分舵，新建的分舵也要和总舵建立 IPSec 隧道。如果继续使用手工方式的话，每个分舵都需要配置各种参数，工作量大也不便于维护。IPSec 也考虑到了这种情况，除了手工方式之外，IPSec 还支持总舵和分舵自动协商建立 IPSec 安全联盟，该招式名为 IKE。欲知 IKE 详情，请听下

回分解。

VPN

🍁 IPsec 携手 IKE 珠联璧合（上篇）

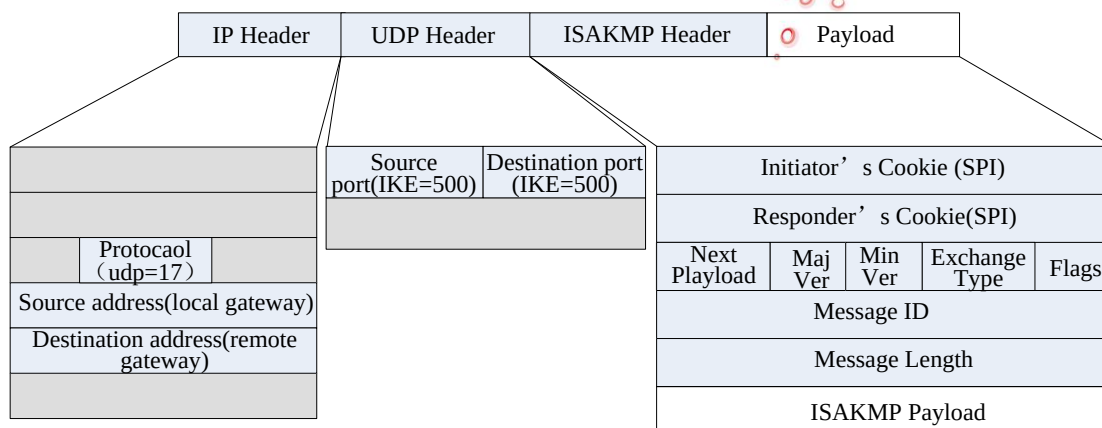
密钥管家 IKE 登场

上回说到手工方式的 IPsec VPN 帮助天地会解决了总舵和分舵之间秘密通信的问题，但随着分舵数量的增加新的问题又出现了。手工方式下防火墙的加密和验证所使用的密钥都是手工配置的，为了保证 IPsec VPN 的长期安全，需要经常修改这些密钥。分舵数量越多，密钥的配置和修改工作量越大。随着天地会的壮大，IPsec VPN 的维护管理工作越来越让人吃不消了。

为了降低 IPsec VPN 管理工作量，天地会总舵主再访高人寻求灵丹妙药。灵丹已练，妙药天成——原来 IPsec 协议框架中早就考虑了这个问题——智能的密钥管家 IKE（Internet Key Exchange）协议可以帮助解决这个问题。IKE 综合了三大协议：ISAKMP（Internet Security Association and Key Management Protocol）、Oakley 协议和 SKEME 协议。ISAKMP 主要定义了 IKE 伙伴（IKE Peer）之间合作关系（IKE SA，跟 IPsec SA 类似）的建立过程。Oakley 协议和 SKEME 协议的核心是 DH（Diffie-Hellman）算法，主要用于在 Internet 上安全地分发密钥、验证身份，以保证数据传输的安全性。有了 IKE 加盟，IPsec VPN 的安全和管理问题不再困扰天地会，各地分舵申请建立 VPN 的流程终于可以进入“实施”状态了。

揭开 ISAKMP 本来面目

IKE 协议的终极目标是通过协商在总舵和分舵之间动态建立 IPsec SA，并能够实时维护 IPsec SA。为建立 IPsec SA 而进行的 IKE 协商工作是由 ISAKMP 报文来完成的，所以在部署 IKE 之前，强叔先领大家认识一下 ISAKMP 报文。ISAKMP 报文封装如下：



🍁 IP 报文头

源地址src: 本端发起IKE协商的IP地址, 可能是接口IP地址, 也可能是通过命令配置的IP地址。

目的IP地址Dst: 对端发起IKE协商的IP地址, 由命令配置。

🍁 UDP 报文头

IKE 协议使用端口号 500 发起协商、响应协商。在总舵和分舵都有固定 IP 地址时, 这个端口在协商过程中保持不变。当总舵和分舵之间有 NAT 设备时 (NAT 穿越场景), IKE 协议会有特殊处理 (后续揭秘)。

🍁 ISAKMP 报文头

Initiator's Cookie (SPI) 和 responder's Cookie (SPI): 在 IKEv1 版本中为 Cookie, 在 IKEv2 版本中 Cookie 为 IKE 的 SPI, 唯一标识一个 IKE SA。¹

Version: IKE 版本号¹。

Exchange Type: IKE 定义的交互类型²。交换类型定义了 ISAKMP 消息遵循的交换顺序。

Next Payload: 标识消息中下一个载荷的类型。一个 ISAKMP 报文中可能装载多个载荷, 该字段提供载荷之间的“链接”能力。若当前载荷是消息中最后一个载荷, 则该字段为 0。

Type Payload: 载荷类型, ISAKMP 报文携带的用于协商 IKE SA 和 IPsec SA 的“参数包”。载荷类型有很多种, 不同载荷携带的“参数包”不同。不同载荷的具体作用我们后面会结合抓包过程逐一分析。

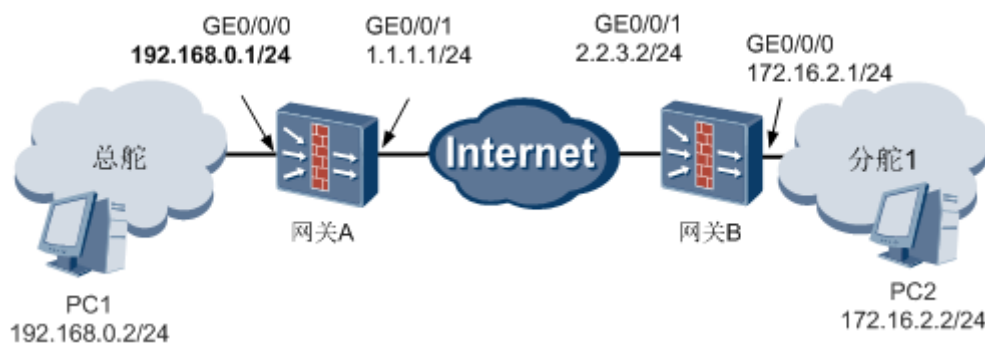
说明:

1: IKE 诞生以来, 有过一次大的改进。老的 IKE 被称为 IKEv1, 改进后的 IKE 被称为 IKEv2。二者可以看做是父子关系, 血脉相承, 基本功能不变; 但青胜于蓝, 后者有了长足的进步。

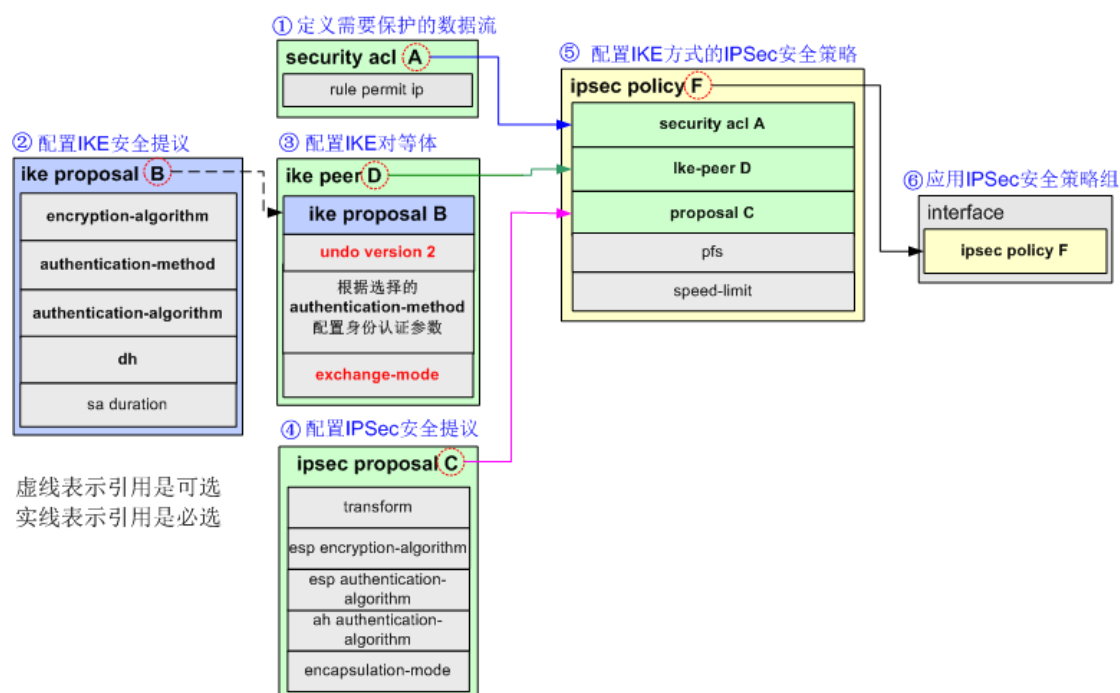
IKEv1 和 IKEv2 的精华之处正是本文的重点, 各位看官莫急, 容强叔一一道来。

IKEv1 小试牛刀

IKE 最适合于在总舵和众多分舵之间建立 IPsec VPN 的场景,分舵越多 IKE 的优势越能凸显。为了讲解方便,这里仅给出总舵和一个分舵之间建立 IPsec VPN 的组网图。



相比手工方式, IKE 方式仅增加了两步: 配置 IKE 安全提议和 IKE 对等体。IKE 安全提议主要用于配置建立 IKE SA 用到的加密和验证算法。IKE 对等体主要配置 IKE 版本、身份认证和交换模式。



配置项	网关 A	网关 B
IKE 安全提议	ike proposal 10	ike proposal 10
IKE 对等体	ike peer b ike-proposal 10 undo version 2 //IKEv1 exchange-mode main//主模式 (缺省) remote-address 2.2.3.2//对端发起 IKE	ike peer a ike-proposal 10 undo version 2 //IKEv1 exchange-mode main//主模式 (缺省) remote-address 1.1.1.1//对端发起 IKE 协商的地址

配置项	网关 A	网关 B
	协商的地址 pre-shared-key tiandihui2	pre-shared-key tiandihui2
ACL	acl number 3001 rule 5 permit ip source 192.168.0.0 0.0.0.255 destination 172.16.2.0 0.0.0.255	acl number 3000 rule 5 permit ip source 172.16.2.0 0.0.0.255 destination 192.168.0.0 0.0.0.255
IPSec 安全提议	ipsec proposal a transform esp encapsulation-mode tunnel esp authentication-algorithm md5 esp encryption-algorithm des	ipsec proposal a transform esp encapsulation-mode tunnel esp authentication-algorithm md5 esp encryption-algorithm des
IPSec 安全策略	ipsec policy policy1 11 isakmp security acl 3001 proposal a ike-peer b	ipsec policy policy1 1 isakmp security acl 3000 proposal a ike-peer a
应用 IPSec 安全策略	interface GigabitEthernet0/0/1 ip address 1.1.1.1 255.255.255.0 ipsec policy policy1	interface GigabitEthernet0/0/1 ip address 2.2.3.2 255.255.255.0 ipsec policy policy1

说明：两条红色命令表示本例采用了 IKEv1 版本主模式。缺省同时开启 IKEv1 和 IKEv2，关闭 IKEv2 后采用 IKEv1 版本进行协商。

配置完成之后，总舵和分舵之间有互访数据流时即可触发建立 IPSec VPN 隧道，下面强叔通过抓包来为大家详解一下 IKEv1 的精妙之处。

IKEv1 版本分两个阶段来完成动态建立 IPSec SA 的任务：

阶段 1-建立 IKE SA：阶段 1 采用主模式或野蛮模式协商。

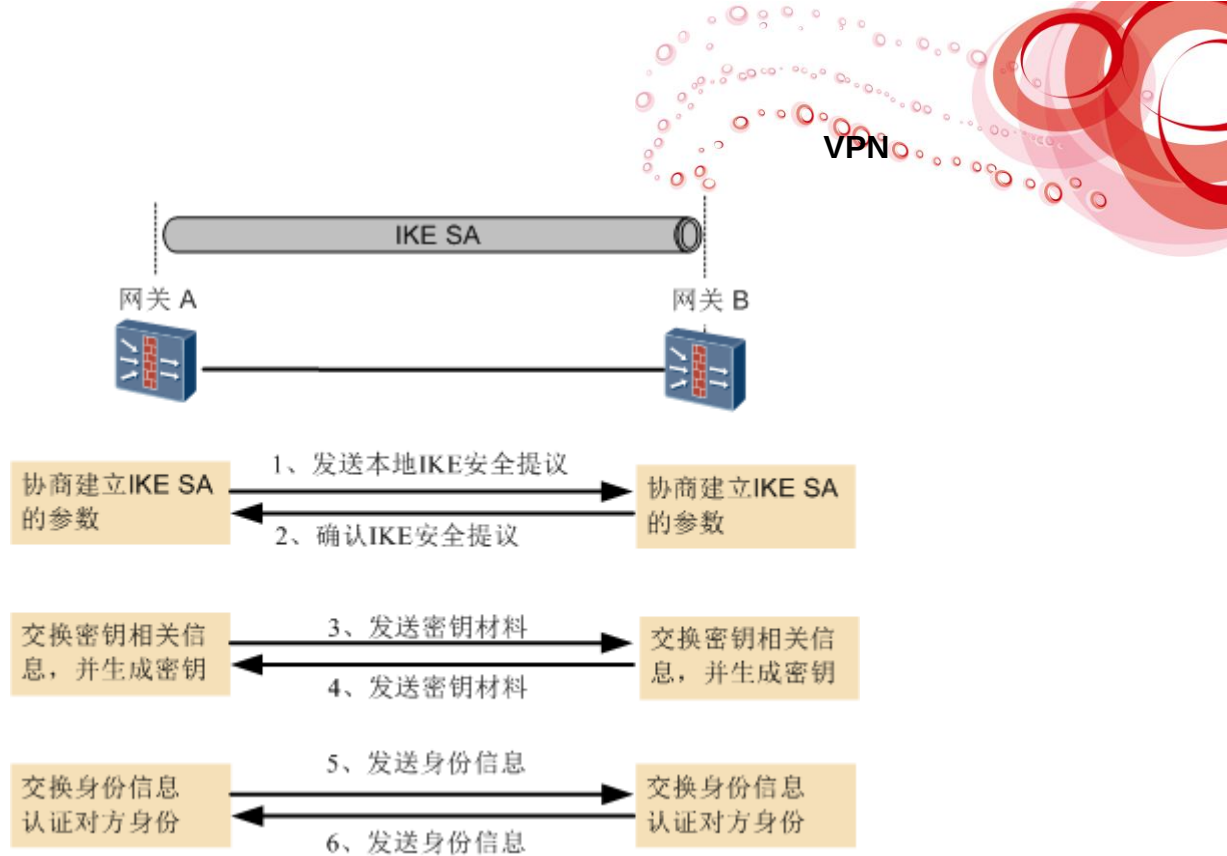
阶段 2-建立 IPSec SA：阶段 2 此采用快速模式协商。

先介绍主模式+快速模式下的 IPSec SA 建立过程

阶段 1-建立 IKE SA（主模式）。

主模式下 IKEv1 采用 3 个步骤 6 条 ISAKMP 消息建立 IKE SA。下面以网关 A 主动发起 IKE 协商为例进行讲解。

302 0.000000	1.1.1.1	2.2.3.2	ISAKMP	Identity Protection (Main Mode)
303 0.000000	2.2.3.2	1.1.1.1	ISAKMP	Identity Protection (Main Mode)
304 0.000000	1.1.1.1	2.2.3.2	ISAKMP	Identity Protection (Main Mode)
305 0.000000	2.2.3.2	1.1.1.1	ISAKMP	Identity Protection (Main Mode)
306 0.000000	1.1.1.1	2.2.3.2	ISAKMP	Identity Protection (Main Mode)
307 0.000000	2.2.3.2	1.1.1.1	ISAKMP	Identity Protection (Main Mode)



1. 协商 IKE 安全提议。

协商分两种情况：

- 发起方的 IKE Peer 中引用了 IKE Proposal
- 发起方的 IKE peer 中没有引用 IKE Proposal

二种情况下响应方都会在自己配置的 IKE 安全提议中寻找与发送方相匹配的 IKE 安全提议，如果没有匹配的安全提议则协商失败。**IKE Peer 双方安全提议匹配的原则为协商双方有相同的加密算法、认证算法、身份认证方法和 DH 组标识（不包括 IKE SA 生存周期）。**

说明：通过 IKEv1 协议协商建立 IPsec 安全联盟时，采用本地生存周期和对端生存周期中较小的一个，不必保证隧道两端设备配置的生存周期相同（sa duration）。

通过抓包可以看出 ISAKMP 消息的 SA 载荷中携带用于协商的 IKE 安全提议。以消息 1 为例：

```

Internet Security Association and Key Management Protocol
Initiator cookie: 98fc99866ccbb551
Responder cookie: 0000000000000000
Next payload: Security Association (1)
Version: 1.0
Exchange type: Identity Protection (Main Mode) (2)
+ Flags: 0x00
Message ID: 0x00000000
Length: 144
Type Payload: Security Association (1)
Next payload: Vendor ID (13)
Payload length: 56
Domain of interpretation: IPSEC (1)
+ Situation: 00000001
Type Payload: Proposal (2) # 1
Next payload: NONE / No Next Payload (0)
Payload length: 44
Proposal number: 1
Protocol ID: ISAKMP (1)
SPI Size: 0
Proposal transforms: 1
Type Payload: Transform (3) # 0
Next payload: NONE / No Next Payload (0)
Payload length: 36
Transform number: 0
Transform ID: KEY_IKE (1)
+ Transform IKE Attribute Type (t=1,l=2) Encryption-Algorithm : DES-CBC
+ Transform IKE Attribute Type (t=2,l=2) Hash-Algorithm : SHA
+ Transform IKE Attribute Type (t=3,l=2) Authentication-Method : PSK
+ Transform IKE Attribute Type (t=4,l=2) Group-Description : Default 768-bit MODP group
+ Transform IKE Attribute Type (t=11,l=2) Life-Type : Seconds
+ Transform IKE Attribute Type (t=12,l=4) Life-Duration : 1

```

2. 使用 DH 算法交换密钥材料，并生成密钥。

网关 A 和 B 利用 ISAKMP 消息的 Key Exchange 和 nonce 载荷交换彼此的密钥材料。Key Exchange 用于交换 DH 公开值，nonce 用于传送临时随机数。由于 DH 算法中 IKE Peer 双方只交换密钥材料，并不交换真正的共享密钥，所以即使黑客窃取了 DH 值和临时值也无法计算出共享密钥，这一点正是 DH 算法的精髓所在。从抓包中可以看到 IKE Peer 双方交换密钥材料，以消息 3 为例：

```

Internet Security Association and Key Management Protocol
Initiator cookie: 98fc99866ccbb551
Responder cookie: 34e0f454337735f7
Next payload: Key Exchange (4)
Version: 1.0
Exchange type: Identity Protection (Main Mode) (2)
+ Flags: 0x00
Message ID: 0x00000000
Length: 196
Type Payload: Key Exchange (4)
Next payload: Nonce (10)
Payload length: 100
Key Exchange Data: 390e8313198d141928d4e8570567588a9687031fad73c0f7...
Type Payload: Nonce (10)
Next payload: NAT-D (RFC 3947) (20)
Payload length: 20
Nonce DATA: b69c95e941c5146abb9937009ef4ff02

```

密钥材料交换完成后，IKE Peer 双方结合自身配置的身份验证方法各自开始复杂的密钥计算过程（预共享密钥或数字证书都会参与到密钥计算过程中），最终会产生三个密钥：

🍁 SKEYID_a: ISAKMP 消息完整性验证密钥——谁也别想篡改 ISAKMP 消息了，只要消

息稍有改动，响应端完整性检查就会发现！

- 🔥 SKEYID_e: ISAKMP 消息加密密钥——再也别想窃取 ISAKMP 消息了，窃取了也看不懂！

以上两个密钥保证了后续交换的 ISAKMP 消息的安全性！

- 🔥 SKEYID_d: 用于衍生出 IPsec 报文加密和验证密钥——最终是由这个密钥保证 IPsec 封装的数据报文的安全性！

整个密钥交换和计算过程在 IKE SA 超时时间的控制下以一定的周期进行自动刷新，避免了密钥长期不变带来的安全隐患。

3. 身份认证。

IKE Peer 通过两条 ISAKMP 消息（5、6）交换身份信息，进行身份认证。目前有两种身份认证技术比较常用：

- 🔥 预共享密钥方式（pre-share）：设备的身份信息为 IP 地址或名称。
- 🔥 数字证书方式：设备的身份信息为证书和通过证书私钥加密的部分消息 Hash 值（俗称签名）。

以上身份信息都由 SKEYID_e 进行加密，所以在抓包中我们只能看到标识为“Encrypted”的 ISAKMP 消息，看不到消息的内容（身份信息）。以消息 5 为例：

```

Internet Security Association and Key Management Protocol
Initiator cookie: 98fc99866ccbb551
Responder cookie: 34e0f454337735f7
Next payload: Identification (5)
Version: 1.0
Exchange type: Identity Protection (Main Mode) (2)
Flags: 0x01
.... ..1 = Encryption: Encrypted
.... ..0. = Commit: No commit
.... .0.. = Authentication: No authentication
Message ID: 0x00000000
Length: 68
Encrypted Data (40 bytes)
  
```

预共享密钥是最简单、最常用的身份认证方法。这种方式下设备的身份信息可以用 IP 地址或名称（包括 FQDN 和 USER-FQDN 两种形式）来标识。当 IKE Peer 两端都有固定 IP 地址的时候，一般都用 IP 地址作为身份标识；当一端为动态获取 IP 地址的时候，没有固定 IP 地址的一端只能用名称来标识。本篇强叔给大家展示的案例为前者，后者晚些再讲。

这里有个小问题提醒大家关注：

总舵收到分舵1发来的身份信息后，需要用密钥（SKEYID_a 和 SKEYID_e）进行完整性验证和解密，只有先找到正确的预共享密钥才能计算出这两个密钥。但总舵网关为每个IKE Peer都配置了一个预共享密钥。怎么找呢？此时只能根据IKE Peer发来的ISAKMP报文的源IP地址（src）来查找预共享密钥，只要报文源地址与本端IKE Peer视图下remote-address命令配置的IP地址一致，就认为该视图下配置的pre-shared-key是分舵1的预共享密钥。

以上是IPSec协议内部处理过程，不需要大家操心。大家只要记住在IKE Peer两端都有固定IP地址的场景下，remote-address命令配置的IP地址要跟对端发起IKE协商的IP地址保持一致即可。这个IP地址的作用不仅仅是指定了隧道对端的IP地址，还参与了预共享密钥的查找。

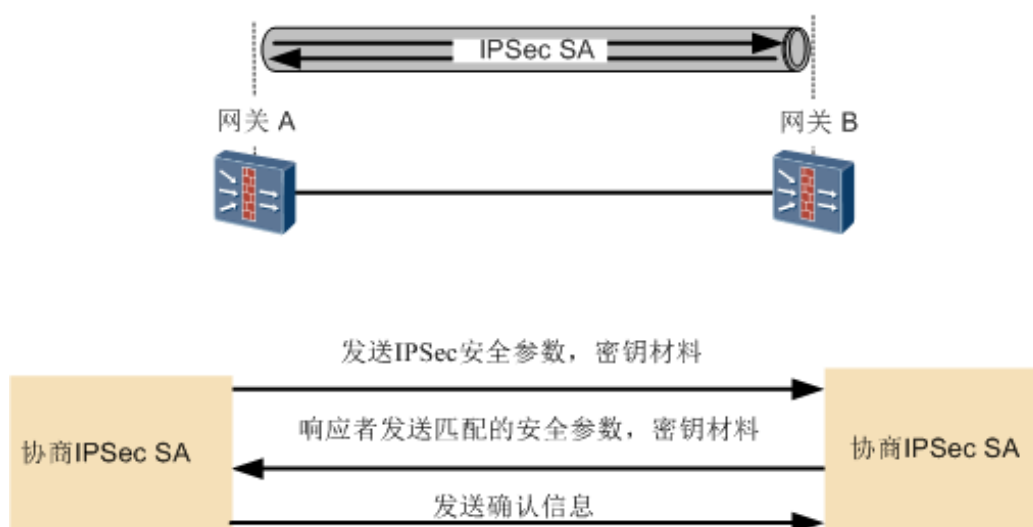
阶段1协商完成后，进入阶段2。

阶段2-建立IPSec SA。

在阶段2中IKEv1采用快速交换模式通过3条ISAKMP消息建立IPSec SA。由于快速交换模式使用IKEv1阶段1中生成的密钥SKEYID_e对ISAKMP消息进行加密，所以我们抓到的报文都是加密的，看不到载荷里面的具体内容。故下面只能文字介绍一下每一步的作用。

下面以网关A发起IPSec协商为例进行讲解。

22	0.000000	1.1.1.1	2.2.3.2	ISAKMP	Quick Mode
23	0.000000	2.2.3.2	1.1.1.1	ISAKMP	Quick Mode
24	0.000000	1.1.1.1	2.2.3.2	ISAKMP	Quick Mode



1. 发起方发送IPSec安全提议、被保护的数据流（ACL）和密钥材料给响应方。
2. 响应方回应匹配的IPSec安全提议、被保护的数据流，同时双方生成用于IPSec SA的密钥。

IPSec对等体两端协商IPSec安全提议的过程跟协商IKE安全提议的过程类似，不再赘述。IKEv1不协商ACL规则，建议两端设备配置的ACL规则互为镜像，避免IPSec SA协商失败。

IPSec 对等体两端交换密钥材料（SKEYID_d、SPI 和协议¹、nonce 等参数），然后各自进行密钥计算生成用于 IPSec SA 加密验证的密钥，这样可以保证每个 IPSec SA 都有自己独一无二的密钥。由于 IPSec SA 的密钥都是由 SKEYID_d 衍生的，一旦 SKEYID_d 泄露将可能导致 IPSec VPN 受到侵犯。为提升密钥管理的安全性，IKE 提供了 PFS（完美向前保密）功能。启用 PFS 后，在进行 IPSec SA 协商时会进行一次附加的 DH 交换，重新生成新的 IPSec SA 密钥，提高了 IPSec SA 的安全性。

说明：协议指 AH 和/或 ESP 协议。

3. 发起方发送确认结果。

协商完成后发送方开始发送 IPSec（ESP）报文。

46	0.000000	1.1.1.1	2.2.3.2	ESP	ESP (SPI=0xca54cc6f)
47	0.000000	1.1.1.1	2.2.3.2	ESP	ESP (SPI=0xca54cc6f)
48	0.000000	2.2.3.2	1.1.1.1	ESP	ESP (SPI=0x4a314106)
49	0.000000	1.1.1.1	2.2.3.2	ESP	ESP (SPI=0xca54cc6f)
50	0.000000	2.2.3.2	1.1.1.1	ESP	ESP (SPI=0x4a314106)
51	0.000000	1.1.1.1	2.2.3.2	ESP	ESP (SPI=0xca54cc6f)

IPSec SA 建立成功后，检查 IPSec VPN 状态信息。以总舵显示信息为例。

 在总舵和分舵上查看 IKE SA 的建立情况。

```
<FW_A> display ike sa
current ike sa number: 2
```

conn-id	peer	flag	phase	vpn
40129	2.2.3.2	RD ST	v1:2	public
40121	2.2.3.2	RD ST	v1:1	public

这里统一显示了 IKE SA (v1:1) 和 IPSec SA (v1:2) 的状态，RD 表示 SA 状态为 READY。

IKE Peer 之间只有一个 IKE SA，IKE SA 是双向逻辑连接（不区分源和目的）。

 在总舵和分舵上查看 IPSec SA 的建立情况。

```
<FW_A> display ipsec sa brief
current ipsec sa number: 2
current ipsec tunnel number: 1
```

Src Address	Dst Address	SPI	Protocol	Algorithm
1.1.1.1	2.2.3.2	4090666525	ESP	E:DES;A:HMAC-MD5-96;
2.2.3.2	1.1.1.1	2927012373	ESP	E:DES;A:HMAC-MD5-96;

IPSec SA 是单向的（区分源和目的），两个方向的 IPSec SA 共同组成一条 IPSec 隧道。

说明：一般来说一条数据流对应一个 IPSec SA。但当 IPSec 同时采用了 ESP+AH 封装时，

一条数据流会对应两个 IPSec SA。

🔥 在总舵和分舵上查看会话表。

```
<FW_A >display firewall session table
11:01:45 2014/07/08
Current Total Sessions : 6
esp VPN:public --> public 1.1.1.1:0-->2.2.3.2:0
icmp VPN:public --> public 172.16.2.2:7264-->192.168.0.2:2048
icmp VPN:public --> public 172.16.2.2:7520-->192.168.0.2:2048
icmp VPN:public --> public 172.16.2.2:7776-->192.168.0.2:2048
icmp VPN:public --> public 172.16.2.2:8032-->192.168.0.2:2048
icmp VPN:public --> public 172.16.2.2:8288-->192.168.0.2:2048
```

防火墙会为 IPSec VPN 建立了会话表 (1.1.1.1:0-->2.2.3.2:0)。

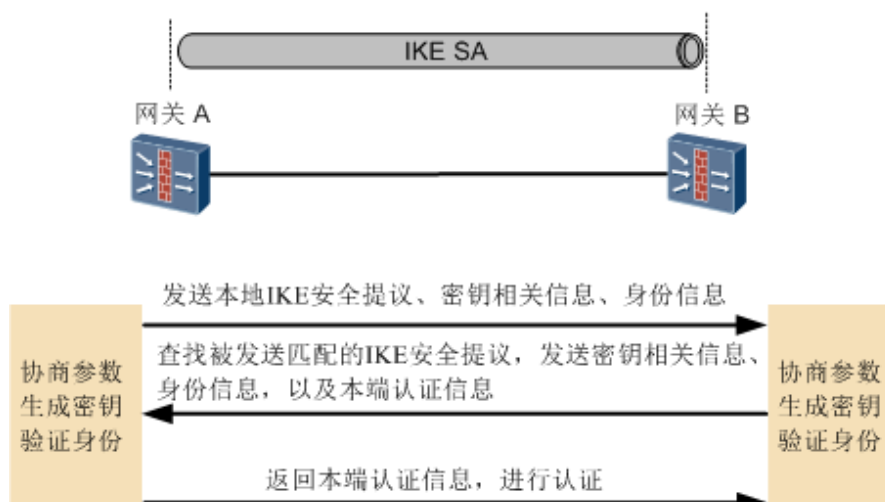
简单介绍一下野蛮模式

IKEv1 版本分两个阶段完成了 IPSec SA 的动态协商, 1 阶段采用主模式协商方式即安全又可靠, 但有些情况下完美的主模式却发挥不了作用。为什么呢? 让我们先看看作为主模式替补的野蛮模式的协商过程吧, 其中的缘由自然不得而知。

在上面配置举例中, 将配置命令 `exchange-mode main` 改为 `exchange-mode aggressive` 即可将 IKEv1 1 阶段的协商模式改为野蛮模式。抓包看一下野蛮模式的情况:

57	0.000000	1.1.1.1	2.2.3.2	ISAKMP	Aggressive
58	0.000000	2.2.3.2	1.1.1.1	ISAKMP	Aggressive
59	0.000000	1.1.1.1	2.2.3.2	ISAKMP	Aggressive
60	0.000000	1.1.1.1	2.2.3.2	ISAKMP	Quick Mode
61	0.000000	2.2.3.2	1.1.1.1	ISAKMP	Quick Mode
62	0.000000	1.1.1.1	2.2.3.2	ISAKMP	Quick Mode

野蛮模式只用了 3 条 ISAKMP 消息就完成了阶段 1 的协商过程, 阶段 2 仍旧是快速模式不变。



发起方和响应方把 IKE 安全提议、密钥相关信息和身份信息一股脑地全放在一个 ISAKMP 消息中发送给对方，IKE 协商效率提高了。但由于身份信息是明文传输，没有加密和完整性验证过程，IKE SA 的安全性降低了。**既然这样不够安全，为什么野蛮模式还会出现？**

在 IPSec VPN 出现的早期，由于主模式+预共享密钥身份认证方式下，IPSec 需要通过对端的 IP 地址来在本地查找预共享密钥（主模式中已经详细解释了这个问题）。这种密钥查找方式在对端没有固定 IP 地址的情况下（比如 IPSec NAT 穿越场景，网络出口动态获取 IP 地址场景）行不通。此时，野蛮模式可以“野蛮”地解决这个问题。野蛮模式中“身份信息”没有加密，IPSec 就直接用对端发送来的身份信息来查找预共享密钥即可。所以在 IPSec VPN 应用初期，野蛮模式主要用于解决没有固定 IP 地址的节点部署 IPSec VPN 的问题。**现在，IPSec VPN 解决这个问题有很多方法，不安全的野蛮模式已经不是最好的选择了。具体方法待后续再呈现给大家。**

🍁 IPsec 携手 IKE 珠联璧合（下篇）

IKEv2 应运而生

IKEv1 似乎已经很完美了，但用得久了仍旧会发现不尽人意之处。

🍁 协商建立 IPsec SA 的时间太长

IKEv1 主模式协商一对 IPsec SA，需要 6（协商 IKE SA）+3（协商 IPsec SA）=9 条消息。

IKEv1 野蛮模式协商一对 IPsec SA，需要 3（协商 IKE SA）+3（协商 IPsec SA）=6 条消息。

🍁 不支持远程用户接入

IKEv1 不能对远程用户进行认证。若想支持远程用户接入，只能借助 L2TP，通过 PPP 来对远程用户进行 AAA 认证。

这些问题怎么解决呢？办法总比问题多！IKEv2 中完美的解决了这些问题。

IKEv2 相比 IKEv1：

🍁 协商建立 IPsec SA 的速度大大提升

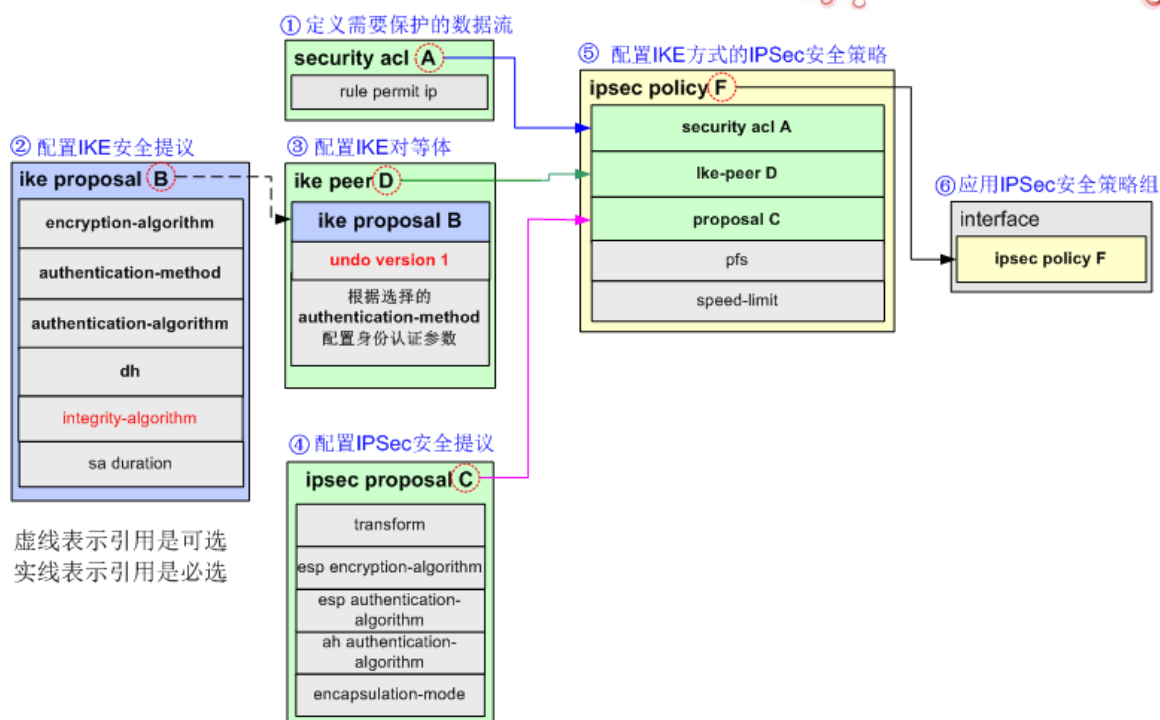
正常情况 IKEv2 协商一对 IPsec SA 只需要 2（协商 IKE SA）+2（协商 IPsec SA）=4 条消息。后续每建立一对 IPsec SA 只会增加 2 条消息。

🍁 增加了 EAP（Extensible Authentication Protocol）方式的身份认证。

IKEv2 通过 EAP 协议解决了远程接入用户认证的问题，彻底摆脱了 L2TP 的牵制。目前 IKEv2 已经广泛应用于远程接入网络中了。今天强叔只介绍 IKEv2 的基本协商过程，EAP 认证在企业网中还没看到应用暂时不着急讲。

IKEv2 配置

IKEv2 的配置思路与 IKEv1 完全相同，只是细节稍有不同：



说明：红色命令与IKEv1不同。缺省情况下，防火墙同时开启IKEv1和IKEv2协议。本端发起协商时，采用IKEv2，接受协商时，同时支持IKEv1和IKEv2。可以不关闭IKEv1。

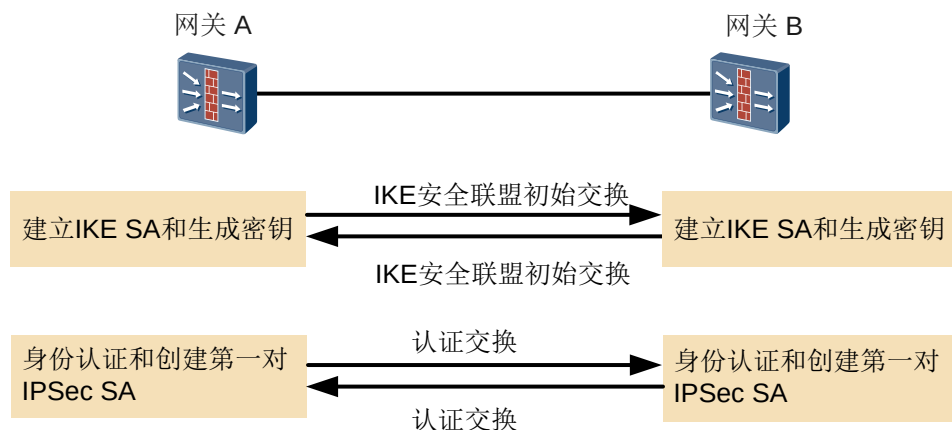
IKEv2 协商过程

IKEv2 协商 IPsec SA 的过程跟 IKEv1 有很大差别：

1. 初始交换 4 条消息同时搞定 IKE SA 和 IPsec SA。

初始交换包括 IKE 安全联盟初始交换（IKE_SA_INIT 交换）和 IKE 认证交换（IKE_AUTH 交换）。

5	0.000000	1.1.1.1	2.2.2.2	ISAKMP	IKE_SA_INIT
6	0.000000	2.2.2.2	1.1.1.1	ISAKMP	IKE_SA_INIT
7	0.000000	1.1.1.1	2.2.2.2	ISAKMP	IKE_AUTH
8	0.000000	2.2.2.2	1.1.1.1	ISAKMP	IKE_AUTH



- 第一个消息对 (IKE_SA_INIT): 负责 IKE 安全联盟参数的协商, 包括 IKE Proposal, 临时随机数 (nonce) 和 DH 值。

```

Frame 5: 310 bytes on wire (2480 bits), 310 bytes captured (2480 bits)
Ethernet II, Src: 00:00:00:b2:f8:01 (00:00:00:b2:f8:01), Dst: HuaweiTe_15:34:ec (00:e0:fc:15:34:ec)
Internet Protocol, Src: 1.1.1.1 (1.1.1.1), Dst: 2.2.2.2 (2.2.2.2)
User Datagram Protocol, Src Port: isakmp (500), Dst Port: isakmp (500)
Internet Security Association and Key Management Protocol
  Initiator cookie: f7ef9cf859be8f3d
  Responder cookie: 0000000000000000
  Next payload: Security Association (33)
  Version: 2.0
  Exchange type: IKE_SA_INIT (34)
  Flags: 0x08
  Message ID: 0x00000000
  Length: 268
    Type Payload: Security Association (33)
    Type Payload: Key Exchange (34)
    Type Payload: Nonce (40)

```

SA 载荷主要用来协商 IKE Proposal。

```

Type Payload: Security Association (33)
  Next payload: Key Exchange (34)
  0... .... = Critical Bit: Not Critical
  Payload length: 44
    Type Payload: Proposal (2) # 1
      Next payload: NONE / No Next Payload (0)
      0... .... = Critical Bit: Not Critical
      Payload length: 40
      Proposal number: 1
      Protocol ID: IKE (1)
      SPI Size: 0
      Proposal transforms: 4
        Type Payload: Transform (3)
          Next payload: Transform (3)
          0... .... = Critical Bit: Not Critical
          Payload length: 8
          Transform Type: Encryption Algorithm (ENCR) (1)
          Transform ID (ENCR): ENCR_DES (2)
        Type Payload: Transform (3)
          Next payload: Transform (3)
          0... .... = Critical Bit: Not Critical
          Payload length: 8
          Transform Type: Pseudo-random Function (PRF) (2)
          Transform ID (PRF): PRF_HMAC_SHA1 (2)
        Type Payload: Transform (3)
          Next payload: Transform (3)
          0... .... = Critical Bit: Not Critical
          Payload length: 8
          Transform Type: Integrity Algorithm (INTEG) (3)
          Transform ID (INTEG): AUTH_HMAC_SHA1_96 (2)
        Type Payload: Transform (3)
          Next payload: NONE / No Next Payload (0)
          0... .... = Critical Bit: Not Critical
          Payload length: 8
          Transform Type: Diffie-Hellman Group (D-H) (4)
          Transform ID (D-H): Default 768-bit MODP group (1)

```

KE (Key Exchange) 载荷和 Nonce 载荷主要用来交换密钥材料。

```

+ Type Payload: Security Association (33)
+ Type Payload: Key Exchange (34)
  Next payload: Nonce (40)
  0... .... = Critical Bit: Not Critical
  Payload length: 104
  DH Group #: Default 768-bit MODP group (1)
  Key Exchange Data: 92d4ee4e8c998aca22f582e58df19b91f1d55991459cbf08...
+ Type Payload: Nonce (40)
  Next payload: Notify (41)
  0... .... = Critical Bit: Not Critical
  Payload length: 36
  Nonce DATA: 3e32b6823e3b1cf46473ea12cfe6bc75b6631940ebd25f43...

```

IKEv2 通过 IKE_SA_INIT 交换后最终也生成三类密钥:

SK_e: 用于加密第二个消息对。

SK_a: 用于第二个消息对的完整性验证。

SK_d: 用于为 Child SA (IPSec SA) 衍生出加密材料。

🔥 第二个消息对 (IKE_AUTH): 负责身份认证, 并创建第一个 Child SA (一对 IPSec SA)。

目前三种身份认证技术比较常用:

预共享密钥方式 (pre-share): 设备的身份信息为 IP 地址或名称。

数字证书方式: 设备的身份信息为证书和通过证书私钥加密的部分消息 Hash 值 (签名)。

EAP 方式: 采用 EAP 认证的交换过程属于扩展交换的内容。这种认证方式在企业网中主要用于总部通过 RADIUS 协议对远程接入用户进行认证, 而这种场景目前强叔还没看到实际应用, 故暂时遗留不写吧。

以上身份信息都通过 SKEYID_e 加密。

创建 Child SA 时, 当然也要协商 IPSec 安全提议、被保护的数据流。IKEv2 通过 TS 载荷 (TSi 和 TSr) 来协商两端设备的 ACL 规则, 最终结果是取双方 ACL 规则的交集 (这点跟 IKEv1 不同, IKEv1 没有 TS 载荷不协商 ACL 规则)。

当一个 IKE SA 需要创建多对 IPSec SA 时, 例如两个 IPSec 对等体之间有多条数据流的时候, 需要使用创建子 SA 交换来协商后续的 IPSec SA。

2. 子 SA 交换 2 条消息建立一对 IPSec SA。

39 0.000000	1.1.1.1	2.2.2.2	ISAKMP	CREATE_CHILD_SA
40 0.000000	2.2.2.2	1.1.1.1	ISAKMP	CREATE_CHILD_SA

子 SA 交换必须在 IKE 初始交换完成之后才能进行, 交换的发起者可以是 IKE 初始交换的发起者, 也可以是 IKE 初始交换的响应者。这 2 条消息由 IKE 初始交换协商的密钥进行保护。

IKEv2 也支持 PFS 功能, 创建子 SA 交换阶段可以重新进行一次 DH 交换, 生成新的 IPSec SA 密钥。

IKEv1 和 IKEv2 大 PK

IKEv1 和 IKEv2 的细节写了这么多，现在总结一下二者的异同点吧：

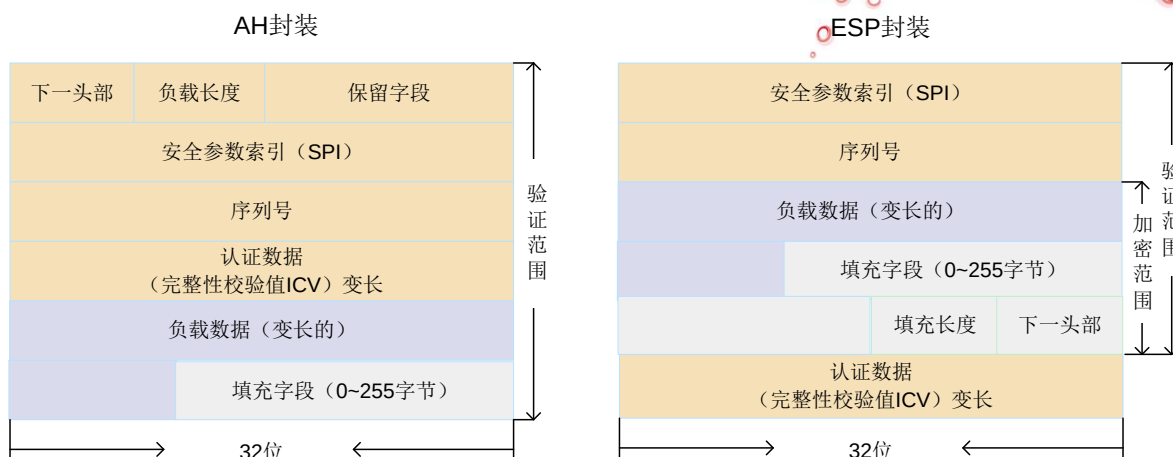
功能项	IKEv1	IKEv2
IPSec SA 建立过程	分两个阶段，阶段 1 分两种模式：主模式和野蛮模式，阶段 2 为快速模式。 主模式+快速模式需要 9 条信息建立 IPSec SA。 野蛮模式+快速模式需要 6 条信息建立 IPSec SA。	不分阶段，最少 4 条消息即可建立 IPSec SA。
ISAKMP	二者支持的载荷类型不同	
认证方法	预共享密钥 数字证书 数字信封（较少使用）	预共享密钥 数字证书 EAP 数字信封（较少使用）
IKE SA 完整性算法	不支持	支持
PFS	支持	支持
远程接入	通过 L2TP over IPSec 来实现	支持

显然 IKEv2 以其更加快捷、更加安全的服务胜出，长江后浪推前浪又成为了没有任何悬念的事实。

IPSec 协议框架

安全协议（AH 和 ESP）、加密算法（DES、3DES、AES）、验证算法（MD5、SHA1、SHA2）、IKE、DH，这么多的概念，大家搞清楚他们之间的关系了么？强叔做了一下总结，看看能否帮助大家：

🍁 安全协议（AH 和 ESP）——IP 报文的安全封装。穿上 AH 或/和 ESP 马甲的 IP 报文称为 IPSec 报文。此“马甲”并非一般马甲，是交织了“加密”和“验证”算法的刀枪不入的“软猬甲”。



说明：AH 封装的验证范围实际要还更大一些，包括新的 IP 头。

- 🍁 加密算法（DES、3DES、AES）——IPSec 报文的易容之术。IPSec 数据报文采用对称加密算法进行加密，但只有 ESP 协议支持加密，AH 协议不支持。另外，IKE 协商报文也会进行加密。
- 🍁 验证（MD5、SHA1、SHA2）——IPSec 报文的验明正身之法。加密后的报文经过验证算法处理生成数字签名，数字签名填写在 AH 和 ESP 报文头的完整性校验值 ICV 字段发送给对端；在接收设备中，通过比较数字签名进行数据完整性和真实性验证。
- 🍁 IKE——手握密钥管理大权的贴心管家。IPSec 使用 IKE 协议在发送、接收设备之间安全地协商密钥、更新密钥。
- 🍁 DH 算法——贴心管家的铁算盘。DH 被称为公共密钥交换方法，它用于产生密钥材料，并通过 ISAKMP 消息进行交换，并最终在收发两端计算出加密密钥和验证密钥。

这些概念一一梳理过来，强叔不得不感叹 IPSec 协议设计者的强大——这么多新的、老的协议、算法拼接起来如此天衣无缝，自此 Internet 的险恶被屏蔽在隧道之外！为了方便大家记忆这些缩写，强叔用一张简图概括之：

安全协议	ESP			AH	
加密	DES	3DES	AES		
验证	MD5	SHA		MD5	SHA
密钥交换	IKE（ISAKMP，DH）				

写到这里，强叔心情舒畅，暗自为天地会网络的顺利推动得意。可是问题又来了——将 IKE



应用于 IPSec VPN 之后，那些有固定公网 IP 地址的大中型分舵的通信问题很快就解决了。但一些小型分舵只有私网 IP 地址，用常规的 IKE 方式部署 IPSec VPN 行不通。这种情况下怎么办呢？别担心，IKE 的精彩之处还在于不出招则已、出招即能克敌；招招有戏、变化无穷，欲听详解请见下回分解。

IPSec 模板海纳百川

IPSec 模板海纳百川

自从天地会采用了 IKE 方式协商 IPSec 安全策略，IPSec 配置和维护的工作量大减。然而之前天地会用过的手工方式 IPSec 安全策略或 IKE 方式 IPSec 安全策略都要求 IPSec 对端的 IP 固定。可是在公网 IP 几近枯竭的 Internet 上要拥有一个固定的公网 IP 谈何容易！天地会每次要申请一个固定公网 IP 地址都要大费周折。于是天地会提出疑问：配置 IPSec VPN 的通信双方必须使用固定的公网 IP 地址吗？

当然不是。下面强叔就给大家介绍一种新的 IPSec 安全策略：模板方式 IPSec 安全策略。

跟 IKE 方式 IPSec 安全策略一样，模板方式 IPSec 安全策略也要依靠 IKE 协商 IPSec 隧道。

模板方式 IPSec 安全策略最大的改进就是不要求对端 IP 地址固定：可以严格指定对端 IP 地址（单个 IP），可以宽泛指定对端 IP 地址（IP 地址段）也可以干脆不指定对端 IP（意味着对端 IP 可以是任意 IP）。

模板方式 IPSec 安全策略就好像一员猛将，根本不把对端 IP 放在眼里，什么固定 IP、动态地址、私网 IP 统统不在话下。只管放马过来，来多少都照单全收。正因为这种大将风范，模板方式 IPSec 安全策略特别适用于天地会总舵，用于响应众多分舵的协商请求。且分舵数量越多越明显：

- ❖ 采用 IKE 方式 IPSec 策略。总舵需要配置 N 个 IPSec 策略，N 个 IKE 对等体。N=分舵数量。
- ❖ 采用模板方式 IPSec 策略。总舵需要配置 1 个 IPSec 策略，一个 IKE 对等体。与 N 的取值无关。

总之，模板方式 IPSec 安全策略的两大优点令他在 IPSec 阵营中始终保持明星地位：

- ❖ 对对端要求甚少，有固定 IP 或者没有无所谓
- ❖ 配置简单，只要配置一个 IKE Peer 即可

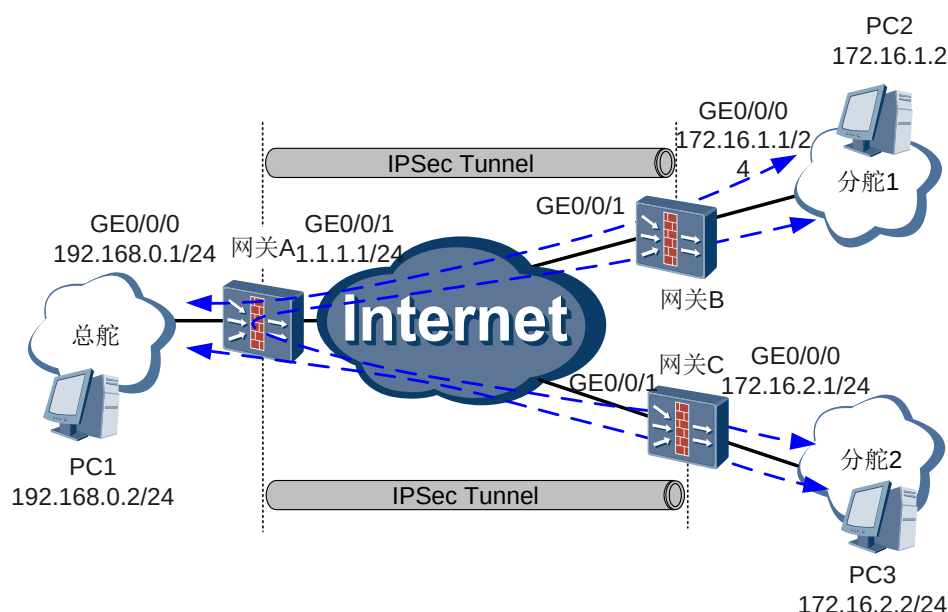
但明星也是有缺点的，这个缺点也可以说是模板方式 IPSec 安全策略“不拘小节”的地方：模板方式 IPSec 安全策略只能响应对端发起的协商请求，不能主动发起协商。

至此强叔已经介绍了三种 IPSec 安全策略：手工方式 IPSec 安全策略、IKE 方式 IPSec 安全

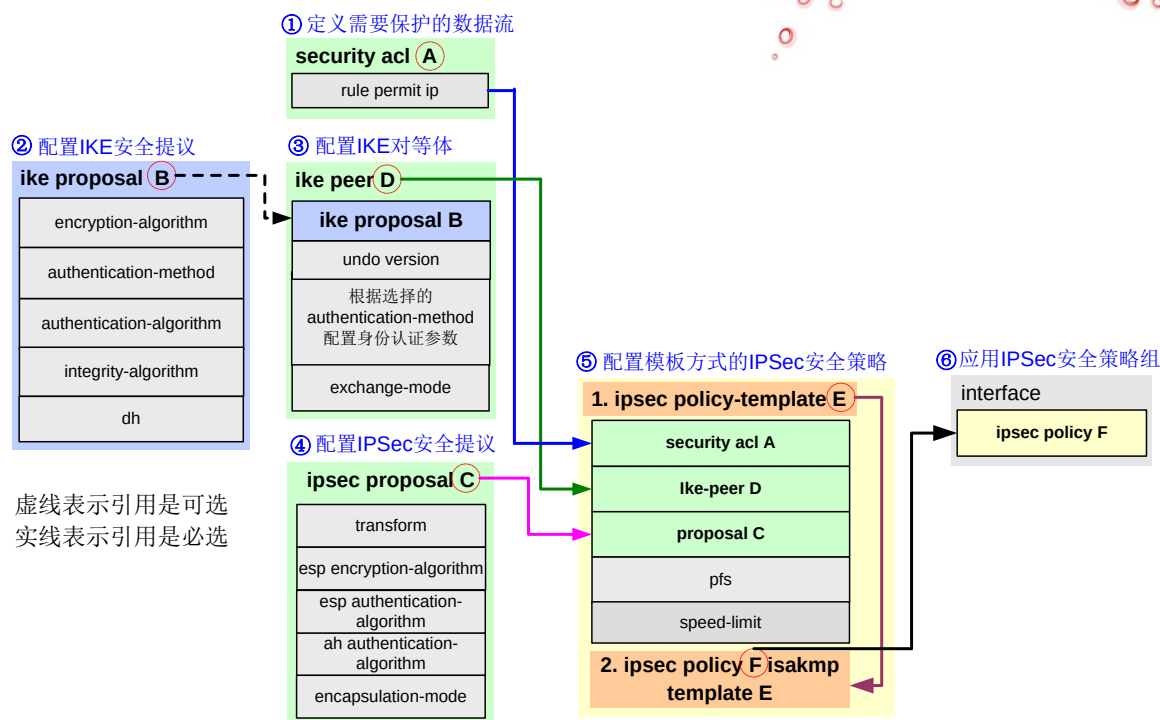
策略、模板方式 IPsec 安全策略。这三种 IPsec 安全策略都可以配置在一个 IPsec 安全策略组中。所谓 IPsec 安全策略组就是一组名称相同的 IPsec 安全策略。在一个 IPsec 安全策略组中最多只能存在一个模板方式 IPsec 安全策略，且其序号必须最大，即优先级最小。否则接入请求被模板接收了，优先级低的 IKE 方式 IPsec 策略则无法施展。切记！

分舵接入 IP 变，模板接收只等闲

总舵跟分舵 1 和分舵 2 之间建立 IPsec VPN 的组网图如下所示。本图中分舵 1 和分舵 2 的出接口采用动态方式获取公网 IP 地址。要求在分舵 1 跟总舵、分舵 2 跟总舵之间建立 IPsec 隧道，分舵 1 跟分舵 2 之间也可以通过 IPsec 通信。



模板方式 IPsec 安全策略配置流程如下：



分舵 2 的配置与分舵 1 类似，请参考分舵 1 的配置。IKE 安全提议和 IPsec 安全提议采用缺省配置（每个版本的缺省配置可能不同，请配置时注意）。

配置项	总舵（采用模板方式 IPsec 策略）	分舵 1（采用 IKE 方式 IPsec 策略）
配置 IKE 安全提议	ike proposal 10	ike proposal 10
配置 IKE 对等体	ike peer a ike-proposal 10 pre-shared-key tiandihui1 <i>// 可以不配置 remote-address，也可以通过 remote-address 指定 IP 地址段。</i>	ike peer a ike-proposal 10 remote-address 1.1.1.1 pre-shared-key tiandihui1
ACL	acl number 3000 rule 5 permit ip destination 172.16.1.0 0.0.0.255 rule 10 permit ip destination 172.16.2.0 0.0.0.255 <i>//配置一条 ACL： rule5 允许分舵 2 和总舵访问分舵 1，source 必须包括分舵 2 和总舵，destination 为分舵 1。本例中不指定 source，表明 source 可以为分舵 2 也可以为总舵或其他 IP 地址段。 rule10 允许分舵 1 和总舵访问分舵 2，source 必须包括分舵 1 和总舵，destination 为分舵 2。本例中不指定 source，表明 source 可以为分舵 1 也可以为总舵或其他 IP 地址段。</i>	acl number 3000 rule 5 permit ip source 172.16.1.0 0.0.0.255 <i>//允许分舵 1 访问分舵 2 和总舵，source 为分舵 1，destination 为分舵 2 和总舵。本例中不指定 destination，表明 source 可以为分舵 2 也可以为总舵或其他 IP 地址段。</i>
IPsec 安全提议	ipsec proposal a	ipsec proposal a
IPsec 安全策略	ipsec policy-template tem1 1 <i>//配置 IPsec 策略模板</i>	ipsec policy policy1 1 isakmp security acl 3000

配置项	总舵（采用模板方式 IPsec 策略）	分舵1（采用 IKE 方式 IPsec 策略）
	<pre>security acl 3000 proposal a ike-peer a ipsec policy policy1 12 isakmp template tem1// 配置模板方式 IPsec 策略</pre>	<pre>proposal a ike-peer a</pre>
应用 IPsec 安全策略	<pre>interface GigabitEthernet0/0/1 ip address 1.1.1.1 255.255.255.0 ipsec policy policy1</pre>	<pre>interface GigabitEthernet0/0/1 ip address 2.2.2.2 255.255.255.0 ipsec policy policy1 auto-neg// 配置 auto-neg 后，不需要流量触发就直接建立 IPsec 隧道。</pre>

部署完成后进行验证：

1. 在总舵 IPsec 网关上可以查看到总舵跟分舵 1 和分舵 2 都正常建立了第一阶段和第二阶段安全联盟。
2. 分舵 1、分舵 2、总舵可以互相通信。

问题：若在接口上应用 IPsec 策略时不配置 **auto-neg** 参数，分舵 1 跟分舵 2 可以直接通信吗？

1. 在分舵 1 和分舵 2 的网关取消接口上应用的 IPsec 策略后，重新应用时不配置 **auto-neg** 参数。由分舵 1 的 PC2 ping 分舵 2 的 PC3，无法 ping 通。
2. 查看分舵 1 上安全联盟的建立情况。

```
<FW_B> display ike sa
current ike sa number: 2
```

```
-----
conn-id    peer                flag                phase vpn
-----
40022      1.1.1.1             RD|ST              v2:2  public
7          1.1.1.1             RD|ST              v2:1  public
```

分舵 1 跟总舵之间的安全联盟正常建立。

3. 查看分舵 2 上安全联盟的建立情况。

```
<FW_C> display ike sa
current sa Num :0
```

分舵 2 跟总部之间没有建立安全联盟。原因在于总部配置了模板方式 IPsec 安全策略，只能响应协商。所以分舵 1 到总舵的安全联盟正常创建了，而总舵到分舵 2 的安全联盟没法建立。在分舵 1 跟分舵 2 上应用 IPsec 安全策略时带上 **auto-neg** 参数后，IPsec 安全联盟自动创建。

由于分舵 1 到总舵、总舵到分舵 2 之间的安全联盟都已创建好，所以分舵 1 跟分舵 2 可以通信。同样总舵可以 ping 通分舵。

模板方式 IPSec 安全策略与 IKE 方式 IPSec 安全策略都需要通过 IKE 来协商 IPSec 隧道。协商的过程一样，这里强叔就不多说了。本篇重点讲讲模板方式 IPSec 安全策略的“特色”之处。

模板神功露破绽，个性化接入化圆满（仅 USG9000 系列防火墙支持）

IPSec 模板中只能引用一个 IKE Peer。而一个 IKE Peer 中只能配置一个预共享密钥，因此所有与之对接的对端都必须配置相同的预共享密钥。于是问题来了，只要有一个 IPSec 网关的预共享密钥泄露，则所有其他网关的 IPSec 安全都受到威胁。

那么在总舵跟多个分舵对接的点到多点的组网中，分舵可以配置不同的预共享密钥吗？既然预共享密钥跟密钥生成和身份认证相关，只要把预共享密钥与设备身份挂钩就可以。预共享密钥认证方式下可以采用本地 IP 地址或设备名称进行身份认证。那通过 IP 地址来指定预共享密钥或通过设备名称来指定预共享密钥就可以为每个接入对端配置“个性化的预共享密钥”了。

🔥 在总舵通过对端 IP 地址为每个分舵指定个性化预共享密钥

此方式适用于分舵出口 IP 地址固定的情况。将总舵在 ike peer 下面配置的 remote-address 和 pre-shared-key 删掉，改为在全局下为每个分舵配置 remote-address 和 pre-shared-key 就成，这样既保留模板的先进性，又巧妙规避了模板的局限性。

配置项	总舵	分舵 1（分舵 2 配置跟分舵 1 类似）
配置 IKE 对等体	ike peer a local-id-type ip ike-proposal 10	ike peer a local-id-type ip ike-proposal 10 remote-address 1.1.1.1 pre-shared-key tiandihui1
配置个性化预共享密钥	ike remote-address 2.2.2.2 pre-shared-key tiandihui1 ike remote-address 2.2.3.2 pre-shared-key tiandihui2	-

🔥 在总舵通过对端设备名称指定预共享密钥

当分舵出口没有固定 IP 地址时，可以通过设备名称来标识身份（ike local-name），此时总舵

在全局下为每个分舵配置 remote-id 和 pre-shared-key 即可。

配置项	总舵	分舵 1 (分舵 2 配置跟分舵 1 类似)
配置 IKE 对等体	<pre>ike peer a local-id-type ip ike-proposal 10</pre>	<pre>ike peer a local-id-type fqdn//通过设备名称进行身份认证时 本地认证类型需配置为 FQDN 或 USER-FQDN。 ike-proposal 10 remote-address 1.1.1.1 pre-shared-key tiandihui1</pre>
配置本地名称	-	ike local-name tdhfd1
配置个性化预共享密钥	<pre>ike remote-id tdhfd1 pre-shared-key tiandihui1 ike remote-id tdhfd1 pre-shared-key tiandihui2</pre>	-

注意：本地配置的“对端 IP”或“对端 ID”必须与对端网关上配置的“本地 ID”一致。

IP 变化不打紧，域名映射获真身

分舵用动态 IP 地址接入的情况，IKE 方式 IPsec 安全策略是否真的束手无策呢？

强叔闭目打坐通过对数通知知识的融会贯通、举一反三，也找到了一个可以帮助 IKE 方式 IPsec 安全策略解决问题的方法：对端 IP 地址不固定，当然也就无法配置 remote-address，但总部可以通过其他方式间接获知 IP 地址，比如通过域名。即总部可以用指定 remote-domain 代替 remote-address；分部配置 DNS 获得域名和 IP 地址之间的映射关系，开启 DDNS 保证映射关系能够实时更新。当然配置动态域名的方式也适用于模板方式 IPsec 策略。

说明：此处仅给出配置差异部分。

配置调整	总舵	分舵
IPsec 配置	仅 IKE Peer 中配置有变化 <pre>ike peer a ike-proposal 10 pre-shared-key tiandihui1 remote-domain www.adcd.3322.org</pre>	无变化
新增配置		1、开启域名解析 <pre>dns resolve dns server 200.1.1.1</pre> 2、配置 DDNS 策略//以下配置需联系 DDNS 服务提供商，并根据 DDNS 服务提供商的说明操作。 <pre>ddns policy abc ddns client www.adcd.3322.org ddns server www.3322.org</pre>

配置调整	总舵	分舵
		<pre>ddns username abc123 password abc123</pre>
		3、应用 DDNS 策略
		<pre>ddns client enable</pre>
		<pre>interface dialer 1</pre>
		<pre>ddns apply policy abc</pre>

此方案的局限在于动态接入方必须有固定域名，另外增加了 DNS 和 DDNS 的配置，有点小复杂，所以强叔至今的感受是“不得不用时才会使用”。

模板方式 IPsec 安全策略很强大吧？实际场景中，他并不是孤军作战的，只有模板方式 IPsec 安全策略和 IKE 方式 IPsec 安全策略联合作战才能全线告捷：

场景	总舵	分舵
总舵 IP 地址固定+分舵 IP 地址固定	IKE 方式 IPsec 策略或模板方式 IPsec 策略	IKE 方式 IPsec 策略
总舵 IP 地址固定+分舵 IP 地址动态获取	IKE 方式 IPsec 策略（指定对端域名）或模板方式 IPsec 策略	IKE 方式 IPsec 策略
总舵 IP 地址动态获取+分舵 IP 地址动态获取	IKE 方式 IPsec 策略（指定对端域名）或模板方式 IPsec 策略	IKE 方式 IPsec 策略（指定对端域名）

看似通过上面三大方案基本可以完美应对总舵-分舵 IPsec VPN 的各种场景了，但实际上是江湖之所以为江湖就是因为永远会有点风浪：分舵申请不到公网 IP 地址，只能配置私网 IP 地址。怎么办，强叔？强叔之所以为强叔，就是因为真的强大，敬请关注下期的强叔侃墙。

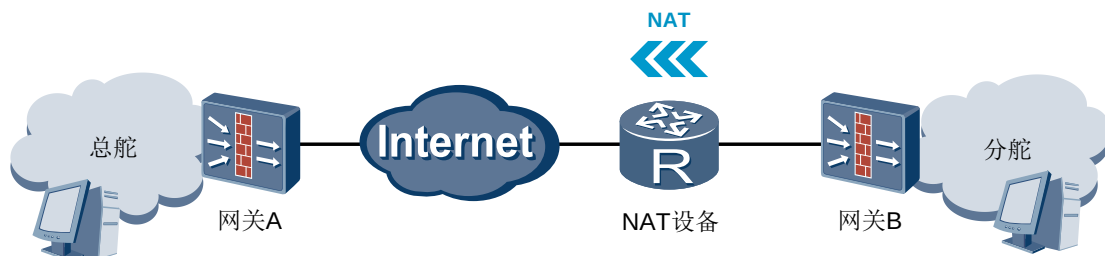
🍁 IPSec 遭遇 NAT 处变不惊

前文说到，使用模板方式可以在总舵与公网 IP 不固定的分舵之间建立 IPSec 隧道。至此，无论是拥有固定公网 IP 的分舵还是动态获得公网 IP 的分舵，都可以通过 IPSec 安全地访问总舵，天地会业务兴隆一片祥和。

但 Internet 的江湖远非如此平静，天地会又面临新的问题。有的分舵连动态的公网 IP 都没有，只能先由网络中的 NAT 设备进行地址转换，然后才能访问 Internet，此时分舵能否正常访问总舵？另外，分舵除了访问总舵之外，还有访问 Internet 的需求，有些分舵在网关上同时配置了 IPSec 和 NAT，两者能否和平共处？如何解决上述这两个问题，且听强叔一一道来。

IPSec 隧道途径 NAT 设备，NAT 穿越力保畅通无阻

先来看网络中存在 NAT 设备的情况，如下图所示，分舵网关 B 的出接口 IP 是私网地址，必须经过 NAT 设备进行地址转换，转换为公网 IP 之后才能与总舵网关 A 建立 IPSec 隧道。



我们都知道，IPSec 是用来保护报文不被修改的，而 NAT 却专门修改报文的 IP 地址，看起来两者水火不容，我们来详细分析一下。首先，协商 IPSec 的过程是由 ISAKMP 消息完成的，而 ISAKMP 消息是经过 UDP 封装的，源和目的端口号均是 500，NAT 设备可以转换该消息的 IP 地址和端口，因此 ISAKMP 消息能够顺利的完成 NAT 转换，成功协商 IPSec 安全联盟。但是数据流量是通过 AH 或 ESP 协议传输的，在 NAT 转换过程中存在问题。下面分别看一下 AH 和 ESP 报文能否通过 NAT 设备。

🍁 AH 协议

因为 AH 对数据进行完整性检查，会对包括 IP 地址在内的整个 IP 包进行 Hash 运算。而 NAT 会改变 IP 地址，从而破坏 AH 的 Hash 值。因此 AH 报文无法通过 NAT 网关。

🍁 ESP 协议

ESP 对数据进行完整性检查，不包括外部的 IP 头，IP 地址转换不会破坏 ESP 的 Hash 值。但 ESP 报文中 TCP 的端口已经加密无法修改，所以对于同时转换端口的 NAT 来说，ESP 没法支持。

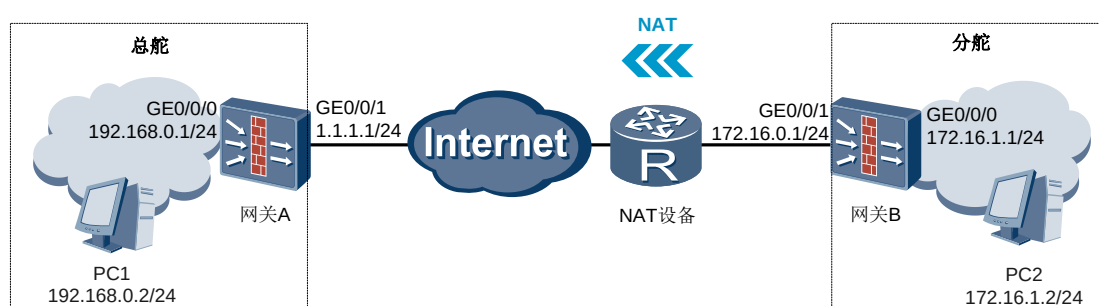
为了解决这个问题，必须在建立 IPsec 隧道的两个网关上同时开启 NAT 穿越功能（对应命令行 `nat traversal`）。开启 NAT 穿越功能后，当需要穿越 NAT 设备时，ESP 报文会被封装在一个 UDP 头中，源和目的端口号均是 4500。有了这个 UDP 头就可以正常进行转换。

根据 NAT 设备所处的位置和地址转换功能的不同，我们从下面三个场景来分别介绍：

场景一：NAT 转换后的分舵公网地址未知，总舵使用模板方式

该场景中，NAT 设备位于分舵网络之外，分舵网关 B 接口 GE0/0/1 的私网 IP 地址，经过 NAT 设备转换后变为公网 IP 地址。由于总舵无从获知经过 NAT 设备转换后的分舵公网 IP 地址，也就无法在总舵网关 A 上明确指定对端分舵的公网地址。因此，总舵网关 A 必须使用模板方式来配置 IPsec，同时总舵和分舵的网关上都要开启 NAT 穿越功能。

总舵既然使用了模板方式，那就无法主动访问分舵，只能由分舵主动向总舵发起访问。



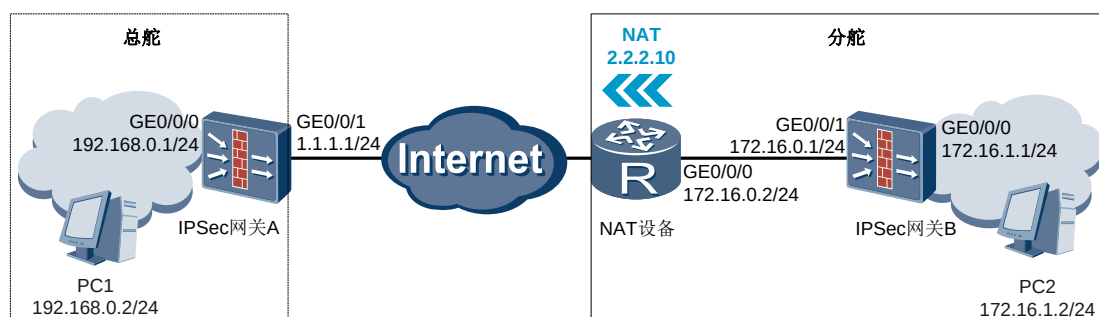
总舵和分舵网关的关键配置如下：

关键配置	总舵	分舵
IPSec 安全提议	<pre>ipsec proposal pro1 transform esp //采用 ESP 协议传输报文</pre>	<pre>ipsec proposal pro1 transform esp //采用 ESP 协议传输报文</pre>
IKE 对等体	<pre>ike peer fenduo pre-shared-key tiandihui1 ike-proposal 10 nat traversal //双方同时开启，默认为开启</pre>	<pre>ike peer zongduo pre-shared-key tiandihui1 ike-proposal 10 remote-address 1.1.1.1 nat traversal //双方同时开启，默认为开启</pre>
IPSec 安全策略	<pre>ipsec policy-template tem1 1 //配置模板方式 security acl 3000 proposal pro1 ike-peer fenduo ipsec policy policy1 1 isakmp template tem1</pre>	<pre>ipsec policy policy1 1 isakmp security acl 3000 proposal pro1 ike-peer zongduo</pre>

场景二：NAT 转换后的分舵公网地址可知，总舵使用模板方式或 IKE 方式

该场景中，NAT 设备位于分舵网络之内，分舵网关 B 接口 GE0/0/1 的私网 IP 地址，经过 NAT 设备转换后变为公网 IP 地址。由于 NAT 设备在分舵控制范围之内，转换后的公网地址可知，所以总舵网关 A 上可以使用模板方式也可以使用 IKE 方式来配置 IPsec。

需要注意的是，即便使用 IKE 方式，总舵也无法主动与分舵建立 IPsec 隧道。这不是 IPsec 的问题，而是 NAT 设备的问题。NAT 设备只提供了源地址转换，实现分舵--->总舵这一方向的访问，分舵“隐藏”在 NAT 设备之后，无法实现总舵--->分舵方向的访问。如果要实现总舵主动访问分舵网关 B 的私网地址，则需要在 NAT 设备上配置 NAT Server 功能，我们会在场景三中详细介绍。



总舵和分舵网关的关键配置如下：

关键配置	总舵	分舵
IPsec 安全提议	ipsec proposal pro1 transform esp //采用 ESP 协议传输报文	ipsec proposal pro1 transform esp //采用 ESP 协议传输报文
IKE 对等体	ike peer fenduo pre-shared-key tiandihui1 ike-proposal 10 nat traversal //双方同时开启，默认为开启 remote-address 2.2.2.10 //对端地址为 NAT 后的地址。采用 IKE 方式时由于对端地址为单个地址，所以要求 NAT 设备的地址池中只有一个地址。模板方式时无此要求。 remote-address authentication-address 172.16.0.1 //认证地址为 NAT 前的地址	ike peer zongduo pre-shared-key tiandihui1 ike-proposal 10 remote-address 1.1.1.1 nat traversal //双方同时开启，默认为开启
IPsec 安全策略	ipsec policy policy1 isakmp security acl 3000 proposal pro1 ike-peer fenduo	ipsec policy policy1 1 isakmp security acl 3000 proposal pro1 ike-peer zongduo

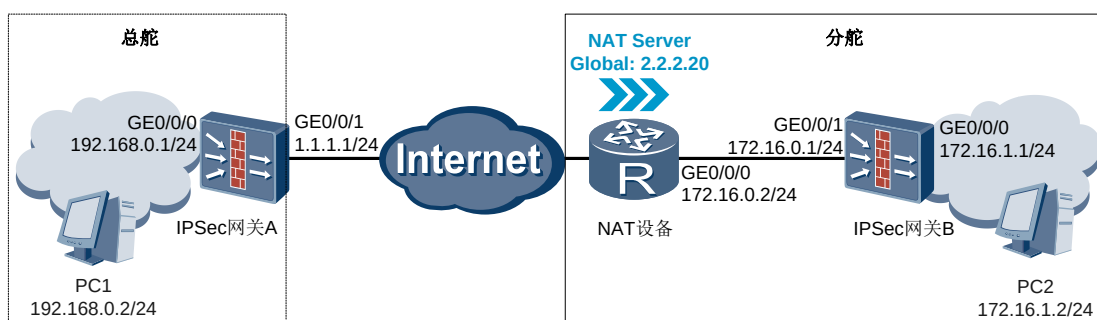
场景三：NAT 设备提供 NAT Server 功能，总舵使用 IKE 方式主动访问分舵

该场景中，NAT 设备位于分舵网络之内，提供 NAT Server 功能，对外发布的地址是 2.2.2.20，映射成私网地址是分舵网关 B 接口 GE0/0/1 的地址 172.16.0.1。总舵网关 A 上使用 IKE 方式来配置 IPsec，即可实现总舵--->分舵方向的访问。在 NAT 设备上配置 NAT Server，将 2.2.2.20 的 UDP 500 端口和 4500 端口分别映射到 172.16.0.1 的 UDP 500 端口和 4500 端口，具体配置如下：

```
nat server protocol udp global 2.2.2.20 500 inside 172.16.0.1 500
```

```
nat server protocol udp global 2.2.2.20 4500 inside 172.16.0.1 4500
```

同时，由于 NAT 设备上配置 NAT Server 时会生成反向 Server-map 表，所以分舵的网关 B 也可以主动向总舵发起访问。报文到达 NAT 设备后，匹配反向 Server-map 表，源地址转换为 2.2.2.20，即可实现分舵--->总舵方向的访问。



总舵和分舵网关的关键配置如下：

关键配置	总舵	分舵
IPSec 安全提议	ipsec proposal pro1 transform esp	ipsec proposal pro1 transform esp
IKE 对等体	ike peer fenduo pre-shared-key tiandihui1 ike-proposal 10 nat traversal //双方同时开启，默认为开启 remote-address 2.2.2.20 //对端地址为 NAT 后的地址 remote-address authentication-address 172.16.0.1 //认证地址为 NAT 前的地址	ike peer zongduo pre-shared-key tiandihui1 ike-proposal 10 remote-address 1.1.1.1 nat traversal //双方同时开启，默认为 开启
IPSec 安全策略	ipsec policy policy1 isakmp security acl 3000 proposal pro1 ike-peer fenduo	ipsec policy policy1 1 isakmp security acl 3000 proposal pro1 ike-peer zongduo

下面以第二种场景为例，分别介绍一下采用 IKEv1 和 IKEv2 时是如何进行 NAT 穿越。

IKEv1 协商 NAT 穿越大揭秘

1. 开启 NAT 穿越时，IKEv1 协商第一阶段的前两个消息会发送标识 NAT 穿越（NAT Traversal，简称 NAT-T）能力的 Vendor ID 载荷（主模式和野蛮模式都是）。用于检查通信双方是否支持 NAT-T。

```

❑ Type Payload: Vendor ID (13) : RFC 3947 Negotiation of NAT-Traversal in the IKE
  Next payload: Vendor ID (13)
  Payload length: 20
  Vendor ID: 4a131c81070358455c5728f20e95452f
  Vendor ID: RFC 3947 Negotiation of NAT-Traversal in the IKE
❑ Type Payload: Vendor ID (13) : draft-ietf-ipsec-nat-t-ike-02\n
  Next payload: Vendor ID (13)
  Payload length: 20
  Vendor ID: 90cb80913ebb696e086381b5ec427b1f
  Vendor ID: draft-ietf-ipsec-nat-t-ike-02\n
❑ Type Payload: Vendor ID (13) : draft-ietf-ipsec-nat-t-ike-00
  Next payload: NONE / No Next Payload (0)
  Payload length: 20
  Vendor ID: 4485152d18b6bbcd0be8a8469579ddcc
  Vendor ID: draft-ietf-ipsec-nat-t-ike-00
  
```

当双方都在各自的消息中包含了该载荷时，才会进行相关的 NAT-T 协商。

2. 主模式消息 3 和消息 4（野蛮模式消息 2 和消息 3）中发送 NAT-D（NAT Discovery）载荷。NAT-D 载荷用于探测两个要建立 IPSec 隧道的网关之间是否存在 NAT 网关以及 NAT 网关的位置。

```

❑ Type Payload: NAT-D (RFC 3947) (20)
  Next payload: NAT-D (RFC 3947) (20)
  Payload length: 24
remote HASH of the address and port: c2954146c97839f5eb42bb0438395a42de5f6aa5
❑ Type Payload: NAT-D (RFC 3947) (20)
  Next payload: NONE / No Next Payload (0)
  Payload length: 24
local HASH of the address and port: 88870822cf707e8da4023106192e8489f0e684ed
  
```

通过协商双方向对端发送源和目的的 IP 地址与端口的 Hash 值，就可以检测到地址和端口在传输过程中是否发生变化。若果协商双方计算出来的 Hash 值与它收到的 Hash 值一样，则表示它们之间没有 NAT。否则，则说明传输过程中对 IP 或端口进行了 NAT 转换。

第一个 NAT-D 载荷为对端 IP 和端口的 Hash 值，第二个 NAT-D 载荷为本端 IP 和端口的 Hash 值。

3. 发现 NAT 网关后，后续 ISAKMP 消息（主模式从消息 5、野蛮模式从消息 3 开始）的端口号转换为 4500。ISAKMP 报文标识了“Non-ESP Marker”。

```

❑ User Datagram Protocol, Src Port: ipsec-nat-t (4500), Dst Port: ipsec-nat-t (4500)
❑ UDP Encapsulation of IPsec Packets
  Non-ESP Marker
❑ Internet Security Association and Key Management Protocol
  
```


4. 在第二阶段会启用 NAT 穿越协商。在 IKE 中增加了两种 IPSec 报文封装模式：UDP 封装隧道模式报文（UDP-Encapsulated-Tunnel）和 UDP 封装传输模式报文（UDP-Encapsulated-Transport）。通过为 ESP 报文封装 UDP 头，当封装后的报文通过 NAT 设备时，NAT 设备对该报文的外层 IP 头和增加的 UDP 头进行地址和端口号转换。UDP 报文端口号修改为 4500。

```

User Datagram Protocol, Src Port: ipsec-nat-t (4500), Dst Port: ipsec-nat-t (4500)
  Source port: ipsec-nat-t (4500)
  Destination port: ipsec-nat-t (4500)
  Length: 100
  Checksum: 0x0000 (none)
  UDP Encapsulation of IPsec Packets
    Encapsulating Security Payload

```

IKEv2 协商 NAT 穿越大揭秘

1. 开启 NAT 穿越后，IKE 的发起者和响应者都在 IKE_SA_INIT 消息对中包含类型为 NAT_DETECTION_SOURCE_IP 和 NAT_DETECTION_DESTINATION_IP 的通知载荷。这两个通知载荷用于检测在将要建立 IPSec 隧道的两个网关之间是否存在 NAT，哪个网关位于 NAT 之后。如果接收到的 NAT_DETECTION_SOURCE 通知载荷没有匹配数据包 IP 头中的源 IP 和端口的 Hash 值，则说明对端位于 NAT 网关后面。如果接收到的 NAT_DETECTION_DESTINATION_IP 通知载荷没有匹配数据包 IP 头中的目的 IP 和端口的 Hash 值，则意味着本端位于 NAT 网关之后。

```

Type Payload: Notify (41)
  Next payload: Notify (41)
  0... .... = Critical Bit: Not Critical
  Payload length: 28
  Protocol ID: RESERVED (0)
  SPI Size: 0
  Notify Message Type: NAT_DETECTION_SOURCE_IP (16388)
  Notification DATA: 6729d884a9f751b8dbca63cab4165e6d137b7cd8
Type Payload: Notify (41)
  Next payload: NONE / No Next Payload (0)
  0... .... = Critical Bit: Not Critical
  Payload length: 28
  Protocol ID: RESERVED (0)
  SPI Size: 0
  Notify Message Type: NAT_DETECTION_DESTINATION_IP (16389)
  Notification DATA: 908595db62cb4f3d4f782f156a6715dd03cfc6c8

```

2. 检测到 NAT 网关后，从 IKE_AUTH 消息对开始 ISAKMP 报文端口号改为 4500。报文标识 “Non-ESP Marker”。


```

User Datagram Protocol, Src Port: ipsec-nat-t (4500), Dst Port: ipsec-nat-t (4500)
  Source port: ipsec-nat-t (4500)
  Destination port: ipsec-nat-t (4500)
  Length: 240
  Checksum: 0xef30 [validation disabled]
UDP Encapsulation of IPsec Packets
  Non-ESP Marker
Internet Security Association and Key Management Protocol

```

IKEv2 中也使用 UDP 封装 ESP 报文，当封装后的报文通过 NAT 设备时，NAT 设备对该报文的外层 IP 头和增加的 UDP 头进行地址和端口号转换。UDP 报文端口号修改为 4500。

```

User Datagram Protocol, Src Port: ipsec-nat-t (4500), Dst Port: ipsec-nat-t (4500)
  Source port: ipsec-nat-t (4500)
  Destination port: ipsec-nat-t (4500)
  Length: 100
  Checksum: 0x0000 (none)
UDP Encapsulation of IPsec Packets
  Encapsulating Security Payload

```

在第二种场景中，配置完成后 PC1 可以 ping 通 PC2。

🔥 在总舵网关 A 上查看 IKE SA 和 IPsec SA 的建立情况：

```
<FW_A> display ike sa
```

```
current ike sa number: 2
```

conn-id	peer	flag	phase	vpn
40014	2.2.2.10:264	RD	v1:2	public
40011	2.2.2.10:264	RD	v1:1	public

🔥 在总舵网关 A 上查看会话：

```
<FW_A> display firewall session table
```

```
Current Total Sessions : 2
```

```
udp VPN:public --> public 2.2.2.10:2050-->1.1.1.1:4500
```

```
udp VPN:public --> public 2.2.2.10:2054-->1.1.1.1:500
```

🔥 在分舵网关 B 上查看会话：

```
<FW_B> display firewall session table
```

```
Current Total Sessions : 2
```

```
udp VPN:public --> public 172.16.0.1:4500-->1.1.1.1:4500
```

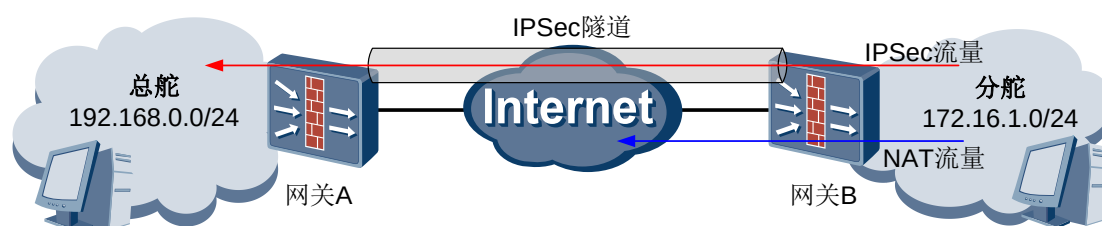
```
udp VPN:public --> public 172.16.0.1:500-->1.1.1.1:500 //刚开始协商时端口号还是 500
```

因为中间 NAT 设备上配置了源 NAT 转换，所以分舵网关 B 上只有分舵到总舵方向的会话，没有总舵到分舵方向的会话。

IPSec 与 NAT 同处一墙，不同流量泾渭分明

前面讲了 IPSec 穿越 NAT 的情况，当 IPSec 和 NAT 同时配置在一台防火墙上时，由于两者处理的流量不同，只要保证两种流量互不干扰，便可相安无事。如下图所示，分舵网关 B 上

同时配置了 IPsec 和 NAT，IPsec 用于保护分舵跟总舵通信的流量，NAT 处理的是分舵访问 Internet 的流量。



按理说两种流量应该是泾渭分明，互不相干。其实不然，在防火墙处理流程中，NAT 在上游，IPsec 在下游，所以 IPsec 流量难免会受到 NAT 的干扰。IPsec 流量一旦命中 NAT 策略就会进行 NAT 转换，转换后的流量不会匹配 IPsec 中的 ACL，也就不会进行 IPsec 处理。

注：强叔后续会推出防火墙处理流程的介绍，敬请期待。

解决这个问题的方法就是在 NAT 策略中配置一条针对 IPsec 流量不进行地址转换的策略，该策略的优先级要高于其他的策略，并且该策略中定义的流量范围是其他策略的子集。这样的话，IPsec 流量会先命中不进行 NAT 转换的策略，地址不会被转换，也就不会影响下面 IPsec 环节的处理，而需要进行 NAT 处理的流量也可以命中其他策略正常转换。

NAT 策略的配置如下：

```
nat-policy interzone trust untrust outbound
policy 0 //需要IPSec保护的流量不进行NAT转换
action no-nat
policy source 172.16.1.0 mask 24
policy destination 192.168.0.0 mask 24
policy 5 //访问Internet的流量进行NAT转换
action source-nat
policy source 172.16.1.0 mask 24
address-group 1
```

若分舵网关 B 上还配置了 NAT Server，配置 NAT Server 时生成的反向 Server-map 表也干扰 IPsec 流量。分舵中的服务器访问总舵时，流量会命中反向 Server-map 表，然后进行地址转换，转换后的流量不会匹配 IPsec 中的 ACL，也就不会进行 IPsec 处理。此时就需要在配置 NAT Server 时指定 **no-reverse** 参数，不生成反向 Server-map 表，关于此问题的具体说明请参见《NAT Server 三十二字真言（上篇）》。

前面的讲解中，总舵和分舵进行身份认证时采用的都是预共享认证方式。预共享密钥还是需要手工配置的，跟总舵对接的分舵数量少的时候没有问题，当分舵数量多的时候，为每个分舵跟总舵规划和配置预共享密钥的工作量还是不小的。对于这种情况，采用证书认证就非常合适。那到底什么是证书，采用证书认证跟预共享认证相比在配置上有什么差别呢。预知详

情，请看下回分解。

VPN

IPsec 引入数字证书

在前面两篇贴子中，天地会总舵和分舵的网关进行身份认证时都是使用预共享密钥方式。单从配置过程来说，该方式配置简单，只需在总舵和分舵的网关上配置相同的密钥即可。但随着分舵数量越来越多，总舵和每个分舵之间形成的对等体都要配置预共享密钥。如果所有对等体都使用同一个密钥，存在安全风险；而每个对等体都使用不同的密钥，又不便于管理和维护。

分舵数量日益增长，天地会亟需一个新的身份信息认证方案来代替预共享密钥方式，降低管理成本。既然天地会的总舵和分舵都是以合法生意（如当铺和票号）作为掩护，不如直接向官府的刑部申请为当铺和票号发放身份凭证，标明当铺和票号的身份信息。因为刑部是公正的、可信赖的官府部门，所以盖着刑部大印的身份凭证也就是可信任的，总舵和分舵就可以直接通过身份凭证来验证双方的身份。

这个身份凭证就叫做数字证书，简称证书，是由第三方机构颁发的代表设备身份信息的“身份证”。通过引入证书机制，总舵和分舵可以简单便捷地进行身份认证。在详细介绍证书的实现原理和获取方法之前，我们先来了解一下公钥密码学和 PKI 框架的知识。

公钥密码学公私分明，PKI 框架深不可测

在《Internet 危机四伏，IPsec 闪亮登场》一篇中，我们提到过对称密码学，即总舵和分舵使用相同的密钥来加密和解密。与之对应的是非对称密码学，即加密和解密数据使用不同的密钥，也叫做**公钥密码学**。目前较常用的公钥密码学算法有 RSA（Ron Rivest、Adi Shamir、LenAdleman）和 DSA（Digital Signature Algorithm）。

公钥密码学中使用了两个不同的密钥：一个可对外界公开的密钥称为“公钥”；只有所有者知道的密钥称为“私钥”。这一对密钥的特点是，使用公钥加密的信息只能用相应的私钥解密，反之使用私钥加密的信息也只能用相应的公钥解密。

利用公钥和私钥的这个特点，就可以实现通信双方的身份认证。例如，某个分舵网关使用自己的私钥对信息进行加密（数字签名），而总舵网关使用分舵网关对外公开的公钥进行解密。因为其他人没有该分舵网关的私钥，所以只要总舵使用对应的公钥可以解密信息就确定信息是由该分舵发出来的，从而实现身份认证功能。

了解公钥密码学的基本概念后，如何在实际环境中应用公钥密码学理论呢？PKI（Public Key

Infrastructure) 正是一个基于公钥密码学理论来实现信息安全服务的基础框架, 数字证书是其核心组成部分, 而 IKE 借用了 PKI 中的证书机制来进行对等体的身份认证。

PKI 框架中包括以下两个重要角色:

- 🍁 终端实体 (EE, End Entity): 证书的最终使用者, 例如总舵和分舵的网关。
- 🍁 证书颁发机构 (CA, Certificate Authority): 是一个权威的、可信任的第三方机构 (类似刑部), 负责证书颁发、查询以及更新等工作。

在 IPSec 中, 总舵和分舵的网关是证书的最终使用者, 要想为网关生成证书, 首先需要保证网关自己的公私密钥对是存在的。我们可以在总舵和分舵的网关上创建公私密钥对, 然后将公钥、网关的信息 (实体信息) 发送给 CA 进行证书申请; 也可以在 CA 上为总舵和分舵的网关生成公私密钥对, 并在此基础上生成证书, 然后在总舵和分舵的网关上分别导入各自的公私密钥对和证书。本篇主要讲解总舵和分舵的网关自己创建公私密钥对并申请证书的内容, CA 为总舵和分舵的网关生成公私密钥对和证书的过程后续介绍。

说明: 本篇中所提到的证书分为两种类型: 网关自身的证书称为本地证书, 代表网关的身份; 而 CA “自签名”的证书称为 CA 证书, 又叫根证书, 代表了 CA 的身份。

上面简单介绍了证书涉及的几个重要概念, 大家先有一个初步了解, 关于公钥密码学的详细介绍后续会给出。下面我们来看一下如何为总舵和分舵的网关获取到证书。

证书重要取之有道, 在线离线灵活选择

总舵和分舵的网关申请证书前, 必须先生成公私密钥对, 然后将公钥以及网关的实体信息提供给 CA, 由 CA 根据这些信息来生成证书。总舵和分舵的网关可以通过下面两种方式申请证书:

🍁 在线方式 (带内方式)

网关和 CA 通过证书申请协议交互报文, 在线申请证书, 申请到的证书直接保存到网关的存储设备中 (Flash 或 CF 卡), 常用的证书申请协议有 SCEP (Simple Certification Enrollment Protocol) 和 CMP (Certificate Management Protocol)。该方式适用于网关支持 SCEP 或 CMP 的情况, 同时依赖于网关与 CA 之间网络的连通状态。

🍁 离线方式 (带外方式)

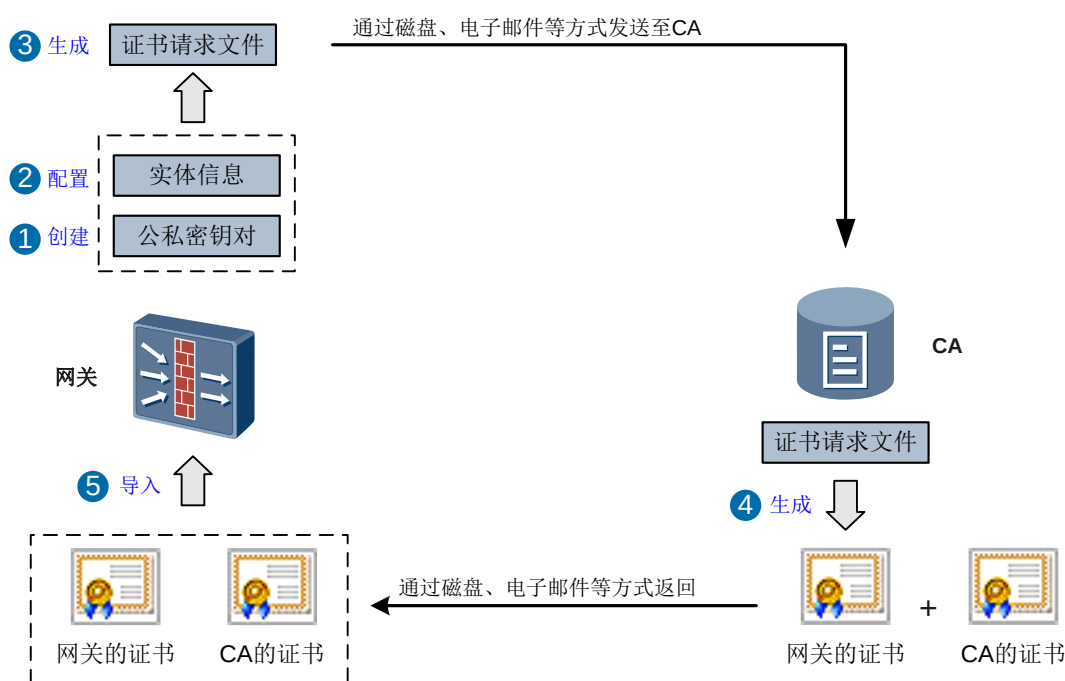
首先, 网关生成证书请求文件 (内容包括公钥和实体信息), 我们将该文件通过磁盘、电子邮件等方式将该文件发送给 CA。然后, CA 根据证书请求文件为网关制作证书, 同样通过磁盘、

电子邮件等方式将证书返回。最后，我们将证书上传到网关的存储设备中。该方式适用于网关不支持 SCEP 或 CMP 的情况，或者网关与 CA 之间无法通过网络互访的情况。

上述两种方式可根据网关的实际情况灵活选择。下面以离线方式为例，介绍天地会总舵和分舵的网关获取证书的过程。



离线方式申请证书的流程如下图所示：



1. 创建公私密钥对

首先，在网关 FWA 和 FWB 上创建各自的公私密钥对，在申请证书时会用到公钥信息。创建过程中，系统会提示输入公钥的位数，位数的长度范围从 512 到 2048。公钥的长度越大，其安全性就越高，但计算速度相对来说比较慢。这里我们要求最高的安全性，所以输入 2048。

在网关 FWA 上创建公私密钥对：

```
[FWA] rsa local-key-pair create
The key name will be: FWA_Host
The range of public key size is (512 ~ 2048).
NOTES: If the key modulus is greater than 512,
It will take a few minutes.
```

Input the bits in the modulus[default = 512]:2048

Generating keys...

.....+++

.....+++

.....+++++++

.+++++++

在网关 FWA 上创建公私密钥对:

[FWA] rsa local-key-pair create

The key name will be: FWB_Host

The range of public key size is (512 ~ 2048).

NOTES: If the key modulus is greater than 512,

It will take a few minutes.

Input the bits in the modulus[default = 512]:2048

Generating keys...

.....+++

.....+++

.....+++++++

.+++++++

2. 配置实体信息

申请证书时, 网关 FWA 和 FWB 必须向 CA 提供能够证明自己身份的信息, 实体信息代表的就是网关的身份信息, 包括: 通用名称 (Common Name)、FQDN (Fully Qualified Domain Name) 名称、IP 地址、电子邮箱地址等。其中, 通用名称是必须配置的, 而其他几项是可选配置的。

上述信息都将会包含在证书中, 在 IKE 对等体中配置 ID 类型时, 就可以根据证书中包含的实体信息来决定使用哪种 ID 类型来进行认证。

实体信息配置完成后, 还需要在 PKI 域中引用实体信息。下面给出网关 FWA 和 FWB 上实体信息和 PKI 域的配置情况:

关键配置	FWA	FWB
实体信息	pki entity fwa common-name fwa //通用名称 fqdn fwa.tdh.com //FQDN 名称 ip-address 1.1.1.1 //IP 地址 email fwa@tdh.com //Email 地址	pki entity fwb common-name fwb //通用名称 fqdn fwb.tdh.com //FQDN 名称 ip-address 3.3.3.3 //IP 地址 email fwb@tdh.com //Email 地址
PKI 域	pki domain fwa certificate request entity fwa //PKI 域中引用实体信息	pki domain fwb certificate request entity fwb //PKI 域中引用实体信息

3. 生成证书请求文件

接下来就可以在网关 FWA 和 FWB 上生成证书请求文件, 生成的证书请求文件以 “PKI 域

名.req” 的名字保存在网关 FWA 和 FWB 的存储设备中，FWA 生成的证书请求文件名字是 **fwa.req**，FWA 生成的证书请求文件名字是 **fwb.req**。

```
[FWA] pki request-certificate domain fwa pkcs10
```

```
Creating certificate request file...
```

```
Info: Create certificate request file successfully.
```

```
[FWB] pki request-certificate domain fwb pkcs10
```

```
Creating certificate request file...
```

```
Info: Create certificate request file successfully.
```

在 FWA 上查看生成的证书请求文件，可以看到里面包含了上面配置的通用名称、FQDN 名称、IP 地址和电子邮箱地址，以及 FWA 的公钥信息。

```
[FWA] display pki cert-req filename fwa.req
```

```
Certificate Request:
```

```
Data:
```

```
Version: 0 (0x0)
```

```
Subject: CN=fwa
```

```
Subject Public Key Info:
```

```
Public Key Algorithm: rsaEncryption
```

```
RSA Public Key: (2048 bit)
```

```
Modulus (2048 bit):
```

```
00:ae:68:50:18:e7:55:32:7a:0e:61:b6:6e:47:45:
```

```
ec:fb:29:d9:1b:4a:9d:6b:b0:00:b0:65:c8:fc:5b:
```

```
b4:68:d7:90:7d:96:f7:1d:e4:62:43:06:bc:d0:a3:
```

```
5b:b4:fa:30:a3:19:7e:6f:7c:05:6b:47:0c:a2:42:
```

```
1b:c4:82:f7:5b:0a:73:a1:0a:8b:00:dd:37:aa:5e:
```

```
21:02:56:b2:e6:55:31:08:8f:71:03:13:92:b9:c1:
```

```
51:7e:51:04:e2:ca:85:2e:45:97:bb:9a:0e:ed:61:
```

```
03:97:d2:1e:44:b2:9f:ff:b9:b1:1d:5d:65:7e:fc:
```

```
e6:13:c3:1e:71:81:d0:fe:a0:60:71:a4:8a:40:93:
```

```
92:e3:b3:b6:cf:56:f1:30:b2:fc:53:31:bd:9d:6f:
```

```
3c:33:1e:4a:a5:6f:83:c7:45:26:8d:c6:9c:84:85:
```

```
b5:8f:b9:e3:86:86:59:ad:9b:58:63:a1:3d:7b:81:
```

```
d7:43:14:3d:98:4a:a2:cb:82:2c:fa:ca:91:32:b1:
```

```
e0:09:de:fa:a8:d6:fc:ea:8e:7e:36:8f:fb:86:31:
```

```
1e:bc:5e:01:71:6b:b4:23:86:7b:05:c1:63:7a:f5:
```

```
bc:a7:9b:a1:da:ff:4f:26:2d:33:44:06:72:f1:7b:
```

```
84:d5:a8:49:1d:be:b4:0e:9c:94:85:34:7b:e5:bb:
```

```
8a:49
```

```
Exponent: 65537 (0x10001)
```

```
Attributes:
```

```
Requested Extensions:
```

```
X509v3 Subject Alternative Name:
```

```
IP Address:1.1.1.1, DNS:fwa.tdh.com, email:fwa@tdh.com
```

```
Signature Algorithm: md5WithRSAEncryption
```

```
4b:a6:fc:91:2a:77:e3:30:02:bb:e4:0f:1a:bf:d2:a1:ad:81:
```

```

3e:44:51:81:b1:26:2d:2e:83:7c:0c:29:70:3c:6a:8a:7a:1a:
27:c8:a4:8d:3b:8f:dc:a7:d7:df:10:be:4c:96:1f:f5:db:96:
4d:e9:28:82:b9:2d:9b:e6:6d:22:52:ca:50:07:c2:7a:2b:17:
c7:49:7a:a6:a5:7c:cc:82:02:15:14:ca:9c:69:39:eb:fb:44:
3a:c9:75:d9:f5:b6:bf:b1:45:e4:e7:f4:db:df:eb:3d:6b:74:
ac:14:e9:51:af:b1:c8:d6:c1:19:48:bc:27:c1:37:59:41:38:
9c:1f:9a:7e:c7:fe:20:c9:e8:1d:94:55:ff:85:3e:8c:5a:f5:
f3:ff:9b:18:36:b1:25:2b:4d:60:2e:13:7b:be:91:c0:a1:1f:
6c:5c:1a:f6:3a:5b:e7:87:2b:43:7f:d8:f6:2b:c8:b1:df:7d:
c8:40:df:07:f9:52:4c:8b:ba:b0:10:f3:34:00:00:74:0b:ae:
c1:7a:9c:dd:de:26:26:28:30:de:e8:6c:dc:0a:c6:8f:27:27:
c6:0d:5e:8e:68:a8:8d:cc:eb:91:9c:59:3d:1e:f3:f3:58:72:
16:bf:cc:f5:df:71:bc:51:fb:98:83:c5:2b:17:73:d7:0a:6c:
f7:93:76:f4

```

4. CA 根据证书请求文件制作证书

证书请求文件生成后，可以将该文件通过磁盘、电子邮件等方式将该文件发送给 CA，由 CA 为网关 FWA 和 FWB 制作证书。除了网关 FWA 和 FWB 的证书之外，CA 还会提供自身的证书，即 CA 证书。CA 将制作好的 FWA 和 FWB 的证书同自身的证书一道通过磁盘、电子邮件等方式返回。

常用的 Windows Server 操作系统就可以作为 CA 来制作和颁发证书，具体的操作步骤在网上都可以搜索到，在此就不作详细介绍了。

5. 导入证书

经过 CA 的处理，最终我们获取到了网关 FWA 的证书 **fwa.cer**，网关 FWB 的证书 **fwb.cer**，以及 CA 本身的证书 **ca.cer**。下图展示了网关 FWA 的证书 **fwa.cer** 的内容：



将 **fwa.cer** 和 **ca.cer** 上传到 FWA 的存储设备中，将 **fwb.cer** 和 **ca.cer** 上传到 FWB 的存储设备中，然后还需要将证书分别导入到 FWA 和 FWB 上。

在 FWA 上导入 CA 证书和本地证书：

```
[FWA] pki import-certificate ca filename ca.cer
```

```
Info: Import file successfully.
```

```
[FWA] pki import-certificate local filename fwa.cer
```

```
Info: Import file successfully.
```

在 FWB 上导入 CA 证书和本地证书：

```
[FWB] pki import-certificate ca filename ca.cer
```

```
Info: Import file successfully.
```

```
[FWB] pki import-certificate local filename fwb.cer
```

```
Info: Import file successfully.
```

证书入主对等体，身份认证更便利

导入证书后，网关 FWA 和 FWB 都是有“身份”的设备了，在 IKE 对等体中引入证书，FWA 和 FWB 就可以通过证书来认证对方的身份。

前面提到过，使用证书进行身份认证时，可以根据证书中包含的实体信息来决定使用哪种 ID 类型。目前可以在 IKE 对等体中使用 DN (Distinguished Name)、FQDN、User-FQDN 和 IP 四种 ID 类型。这四种 ID 类型对应的证书中字段以及 FWA 和 FWB 上的取值如下表所示。

ID 类型	对应证书中的字段	FWA 的取值	FWB 的取值
DN	Subject	本端 ID: /CN=fwa 对端 ID: /CN=fwb	本端 ID: /CN=fwb 对端 ID: /CN=fwa
FQDN	DNS	本端 ID: fwa.tdh.com 对端 ID: fwb.tdh.com	本端 ID: fwb.tdh.com 对端 ID: fwa.tdh.com
User-FQDN	email	本端 ID: fwa@tdh.com 对端 ID: fwb@tdh.com	本端 ID: fwb@tdh.com 对端 ID: fwa@tdh.com
IP	IP Address	本端 ID: 1.1.1.1 对端 ID: 3.3.3.3	本端 ID: 3.3.3.3 对端 ID: 1.1.1.1

下面以 ID 类型是 DN 为例，介绍网关 FWA 和 FWB 的关键配置：

关键配置	FWA	FWB
IKE 安全提议	ike proposal 10 authentication-method rsa-sig //采用证书 认证方式	ike proposal 10 authentication-method rsa-sig //采用证书认证方 式
IKE 对等体	ike peer fwb certificate local-filename fwa.cer //FWA 的证书 ike-proposal 10 local-id-type dn //ID 类型为 DN remote-id /CN=fwb //FWB 的 DN remote-address 3.3.3.3 //FWB 的 IP 地址	ike peer fwa certificate local-filename fwb.cer //FWB 的证书 ike-proposal 10 local-id-type dn //ID 类型为 DN remote-id /CN=fwb //FWA 的 DN remote-address 1.1.1.1 //FWA 的 IP 地址
证书属性访问 控制策略	pki certificate access-control-policy default permit	pki certificate access-control-policy default permit

采用证书方式的 IKE 协商过程与采用预共享密钥方式的 IKE 协商过程大致相同，不同之处在于采用证书方式时，两端交互身份信息的 ISAKMP 消息（主模式是第 5、6 条 ISAKMP 消息；野蛮模式是第 1、2 条 ISAKMP 消息）中多了证书载荷和签名载荷，具体协商过程不再赘述。至此，天地会找到了可以代替预共享密钥方式的身份信息认证方案。当新的分舵与总舵建立 IPsec 连接时，只需要在同一个 CA 为该分舵申请证书，然后分舵和总舵就可以通过证书来进行身份认证，从而无需再为每个分舵和总舵之间的对等体都维护一个预共享密钥，减少了天地会的管理成本。

说明：数字证书除了在 IPsec 中使用之外，还可以应用于 SSL VPN 中客户端和服务器的身份认证，详细内容将会在 SSL VPN 篇中介绍。

除了新发展的分舵，天地会还有一些老的分舵，已经通过 GRE 或 L2TP 方式接入总舵。如何在不改变原有接入方式的基础上使这些分舵和总舵之间可以安全通信呢？下一篇贴子中将介绍 GRE、L2TP 以及移动接入场景中 IPsec 的相关内容，敬请期待。

🍁 IPsec 兼容 GRE 和 L2TP

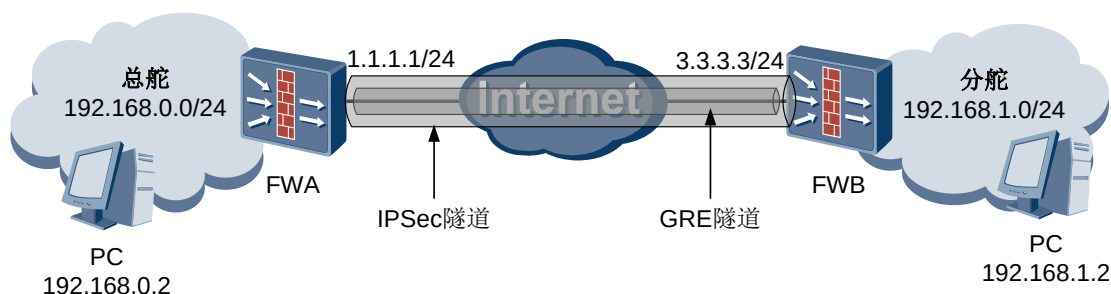
对于新发展的分舵，天地会使用 IPsec 来保护分舵与总舵之间的通信。而对于一些很早就存在的分舵，它们已经通过 GRE 或 L2TP 方式接入总舵，如何在不改变原有接入方式的基础上使这些分舵也可以使用 IPsec 与总舵安全通信呢？

好在 IPsec 兼容并济胸襟宽广，GRE 和 L2TP 皆可纳入到 IPsec 中。IPsec 将 GRE 隧道和 L2TP 隧道视为受保护对象，即报文先进行 GRE 或 L2TP 封装，然后再进行 IPsec 封装。这样分舵和总舵之间的通信非但不用改动原有的接入方式，还可以受到 IPsec 的保护。这种方式相当于是两种不同类型的隧道叠加，也叫做 GRE over IPsec 和 L2TP over IPsec。

下面先来看一下在分舵通过 GRE 接入总舵的场景中，如何使用 IPsec 来保护 GRE 隧道。

分舵通过 GRE over IPsec 接入总舵

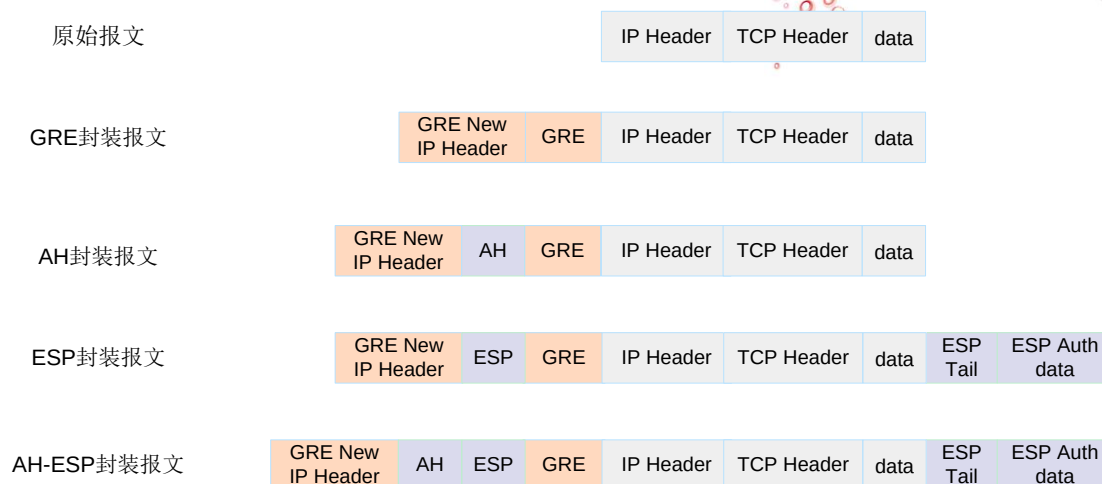
如下图所示，总舵网关 FWA 和分舵网关 FWB 已经建立了 GRE 隧道，现需要在 GRE 隧道之外再封装 IPsec 隧道，对总舵和分舵的通信进行加密保护。



前面我们介绍过，IPsec 有两种封装模式：传输模式和隧道模式。IPsec 对 GRE 隧道进行封装时，这两种模式的封装效果也不尽相同。

🍁 传输模式

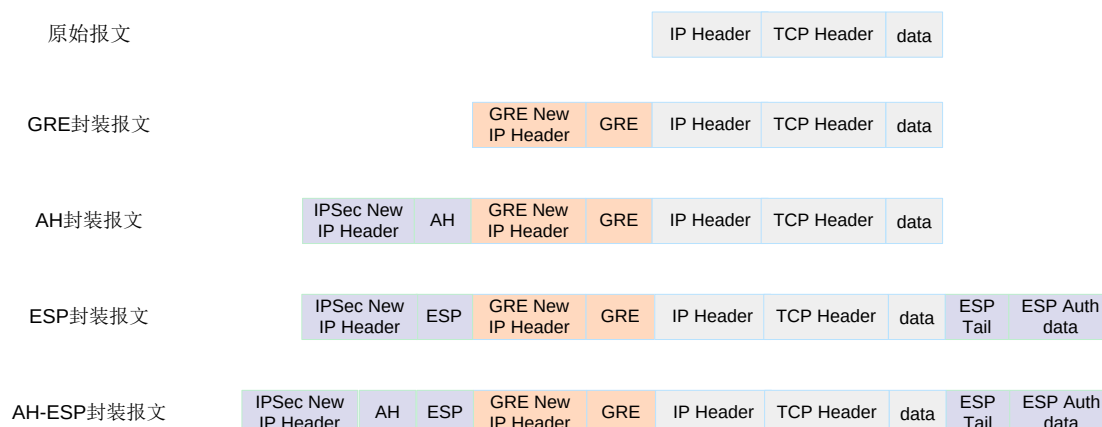
在传输模式中，AH 头或 ESP 头被插入到新的 IP 头与 GRE 头之间：



传输模式不改变 GRE 封装后的报文头，IPSec 隧道的源和目的地址就是 GRE 封装后的源和目的地址。

隧道模式

在隧道模式中，AH 头或 ESP 头被插到新的 IP 头之前，另外再生成一个新的报文头放到 AH 头或 ESP 头之前：



隧道模式使用新的 IPSec 报文头来封装经过 GRE 封装后的消息，封装后的消息共有三个报文头：原始报文头、GRE 报文头和 IPSec 报文头，Internet 上的设备根据最外层的 IPSec 报文头来转发该消息。

封装 GRE 报文头时，源和目的地址可以与 IPSec 报文头中的源和目的地址相同，即使用公网地址来封装；也可以使用私网地址封装 GRE 报文头，例如，创建 Loopback 接口并配置私网地址，然后在 GRE 中借用 Loopback 接口的地址来封装。

在 GRE over IPSec 中，无论 IPSec 采用传输模式还是隧道模式，都可以保护两个网络之间通信的消息。这是因为 GRE 已经进行了一次封装，原始报文就可以是两个网络之间的报文。

注意：隧道模式与传输模式相比多增加了新的 IPSec 报文头，导致报文长度更长，更容易导

致分片。如果网络环境要求报文不能分片，推荐使用传输模式。

下面以隧道模式下 ESP 封装为例，来对总舵和分舵之间的 GRE 隧道进行加密保护。首先给出总舵网关 FWA 和分舵网关 FWB 的关键配置。

关键配置	总舵 FWA	分舵 FWB
GRE 配置	<pre>interface Tunnel1 ip address 10.1.1.1 255.255.255.0 tunnel-protocol gre source 1.1.1.1 //使用公网地址封装 destination 3.3.3.3 //使用公网地址封装 如果使用私网地址进行封装，进行如下配置： interface LoopBack1 ip address 172.16.0.1 255.255.255.0 interface Tunnel1 ip address 10.1.1.1 255.255.255.0 tunnel-protocol gre source LoopBack 1 destination 172.16.0.2 //FWB 的 Loopback1 接口地址</pre>	<pre>interface Tunnel1 ip address 10.1.1.2 255.255.255.0 tunnel-protocol gre source 3.3.3.3 //使用公网地址封装 destination 1.1.1.1 //使用公网地址封装 如果使用私网地址进行封装，进行如下配置： interface LoopBack1 ip address 172.16.0.2 255.255.255.0 interface Tunnel1 ip address 10.1.1.2 255.255.255.0 tunnel-protocol gre source LoopBack 1 destination 172.16.0.1 //FWA 的 Loopback1 接口地址</pre>
路由配置	<pre>ip route-static 0.0.0.0 0.0.0.0 1.1.1.2 //假设下 一跳为 1.1.1.2 ip route-static 192.168.1.0 255.255.255.0 Tunnel1</pre>	<pre>ip route-static 0.0.0.0 0.0.0.0 3.3.3.1 //假设下 一跳为 3.3.3.1 ip route-static 192.168.0.0 255.255.255.0 Tunnel1</pre>
ACL	<pre>acl number 3000. rule 5 permit ip source 1.1.1.1 0 destination 3.3.3.3 0 //定义 GRE 封装后的源地址和目的地址 如果使用私网地址进行封装，此处的源地址应为 172.16.0.1，目的地址应为 172.16.0.2</pre>	<pre>acl number 3000 rule 5 permit ip source 3.3.3.3 0 destination 1.1.1.1 0 //定义 GRE 封装后的源地址和目的地址 如果使用私网地址进行封装，此处的源地址应为 172.16.0.2，目的地址应为 172.16.0.1</pre>
IKE 安全提议	<pre>ike proposal 1 //使用默认参数</pre>	<pre>ike proposal 1 //使用默认参数</pre>
IKE 对等体	<pre>ike peer fwb pre-shared-key tiandihui ike-proposal 1 remote-address 3.3.3.3</pre>	<pre>ike peer fwa pre-shared-key tiandihui ike-proposal 1 remote-address 1.1.1.1</pre>
IPSec 安全提议	<pre>ipsec proposal 1 transform esp encapsulation-mode tunnel esp authentication-algorithm sha1 esp encryption-algorithm aes</pre>	<pre>ipsec proposal 1 transform esp encapsulation-mode tunnel esp authentication-algorithm sha1 esp encryption-algorithm aes</pre>
IPSec 安全策略	<pre>ipsec policy policy1 1 isakmp security acl 3000 ike-peer fwb proposal 1</pre>	<pre>ipsec policy policy1 1 isakmp security acl 3000 ike-peer fwa proposal 1</pre>
应用 IPSec 安全	<pre>interface GigabitEthernet0/0/1</pre>	<pre>interface GigabitEthernet0/0/1</pre>

关键配置	总舵 FWA	分舵 FWB
策略	ip address 1.1.1.1 255.255.255.0 ipsec policy policy1	ip address 3.3.3.3 255.255.255.0 ipsec policy policy1

从上面的表格可以看出，配置 GRE over IPsec 时，与单独配置 GRE 和 IPsec 没有太大的区别。唯一需要注意的地方是，通过 ACL 定义需要保护的数据流时，不能再以总舵和分舵内部私网地址为匹配条件，而是必须匹配经过 GRE 封装后的报文，即定义报文的源地址为 GRE 隧道的源地址，目的地址为 GRE 隧道的目的地址。

配置完成后，在分舵中的 PC 可以 ping 通总舵的 PC，在 Internet 上抓包只能看到加密后的信息，无法获取到真实的 ping 消息：

No.	Time	Source	Destination	Protocol	Info
1	0.000000	3.3.3.3	1.1.1.1	ESP	ESP (SPI=0x6f2124bb)
2	0.032000	1.1.1.1	3.3.3.3	ESP	ESP (SPI=0xbcc06764)

Frame 1: 150 bytes on wire (1200 bits), 150 bytes captured (1200 bits)	
Ethernet II, Src: 00:00:00:d7:95:00 (00:00:00:d7:95:00), Dst: HuaweiTe_f0:1d:28 (54:89:98:f0:1d:28)	
Internet Protocol, Src: 3.3.3.3 (3.3.3.3), Dst: 1.1.1.1 (1.1.1.1)	
Encapsulating Security Payload	
ESP SPI: 0x6f2124bb	
ESP Sequence: 10	

0000	54 89 98 f0 1d 28 00 00	00 d7 95 00 08 00 45 00	T....(.....E.
0010	00 88 00 09 00 00 ff 32	b3 33 03 03 03 03 01 01	2.3.....
0020	01 01 6f 21 24 bb 00 00	00 0a e0 16 8e 05 4b 57	..O!\$....R-.I..KW
0030	f7 b4 d9 8b c1 dc ab 66	22 52 2d 13 49 8e 2c f8	f...vWN.
0040	46 41 df 97 96 5b 5a 63	2e d8 99 b6 76 57 4e 87	FA...[Zc...vWN.
0050	e0 15 ca 8e 4a bc 15 80	d5 e4 8f 07 a5 3b 1e a8	J.....
0060	e0 f8 06 99 f0 6b bf be	64 f6 c0 92 79 1e e8 8a	k...d...y..
0070	a5 bf 59 a8 08 30 f8 34	e4 3e 10 88 16 6c 36 69	..Y..0.4...>...16i
0080	2d 0b 0c 7d 38 d4 20 26	e7 16 cd 65 d1 9b 25 81	...}8. & ...e..%.
0090	ff 5d c8 8f 61 b7		].a.

在总舵的网关 FWA 上可以查看到如下会话信息，会话信息中包括了原始的 ICMP 报文、第一层封装即 GRE 封装、第二层封装即 IPsec 封装，其中 GRE 封装和 IPsec 封装使用了相同的源和目的地址。

[FWA] display firewall session table

Current Total Sessions : 4

```

udp  VPN:public --> public 3.3.3.3:500-->1.1.1.1:500
esp  VPN:public --> public 3.3.3.3:0-->1.1.1.1:0
gre  VPN:public --> public 3.3.3.3:0-->1.1.1.1:0
icmp VPN:public --> public 192.168.1.2:2862-->192.168.0.2:2048

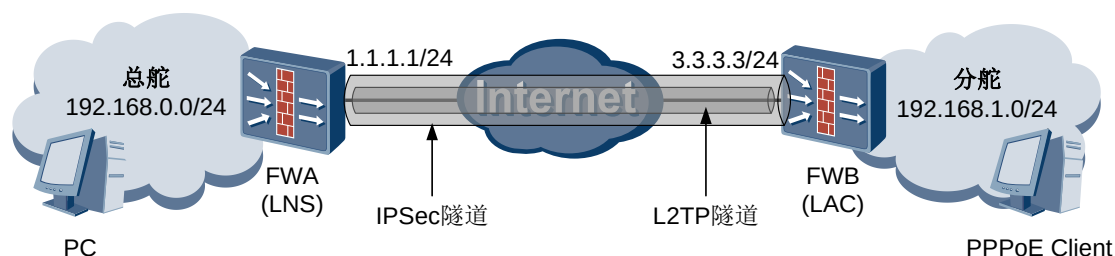
```

介绍完 GRE over IPsec 后，下面来看一下使用 IPsec 来保护 L2TP 隧道的情况。

分舵通过 L2TP over IPsec 接入总舵

在之前的 L2TP 系列贴子中我们介绍了 L2TP 有三种类型，分别是 Client-Initiated VPN、NAS-Initiated VPN 和 LAC-Auto-Initiated VPN。其中 Client-Initiated VPN 属于单独的移动用

户远程接入,我们将在下文中的远程接入部分介绍。NAS-Initiated VPN 和 LAC-Auto-Initiated VPN 都可以实现两个网络之间的通信,在这里我们重点以 NAS-Initiated VPN 为例来介绍。如下图所示,总舵网关 FWA 和分舵网关 FWB 已经建立了 NAS-Initiated 方式的 L2TP 隧道,现需要在 L2TP 隧道之外再封装 IPSec 隧道,对总舵和分舵的通信进行加密保护。



IPSec 对 L2TP 隧道进行封装时,传输模式和隧道模式的封装效果如下:

🍁 传输模式

在传输模式中,AH 头或 ESP 头被插入到新的 IP 头与 UDP 头之间:

原始报文

IP Header	TCP Header	data
-----------	------------	------

L2TP封装报文

L2TP New IP Header	UDP	L2TP	PPP	IP Header	TCP Header	data
--------------------	-----	------	-----	-----------	------------	------

AH封装报文

L2TP New IP Header	AH	UDP	L2TP	PPP	IP Header	TCP Header	data
--------------------	----	-----	------	-----	-----------	------------	------

ESP封装报文

L2TP New IP Header	ESP	UDP	L2TP	PPP	IP Header	TCP Header	data	ESP Tail	ESP Auth data
--------------------	-----	-----	------	-----	-----------	------------	------	----------	---------------

AH-ESP封装报文

L2TP New IP Header	AH	ESP	UDP	L2TP	PPP	IP Header	TCP Header	data	ESP Tail	ESP Auth data
--------------------	----	-----	-----	------	-----	-----------	------------	------	----------	---------------

传输模式不改变 L2TP 封装后的报文头,IPSec 隧道的源和目的地址就是 L2TP 封装后的源和目的地址。

🍁 隧道模式

在隧道模式中,AH 头或 ESP 头被插到新的 IP 头之前,另外再生成一个新的报文头放到 AH 头或 ESP 头之前:

原始报文

IP Header	TCP Header	data
-----------	------------	------

L2TP封装报文

L2TP New IP Header	UDP	L2TP	PPP	IP Header	TCP Header	data
--------------------	-----	------	-----	-----------	------------	------

AH封装报文

IPSec New IP Header	AH	L2TP New IP Header	UDP	L2TP	PPP	IP Header	TCP Header	data
---------------------	----	--------------------	-----	------	-----	-----------	------------	------

ESP封装报文

IPSec New IP Header	ESP	L2TP New IP Header	UDP	L2TP	PPP	IP Header	TCP Header	data	ESP Tail	ESP Auth Data
---------------------	-----	--------------------	-----	------	-----	-----------	------------	------	----------	---------------

ESP封装报文

IPSec New IP Header	AH	ESP	L2TP New IP Header	UDP	L2TP	PPP	IP Header	TCP Header	data	ESP Tail	ESP Auth Data
---------------------	----	-----	--------------------	-----	------	-----	-----------	------------	------	----------	---------------

隧道模式使用新的 IPSec 报文头来封装经过 L2TP 封装后的消息，封装后的消息共有三个报文头：原始报文头、L2TP 报文头和 IPSec 报文头，Internet 上的设备根据最外层的 IPSec 报文头来转发该消息。

在 L2TP over IPSec 中，由于 L2TP 已经进行了一次封装，原始报文就是两个网络之间的报文，所以无论 IPSec 采用传输模式还是隧道模式，都可以保护两个网络之间通信的消息。

注意：隧道模式与传输模式相比多增加了新的 IPSec 报文头，导致报文长度更长，更容易导致分片。如果网络环境要求报文不能分片，推荐使用传输模式。

下面以隧道模式下 ESP 封装为例，来对总舵和分舵之间的 L2TP 隧道进行加密保护。首先给出总舵网关 FWA 和分舵网关 FWB 的关键配置。

关键配置	总舵 FWA (LNS)	分舵 FWB (LAC)
L2TP 配置	<pre>l2tp enable interface Virtual-Template1 ppp authentication-mode chap ip address 10.1.1.1 255.255.255.0 remote address pool 0 l2tp-group 1 tunnel authentication tunnel password cipher Tiandihui123 allow l2tp virtual-template 1 remote lac tunnel name lns aaa local-user l2tpuser password cipher Password1 local-user l2tpuser service-type ppp ip pool 0 192.168.1.2 192.168.1.10</pre>	<pre>l2tp enable l2tp-group 1 tunnel authentication tunnel password cipher Tiandihui123 start l2tp ip 1.1.1.1 fullusername l2tpuser tunnel name lac</pre>
路由配置	<pre>ip route-static 0.0.0.0 0.0.0.0 1.1.1.2 //假设下一跳为 1.1.1.2</pre>	<pre>ip route-static 0.0.0.0 0.0.0.0 3.3.3.1 //假设下一跳为 3.3.3.1</pre>
ACL	<pre>acl number 3000 rule 5 permit ip source 1.1.1.1 0 destination</pre>	<pre>acl number 3000 rule 5 permit ip source 3.3.3.3 0 destination</pre>

关键配置	总舵 FWA (LNS)	分舵 FWB (LAC)
	3.3.3.3 0 //定义 L2TP 封装后的源地址和目的地地址	1.1.1.1 0 //定义 L2TP 封装后的源地址和目的地地址
IKE 安全提议	ike proposal 1 //使用默认参数	ike proposal 1 //使用默认参数
IKE 对等体	ike peer fwb pre-shared-key tiandihui ike-proposal 1 remote-address 3.3.3.3	ike peer fwa pre-shared-key tiandihui ike-proposal 1 remote-address 1.1.1.1
IPSec 安全提议	ipsec proposal 1 transform esp encapsulation-mode tunnel esp authentication-algorithm sha1 esp encryption-algorithm aes	ipsec proposal 1 transform esp encapsulation-mode tunnel esp authentication-algorithm sha1 esp encryption-algorithm aes
IPSec 安全策略	ipsec policy policy1 1 isakmp security acl 3000 ike-peer fwb proposal 1	ipsec policy policy1 1 isakmp security acl 3000 ike-peer fwa proposal 1
应用 IPSec 安全策略	interface GigabitEthernet0/0/1 ip address 1.1.1.1 255.255.255.0 ipsec policy policy1	interface GigabitEthernet0/0/1 ip address 3.3.3.3 255.255.255.0 ipsec policy policy1

同 GRE over IPSec 类似，在 L2TP over IPSec 中定义 ACL 时，不能再以总舵和分舵内部私网地址为匹配条件，而是必须匹配经过 L2TP 封装后的报文，即定义报文的源地址为 L2TP 隧道的源地址，目的地址为 L2TP 隧道的目的地址。

配置完成后，分舵中的 PPPoE Client 发起拨号访问，分舵网关 FWB 和总舵网关 FWA 先进行 IPSec 协商，建立 IPSec 隧道，然后在 IPSec 隧道的保护下进行 L2TP 协商，建立 L2TP 隧道。同样，在 Internet 上抓包也只能看到加密后的信息，无法获取到 L2TP 隧道中传输的消息。

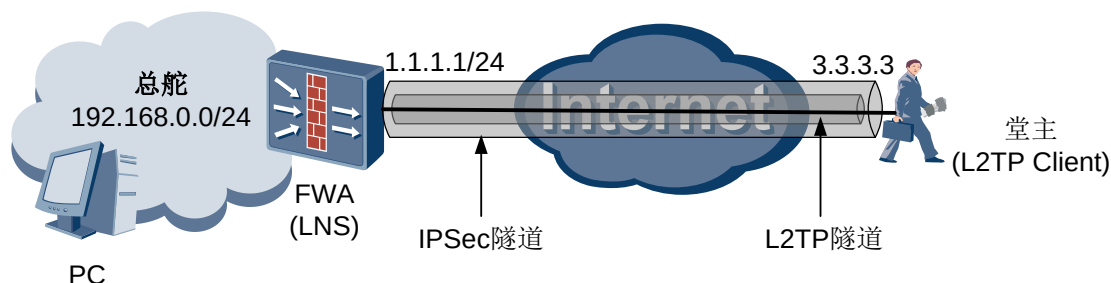
通过部署 IPSec 对 GRE 和 L2TP 隧道进行加密保护，天地会这些老旧的分舵又焕发了青春。接下来天地会又面临新的问题：总舵的堂主经常会出差到各个分舵指导工作，而总舵中有一些紧急帮务需要他们及时处理。如果堂主恰巧在分舵中，就可以通过分舵接入总舵；但是如果堂主正在路途中，就只能通过 Client-Initiated VPN 方式的 L2TP 接入总舵，但是此时通信的安全性无法保证。而借助于 IPSec，就可以保证移动用户远程接入场景中的通信安全。

移动用户使用 L2TP over IPSec 远程接入总舵

在 L2TP 中，Client-Initiated VPN 方式专门适用于移动用户远程接入的场景。在此基础上，

使用 IPSec 来对 L2TP 隧道进行加密保护，这也是一种 L2TP over IPSec 的应用。

如下图所示，总舵网关 FWA 和分舵网关 FWB 已经建立了 Client-Initiated VPN 方式的 L2TP 隧道，现需要在 L2TP 隧道之外再封装 IPSec 隧道，对总舵和分舵的通信进行加密保护。



堂主可以使用 Windows 系统自带的客户端来拨号，也可以使用第三方的拨号软件（如华为 VPN Client 软件）来拨号。如果使用 Windows 系统自带的客户端来拨号，因为其只支持传输模式，所以在总舵网关 FWA 上也只能配置传输模式的 IPSec。IPSec 对 Client-Initiated VPN 方式的 L2TP 隧道的封装效果与上文介绍过的 NAS-Initiated VPN 方式相同，此处不再赘述。下面以堂主使用 Windows 7 系统自带的客户端拨号接入总舵为例，给出总舵网关 FWA 和 L2TP Client 的关键配置。

设备	配置步骤	配置命令
FWA (LNS)	L2TP	<pre> l2tp enable interface Virtual-Template1 ppp authentication-mode chap ip address 10.1.1.1 255.255.255.0 remote address pool 0 l2tp-group 1 //使用 L2TP 组 1 undo tunnel authentication //关闭隧道验证 allow l2tp virtual-template 1 //在 L2TP 组 1 中无需设置隧道对端名称 aaa local-user l2tpuser password cipher Password1 local-user l2tpuser service-type ppp ip pool 0 192.168.1.2 192.168.1.10 </pre>
	ACL	<pre> acl number 3000 rule 5 permit udp source-port eq 1701 //定义 L2TP 封装后的源端口 </pre>
	IKE 安全提议	<pre> ike proposal 1 encryption-algorithm 3des-cbc dh group2 </pre>
	IKE 对等体	<pre> ike peer client pre-shared-key tiandihui ike-proposal 1 </pre>
	IPSec 安全提议	<pre> ipsec proposal 1 transform esp </pre>

设备	配置步骤	配置命令
		encapsulation-mode transport //使用传输模式 esp authentication-algorithm sha1 esp encryption-algorithm 3des
	IPSec 安全策略	ipsec policy-template tem1 1 security acl 3000 ike-peer client proposal 1 ipsec policy policy1 1 isakmp template tem1
	应用 IPSec 安全策略	interface GigabitEthernet0/0/1 ip address 1.1.1.1 255.255.255.0 ipsec policy policy1
L2TP Client	IPsec Policy Agent 状态: 已启动 用户名: l2tpuser 密码: Password1 ----- 目的地的主机名或 IP 地址: 1.1.1.1 VPN 类型: 使用 IPsec 的第 2 层隧道协议(L2TP/IPSec) 身份验证: CHAP ----- IKE 加密算法: 3DES IKE 完整性算法: SHA1 DH 组: 中(2) ESP 加密算法: 3DES ESP 完整性算法: SHA1 预共享密钥: tiandihui 注意: 此处只给出了 Windows 7 系统自带的客户端上默认的 IPsec 配置, 如需调整这些 IPsec 参数, 请在 Windows 7 系统的“控制面板—>系统和安全—>管理工具—>本地安全策略”中设置 IP 安全策略。	

因为堂主都是在 Internet 上动态接入, 公网 IP 地址不确定, 所以在总舵网关 FWA 上定义 ACL 时, 以源端口 1701 来匹配经过 L2TP 封装后的报文。此外, 由于 Windows 系统自带的客户端不支持隧道验证, 所以还需要在总舵网关 FWA 上关闭 L2TP 的隧道验证功能。

配置完成后, 堂主就可以使用 Windows 7 系统自带的客户端随时随地接入总舵, 在 IPsec 隧道的保护下处理紧急事务。而在 Internet 上抓包也只能看到加密后的信息, 无法获取到 L2TP 隧道中传输的消息。

虽然 IPsec 可以对 Client-Initiated VPN 方式的 L2TP 隧道加密, 但是 L2TP 和 IPsec 一块使用, 配置起来较为繁琐。其实对于远程接入的场景, 还可以使用一种轻量级的 VPN 即 SSL VPN 来实现。部署 SSL VPN 后, 堂主直接使用浏览器就可以接入到总舵, 操作简单效果直观。

关于 SSL VPN 的内容，我们将在 IPSec 系列技术贴完成后推出。

至此，IPSec 的主要应用场景都介绍完毕。借助 IPSec，天地会解决了一个又一个的问题，终于搭建起来了涵盖分舵接入、移动用户远程接入的加密通信网络。网络虽然搭建起来，但是还面临稳定运行的问题，下一篇我们就来介绍 IKE 对等体检测、主备链路、隧道化链路等提高可靠性的内容，这也将是 IPSec 系列的最后一篇贴子，敬请期待。

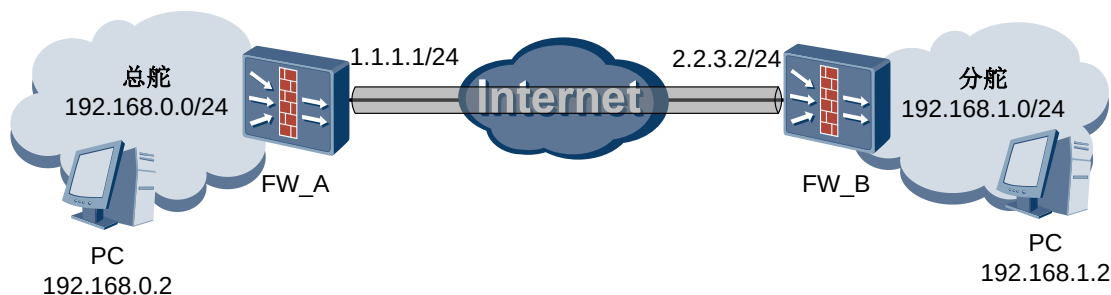
对等体检测和双链路备份

天地会应用了 IPsec 后，通信安全得到保障。当网络上突发问题时，比如路由问题、对端设备重启等，会导致 IPsec 通信中断或时断时续，那么如何才能快速的检测到故障并迅速恢复 IPsec 通信呢？这就需要用到 IKE 对等体检测机制和 IPsec 可靠性了。

对等体检测洞察先机

当两个对等体之间采用 IKE 和 IPsec 进行通信时，对等体之间可能会由于路由问题、IPsec 对等体设备重启或其他原因导致连接断开。由于 IKE 协议本身没有提供对等体状态检测机制，一旦发生对等体不可达的情况，只能等待安全联盟老化。只有快速检测到对等体的状态变化，才可以尽快恢复 IPsec 通信。

现在强叔给大家展示一下由于对端设备重启导致 IPsec 中断的情况。



网关 FW_A 跟网关 FW_B 之间采用 IPsec 进行通信。重启网关 FW_B 后，查看两端 IPsec 隧道情况。

1. 发现网关 FW_A 上的 IPsec 隧道还在。

```
<FW_A> display ike sa
current ike sa number: 2
```

conn-id	peer	flag	phase	vpn
40015	2.2.3.2	RD	v1:2	public
40014	2.2.3.2	RD	v1:1	public

2. 发现网关 FW_B 重启后其上的 IPsec 隧道不在了。

```
<FW_B> display ike sa
current sa Num :0
```

这就会导致如下的问题：

- 总舵的PC1先访问分舵的PC2，无法通信。

原因就在于 FW_A 上的安全联盟还存在，但 FW_B 上的安全联盟已经没有了。从 PC1 访问 PC2 会采用原有的安全联盟，而无法触发 FW_A 与 FW_B 之间建立新的安全联盟。从而导致访问失败。

- 分舵的PC2先访问总舵的PC1，可以正常通信。

因为 FW_B 上不存在安全联盟，因此可以触发建立安全联盟。安全联盟建立后，双方可以正常通信。

针对这种问题的解决办法就是需要开启 IPSec 检测机制。开启 IPSec 检测机制后，当一端设备故障后，另一端设备的安全联盟也同时删除。

USG 系列防火墙支持两种 IPSec 检测机制：

🔥 Keepalive 机制

Keepalive 机制通过 IKE 对等体周期性发送 Hello 消息来告诉对方自己处于活动状态。若在超时时间内没有收到 Hello 报文，则认为对端不可达。此时将删除该 IKE SA 及由它协商的 IPSec SA。

Keepalive 机制默认关闭。配置相应参数后，keepalive 机制生效。

🔥 DPD 机制

DPD (Dead Peer Detection) 机制不用周期性发送 Hello 消息，而是通过使用 IPSec 流量来最小化对等体状态检测所需消息报文的数量。若可以收到对端发来的流量，则认为对方处于活动状态；只有当一定时间间隔内没有收到对端发来的流量时，才会发送 DPD 报文探测对端的状态。若发送几次 DPD 报文后一直没有收到对端的回应，则认为对端不可达，此时将删除该 IKE SA 及由它协商的 IPSec SA。

- IKEv1时配置DPD机制

IKEv1 默认不支持 DPD。需要在隧道两端同时配置。两端配置的 DPD 参数彼此独立，不需要匹配。

- IKEv2时配置DPD机制

IKEv2 默认支持 DPD。只要在隧道一端配置 DPD，另一端就可以响应 DPD 报文。

实际应用中 keepalive 很少使用。主要原因是 Keepalive 存在如下两个方面的缺陷：

- Keepalive机制周期性发送Hello报文将消耗大量的CPU资源，会限制可建立的IPSec会话的数量。

- Keepalive没有统一标准，各厂商的设备可能无法对接。

说明：对等体检测机制虽然不是必配项，但对IPSec故障恢复有益，推荐大家配置IPSec时一定要配置其中一种检测机制。

下面以配置DPD机制为例，说明FW_B发生故障后FW_A上的变化。

配置	总舵	分舵
配置DPD	ike dpd interval 10 2	ike dpd interval 10 2

说明：采用IKEv2是只要一端配置就可以了。

配置完成后，设备检测到链路状态的变化。此时查看IKE SA和IPSec SA的情况，发现FW_A上的安全联盟已经删除了。此时可以正常建立IPSec隧道了。

```
[FW_A] display ike sa
```

```
current sa Num :0
```

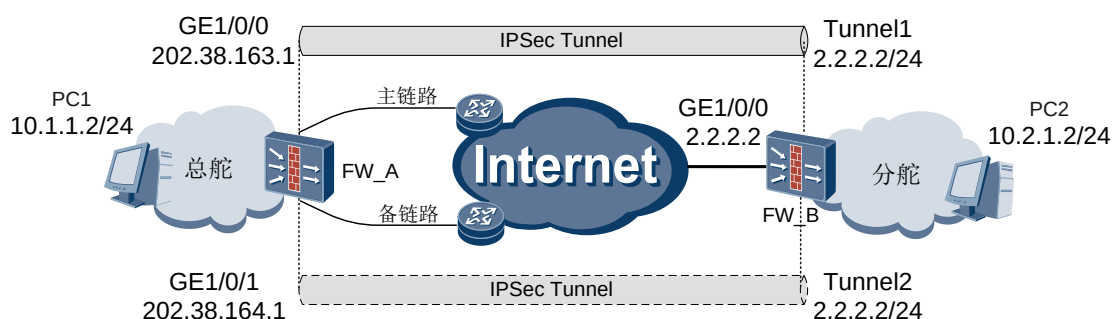
双链路备份有备无患

IPSec可靠性可以分为设备可靠性和链路可靠性。设备可靠性主要是双机热备，由于双机热备非常复杂，不是一句两句可以说清的，强叔将在后面专门讲解。本次讲解的重点是链路可靠性。

由于天地会的发展壮大，总舵要与所有分舵进行通信，网络带宽倍感压力。一旦网络出问题，就完全不能通信了。为避免此等悲剧，陈总舵主决定为总舵再购买一条链路，既可缓解带宽压力，又可以双链路备份。针对双链路的场景可以采用如下两种方案进行配置：

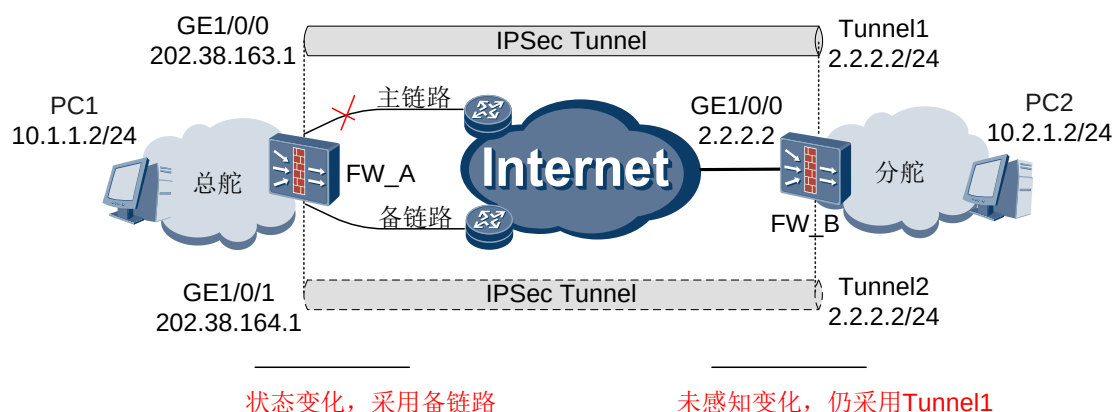
方案一：IPSec主备链路备份

总舵采用双链路与分舵通信，采用IPSec主备链路备份方式的典型组网如下图所示。

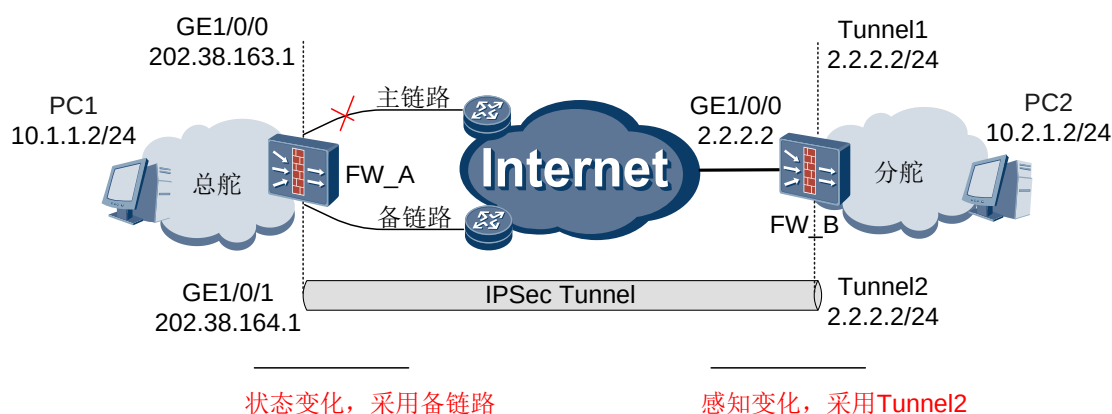


FW_B上配置两个Tunnel接口(借用GE1/0/0的IP地址)分别于FW_A的主链路接口GE1/0/0和备链路接口GE1/0/1进行IPSec对接。其配置的关键在于必须配置IP-Link,并采用IP-Link检测主链路的状态。

- ❗ 不配置 IP-Link 时，当 FW_A 的主链路故障时，FW_B 无法感知。因此 FW_B 上的路由表不会变化，其出接口仍然是与主链路对接的 Tunnel1。这会导致 IPsec 隧道建立失败。



- ❗ 配置 IP-Link 时，FW_B 上可以感知 FW_A 上的主链路状态的变化。当 FW_A 的主链路故障时，FW_B 上的路由表也会同步发生改变。此时可以通过备链路建立 IPsec 隧道，恢复通信。



总舵和分舵网关的关键配置如下：

关键配置	总舵	分舵
IP-Link //用于监控主链路的情况	<pre>ip-link check enable ip-link 1 destination 2.2.2.2 interface GigabitEthernet1/0/0 mode icmp next-hop 202.38.163.2</pre>	<pre>ip-link check enable ip-link 1 destination 202.38.163.1 interface GigabitEthernet1/0/0 mode icmp next-hop 2.2.2.1</pre>
路由配置	<pre>ip route-static 10.2.1.0 24 202.38.163.2 preference 10 track ip-link 1 //到分支的路由，GE1/0/0出口的链路为主链路 ip route-static 10.2.1.0 24 202.38.164.2 preference 20 //到分支的路由，GE1/0/1出口的链路为备链路 ip route-static 0.0.0.0 0.0.0.0 202.38.163.2 preference 10 track ip-link 1 //缺省路由，GE1/0/0出</pre>	<pre>ip route-static 10.1.1.0 255.255.255.0 Tunnel 1 preference 10 track ip-link 1 //Tunnel1跟主链路对接 ip route-static 10.1.1.0 255.255.255.0 Tunnel 2 preference 20 //Tunnel2跟备链路对接 ip route-static 0.0.0.0 0.0.0.0 2.2.2.1 //配置缺省路由，下一跳为2.2.2.1</pre>

关键配置	总舵	分舵
	口的链路为主链路 <pre>ip route-static 0.0.0.0 0.0.0.0 202.38.164.2 preference 20</pre> //缺省路由, GE1/0/1 出口的链路为备链路	
定义被保护的数据流 // 定义两个 ACL, 配置相同的规则	<pre>acl 3000 rule 5 permit ip source 10.1.1.0 0.0.0.255 destination 10.2.1.0 0.0.0.255 acl 3001 rule 5 permit ip source 10.1.1.0 0.0.0.255 destination 10.2.1.0 0.0.0.255</pre>	<pre>acl 3000 rule 5 permit ip source 10.2.1.0 0.0.0.255 destination 10.1.1.0 0.0.0.255 acl 3001 rule 5 permit ip source 10.2.1.0 0.0.0.255 destination 10.1.1.0 0.0.0.255</pre>
配置 IPsec 安全提议	ipsec proposal pro1	ipsec proposal pro1
配置 IKE 安全提议	ike proposal 10	ike proposal 10
IKE 对等体	<pre>ike peer fenduo pre-shared-key tiandihui1 ike-proposal 10</pre>	<pre>ike peer a1 pre-shared-key tiandihui1 ike-proposal 10 remote-address 202.38.163.1</pre> //原有链路的出接口 IP 地址 <pre>ike peer a2 pre-shared-key tiandihui1 ike-proposal 10 remote-address 202.38.164.1</pre> //原有链路的出接口 IP 地址
IPsec 安全策略 //创建两条 IPsec 安全策略	<pre>ipsec policy-template tem1 1 security acl 3000 proposal pro1 ike-peer fenduo ipsec policy policy1 1 isakmp template tem1 ipsec policy-template tem2 1 security acl 3001 proposal pro1 ike-peer fenduo ipsec policy policy2 1 isakmp template tem2</pre>	<pre>ipsec policy policy1 1 isakmp security acl 3000 proposal pro1 ike-peer a1 ipsec policy policy1 1 isakmp security acl 3001 proposal pro1 ike-peer a2</pre>
应用 IPsec 安全策略 // 在两个接口上分别应用 IPsec 安全策略	<pre>interface GigabitEthernet1/0/0 ip address 202.38.163.1 24 ipsec policy policy1 interface GigabitEthernet1/0/1 ip address 202.38.164.1 24 ipsec policy policy2</pre>	<pre>interface Tunnel1 ip address unnumbered interface GigabitEthernet1/0/0</pre> //借用物理口 GE1/0/0 的 IP 地址 <pre>tunnel-protocol ipsec ipsec policy policy1 interface Tunnel2 ip address unnumbered interface GigabitEthernet1/0/0</pre> //借用物理口 GE1/0/0 的 IP 地址 <pre>tunnel-protocol ipsec</pre>

关键配置	总舵	分舵
		ipsec policy policy2

配置完成后，建立 IPsec VPN。从 PC2 ping PC1，可以 ping 通。查看 IKE SA 的建立情况如下：

```
<FW_B> display ike sa
```

```
current ike sa number: 2
```

```
-----
conn-id    peer                flag                phase vpn
-----
40003      202.38.163.1        RD|ST               v2:2  public
3          202.38.163.1        RD|ST               v2:1  public
```

断开主链路 (shutdown GE1/0/0 接口) 后，可以使用备链路建立 IPsec VPN。从分舵 PC ping 总舵 PC，可以 ping 通。查看 IKE SA 建立情况如下：

```
<FW_B> display ike sa
```

```
current ike sa number: 2
```

```
-----
conn-id    peer                flag                phase vpn
-----
40009      202.38.164.1        RD|ST               v2:2  public
9          202.38.164.1        RD|ST               v2:1  public
```

说明 IP-Link 检测到主链路的接口故障后，修改了 FW_A 上和 FW_B 上使用的路由。

主链路正常情况下 FW_B 上的路由表 (只给出静态路由)：

```
<FW_B> display ip routing-table
```

```
Route Flags: R - relay, D - download to fib
```

```
-----
Routing Tables: Public
```

```
Destinations : 10 Routes : 12
```

```
Destination/Mask    Proto Pre  Cost    Flags NextHop        Interface
0.0.0.0/0            Static 60   0      RD   2.2.2.1        GigabitEthernet1/0/0
10.1.1.0/24          Static 10   0      D    2.2.2.2        Tunnel1
```

主链路断开后 FW_B 上的路由表 (只给出静态路由)：

```
<FW_B> display ip routing-table
```

```
Route Flags: R - relay, D - download to fib
```

```
-----
Routing Tables: Public
```

```
Destinations : 10 Routes : 12
```

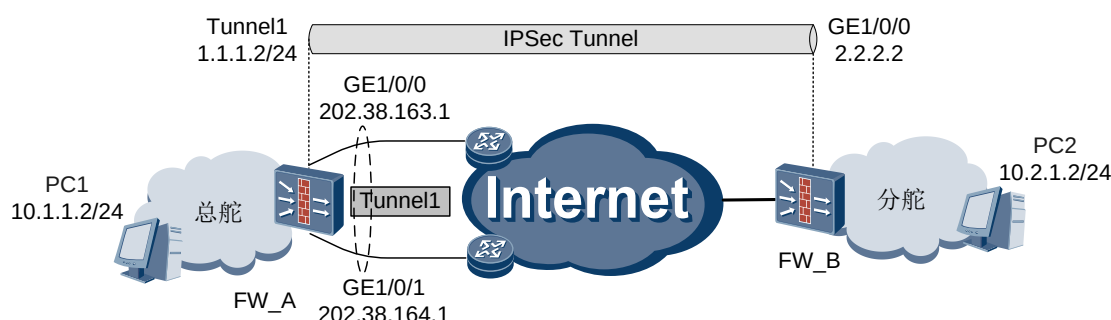
```
Destination/Mask    Proto Pre  Cost    Flags NextHop        Interface
0.0.0.0/0            Static 60   0      RD   2.2.2.1        GigabitEthernet1/0/0
10.1.1.0/24          Static 20   0      D    2.2.2.2        Tunnel2
```

方案二：IPsec 隧道化链路备份

IPSec 主备链路备份需要分别在两个物理接口上应用 IPSec 策略，配置复杂；且需要通过 IP-Link 跟踪路由状态，以便正确的进行 IPSec 隧道切换。那能否不直接在物理接口上应用 IPSec 策略呢，这样是否就可以避免隧道切换的问题呢？

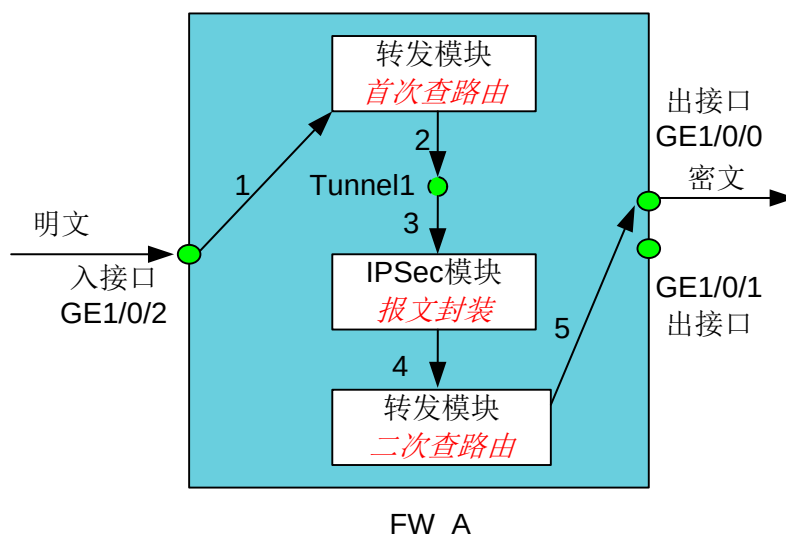
当然是可以的。方法就是将 IPSec 策略应用到一个虚拟的 Tunnel 接口上。由于策略不是应用到实际物理接口，那么 IPSec 并不关心有几条链路可以到达对端，也不关心哪条链路发生故障，只有可以查找到可以到达对端的路由，IPSec 通信就不会中断。

总舵采用双链路与分舵通信，采用 IPSec 隧道化链路备份方式组网如下图所示。



下面强叔就讲一下 IPSec 隧道化链路备份时报文是如何封装和解封装的。

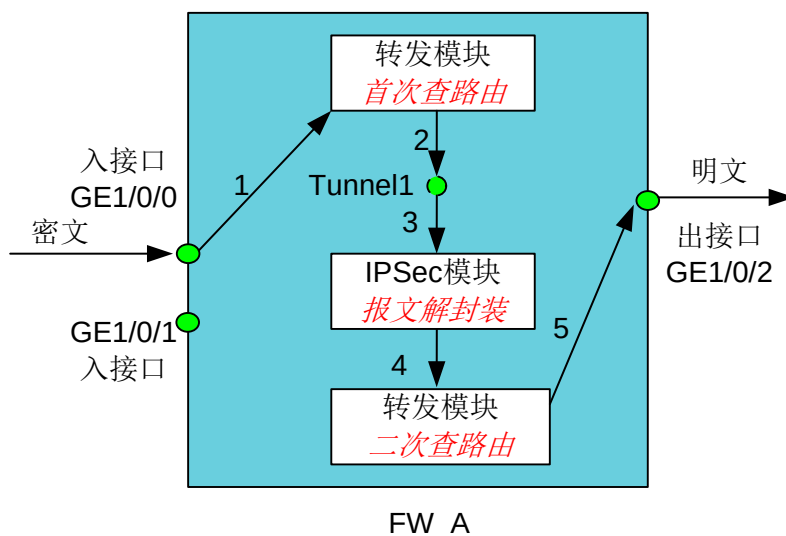
FW_A 上报文加封装的过程如下图所示：



1. FW_A 收到需要进行 IPSec 处理的报文后，将收到的 IP 明文送到转发模块进行处理。
2. 转发模块通过查找路由，发现路由出接口为 Tunnel 接口。转发模块依据路由查询结果将 IP 明文发送到 Tunnel 接口。
3. 由于 Tunnel 接口应用了 IPSec 策略，报文在 Tunnel 接口上进行 IPSec 加封装。封装后的报文的源 IP 和目的 IP 分别为两端隧道接口（应用 IPSec 策略的接口）的 IP 地址。

- 封装后的 IP 报文被送到转发模块进行处理。转发模块再次对密文查找路由。
- 转发模块通过查找路由，发现路由下一跳为物理接口的路由。根据路由的优先级等选择合适的路由，将 IP 密文从设备的某个实际物理接口转发出去。

FW_A 上报文解封封装的过程如下图所示：



- FW_A 收到加密后的 IPSec 报文后，将收到的 IP 密文送到转发模块进行处理。
- 转发模块识别到此 IP 密文的地址为本设备 Tunnel 接口的 IP 地址且 IP 协议号为 AH 或 ESP，将 IP 密文发送到 Tunnel 接口进行解封封装。
- IPSec 报文在 Tunnel 接口上进行解封封装。
- 解封封装后的 IP 明文被再次送到转发模块进行处理。转发模块再次对明文查找路由。
- 转发模块通过查找路由，将 IP 明文从隧道的实际物理接口转发出去。

总舵和分舵网关的关键配置如下：

关键配置	总舵	分舵
路由配置	<pre>ip route-static 10.2.1.0 24 tunnel1//到分舵的路由 ip route-static 2.2.2.2 32 202.38.163.2//到分舵网关的路由 ip route-static 2.2.2.2 32 202.38.164.2//到分舵网关的路由</pre>	<pre>ip route-static 10.1.1.0 255.255.255.0 2.2.2.1//到总舵的路由 ip route-static 1.1.1.2 255.255.255.0 2.2.2.1//到总舵网关 Tunnel1 接口的路由</pre>
定义被保护的数据流	<pre>acl 3000 rule 5 permit ip source 10.1.1.0 0.0.0.255 destination 10.2.1.0 0.0.0.255</pre>	<pre>acl 3000 rule 5 permit ip source 10.2.1.0 0.0.0.255 destination 10.1.1.0 0.0.0.255</pre>
配置 IPSec 安全提议	ipsec proposal pro1	ipsec proposal pro1
配置 IKE 安全提议	ike proposal 10	ike proposal 10

关键配置	总舵	分舵
IKE 对等体	ike peer fenduo pre-shared-key tiandihui1 ike-proposal 10	ike peer zongduo pre-shared-key tiandihui1 ike-proposal 10 remote-address 1.1.1.2//Tunnel 接口的 IP 地址
IPSec 安全策略	ipsec policy-template tem1 1 //配置模板方式 security acl 3000 proposal pro1 ike-peer fenduo ipsec policy policy1 1 isakmp template tem1	ipsec policy policy1 1 isakmp security acl 3000 proposal pro1 ike-peer zongduo
应用 IPSec 安全策略	interface Tunnel1 ip address 1.1.1.2 24 tunnel-protocol ipsec ipsec policy policy1	interface GigabitEthernet0/0/0 ip address 2.2.2.2 24 ipsec policy policy1

配置完成后，建立 IPSec VPN。从分舵 PC ping 总舵 PC，可以 ping 通。查看 IKE SA 的建立情况如下：

```
<FW_B> display ike sa
```

```
current ike sa number: 2
```

```
-----
conn-id    peer          flag          phase  vpn
-----
40003      1.1.1.2       RD|ST         v2:2   public
3          1.1.1.2       RD|ST         v2:1   public
```

主链路正常情况下 FW_A 上的路由表（只给出静态路由）：

```
<FW_B> display ip routing-table
```

```
Route Flags: R - relay, D - download to fib
```

```
-----
Routing Tables: Public
```

```
Destinations : 17      Routes : 18
```

```
-----
Destination/Mask    Proto  Pre  Cost    Flags NextHop          Interface
-----
2.2.2.2/0           Static 60   0      RD    202.38.163.1 GigabitEthernet1/0/0
2.2.2.2/0           Static 60   0      RD    202.38.164.2 GigabitEthernet1/0/1
10.2.1.0/24         Static 60   0      D     1.1.1.2      Tunnel1
```

主链路中断情况下 FW_A 上的路由表（只给出静态路由）：

```
<FW_B> display ip routing-table
```

```
Route Flags: R - relay, D - download to fib
```

```
-----
```


Routing Tables: Public

Destinations : 17 Routes : 18

Destination/Mask	Proto	Pre	Cost	Flags	NextHop	Interface
2.2.2.2/0	Static	60	0	RD	202.38.164.2	GigabitEthernet1/0/1
10.2.1.0/24	Static	60	0	D	1.1.1.2	Tunnel1

由于分舵是与总舵 Tunnel 接口的 IP 地址建立 IPsec 隧道,所以在总舵 FW_A 上断开 GE1/0/0 接口所属链路后,分舵 PC ping 总舵 PC 的业务流量不会中断。在 FW_B 上查看 IKE SA, 可以看到对端的 IP 地址还是 1.1.1.2。

IPsec 隧道化链路备份跟 IPsec 主备链路备份相比有如下优势,推荐采用 IPsec 隧道化备份。

IPsec 链路备份方案	IPsec 主备链路备份	IPsec 隧道化链路备份
配置	复杂	简单
平滑切换	IPsec 隧道切换, 需重新协商隧道	不需要隧道切换
适用场景	双链路	双链路或多链路

至此 IPsec 专题就讲完了。接下来强叔将介绍另一种安全的 VPN: SSL VPN。由于 SSL VPN 配置简单,且通信安全,一经问世就开始攻城掠地,抢占 L2TP 和 IPsec 的市场,大有后生可畏的劲头。那么 SSL VPN 跟前面讲的几种 VPN 有什么差别,适用于哪些场景,又该如何配置呢?敬请关注下期的强叔侃墙。

SSL VPN 登上历史舞台

强叔耗费长达七篇的笔墨介绍了 VPN 江湖中大佬地位的 IPSec，终于将其事迹介绍完毕，不必再“且听下回分解”。然，江湖从来不曾风平浪静，又是风起云涌时；江山亦从来代有才人出，各领风骚数百年。

随着互联网不可预测的高速发展，如今可谓已“无处不网络，随时可接入”，PC、笔记本人人具备但已被嫌弃厚重不便，人人手持智能手机或平板随时随地上网。时代始终不断变迁，科技始终不断进步，不变的是科技以人为本的理念——人们追求的是更便捷、更简单、更安全。

在远程接入、访问内网这一具体应用场景中，传统 VPN 巨头 IPSec 也暴露出自身一些短板：

- ❖ 组网不灵活。建立 IPSec VPN，如果增加设备或调整用户的 IPSec 策略，需要调整原有 IPSec 配置。
- ❖ 需要安装客户端软件，导致在兼容性、部署和维护方面都比较麻烦。
- ❖ IPSec VPN 对用户的访问控制不够严格，只能进行网络层的控制，无法进行细粒度的、应用层资源的访问控制。

所谓车到山前必有路，有问题就会有解决办法，一种新的技术开始登上历史舞台——SSL VPN 作为新型的轻量级远程接入方案，有效地解决了上述问题，在实际远程接入方案中应用非常广泛。

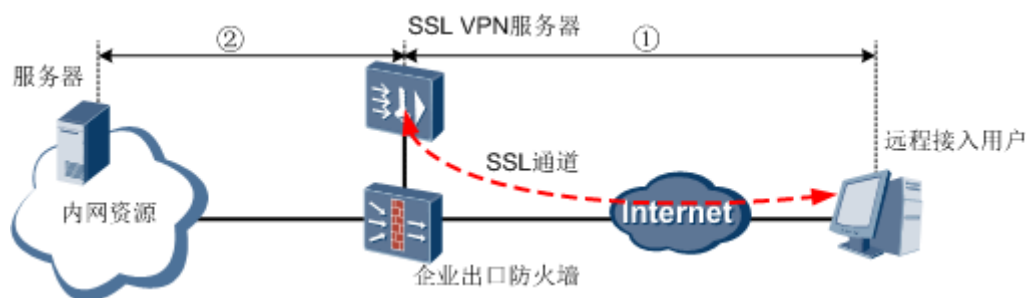
- ❖ SSL VPN 工作在传输层和应用层之间，不会改变 IP 报文头和 TCP 报文头，不会影响原有网络拓扑。如果网络中部署了防火墙，只需放行传统的 HTTPS (443) 端口。
- ❖ SSL VPN 基于 B/S 架构，无需安装客户端，只需要使用普通的浏览器进行访问，方便易用。
- ❖ 更重要的，相对于 IPSec 网络层的控制，SSL VPN 的所有访问控制都是基于应用层，其细分程度可以达到 URL 或文件级别，可以大大提高企业远程接入的安全级别。

下面强叔就为小伙伴带来详细的 SSL VPN 技术介绍。

所谓 SSL VPN 技术，实际是 VPN 设备厂商创造的名词，指的是远程接入用户利用标准 Web 浏览器内嵌的 SSL(Security Socket Layer)封包处理功能，连接企业内部的 SSL VPN 服务器，然后 SSL VPN 服务器可以将报文转向给特定的内部服务器，从而使得远程接入用户在通过

验证后，可访问企业内网特定的服务器资源。其中，远程接入用户与 SSL VPN 服务器之间，采用标准的 SSL 协议对传输的数据包进行加密，这相当于远程接入用户与 SSL VPN 服务器之间建立起隧道。<注：远程接入用户以下简称用户>

SSL VPN 典型应用场景如下图所示：



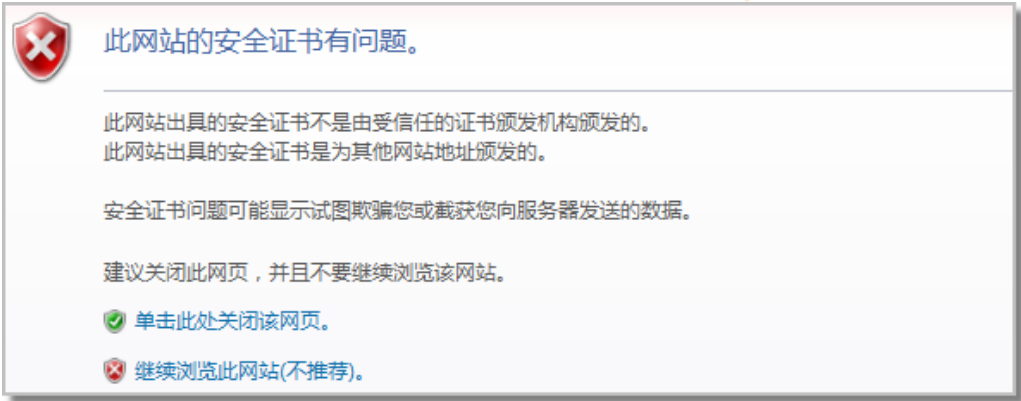
一般来说，SSL VPN 服务器通常部署在企业出口防火墙的内部，而华为 USG2000 和 USG5000 系列产品可以直接作为 SSL VPN 服务器，two in one，节省网络建设和管理成本。顺便打个广告，华为亦推出了 SVN2000 和 SVN5000 系列专门产品，可以提供支持更多用户数、支持更多应用场景的 SSL VPN 解决方案。

本篇强叔主要介绍远程接入用户与 SSL VPN 服务器建立连接、成功登录 SSL VPN 服务器的过程①。而 SSL VPN 服务器如何将远程接入用户的请求转向给各内部服务器②，将在下篇介绍。

在开始枯燥的原理讲解之前，强叔先给小伙伴演示下登录 SSL VPN 服务器的步骤，直观感受下 SSL VPN 的便捷。

作为远程接入用户，访问 SSL VPN 服务器的典型步骤非常简单：

步骤	具体描述
1	打开浏览器，输入 <code>https://SSL VPN 服务器的地址:端口</code> 或 <code>https://域名</code> ，发起连接。 
2	Web 页面可能会提示即将访问的网站安全证书有问题，我们选择“继续浏览该网站”。

步骤	具体描述
	 此网站的安全证书有问题。 此网站出具的安全证书不是由受信任的证书颁发机构颁发的。 此网站出具的安全证书是为其他网站地址颁发的。 安全证书问题可能显示试图欺骗您或截获您向服务器发送的数据。 建议关闭此网页，并且不要继续浏览该网站。 单击此处关闭该网页。 继续浏览此网站(不推荐)。
3	成功刷新出 SSL VPN 服务器登录界面，界面右侧提示输入用户名/密码。 
4	输入预先从企业网络管理员处获得的用户名/密码，成功登录 SSL VPN 服务器，进入访问内网资源页面。 

这几个步骤就能保证 SSL VPN 建立、保证访问安全？为什么有提示“访问的网址安全证书有问题”？小伙伴心中可能飞过了无数问号。带着这些疑问，我们就研究一下在这简短的几个步骤中，远程用户是如何与 SSL VPN 服务器进行报文交互的。

强叔认为这其中有两个关键之处，亦是 SSL VPN 技术两个基本安全性的体现：

🔥 传输过程安全。

在上文 SSL VPN 定义中，我们提到，远程接入用户与 SSL VPN 服务器之间，采用标准的 SSL 协议对传输的数据包进行加密。那么从用户打开浏览器、访问 SSL VPN 服务器地址开始，SSL 协议就开始运行。为此我们要详细地研究下 **SSL 协议的运行机制**。

🔥 用户身份安全。

在上文演示登录 SSL VPN 服务器时，用户访问 SSL VPN 服务器登录界面，SSL VPN 服务器要求输入用户名/密码。这实际是 **SSL VPN 服务器要求对用户身份进行认证**。SSL VPN 服务器往往支持多种用户认证方式，来保证访问的安全性、合法性。华为 USG 系列产品支持用户名/密码的本地认证；服务器认证；证书认证；用户名/密码+证书双重因素认证等。

SSL 协议的运行机制

SSL(Security Socket Layer, 安全套接层)是一种在客户端和服务端之间建立安全通道的协议，是 Netscape 公司提出的基于 web 应用的安全协议，它为基于 TCP/IP 连接的应用程序协议（如 HTTP、Telnet 和 FTP 等）提供数据加密、服务器认证、消息完整性以及可选的客户端认证，也就是说，SSL 协议具备如下特点：

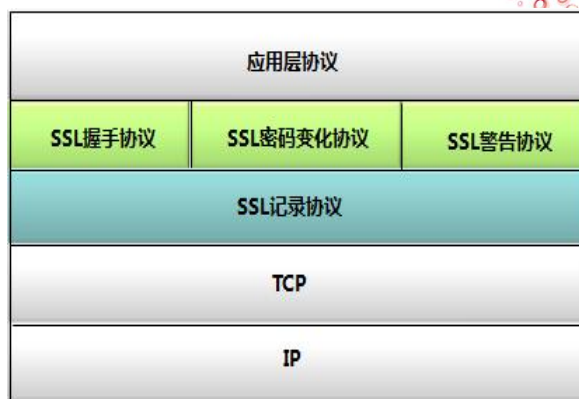
🔥 所有要传输的数据信息都是加密传输，第三方无法窃听。

🔥 具有校验机制，信息一旦被篡改，通信双方会立刻发现。

🔥 配备身份证书，防止身份被冒充。

SSL 协议自 1994 年被提出以来一直在不断发展。Netscape 公司发布的 SSL2.0 和 SSL3.0 版本得到了大规模应用。而后，互联网标准化组织基于 SSL3.0 版本推出了 TLS1.0 协议(又称 SSL3.1)，之后又推出了 TLS1.1 和 TLS1.2 版本。当前，主流浏览器大多已实现了对 TLS1.2 的支持。华为防火墙支持 SSL2.0、SSL3.0 和 TLS1.0 版本。

SSL 协议结构分为两层，底层为 SSL 记录协议，上层为 SSL 握手协议、SSL 密码变化协议、SSL 警告协议，各协议的作用简介如下。



🍁 **SSL 握手协议：**用于在实际的数据传输开始前，通信双方进行身份认证、协商加密算法、交换加密密钥等。相当于完成所有传输准备工作，比较复杂。

🍁 **SSL 密码变化协议：**这个协议只包含一条消息（ChangeCipherSpec）。客户端和服务端双方都可以发送，目的是通知对方后面发送的数据将启用新协商的算法和密钥。

🍁 **SSL 记录协议：**建立在可靠的传输协议（如 TCP）之上，负责把上层的数据分块进行

可以看到，SSL 连接的建立，主要依靠 SSL 握手协议，下面我们来详细研究 SSL 握手协议。

简短一句话就可以概括 SSL 握手协议的基本设计思路：**采用公钥加密法进行密文传输**。也就是说，服务器端将其公钥告诉客户端，然后客户端用服务器的公钥加密信息，服务器收到密文后，用自己的私钥解密。

公钥加密法，又称非对称加密法。其工作原理如下。

1. 客户端 A 要向服务器 B 发送信息，B 要产生一对用于加密和解密的公钥和私钥。
2. B 的私钥自己保密，B 的公钥告诉 A。
3. A 要给 B 发送信息时，用 B 的公钥加密信息，因为 A 知道 B 的公钥。
4. B 收到这个信息后，自己的私钥解密。其他所有收到这个报文的人都无法解密，因为只有 B 才有自己的私钥。
5. B 要给 A 发送消息时，亦同理加密。

这个设计思路有两个问题，需要继续细化解决：

🍁 服务器将其公钥告诉客户端时，如何保证该公钥不被篡改？

解决办法：引入数字证书。将服务器公钥放入服务器证书中，由服务器将证书传给客户端。只要证书可信，公钥就可信。

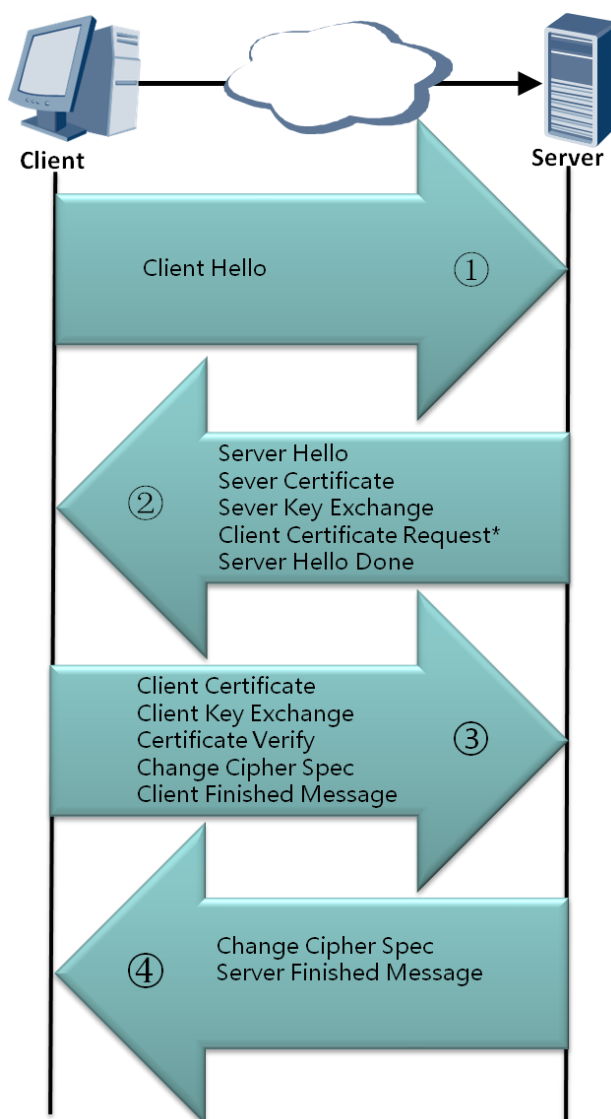
🍁 公钥加密法安全性高，但也由于两端各自用私钥解密，导致算法较复杂、加解密计算量

大，如何提升效率？

解决办法：引入一个新的“会话密钥”。客户端与服务器采用公钥加密法协商出此“会话密钥”，而后续的数据报文都使用此“会话密钥”进行加密和解密（即对称加密法）。对称加密法运算速度很快，这样就大大提升了加解密运算效率。还得罗嗦一下，密钥实际就是一个服务器和客户端共享的密钥，叫“会话密钥”是因为引入了会话 Session 的概念，就是基于 TCP 的 SSL 每个连接与一个会话相关联，会话由 SSL 握手协议来创建，为每个连接提供完整的传输加密，即握手过程包含在会话之中。

既然 SSL 握手协议的设计已经解决了关键问题，下面就要落地具地设计细节了：**通过服务器与客户端的四次通信**来实现上述设计思路，从而能够保证握手阶段之后能够进行高效、安全的加密报文传输。

SSL 握手涉及的四次通信具体内容如下。需要注意的是，此阶段的所有通信都是明文的。



①客户端发出请求 (Client Hello)

客户端 (通常是浏览器) 首先向服务器发出加密通信的请求, 此步主要向服务器提供以下信息。

- (1) 支持的协议版本, 比如 TLS 1.0 版。
- (2) 一个客户端生成的随机数, 稍后用于生成“会话密钥”。
- (3) 支持的加密套件。包括数据加密算法等, 按客户端优先原则排序。
- (4) 支持的压缩方法。

②服务器回应 (Server Hello)

服务器收到客户端请求后, 向客户端发出回应, 此步包含以下信息。

- (1) 确认使用的加密通信协议版本, 比如 TLS 1.0 版本。如果浏览器与服务器支持的版本不一致, 则服务器关闭加密通信。
- (2) 一个服务器生成的随机数, 稍后用于生成“会话密钥”。
- (3) 确认加密套件。
- (4) 服务器证书, 装有服务器公钥。

*SSL 握手协议支持客户端与服务器双向互认, 也就是服务器可以发出对客户端的证书验证请求。本文按华为产品, 仅进行客户端验证服务器。

③客户端回应

客户端收到服务器回应以后, 首先验证服务器证书。如果证书不是可信机构颁布、或者证书中的域名与实际域名不一致、或者证书已过期, 就会向访问者显示一个警告, 由其选择是否还要继续通信。如果证书没有问题, 客户端就会从证书中取出服务器的公钥。然后向服务器发送下面三项信息。

- (1) 一个随机数 pre-master-key, 用服务器公钥加密, 防止被窃听, 稍后用于生成“会话密钥”。<<此时客户端已经拥有三个随机数, 会计算出本次会话所用的“会话密钥”。
- (2) 编码改变通知, 表示随后信息都将用双方商定的加密方法和密钥发送。
- (3) 客户端握手结束通知, 表示客户端的握手阶段已经结束。这一项同时也是前面发送的所有内容的 hash 值, 用来供服务器校验。

④服务器最后回应

服务器收到客户端的随机数 pre-master key 之后, 计算生成本次会话所用的“会话密钥”。然后向客户端最后发送下面信息。<<与客户端计算结果相同。

- (1) 编码改变通知, 表示随后信息都将用双方商定的加密方法和密钥发送。
 - (2) 服务器握手结束通知, 表示服务器的握手阶段已经结束。这一项同时也是前面发送的所有内容的 hash 值, 用来供客户端校验。
- 至此, 整个握手阶段全部结束。接下来, 客户端与服务器进入加密通信传输数据, 就完全是使用普通的 HTTP 协议, 只是用“会话密钥”加密内容。

小伙伴们在看完 SSL 握手协议四次通信的具体内容之后, 估计又会产生几个问号, 强叔已在此恭候:

🍁 上述当随机数 pre-master-key 出现时, 已经是第三个随机数。有了它以后, 客户端和服务端就同时有了三个随机数, 接着双方就用事先商定的加密方法, 各自生成本次会话所用的同一把“会话密钥”。为什么要用三个随机数来生成“会话密钥”呢?

答: 对于公钥加密法 (RSA 密钥交换算法) 来说, 三个随机数通过一个密钥导出器, 最终导出一个对称密钥, 显然是为了增强安全性。pre-master-key 之所以存在, 是因为 SSL 协议不信任每个主机都能产生“完全随机”的随机数。而如果随机数不随机, 就有可能被猜测出来,

安全性就存在问题。而三个伪随机数凑在一起就十分接近随机了，每增加一个数，随机性增加的可不是一。

🍁 上述 SSL 握手协议第②次通信：服务器回应（Sever Hello）时发出了自己的证书，而马上客户端会对服务器的证书进行验证，也就是客户端在验证服务器的合法性。这是否与演示登录 SSL VPN 服务器时，第 2 步骤中曾遇到了一个警告，即“为什么有提示访问的网址安全证书有问题？”相关联？

答：其实从第 1 步骤客户端（远程用户）发起 https 访问 SSL VPN 服务器起，SSL 协议已开始运行，第 2 步骤的提示，恰恰可以与 SSL 握手协议的第②次通信相对应：此时服务器将自己的本地证书传给了客户端，客户端要对服务器的证书进行认证。提示警告，说明客户端认为该服务器证书不可信。如果平时我们访问网银等界面出现此提示时，要提高警惕，防止误入钓鱼网站。而此时，我们选择强制相信该网站。

提示安全证书有问题的警告可以消除，一个办法就在登录 SSL 服务器的用户名/密码界面上，提示可以下载并安装证书。

下载安装的是什么证书？正是为服务器颁发本地证书的 CA 机构自身的证书（称为服务器 CA 证书）。客户端安装了服务器 CA 证书，则在收到服务器本地证书时会认定该证书合法—因为是由自己已知的 CA 机构颁发的。

当然，客户端验证服务器本地证书是否合法还有两个附加条件，三个条件完全满足才会确认服务器证书合法：

- 🍁 证书在有效期内
- 🍁 证书中的关键字段 CN 与该 SSL VPN 服务器的地址或域名相一致（即该证书确为颁发给该服务器的身份证，不是冒用别人的）
- 🍁 证书是否是由自己已知的 CA 机构颁发

践这个唯一标准来检验真理。

<步骤 1>以防火墙作为 SSL VPN 服务器，在防火墙上完成相关配置。

SSL 服务器的功能在防火墙上被称为**虚拟网关**。虚拟网关的地址/域名即为 SSL VPN 服务器的地址/域名。

- a. 配置虚拟网关，启用 SSL VPN 服务器功能，配置服务器地址等。
- b. 配置认证授权方式为本地认证、并创建用户包括配置用户名和密码。
- c. 配置防火墙安全策略，保证网络互通。

<步骤 2>按本文开篇演示的客户端登录 SSL VPN 服务器步骤、刚刚配置好的用户名/密码，登录 SSL VPN 服务器。

- a) 客户端 10.108.84.93 以 IE 浏览器向防火墙的虚拟网关 <https://10.174.64.61> 发起链接请

求。如下图所示，No.21-No.29 展示了完 SSL 握手的四次通信过程，至 No.29 服务器回应 Server Finished Message（已被加密，提示为加密的握手消息）后，对应登录到提示安全证书有问题的警告界面。此时客户端与服务器实际并未开始正常通信，而是在 SSL 握手阶段，客户端验证服务器不合法，在其 Web 界面提示用户是否继续浏览该网站。

No.	Time	Source	Destination	Protocol	Length	Info
21	12.152928000	10.108.84.93	10.174.64.61	SSLv2	126	Client Hello
23	12.204784000	10.174.64.61	10.108.84.93	TLSv1	1514	Server Hello
24	12.204904000	10.174.64.61	10.108.84.93	TLSv1	529	Certificate
26	12.205857000	10.108.84.93	10.174.64.61	TLSv1	380	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
27	12.291021000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec
29	12.544490000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message
103	45.686218000	10.108.84.93	10.174.64.61	TLSv1	149	Client Hello
105	45.737969000	10.174.64.61	10.108.84.93	TLSv1	1514	Server Hello
106	45.738171000	10.174.64.61	10.108.84.93	TLSv1	529	Certificate
108	45.739538000	10.108.84.93	10.174.64.61	TLSv1	380	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
109	45.823791000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec
111	46.083392000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message
120	46.141764000	10.108.84.93	10.174.64.61	TLSv1	149	Client Hello
123	46.143644000	10.174.64.61	10.108.84.93	TLSv1	1514	Server Hello
Frame 29: 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interface 0						
Ethernet II, Src: IETF-VRRP-VRID_a9 (00:00:5e:00:01:a9), Dst: RealtekS_98:29:04 (00:e0:4c:98:29:04)						
Internet Protocol Version 4, Src: 10.174.64.61 (10.174.64.61), Dst: 10.108.84.93 (10.108.84.93)						
Transmission Control Protocol, Src Port: https (443), Dst Port: 57814 (57814), Seq: 1942, Ack: 399, Len: 53						
Secure Sockets Layer						
TLSv1 Record Layer: Handshake Protocol: Encrypted Handshake Message						

b) 强制信任，选择“继续浏览该网站”，从 No.103 起客户端要求使用新会话，重新发起 SSL 握手协议。握手完成之后，开始正常的加密通信，直至用户浏览器上成功加载出防火墙虚拟网关的用户登录界面。

No.	Time	Source	Destination	Protocol	Length	Info
1128	49.159026000	10.174.64.61	10.108.84.93	TLSv1	1514	Application Data
1145	49.211475000	10.174.64.61	10.108.84.93	TLSv1	1514	Application Data
1150	49.328483000	10.174.64.61	10.108.84.93	TLSv1	1018	Application Data
1152	49.469116000	10.174.64.61	10.108.84.93	TLSv1	704	Application Data
1157	50.596904000	10.108.84.93	10.174.64.61	TLSv1	555	Application Data
1158	50.648566000	10.174.64.61	10.108.84.93	TLSv1	219	Application Data
1159	50.648837000	10.174.64.61	10.108.84.93	TCP	1514	[TCP segment of a reassembled PDU]
1163	50.903890000	10.174.64.61	10.108.84.93	TLSv1	803	Application Data
1561	121.916595000	10.108.84.93	10.174.64.61	TLSv1	149	Client Hello
1563	121.968498000	10.174.64.61	10.108.84.93	TLSv1	1514	Server Hello
1564	121.968686000	10.174.64.61	10.108.84.93	TLSv1	529	Certificate
1566	121.970514000	10.108.84.93	10.174.64.61	TLSv1	380	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
1567	122.058709000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec
1571	122.163832000	10.108.84.93	10.174.64.61	SSLv2	126	Client Hello
1573	122.215714000	10.174.64.61	10.108.84.93	TLSv1	1514	Server Hello
1574	122.215940000	10.174.64.61	10.108.84.93	TLSv1	529	Certificate
1576	122.217712000	10.108.84.93	10.174.64.61	TLSv1	380	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
1581	122.305165000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message
1583	122.307699000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec
1588	122.312994000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message
1597	122.314132000	10.108.84.93	10.174.64.61	SSLv2	126	Client Hello
1599	122.365835000	10.174.64.61	10.108.84.93	TLSv1	1514	Server Hello
1600	122.366050000	10.174.64.61	10.108.84.93	TLSv1	529	Certificate
1602	122.367392000	10.108.84.93	10.174.64.61	TLSv1	380	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
1603	122.450535000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec
1605	122.710145000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message
1606	122.712813000	10.108.84.93	10.174.64.61	TLSv1	939	Application Data
1607	122.769454000	10.174.64.61	10.108.84.93	TLSv1	363	Application Data
1608	122.772244000	10.108.84.93	10.174.64.61	TLSv1	843	Application Data
1609	122.827314000	10.174.64.61	10.108.84.93	TCP	1514	[TCP segment of a reassembled PDU]
1628	122.879319000	10.174.64.61	10.108.84.93	TLSv1	1514	Application Data
1645	122.934144000	10.174.64.61	10.108.84.93	TLSv1	1036	Application Data
1647	122.936277000	10.108.84.93	10.174.64.61	TLSv1	651	Application Data
Frame 1163: 803 bytes on wire (6424 bits), 803 bytes captured (6424 bits) on interface 0						
Ethernet II, Src: IETF-VRRP-VRID_a9 (00:00:5e:00:01:a9), Dst: RealtekS_98:29:04 (00:e0:4c:98:29:04)						
Internet Protocol Version 4, Src: 10.174.64.61 (10.174.64.61), Dst: 10.108.84.93 (10.108.84.93)						
Transmission Control Protocol, Src Port: https (443), Dst Port: 57818 (57818), Seq: 102640, Ack: 2602, Len: 749						
[3 Reassembled TCP Segments (3669 bytes): #1159(1460), #1160(1460), #1163(749)]						
Secure Sockets Layer						
TLSv1 Record Layer: Application Data Protocol: ssl						

c) 输入用户名/密码，从 No.1561 起继续发起 SSL 握手协议，经过四次通信协商出“会话密钥”，从携带用户名/密码的报文起所有用户与服务器之前的数据被加密（显示为 Application Data）发送给服务器。

用户身份认证

为了保证 SSL VPN 接入用户的合法性、提升系统安全性，SSL VPN 服务器往往支持多种认证方式，上文我们一直在以配置并存储在防火墙上的用户名/密码为例，这是最基本最简单的认证方式。实际华为防火墙支持以下认证方式：

- 🍁 用户名/密码的本地认证：即上文各种举例，指将用户名/密码在防火墙上配置并存储，用户输入与之匹配的用户名/密码即可成功登录。
- 🍁 用户名/密码的服务器认证：指将用户名/密码存储在专门的认证服务器上，用户输入用户名/密码后，防火墙将其转至认证服务器认证。当前支持的认证服务器类型包括：RADIUS、HWTACACS、SecurID、AD、LDAP。
- 🍁 证书匿名认证：指用户的客户端配备客户端证书，防火墙通过验证客户端的证书来认证用户身份。
- 🍁 证书挑战认证：指服务器通过用户名/密码+客户端证书双重因素认证用户身份。可以看出，此种方式是最安全的。

单用客户端证书认证，当客户端丢失或被非法使用时就无法保证安全；单用用户名/密码认证，当使用不同客户端时，客户端本身可能存在安全隐患。双重因素认证方式，保证了指定用户、使用指定客户端登录 SSL VPN 服务器，进而合法访问内网资源。

用户名/密码的服务器认证和本地认证，即为最为常见的用户认证实现，此处不再赘述。下面强叔介绍下证书认证。

证书挑战认证比证书匿名认证多了一次用户名/密码认证，原理基本一致，可以一并讲解。



A decorative graphic featuring a large, stylized red number '119' in the center. The number is composed of overlapping, semi-transparent red circles and rings, giving it a dynamic, swirling appearance. To the left of the number, there is a trail of smaller, solid red circles of varying sizes, some of which are slightly blurred, suggesting motion or a path leading towards the main graphic. The background is white.

在防火墙虚拟网关界面，选择客户端要使用的证书后，按回车确认，正常抓包如下，SSL 握手协议后，又有一堆加密的握手信息和数据。这无法分辨到底是什么报文，于是我们导入防火墙（SSL 服务器）的私钥、利用抓包工具对抓到的报文进行解密。

如下图所示，左右对比之下，我们可以看到，从 No.895 开始，显示为 Euncrypted Hanashake Message 的第一条消息，实际是服务器 10.174.64.61 对客户端 10.108.84.93 发出 Hello Request，然后客户端回应，随后服务器发出 Server Hello，并且从这条之后，服务器对客户端发出认证其证书的请求。从抓包来看，由于某些原因，这次协商没有成功，客户端和服务器的协商会继续进行。

在 No.1045 开始，服务器再次发现 Hello Request，然后继续执行，No.1085 服务器要求客户端提供证书，然后 No.1088 客户端发出其证书给服务器，No.1097 服务器对客户端证书进行认证，抓包显示为证书非法，无法通过认证。非法没关系，Alert 多么显眼，突出反映了服务器对客户端证书认证的完整过程，请小伙伴左右对比，一揭真相^_^

No.	Time	Source	Destination	Protocol	Length	Info	Info
766	3.541675000	10.108.84.93	10.174.64.61	TLSv1	587	Application Data	Application Data
775	3.604183000	10.174.64.61	10.108.84.93	TLSv1	779	Application Data	Application Data
823	3.884566000	10.108.84.93	10.174.64.61	SSLv2	126	Client Hello	Client Hello
837	3.947013000	10.174.64.61	10.108.84.93	TLSv1	1183	Server Hello, Certificate, Server Hello Done	Server Hello, Certificate, Server Hello Done
838	3.947958000	10.108.84.93	10.174.64.61	TLSv1	252	Client Key Exchange, Change Cipher Spec, Finished	Client Key Exchange, Change Cipher Spec, Finished
853	4.017627000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec	Change Cipher Spec
887	4.281916000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message	Finished
888	4.282603000	10.108.84.93	10.174.64.61	TLSv1	363	Application Data	Ignored Unknown Record
895	4.345197000	10.174.64.61	10.108.84.93	TLSv1	91	Encrypted Handshake Message	Hello Request
896	4.346349000	10.108.84.93	10.174.64.61	TLSv1	187	Encrypted Handshake Message	Client Hello
906	4.408599000	10.174.64.61	10.108.84.93	TLSv1	155	Encrypted Handshake Message	Server Hello
952	4.673126000	10.174.64.61	10.108.84.93	TLSv1	1200	Encrypted Handshake Message, Encrypted Handshake Message	Certificate, Certificate Request, Server Hello Done
980	4.794549000	10.108.84.93	10.174.64.61	SSLv2	126	Client Hello	Client Hello
994	4.857173000	10.174.64.61	10.108.84.93	TLSv1	1183	Server Hello, Certificate, Server Hello Done	Server Hello, Certificate, Server Hello Done
995	4.858409000	10.108.84.93	10.174.64.61	TLSv1	252	Client Key Exchange, Change Cipher Spec, Finished	Client Key Exchange, Change Cipher Spec, Finished
1002	4.924322000	10.174.64.61	10.108.84.93	TLSv1	60	Change Cipher Spec	Change Cipher Spec
1033	5.187501000	10.174.64.61	10.108.84.93	TLSv1	107	Encrypted Handshake Message	Finished
1034	5.188340000	10.108.84.93	10.174.64.61	TLSv1	395	Application Data	Ignored Unknown Record
1045	5.250574000	10.174.64.61	10.108.84.93	TLSv1	91	Encrypted Handshake Message	Hello Request
1046	5.251291000	10.108.84.93	10.174.64.61	TLSv1	187	Encrypted Handshake Message	Client Hello
1054	5.315473000	10.174.64.61	10.108.84.93	TLSv1	155	Encrypted Handshake Message	Server Hello
1085	5.572368000	10.174.64.61	10.108.84.93	TLSv1	1200	Encrypted Handshake Message, Encrypted Handshake Message	Certificate, Certificate Request, Server Hello Done
1088	5.576844000	10.108.84.93	10.174.64.61	TLSv1	981	Encrypted Handshake Message, Change Cipher Spec, Finished	Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Finished
1097	5.639377000	10.174.64.61	10.108.84.93	TLSv1	91	Encrypted Alert	Alert (Level: Fatal, Description: Bad Certificate)
1098	5.639974000	10.174.64.61	10.108.84.93	TLSv1	91	Encrypted Alert	Alert (Level: Fatal, Description: Bad Certificate)
1119	5.742264000	10.108.84.93	10.174.64.61	SSLv2	126	Client Hello	Client Hello
1132	5.804786000	10.174.64.61	10.108.84.93	TLSv1	1183	Server Hello, Certificate, Server Hello Done	Server Hello, Certificate, Server Hello Done

正常抓包

对抓到的报文进行解密

Frame 895: 91 bytes on wire (728 bits), 91 bytes captured (728 bits) on interface 0

Ethernet II, Src: IETF-VRRP-VRID_a9 (00:00:5e:00:01:a9), Dst: RealtekS_98:29:04 (00:e0:4c:98:29:04)

Internet Protocol Version 4, Src: 10.174.64.61 (10.174.64.61), Dst: 10.108.84.93 (10.108.84.93)

Transmission Control Protocol, Src Port: https (443), Dst Port: 59028 (59028), Seq: 1189, Ack: 580, Len: 37

Secure Sockets Layer

TLSv1 Record Layer: Handshake Protocol: Encrypted Handshake Message

Content Type: Handshake (22)

Version: TLS 1.0 (0x0301)

Length: 32

Handshake Protocol: Encrypted Handshake Message

以上，强叔将远程用户与 SSL VPN 服务器建立连接、成功登录 SSL VPN 服务器的过程就全部介绍完了。开篇时强叔提到，后续将继续介绍 SSL VPN 服务器如何将远程用户的请求转向给各内部服务器。在未来的几篇 SSL VPN 系列里，强叔将排出兵器谱，从访问控制粒度由小到大，首先对办公应用中的最常见的 Web 访问（E-mail、电子流等）和文件访问进行介绍，敬请期待。

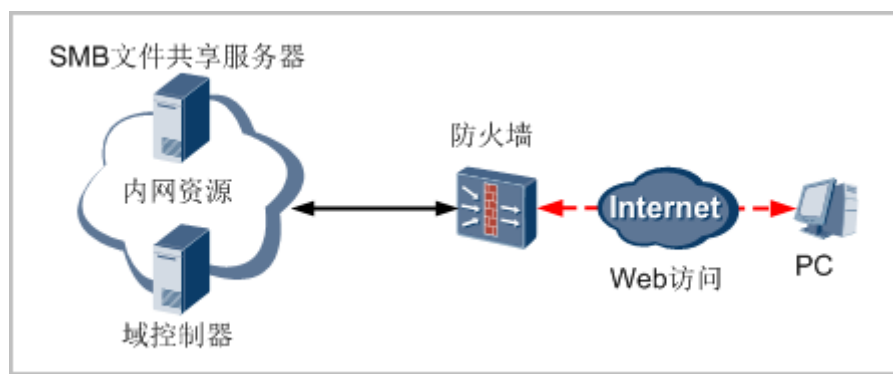
🍁 URL 地址隐藏改写各千秋

通过第一篇的介绍，大家了解到 SSL VPN 与 IPSec 相较的巧妙之处在于可以将远程用户的访问粒度细化到指定的资源对象，例如一个文件或者一个 URL。为了让远程用户对自己的访问权限一目了然，虚拟网关提供了一个特别友好、特别人性化的平台：将若干文件和 URL 组合成“私人定制”资源清单呈现给远程用户。这就好比虚拟网关是新潮的时尚餐厅，不仅卖美食还卖定制化服务，可以为不同口味的顾客定制不同的菜单。不仅如此，由于大多数企业出于安全考虑都不想对外公开内网服务器资源地址（URL 或文件路径），为此 SSL VPN 还提供了“资源地址加密”服务--对资源路径进行了巧妙改写，让远程用户既能顺畅访问内网资源，又很难找到内网资源地址。这就好比给白菜豆腐起名为翡翠白玉，红椒绿椒冠以绝代双骄，乍一看挺玄乎费点脑筋才能搞懂其中奥妙。

华为防火墙 SSL VPN 为大家准备了丰盛的菜肴，提供了定制化菜单服务，还附赠了脑筋急转弯似的小节目，希望顾客在品尝美味的同时能收获美丽的心情！

口味：对文件的访问——菜系：文件共享功能

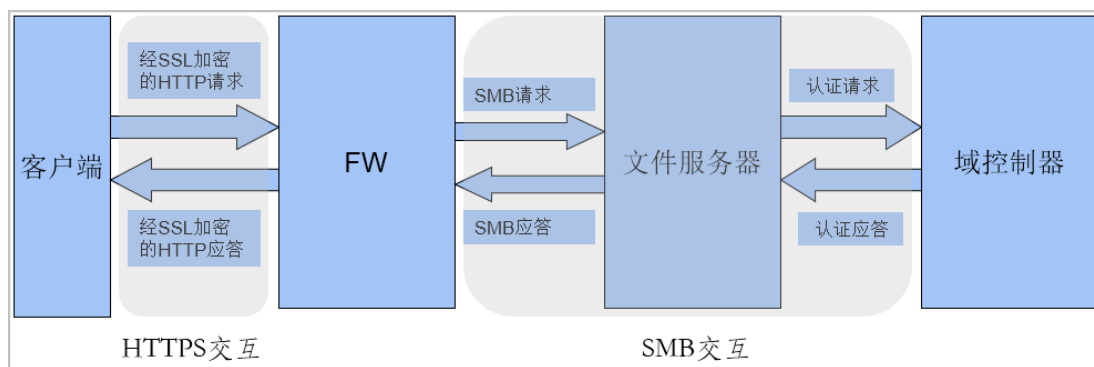
菜系简介：SSL VPN 的文件共享功能，简单说就是能够让远程接入用户能够直接通过浏览器安全的访问企业内部的文件服务器，而且支持新建、修改、上传、下载等常见的文件操作。目前，在企业中较为流行的文件共享协议包括 SMB(Server Message Block)和 NFS(Network File System)，前者主要应用于 Windows 操作系统，后者主要应用在 Linux 操作系统。华为防火墙的 SSL VPN 对这两种协议已经兼顾到了，小伙伴们不必担心。接下来的内容以 SMB 协议为例，并借助域控制器这种常用的鉴权方式，介绍文件共享的交互过程。



如下图所示，可以看出防火墙作为代理设备，与客户端之间的通信始终是通过安全 HTTPS（HTTP+SSL）协议加密传输，当加密报文抵达防火墙后，防火墙对其解密后并进行协议转换，最终作为 SMB 客户端，向相应的 SMB 文件共享服务器发起请求，其中还包含了文件服务器认证的过程。从通信所使用协议的角度，以上过程可以概况为两个阶段：

远程接入用户作为 Web Client 与防火墙作为 Web Server 之间的 HTTPS 交互。

2) 防火墙作为 SMB Client 与文件服务器 SMB Server 之间的 SMB 交互。



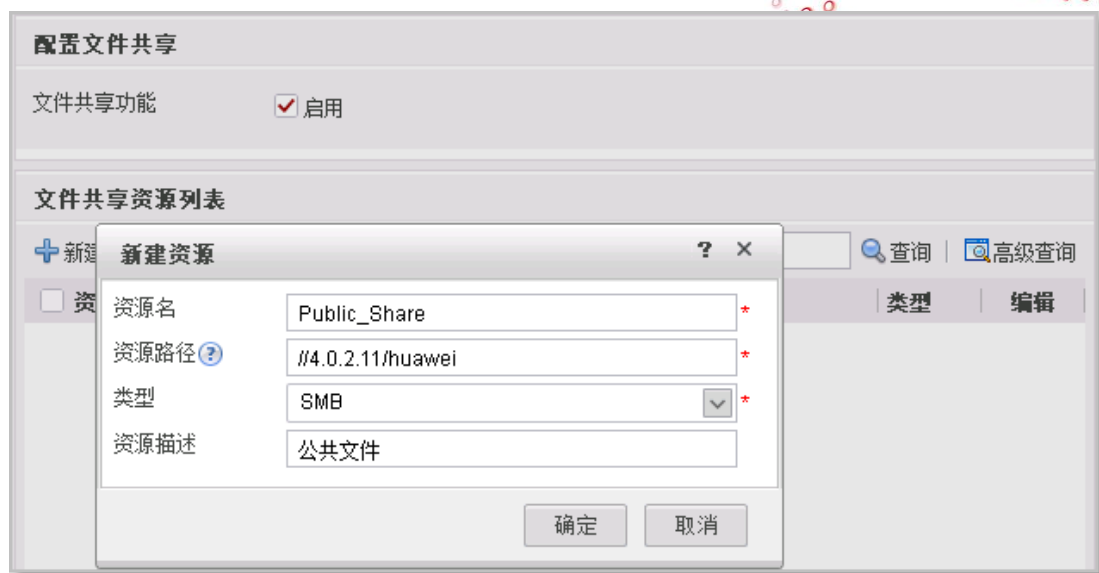
准备文件共享资源菜单（配置文件共享资源）

在正式介绍交互之前，我们首先假设在 SMB 文件服务器（以 Windows Server 2008 为例）中已配置好文件共享资源，并在域控制器中配置好权限分配：

资源访问地址：<\\4.0.2.11\\huawei>

使用者权限分配：admin 具有读取/写入权限；usera 仅具有读取权限

虚拟网关作为 SSL VPN 的所有资源入口，任何需要访问的资源都必须在 SSL VPN 的配置中体现，这也体现了 SSL VPN 可以细化访问控制粒度的设计理念。对于文件共享，首先需要开启文件共享功能，并新建文件共享资源，目的是为远程用户提供可视化的文件共享资源“菜单”。



远程用户轻松点菜（远程用户与防火墙的 HTTPS 交互）

登录成功后，虚拟网关对用户开放的资源都会在此界面上体现。将鼠标置于资源上，在浏览器的状态栏中可以看到此资源对应的 Web 链接，包含了需要向防火墙请求的页面和需要传递的参数，可不要小瞧这个 URL，它代表了远程用户请求的文件资源信息以及相应的操作指令，不同的目录和操作，均会对应不同的 URL。



[https://4.1.64.12/protocoltran/Login.html?VTID=0&UserID=4&SessionID=2141622535&ResourceType=1&ResourceID=4&PageSize=20&%22,1\)](https://4.1.64.12/protocoltran/Login.html?VTID=0&UserID=4&SessionID=2141622535&ResourceType=1&ResourceID=4&PageSize=20&%22,1))

Q: 上文的提到的文件共享资源\\4.0.2.11\huawei 这里为什么看不到？

A: 因为防火墙已经将其隐藏，使用 ResourceID 唯一确定资源地址，ResourceID 与资源地址的对应关系保存在防火墙的大脑（内存）中，这样便可以隐藏了内网服务器的真实地址，保护服务器安全。

对这个 Web 链接进行深入剖析，除去很明显的 4.1.64.12 是虚拟网关地址外，强叔将剩下的链接结构分解为 3 个部分：

1. **protocoltran**，文件共享的专属目录。从字面猜测是 protocol+transform，表示将 HTTPS 协议和 SMB/NFS 协议相互转换。

2. [Login.html](#), 请求的页面, 通常情况下不同的操作会对应不同的请求页面, 强叔整理了所有可能用到的请求页面和请求结果页面。

页面名称	含义
login.html	SMB 文件服务器鉴权页面。
loginresult.html	
dirlist.html	显示文件共享资源的详细列表, 文件夹结构。
downloadresult.html	下载文件。
downloadfailed.html	
create.html	创建文件夹。
result.html	
deleterestult.html	删除文件、文件夹。
result.html	
rename.html	重命名文件、文件夹。
result.html	
upload.html	上传文件。
uploadresult.html	

3. [?VTID=0&UserID=4&SessionID=2141622535&ResourceType=1&ResourceID=4&PageSize=20&%22,1](#), 向请求页面传递的参数。强叔这里先给出一张参数明细表, 除了已经给出这条 URL 涵盖的参数外, 还包括了之后其他操作的请求参数, 供大家对照理解。

参数	含义
VTID	虚拟网关 ID, 用以区分同一台防火墙上的多个虚拟网关。
UserID	用户 ID, 标识当前登录用户。为了安全起见, 同一用户每次登录的 ID 不同, 防止中间攻击人伪造假的数据包。
SessionID/RandomID	会话 ID, 同一次登录虚拟网关的所有会话 ID 均相同。
ResourceID	资源 ID, 标识每个文件共享资源。
CurrentPath	当前操作所在的文件路径。
MethodType	操作类型: 1: 删除文件夹

参数	含义
	2: 删除文件 3: 显示目录 4: 重命名目录 5: 重命名文件 6: 新建目录 7: 上传文件 8: 下载文件
ItemNumber	操作对象数量。
ItemName1	操作对象名称，可以包含多个操作对象，例如删除多个文件。
ItemType1	操作对象类型： 0: 文件 1: 文件夹
NewName	新名称。
ResourceType	资源类型： 1: SMB 资源 2: NFS 资源
PageSize	每页显示资源条目数量。

为了让大家对文件共享功能的全景有所了解，强叔将结合文件共享的具体功能对以上各种操作指令进行逐一验证。

1) 首次访问文件共享资源，要先通过文件服务器的认证。这里所说的认证一定要和 SSL VPN 登录时的认证区分开，在登录阶段，接入用户首先要通过的是防火墙认证。而此时要访问文件共享资源，当然要看文件服务器是否答应。在点击资源列表中的“Public_share”时会弹出认证页面：

文件共享登录

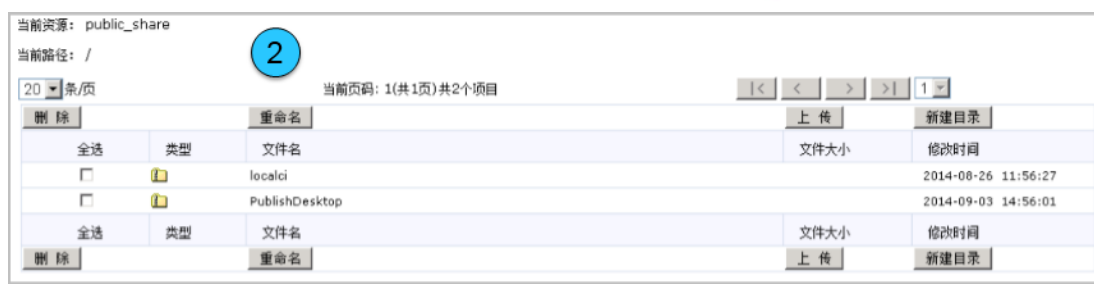
1

用户名：

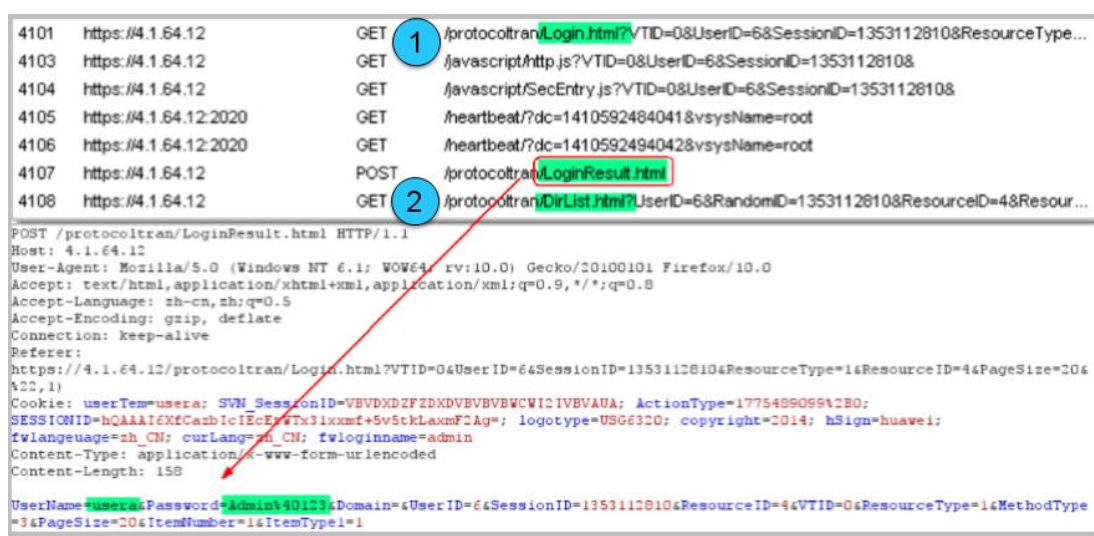
密码：

确定

鉴权通过后显示文件资源页面：



上面的访问，强叔理解可以分解为认证和显示文件夹两个过程，那么真正的交互过程是否是这样呢？



没错，看来强叔的理解是正确的，Login.html/LoginResult.html 均是鉴权认证的页面，而且将加密报文解密后，LoginResult.html 还包括了待文件服务器认证的用户名和密码。另外的Dirlist.html 正是显示文件夹结构的页面。

2) 下载文件的验证。



可以对照强叔给出的表格，描述出如下语句：下载(MethodType=8)根目录(CurrentPath=2F)下的文件(ItemType1=0)，名称为 readme_11 (ItemName1=%r%e%a%d%m%e_%1%1)。

但是要注意，这里有一点 URL 解码的内容，例如 CurrentPath 的取值 2F 解码之后是 /，表示当前资源的根目录。

3) 重命名文件夹的验证。

当前资源: Public_share
当前路径: /

20 条/页 当前页码: 1(共1页)共4个项目

删除	重命名	上传	新建目录
全选	类型	文件名	文件大小 修改时间
☐	文件夹	localci	2014-08-26 11:56:27
☐	文件夹	PublishDesktop	2014-09-03 14:56:01
☐	文件	readme_11	0.01 KB 2014-09-11 12:10:08
☒	文件夹	userb	2014-09-11 21:21:40
全选	类型	文件名	文件大小 修改时间
删除	重命名	上传	新建目录

新名称: 长度: 1~253

重命名目录失败: 没有足够的权限进行操作。

这里因为用户 usera 只有可读的权限，所以提示失败了，但不妨碍我们继续验证：重命名（MethodType=4）根目录（CurrentPath=2F）下的文件（ItemType1=1）userb（ItemNmame1=%u%s%e%r%b）为 usera（NewName=%u%s%e%r%a）。

```
POST
/protocoltran/Result.html?VTID=0&UserID=6&SessionID=1353112810&CurrentPath=2F&ItemName1=%u%s%e%r%b&ResourceID=4&ResourceType=1&MethodType=4&ItemNumber=1&ItemType1=1 HTTP/1.1
Host: 4.1.64.12
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:10.0) Gecko/20100101 Firefox/10.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-cn,zh;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: https://4.1.64.12/protocoltran/Rename.html?VTID=0&UserID=6&SessionID=1353112810&ResourceID=4&Parameter=2F,%u%s%e%r%b,1&ResourceType=1&ItemNumber=1&
Cookie: userTem=usera; SVN_SessionID=VBVDXDZFDZDXDVBVBVCW12IVBVAUA; ActionType=1775489099%2B0; SESSIONID=hQAAAI6XfCazb1cIEcEtWtX31xxmf+5v5tkLaxmF2Ag=; logotype=USG6320; copyright=2014; hSign=huawei; fwlanguage=zh_CN; curLang=zh_CN; fwloginname=admin
Content-Type: Text/plain
Content-Length: 172

RenameParam=VTID=0&UserID=6&SessionID=1353112810&CurrentPath=2F&ItemName1=%u%s%e%r%b&ResourceID=4&ResourceType=1&MethodType=4&ItemNumber=1&ItemType1=1&NewName=%u%s%e%r%a;
```

通过以上介绍相信大家已经明白，防火墙构建这些链接，首先是为了隐藏真实的内网文件资源路径（\\4.0.2.11\\huawei\\），再者就是作为 SSL VPN 网关为远程用户访问牵线搭桥：作为 SMB Client 向 SMB Server 发起文件访问（明确访问的文件对象和操作）。

虚拟网关为远程用户文件访问牵线搭桥（防火墙与文件服务器的 SMB 交互）

112	10:58:10.456	4.1.64.12	4.0.2.11	SMB	105 Negotiate Protocol Request
113	10:58:10.456	4.0.2.11	4.1.64.12	SMB	181 Negotiate Protocol Response
114	10:58:10.457	4.1.64.12	4.0.2.11	SMB	268 Session Setup Andx Request, User: ?\admin; Tree Connect Andx, Path: \\4.0.2.11\huawei
115	10:58:10.459	4.0.2.11	4.1.64.12	SMB	296 Session Setup Andx Response; Tree Connect Andx
116	10:58:10.459	4.1.64.12	4.0.2.11	SMB	154 Trans2 Request, FIND_FIRST2, Pattern: \userb*
117	10:58:10.459	4.0.2.11	4.1.64.12	SMB	330 Trans2 Response, FIND_FIRST2, Files: ...
118	10:58:10.460	4.1.64.12	4.0.2.11	SMB	95 Find Close2 Request
119	10:58:10.460	4.0.2.11	4.1.64.12	SMB	93 Find Close2 Response

1) 防火墙 (4.1.64.12) 作为客户端向文件服务器 (4.0.2.11) 发起协商请求, 首先协商使用的 SMB 版本 (Dialect), 防火墙目前仅支持使用 SMB1.0 (NT LM 0.12) 作为客户端与服务进行交互。

112	10:58:10.456	4.1.64.12	4.0.2.11	SMB	105 Negotiate Protocol Request
113	10:58:10.456	4.0.2.11	4.1.64.12	SMB	181 Negotiate Protocol Response
114	10:58:10.457	4.1.64.12	4.0.2.11	SMB	268 Session Setup Andx Request, User: ?\admin; Tree Connect Andx, Path: \\4.0.2.11\huawei
115	10:58:10.459	4.0.2.11	4.1.64.12	SMB	296 Session Setup Andx Response; Tree Connect Andx
116	10:58:10.459	4.1.64.12	4.0.2.11	SMB	154 Trans2 Request, FIND_FIRST2, Pattern: \userb*
117	10:58:10.459	4.0.2.11	4.1.64.12	SMB	330 Trans2 Response, FIND_FIRST2, Files: ...
118	10:58:10.460	4.1.64.12	4.0.2.11	SMB	95 Find Close2 Request
119	10:58:10.460	4.0.2.11	4.1.64.12	SMB	93 Find Close2 Response

Frame 112: 105 bytes on wire (840 bits), 105 bytes captured (840 bits) on interface 0	
Ethernet II, Src: HuaweiTe_a2:39:bb (70:54:f5:a2:39:bb), Dst: HuaweiTe_e8:75:de (f8:4a:bf:e8:75:de)	
Internet Protocol Version 4, Src: 4.1.64.12 (4.1.64.12), Dst: 4.0.2.11 (4.0.2.11)	
Transmission Control Protocol, Src Port: 10137 (10137), Dst Port: microsoft-ds (445), Seq: 1, Ack: 1, Len: 51	
NetBIOS Session Service	
SMB (Server Message Block Protocol)	
SMB Header	
Negotiate Protocol Request (0x72)	
Word Count (WCT): 0	
Byte Count (BCC): 12	
Requested Dialects	
Dialect: NT LM 0.12	

2) 服务器响应信息中包含接下来使用的认证方式以及 16 位挑战随机数。这里使用了一种安全的认证机制: NT 挑战/响应机制, 即 NTLM。

112	10:58:10.456	4.1.64.12	4.0.2.11	SMB	105 Negotiate Protocol Request
113	10:58:10.456	4.0.2.11	4.1.64.12	SMB	181 Negotiate Protocol Response
114	10:58:10.457	4.1.64.12	4.0.2.11	SMB	268 Session Setup Andx Request, User: ?\admin; Tree Connect Andx, Path: \\4.0.2.11\huawei
115	10:58:10.459	4.0.2.11	4.1.64.12	SMB	296 Session Setup Andx Response; Tree Connect Andx
116	10:58:10.459	4.1.64.12	4.0.2.11	SMB	154 Trans2 Request, FIND_FIRST2, Pattern: \userb*
117	10:58:10.459	4.0.2.11	4.1.64.12	SMB	330 Trans2 Response, FIND_FIRST2, Files: ...
118	10:58:10.460	4.1.64.12	4.0.2.11	SMB	95 Find Close2 Request
119	10:58:10.460	4.0.2.11	4.1.64.12	SMB	93 Find Close2 Response

Frame 113: 181 bytes on wire (1448 bits), 181 bytes captured (1448 bits) on interface 0	
Ethernet II, Src: HuaweiTe_e8:75:de (f8:4a:bf:e8:75:de), Dst: HuaweiTe_a2:3b:7b (70:54:f5:a2:3b:7b)	
Internet Protocol Version 4, Src: 4.0.2.11 (4.0.2.11), Dst: 4.1.64.12 (4.1.64.12)	
Transmission Control Protocol, Src Port: microsoft-ds (445), Dst Port: 10137 (10137), Seq: 1, Ack: 52, Len: 127	
NetBIOS Session Service	
SMB (Server Message Block Protocol)	
SMB Header	
Negotiate Protocol Response (0x72)	
Word Count (WCT): 17	
Selected Index: 0: NT LM 0.12	
Security Mode: 0x03	
... ..1 = Mode: USER security mode	
... ..1. = Password: ENCRYPTED password, Use challenge/response	
... ..0.. = Signatures: Security signatures NOT enabled	
... ..0... = Sig Req: Security signatures NOT required	
Max Mpx Count: 50	
Max VCS: 1	
Max Buffer size: 16644	
Max Raw Buffer: 65536	
Session Key: 0x00000000	
Capabilities: 0x0001e3fc	
System Time: Sep 12, 2014 10:58:33.803746600	
Server Time Zone: -480 min from UTC	
Key Length: 8	
Byte Count (BCC): 54	
Encryption Key: 00d1c20882204c11	

过程大致如下:

1. 服务器产生一个 16 位随机数字发送给防火墙, 作为一个挑战随机数。
2. 防火墙使用散列算法生成用户密码的哈希值, 并对收到的挑战随机数进行加密。同时将自己的用户名, 使用明文传输, 一并返回给服务器。
3. 服务器将用户名、挑战随机数和防火墙返回的加密后的挑战随机数, 发送给域控制器。
4. 域控制器按用户名在密码管理库中找到用户密码的哈希值, 也用来加密挑战随机数。域控制器比对两个加密的挑战随机数, 如果相同, 那么认证成功。

认证通过后就是访问指定的文件或文件夹，暂不赘述。

综上，我们可以看出防火墙在文件共享功能中的作用其实就是代理，作为远程用户和 SMB server 的中介：在 HTTPS 阶段，作为 Web Server 接收远程用户的文件访问请求，并翻译为 SMB 请求；在 SMB 阶段，作为 SMB Client 发起请求、接收应答，并翻译给远程用户。有了文件共享功能，远程用户访问内网文件服务器就像访问普通 Web 网页一样方便，不用安装文件共享客户端、不用记服务器的 IP 地址，也不会众多服务器中迷航。

看似简单的文件共享菜系，没想到包含了这么多道工序：接收请求 -> 翻译并转发请求 -> 接收响应 -> 翻译并转发响应，防火墙如同技艺精湛的厨师，精心料理每一个环节，现在文件访问这道大餐已经火热出炉，那么对于另外一种常用服务：Web 访问服务，防火墙又是如何发挥作用的呢？

口味：对 URL 的访问——菜系：Web 代理功能

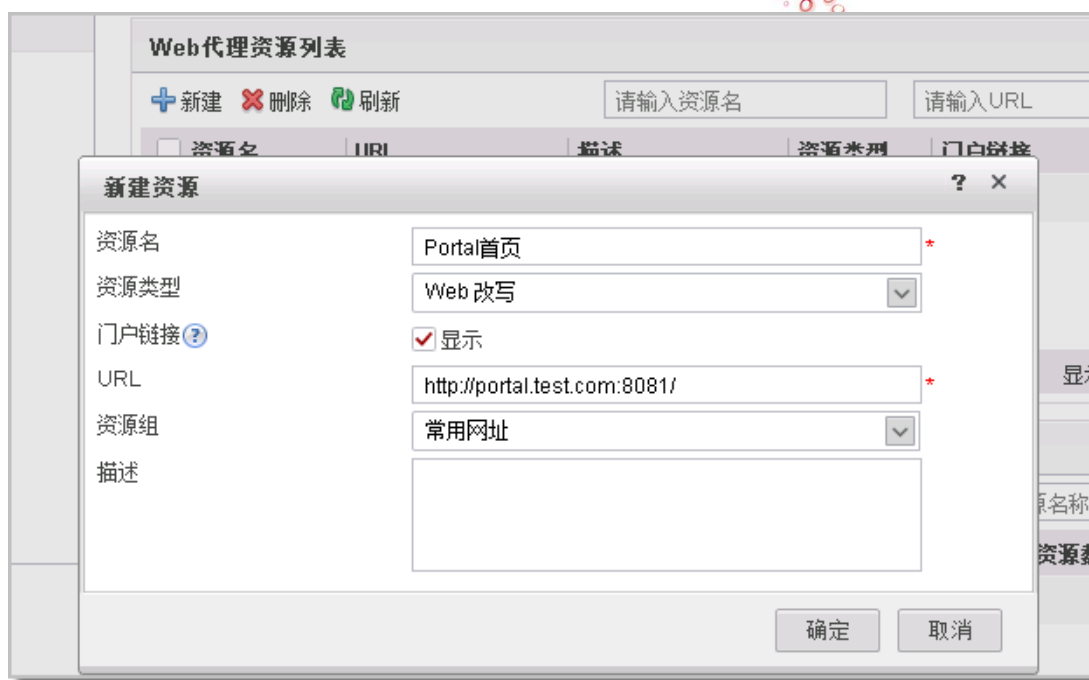
菜系简介：尽管都是对象级别的资源访问，但 URL 与文件共享不同，访问 URL 本身使用的就是 HTTP 协议，而 SSL 协议生来就和 HTTP 是天生一对，因此在 Web 代理功能中，已经不再涉及协议转换的内容。但是我们还是要围绕两个最核心的内容进行阐述：URL 级别的访问控制和隐藏真实的 URL 地址。

Web 代理，菜如其名，也就是通过防火墙做代理访问内网的 Web 服务器资源，也就是 URL。说到这有的朋友可能会问了，这不就是普通的代理功能吗：使用一台服务器做跳板访问目的 URL 地址，这台服务器就是起的代理作用，防火墙跟这个实现一样吗？答案是不完全一样，防火墙在整个过程中不仅做了代理，而且对真实的 URL 进行了改写处理，从而达到隐藏真实的内网 URL 的目的，进一步保护了内网 Web 服务器的安全。

准备 Web 代理资源菜单（配置 Web 代理资源）

假设企业已架设好 Web 服务器，并对企业内网提供了 Portal 门户地址：<http://portal.test.com:8081/>，希望通过 Web 代理功能为远程用户提供访问。

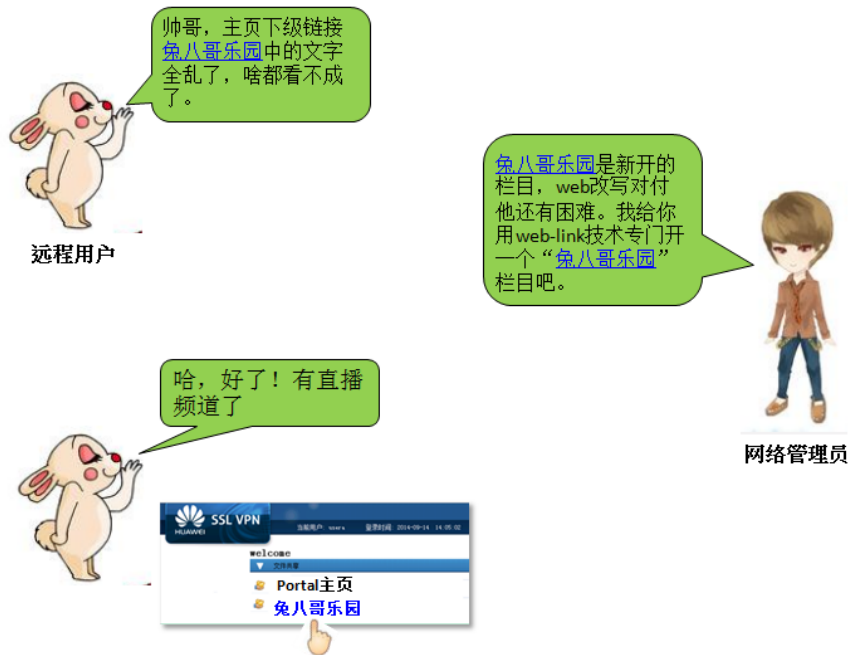
与文件共享资源一样，为了细化访问控制粒度到 URL 级别，需要在虚拟网关中配置相应的 Web 代理资源。



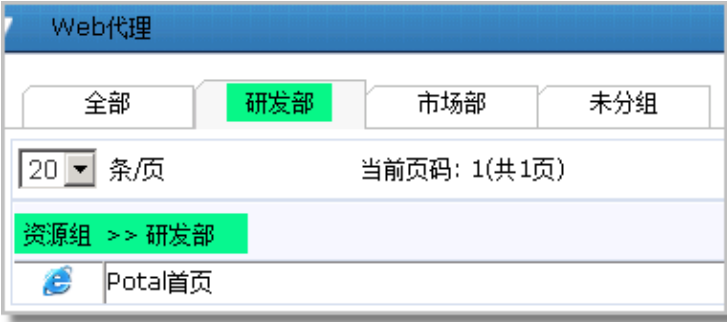
在以上配置中，最重要的参数要属资源类型，它定义了 Web 代理方式。代理方式包括 Web 改写和 Web-Link，二者的差异请参见下表：

对比项	Web 改写	Web-Link
安全性	对真实的 URL 进行改写,隐藏内网服务器地址, 安全性较高。	对 URL 不会进行改写, 直接转发 Web 请求和响应, 会暴露内网服务器的真实地址
易用性	不依赖 IE 控件, 在非 IE 环境的浏览器中可以正常使用。	依赖 IE 控件, 在非 IE 环境中无法正常使用。
兼容性	由于 Web 技术发展非常迅速, 防火墙对于各类 URL 资源的改写无法做到面面俱到, 可能会出现图片错位, 字体显示不正常等问题。	无需对资源进行改写, 由防火墙直接对请求和响应进行转发, 所以没有页面兼容性的问题。
使用建议	优先推荐使用 Web 改写, 因为这是最安全、最方便的一种访问方式。如果出现页面显示异常, 再考虑 Web-Link 方式。	Web-Link 作为 Web 改写的最佳替补, 但由于依赖 IE 控件, 必然在使用上存在局限性。而且没有对内网 URL 进行改写, 存在安全风险。

看到这里大家知道什么时候用 Web-Link 了么？



在下表中强叔还列出了其他一些参数的含义：

参数	说明
URL	内网可以直接访问的 Web 应用地址。如果是域名形式的话，那么需要在虚拟网关中配置相应的 DNS 服务器地址。
资源组	相当于 Web 应用地址的自定义分类，远程用户登录后可以通过资源组筛选需要的资源，好比菜单中的主食、酒水分类一样。 
门户链接	选择 Web 代理资源是否显示在登录后的虚拟网关首页上。如果不选中，相当于为老顾客准备了菜单之外的私房菜。这时老用户可以在登录后的右上角地址栏中手动输入 URL 地址，访问一些比较机密的 URL 资源。 



Web 改写究竟是如何工作的，强叔会带着大家一起继续学习。对于 Web-Link 这里先埋个小小的伏笔，强叔会在后续的帖子会重点提到它。

远程用户轻松点菜（对 URL 地址的改写）

对于前面已经配置的 Web 代理资源 URL <http://portal.test.com:8081/>，给用户实际呈现的 URL 地址我们可以看到已经被改写成：

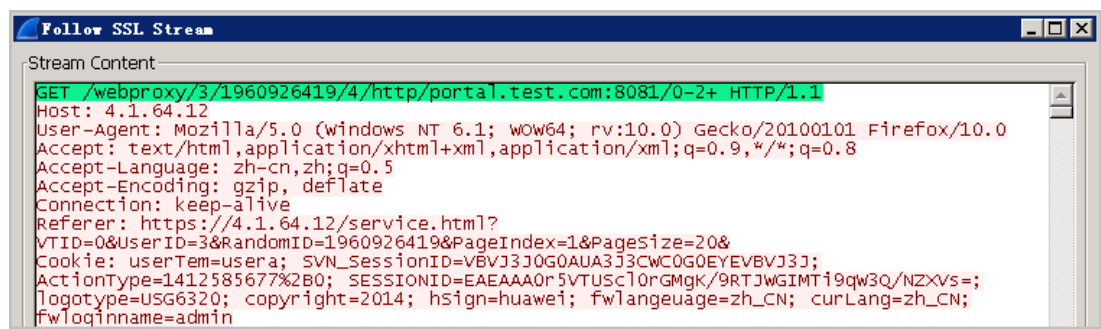


对改写结果进行逆向分析，其中 4.1.64.12 为虚拟网关地址，剩余部分大致可以分为：

1. **webproxy**: Web 代理的专属目录。
2. **1/1412585677/4**: UserID/SessionID/ResourceID，这几个参数在介绍文件共享的时候已经提到过。
3. **http/portal.test.com:8081/0-2+**: 原始 URL 地址的变形。

当用户访问改写后的地址时，发生了如下交互。

- 1) 远程用户向防火墙请求改写后的 URL 地址。



请求报文到达防火墙之前均为加密状态，上图是经过解密处理后的，所以也可以理解为防火墙收到的真实请求。

- 2) 防火墙对收到的报文进行解密后，向内网服务器发送请求之前，继续对原始报文作如下处理：

需要删除原始报文头中的 Accept-Encoding 字段, 否则 Web 服务器可能会将响应报文加密发给虚防火墙, 而防火墙对其无法解密处理, 无法进行进一步的转发。在下图中, 可见防火墙已经将原始报文中的 Accept-Encoding 字段删除了。

将 host 字段替换为真实的内网 Web 资源地址。

```

91 22:03:53.592 4.1.64.12 4.0.2.11 HTTP 567 GET / HTTP/1.1
92 22:03:53.593 4.0.2.11 4.1.64.12 HTTP 265 HTTP/1.1 304 Not Modified
669 22:03:53.654 4.1.64.12 4.0.2.11 HTTP 512 GET /static/img/retweet.png HTTP/1.1
670 22:03:53.654 4.0.2.11 4.1.64.12 HTTP 948 HTTP/1.1 200 OK (PNG)
701 22:03:53.857 4.1.64.12 4.0.2.11 TCP 60 10030 -> sunproxysadmin [ACK] Seq=972 Ack=1106 win=3276
702 22:03:53.858 4.1.64.12 4.0.2.11 HTTP 473 GET /static/img/favicon.ico HTTP/1.1

Frame 91: 567 bytes on wire (4536 bits), 567 bytes captured (4536 bits) on interface 0
Ethernet II, Src: HuaweiTe_a2:39:bb (70:54:f5:a2:39:bb), Dst: HuaweiTe_e8:75:de (f8:4a:bf:e8:75:de)
Internet Protocol Version 4, Src: 4.1.64.12 (4.1.64.12), Dst: 4.0.2.11 (4.0.2.11)
Transmission Control Protocol, Src Port: 10030 (10030), Dst Port: sunproxysadmin (8081), Seq: 1, Ack: 1, Len: 513
Hypertext Transfer Protocol
GET / HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET / HTTP/1.1\r\n]
Request Method: GET
Request URI: /
Request Version: HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:10.0) Gecko/20100101 Firefox/10.0\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: zh-cn,zh;q=0.5\r\n
Connection: keep-alive\r\n
Cookie: SESSIONID=EAEAA0r5VTUScl0rGMgK/9RTJWGIMT19qW3Q/NZXvs=; logotype=USG6320; copyright=2014; hsign=huawei; fwla
If-None-Match: "e3fb613ddfd0cf1:0"\r\n
If-Modified-Since: Mon, 15 Sep 2014 12:19:04 GMT\r\n
Host: portal.test.com:8081\r\n
\r\n
[Full request URI: http://portal.test.com:8081/]
[HTTP request 1/3]

```

修改与此 Web 资源相关的一些 URL 的 Referer 字段为真实的内网 Web 资源地址。

```

669 22:03:53.654 4.1.64.12 4.0.2.11 HTTP 512 GET /static/img/retweet.png HTTP/1.1
670 22:03:53.654 4.0.2.11 4.1.64.12 HTTP 948 HTTP/1.1 200 OK (PNG)
702 22:03:53.858 4.1.64.12 4.0.2.11 HTTP 473 GET /static/img/favicon.ico HTTP/1.1
703 22:03:53.860 4.0.2.11 4.1.64.12 HTTP2 1454 HEADERS, HEADERS, Unknown type (22), Unknown type (1
710 22:03:53.894 4.1.64.12 4.0.2.11 HTTP 509 GET /static/img/icon.png HTTP/1.1
714 22:03:53.895 4.1.64.12 4.0.2.11 HTTP 508 GET /static/img/app.png HTTP/1.1
715 22:03:53.895 4.0.2.11 4.1.64.12 HTTP 771 HTTP/1.1 200 OK (PNG)

Frame 669: 512 bytes on wire (4096 bits), 512 bytes captured (4096 bits) on interface 0
Ethernet II, Src: HuaweiTe_a2:39:bb (70:54:f5:a2:39:bb), Dst: HuaweiTe_e8:75:de (f8:4a:bf:e8:75:de)
Internet Protocol Version 4, Src: 4.1.64.12 (4.1.64.12), Dst: 4.0.2.11 (4.0.2.11)
Transmission Control Protocol, Src Port: 10030 (10030), Dst Port: sunproxysadmin (8081), Seq: 514, Ack: 212, Len: 458
Hypertext Transfer Protocol
GET /static/img/retweet.png HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET /static/img/retweet.png HTTP/1.1\r\n]
Request Method: GET
Request URI: /static/img/retweet.png
Request Version: HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:10.0) Gecko/20100101 Firefox/10.0\r\n
Accept: image/png,image/*;q=0.8,*/*;q=0.5\r\n
Accept-Language: zh-cn,zh;q=0.5\r\n
Connection: keep-alive\r\n
Referer: http://portal.test.com:8081/\r\n
Cookie: SESSIONID=EAEAA0r5VTUScl0rGMgK/9RTJWGIMT19qW3Q/NZXvs=; logotype=USG6320; copyright=2014; hsign=huawei; fwla
Host: portal.test.com:8081\r\n
\r\n
[Full request URI: http://portal.test.com:8081/static/img/retweet.png]
[HTTP request 2/3]
[Prev request in frame: 91]
[Response in frame: 670]

```

3) 防火墙作为 Web Client 将改写后的数据发送给真实的 Web Server。

接下来就是正常的 HTTP 交互了, 此处不再赘述。

对 URL 中资源路径的改写

防火墙接收到的应答报文, 也就是需要呈现给用户的页面(以首页 <http://portal.test.com:8081/> 为例), 对于页面中的一些资源路径也需要进行改写, 如果不对资源路径进行改写, 客户端就会使用错误/不存在的地址获取资源, 最终导致相应的内容无法正常显示。目前防火墙支持对

如下页面资源进行改写：

- HTML 属性
- HTML 事件
- JavaScript
- VBScript
- ActiveX
- CSS
- XML

防火墙可以对这些资源的内部路径进行改写，目的是为了页面的正常显示和功能的正常使用。

对 URL 包含的文件改写

其实在上一小节已经对文件的改写已经作了一部分介绍，但均是基于所请求页面的资源改写，也就是说所改写的内容是用户无需感知的，用户所关心的是页面是否能正常显示，Web 的功能是否正常。而接下来要说的，正是用户切实关心的文件，包括 PDF、Java Applet 和 Flash。

以 PDF 为例，我们将 a.pdf 内嵌到 <http://portal.test.com:8081/> 中，以链接的形式供用户下载使用。PDF 中的内容如下，包括只有在内网可以访问的链接（<http://support.test.com/enterprise>），如果防火墙不对其进行改写，接入用户打开下载后的 PDF 文件并访问其中的链接，会无法访问。

介绍如何获取华为技术有限公司 Huawei Technologies Co., Ltd. 的技术支持。

华为技术有限公司 Huawei Technologies Co., Ltd. 提供了丰富的技术支持渠道。如果您是我们的客户，并且拥有[技术支持网站](http://support.test.com/enterprise)账号，您可以在线访问我们的工具和资源。

- 查阅产品文档：进入[技术支持网站](http://support.test.com/enterprise)，在产品支持版块中选择产品。
- 下载最新的软件版本，查阅版本文档：进入技术支持网站，在软件下载版块中选择产品。
- 在知识库中查找答案：进入技术支持网站，在快速链接中选择知识库。

而通过虚拟网关下载 Web 代理资源中的 PDF 文件，本地打开后显示如下，可见文件中原来的内网 URL 已经被改写，改写后的 URL 为虚拟网关地址开头，这样外网用户就可以访问内嵌在 PDF 文件中的内网资源。



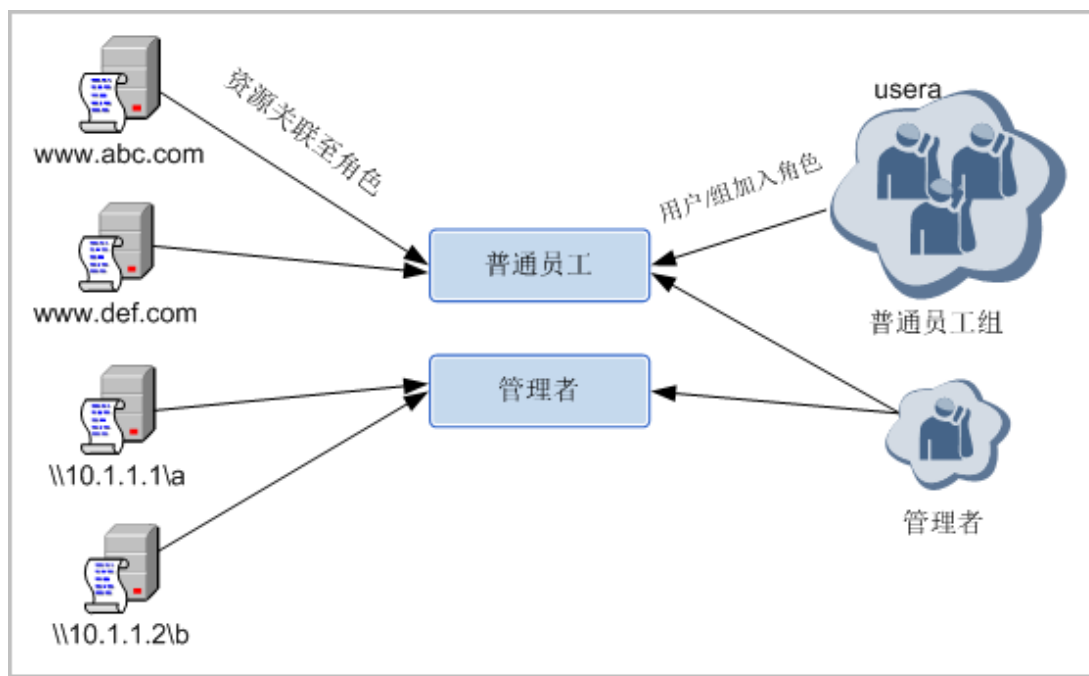
华为技术有限公司 Huawei Technologies Co., Ltd.提供了丰富的技术支持渠道。如果您是我们的客户，并且拥有[技术支持网站](#)账号，您可以在线访问我们的工具和资源。

- 查阅产品文档：进入技术支持<https://4.1.64.12/webproxy/7/170957690/4/http/support.test.com/0-0+enterprise>选择产品。
- 下载最新的软件版本，查阅版本文档：进入技术支持网站，在软件下载版块中选择产品。
- 在知识库中查找答案：进入技术支持网站，在快速链接中选择知识库。

定制化菜单服务-角色授权

企业管理员可以为不同的用户定制不同的“菜单”，实现细粒度的资源访问控制。在华为防火墙中，对多个资源的访问控制是通过角色授权完成的，一个角色中的所有用户拥有相同的权限。角色是连接用户/组与业务资源的桥梁，管理员可以将权限相同的用户或组加入到某个角色，然后在角色中关联业务资源。

如下图所示，角色中可以包含多个用户/组，同时还可以关联多个业务资源项。



普通员工 usera 登录后的资源界面： 管理者 master 登录后的资源界面：



说明：由于管理者 master 加入了两个角色中，所以看到的菜单是两个角色的合集。

角色授权的配置界面如下所示，其中角色具体可以关联的控制项如下：

业务授权：指定角色内用户可以使用的业务，包括 Web 代理、文件共享，以及后面将要介绍的端口转发和网络扩展。

资源授权：在启用某个业务的前提下，指定具体可以访问的资源。如果不指定具体资源，角色内用户无法访问任何资源。

普通员工角色配置：

管理者角色配置：



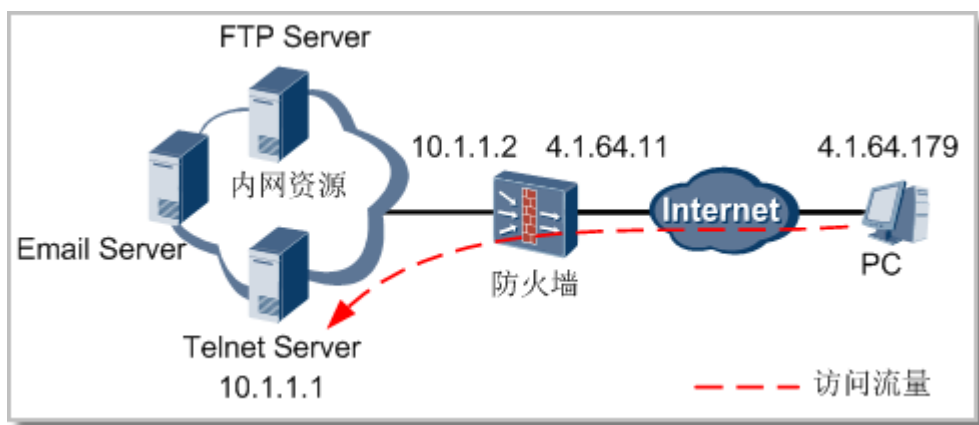
本篇中，强叔介绍了 SSL VPN 时尚餐厅中两个热销的菜系：文件共享和 Web 代理，其中详细介绍了两种菜系各自不同的料理工序—隐藏或改写 URL。另外针对不同口味的顾客，还推出了定制化菜单服务--角色授权。文件共享和 Web 代理各自对应了文件和 URL 这两类最细粒度的资源访问方式，而且均是基于 Web 进行访问。但是在有些远程办公场景中，使用的是 Non-Web 的应用方式，例如 Telnet，所以在下一篇中，强叔将对此类访问需求进行详细介绍。

🍁 端口转发又一村

上一篇贴子中，强叔介绍了文件和 URL 这两种最为常见的细粒度资源在 SSL VPN 中的访问方法。但是在有些情况下，例如对于 Telnet、SSH、Email 等基于 TCP 的非 Web 应用的访问，文件共享和 Web 代理两个功能便无法满足。此时管理员需要考虑其他合理的方案，要求既能让远程用户无障碍（不依赖 Web）访问内网的应用，又不能像 L2TP VPN 那样直接将远程用户置于内网，产生安全隐患。

我们第一篇中介绍过，SSL VPN 的特色及优势便是可以借助 Web 覆盖所有的远程访问需求，而现在这些非 Web 的应用使得原本基于 Web 的 SSL VPN 看似束手无策了。但事实并非如此，在本篇技术贴中，强叔就是要带着大家跨过山和大海，共同见证 SSL VPN 展示出的神奇力量——端口转发功能。

端口转发，一言以蔽之，就是端口转发客户端程序在远程用户侧获取用户的访问请求，再通过虚拟网关转发到内网相应的服务器。接下来以最常见的 Telnet 访问为例，介绍远程客户端通过 SSL VPN 访问内网的 Telnet 应用。



端口转发业务的配置如下，与前面介绍的 SSL VPN 其他功能一样，无论访问什么样的应用，首先都要在虚拟网关中添加相应的资源，对 Telnet 而言，只需要在虚拟网关中配置 Telnet 服务器的 IP 地址和端口即可。

新建资源

资源名	Telnet	*
主机地址类型	主机IP地址	
主机IP地址	10.1.1.1	*
端口	23	*≤0-65535>
资源描述	Telnet应用	

远程用户在登录后的虚拟网关界面中，可以选择手动启用端口转发功能，也可以设置为登录后客户端自动启用，总之全凭虚拟网关一句话；除了可以自动启动端口转发功能，还可以选择是否保持端口转发长连接，这是由于有些应用的访问持续时间较长（例如管理员正操作 Telnet 过程中突然要离开一段时间），选中后可以防止因 SSL 连接超时断开而中断端口转发业务。

配置端口转发

端口转发功能 ☒ 启用

客户端自动启用 ☐ 启用

保持连接 ☒ 启用

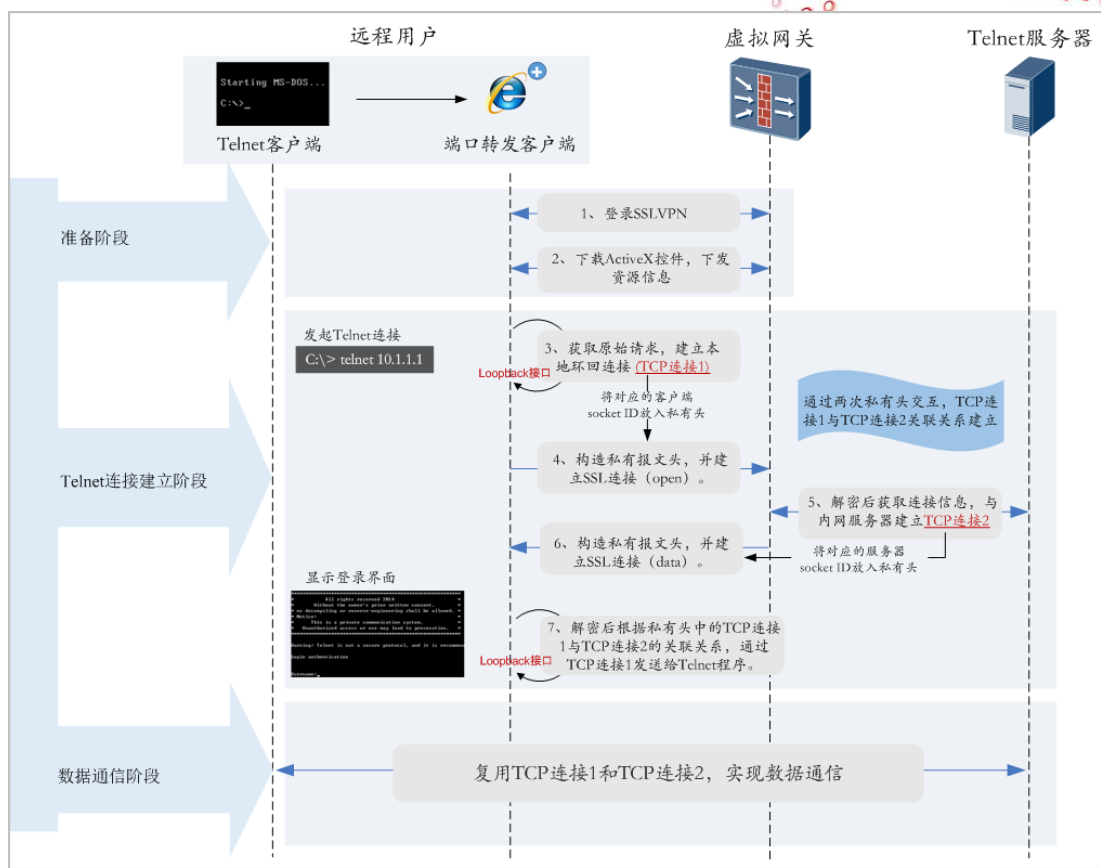
端口转发资源列表

+ 新建 × 删除 复制 刷新

请输入资源名 查询 高级查询

资源名	主机名/IP地址	端口	资源描述	编辑
telnet	10.1.1.1	23		

端口转发的数据处理流程相对复杂，强叔这里给出一张简图，后文将对各个阶段逐一介绍。



说明: 这里的端口转发客户端包含了 SSL VPN 客户端功能, 只是为了强调端口转发业务才特地如此称谓。

准备阶段

1. 登录 SSL VPN

此过程在 SSL VPN 第一篇贴子《远程接入 IPSec 现短板, SSL VPN 登上历史舞台》中已经介绍, 不再赘述。

另外需要大家知悉, 在上一篇贴子中强叔是围绕 URL 展开分析的, 而端口转发的分析将有所不同, 尽管也登录了虚拟网关, 但因为是 Web 类的应用访问, 所以对应的资源访问也不再使用 Web, 而是借助其他应用程序, 如 Putty (Telnet/SSH 工具)、Filezilla (FTP 工具)、Foxmail (邮件程序) 等。这时问题也随之而来了, 非 web 应用程序是如何利用已经登录的 SSL VPN 连接的呢?

2. 顺风耳收到指令, 进入聆听状态

使用非 web 应用程序进行数据访问时, 看似与 SSL VPN 没什么关系, 但实际上, 端口转发的关键的技术点已在此恭候多时: 用户使用 Windows 系统的 IE 浏览器登录虚拟网关后, 会在本地 PC 的 IE 浏览器上自动运行端口转发客户端 (ActiveX 控件)。这个客户端的作用就

是像顺风耳一样时刻“聆听”其他程序的所有请求，并在千钧一发之际将远程用户发给内网服务器的请求“拦截”下来，然后再通过 SSL 连接发送给虚拟网关。对于“聆听”到的请求，选择哪些请求进行拦截，这位顺风耳并不是一意孤行，是根据虚拟网关这位上级的指示来严格执行的。那么指令是什么呢？

前面配置的端口转发资源，实际上就是虚拟网关给端口转发客户端下发的指令——“有用户要访问这些资源，你协助他们完成访问任务”。在端口转发功能中，下发的指令便是**目的主机 IP 地址+目的端口**，以上信息可以唯一确定远程用户要访问的应用信息。

如下图所示，远程用户手动启用端口转发功能后，端口转发客户端会自动向虚拟网关请求资源信息，客户端请求到的资源信息会保存在远程用户 PC 的内存中，随时待命，便于在接下来选择“拦截”哪些请求。



为了不暴露内网服务器的地址，具体资源信息无法在端口转发客户端上进行检查，菜单中的资源也无法直接点击，只有简单的提示作用。不过我们可以间接通过查看端口转发客户端日志（存放路径为%appdata%\Roaming\svnclient\log），了解资源信息下发的过程。

```
2014-10-13 9:21:48.647 CPM_SynServe:receive 1 resources from GateWay
```

Telnet 连接建立阶段

3. 顺风耳精准拦截，施展代理绝活

在顺风耳的脑海中，已经明确要“拦截”哪些请求了，当下要做的就是竖起耳朵，聆听自己关心的内容。当用户使用 Telnet 程序对 10.1.1.1 的 23 端口请求连接时（一个 TCP SYN 报文），顺风耳发现与上级虚拟网关下发的资源信息（目的 IP+目的端口）匹配，立即将此 TCP SYN 报文“拦截”。如果按照普通的人的做法这时就可以将请求报文发给虚拟网关交差了，但这里顺风耳考虑到如果不经处理便发送给虚拟网关，即会导致每个 Telnet 请求（即每条 TCP 连接）都会对应建立一条新的 SSL 连接，这样做会不但会占用过多的系统资源，而且响应速

度也会有延迟。为了节省虚拟网关的会话和内存资源，提高用户体验，顺风耳决定先乔装打扮成接收方，模拟接收一次 Telnet 业务请求（TCP 连接），弄清楚用户需要访问什么样的资源，采用“集中代理的方式，一条 SSL 连接搞定”的战术，减轻“上级”（虚拟网关）的压力。

如何模拟接收 Telnet 业务？如何集中代理呢？顺风耳这个优秀的代理业务员自有妙计：

端口转发收到 Telnet 请求后对报文进行改造，将原来要发送给 10.1.1.1 的请求改为发送给自己（127.0.0.1），这样就等于自己代替 Telnet 服务器接受了请求。不过，模拟归模拟，模拟的同时必须记录改造前后的对应关系（本地换回记录表），便于后续可以代替 Telnet 服务器应答真正的用户（4.1.64.179）。

本地环回记录表

改造项目	改造前	改造后
源 IP 地址	4.1.64.179	127.0.0.1
源端口	随机	随机
目的 IP 地址	10.1.1.1	127.0.0.1
目的端口	23	1047 (23+1024)

端口转发客户端与自己建立了 TCP 连接 1(也叫本地环回连接),使用 netstat 命令验证如下。

```
C:\Users\>netstat -anp tcp
```

活动连接

协议	本地地址	外部地址	状态
TCP	127.0.0.1:1047	0.0.0.0:0	LISTENING
TCP	127.0.0.1:1047	127.0.0.1:7319	ESTABLISHED
TCP	127.0.0.1:5037	0.0.0.0:0	LISTENING
TCP	127.0.0.1:5999	0.0.0.0:0	LISTENING
TCP	127.0.0.1:7319	127.0.0.1:1047	ESTABLISHED

4. 构造私有报文头，提交“端口转发业务单”

模拟接收 Telnet 业务请求后，端口转发客户端对用户的请求已经了然于胸，按流程要求需要填写“端口转发业务单”提交上级领导。业务单中必须包括用户要请求的目的地址（10.1.1.1）和端口（23），以及命令字（建立连接、传输数据报文或关闭连接等），这样上级才能进一步处理。

此处需要注意：由于端口转发客户端自己模拟了接收方建立了 TCP 链接 1，所以业务单中必须对该 TCP 链接做一个标记（TCP 连接 1 的 socket ID，称为客户端 socket ID），只有这样当上级将业务受理结果返回时，端口转发客户端才能根据标记找到 TCP 连接 1，将返回结果

发给对应的 Telnet 客户端。

“端口转发业务单”在端口转发业务中被称为私有报文头（简称私有头）。这里仅以 Telnet 请求连接的报文为例，对私有报文头的主要字段进行说明。Telnet 连接建立阶段报文载荷为空，所以传输时只有私有报文头，在数据传输阶段报文才会有载荷。

字段名称	字段说明
用户标识	标识用户身份，虚拟网关自动为用户分配。可以理解为端口转发业务单号。
命令字	1. Open-新建连接 2. Data-数据命令 3. Close-关闭连接
业务类型	1. 端口转发 2. Web-Link Web-Link 实际上就是 HTTP/HTTPS 的端口转发业务，也就是说 Web-Link 资源，同样也可以配置为端口转发资源。但是请注意，在 Web-Link 资源中知名端口可以不指定，例如 http://www.huawei.com/，但是在端口转发的配置中端口号是必选，例如 HTTP 的知名端口号 80，HTTPS 是 443。
源 IP 地址	原始请求中的源 IP 地址，本例中为远程用户客户端的地址：4.1.64.179
目的 IP 地址	原始请求中的目的 IP 地址，本例中为内网 Telnet 地址：10.1.1.1
协议类型	目前仅支持 TCP 协议。
目的端口	原始请求中的目的端口，本例中为内网 Telnet 端口：23
客户端 socket ID	远程用户与自己建立连接使用的 socket ID，用于标识此次会话，后续的报文会继续使用这个 socket ID。
服务器 socket ID	防火墙作为 Telnet Client 与内网服务器建立连接使用的 socket ID，作用与客户端 socket ID 一样，均用来标识会话。

“端口转发业务单”整理完成后通过 SSL 连接加密发送给虚拟网关。

需要注意的是，此处建立的是一条专门用于端口转发业务的 SSL 连接，而非登录时已经建立好的 SSL 连接。当 Telnet 对其它资源发起访问时情况类似，建立新的 TCP 连接后，端口转发客户端也会再填写一份“端口转发业务单”，并共享这条 SSL 连接进行发送。这样，在客户端和虚拟网关之间会始终保持一条唯一的 SSL 连接。总之，所有的“端口转发业务单”都

将送往这条 SSL 连接，经过加密后再发往虚拟网关，大大减轻了虚拟网关的工作量。

5. 虚拟网关与内网服务器建立连接

虚拟网关收到加密后的报文，对其进行解密，在“端口转发业务单”中获取到 Telnet 真实的
目的 IP 地址和端口、命令字等信息，此时虚拟网关将作为 Telnet 客户端与内网服务器进行交
互建立 Telnet 连接。通过查看防火墙的会话表，可以得出防火墙随机启用了端口 10010 向
10.1.1.1:23 发起访问请求，建立了 TCP 连接 2。

```
Telnet VPN:public --> public 10.1.1.2:10010-->10.1.1.1:23
```

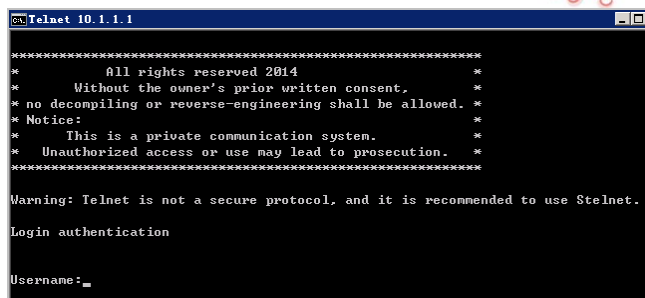
6+7 内网服务器响应报文返回 Telnet 客户端

虚拟网关收到内网服务器的响应报文（登录界面），在发给远程客户之前，虚拟网关依然会构造私有报文头， 填写 TCP 连接 2 的 socket ID（服务器 socket ID），这样便可以与 TCP 连接 1 建立对应关系。最终，虚拟网关把经过 SSL 加密后的私有报文头+数据发送给端口转发客户端；端口转发客户端根据私有头中客户端 socket ID 找到 TCP 连接 1，再根据本地环回记录表找到 Telnet 客户端真实 IP 地址，最终返回真实的数据。

端口转发客户端收到经 SSL 解密后的数据如下，从截图中标识的部分已经依稀看到登录页面的文字了，也就是 Telnet 数据报文，而上半部分的内容正是私有头的内容。



Telnet 客户端收到的 Telnet 登录界面如下。



数据通信阶段

后续的 Telnet 数据报文，会继续使用之前建立的 TCP 连接 1 和 TCP 连接 2，然后通过私有报文头将两个连接进行关联，最终打通“Telnet 客户端—端口转发客户端—虚拟网关—Telnet 服务器”之间的传输通道，实现数据通信。

Telnet 应用的端口转发流程就介绍到这里，Telnet 协议只是最简单的单通道协议，除此之外，端口转发还支持如下类型应用：

- 🍁 多通道协议，支持 FTP 和 Oracle SQL NET。在实际的配置中，对于 FTP 协议只需要指定控制通道的端口 21，协商后的数据端口会被自动“聆听”，无需额外配置。
- 🍁 多协议应用。对于有些应用，需要多个协议支持，例如 Email，需要在配置端口转发业务前，弄清楚使用发送协议（SMTP:25）和接收协议（POP3:110 或者 IMAP:143）的端口号，并为每一种协议配置一条端口转发资源。
- 🍁 多 IP 固定端口应用。例如 IBM Lotus Notes，对应的数据库存在于多个服务器上，但是对外提供服务时，均使用 1352 端口。这类应用在配置端口转发业务时，无需遍历配置所有服务器，只需要在“主机地址类型”中选择“任意 IP 地址”即可。

在实际应用中，端口转发的配置和使用都十分简单（SSL VPN 的特色嘛），但殊不知神功练就的背后夹杂了多少泪水，也许在实际的使用中用不到这些生僻的内容，但是相信有那么一瞬间，强叔会助你灵光一现，发现更大的世界。那么接下来，将进入 SSL VPN 的最后一篇，与 L2TP VPN 十分类似的三层 VPN 功能，将 SSL VPN 的资源访问控制，延伸无界……

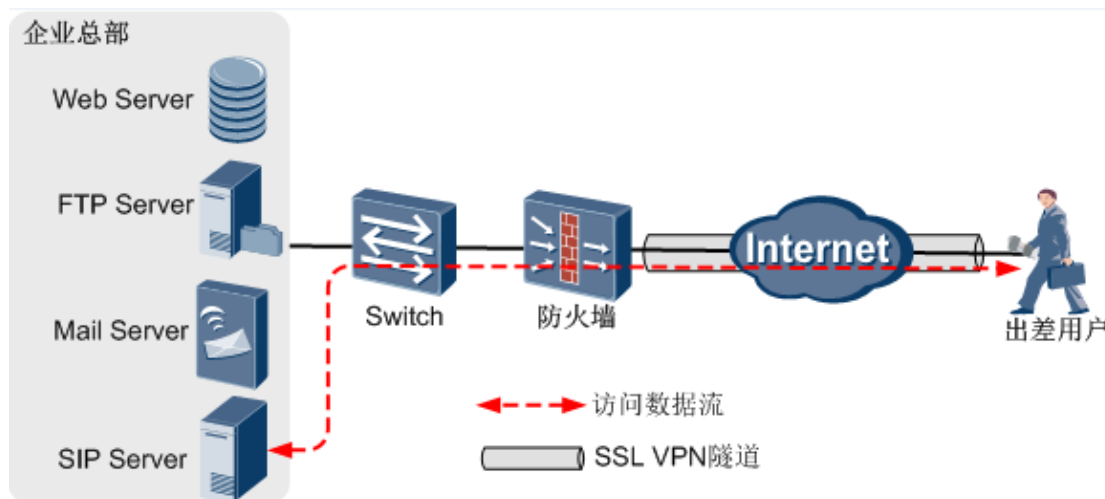
网络扩展惊鸿出世一统乾坤（上）

在 SSL VPN 开篇伊始，强叔说到 SSL VPN 提供了四种业务，Web 代理、文件共享、端口转发和网络扩展。前面的贴子已经对前三种业务做了详尽的介绍，今天强叔就带着各位小伙伴一起来认识一下这第四种业务---网络扩展。

话说吃饭用筷子，喝汤用勺子（此处为废话），讲的是“物有所归，各有其能”。SSL VPN 的这四大业务也是如此，用户要访问 Web 资源就要用 Web 代理业务；要访问文件资源，就要用文件共享业务。。。以此推理，好奇的小伙伴一定会产生几个问题，强叔今天要讲的网络扩展又会用在什么场景下呢？还有它的工作原理是什么样的？为什么这个业务要叫“网络扩展”呢？名字老让人费解了。。。或许小伙伴们的问题还很多，不过没关系，强叔会在本篇中对这些问题逐一进行解释。

资源访问类型受限，网络扩展化解难题

下图是出差用户访问企业内网资源的一个场景，即出差用户需要访问企业内部的语音服务器（SIP Server）参加电话会议。面对这样一个需求，SSL VPN 的前三个业务是否能够搞定呢？

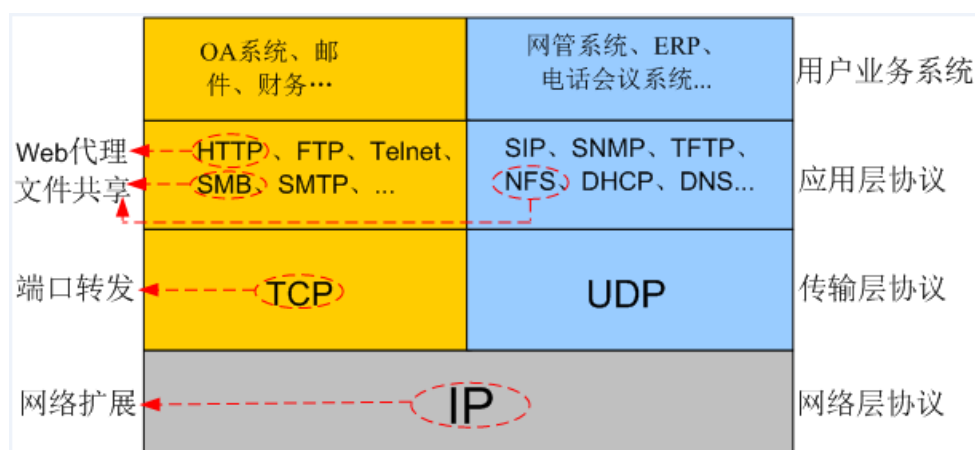


我们先来分析一下，出差用户要访问语音服务器，两者之间采用的是 SIP 协议通信，而 SIP 通常是基于 UDP 之上的一种应用协议。Web 代理、文件共享解决的是出差用户访问 Web 资源和文件资源的具体问题，这两个业务和语音资源就不沾边，所以肯定搞不定这个需求。那端口转发业务能不能解决这个问题呢？答案也是 No。原因在于端口转发只能解决基于 TCP 的应用协议，而 SIP 通常是基于 UDP 的协议，所以端口转发业务在这个需求面前它也是爱

莫能助。难道堂堂 SSL VPN 连这个需求都搞不定？非也，这就要用到我们今天讲的网络扩展业务了。

在防火墙上开启网络扩展业务就可以实现出差用户的这个需求了，因为网络扩展业务可以满足出差用户访问企业内网的全部 IP 资源，而上面需求中提到的基于 SIP 的语音资源就是 IP 资源的一种。

或许有些小伙伴对于网络扩展业务可以满足出差用户访问企业内网全部 IP 资源这个说法理解的还不透彻，这里强叔就结合下面的图再解释一下。

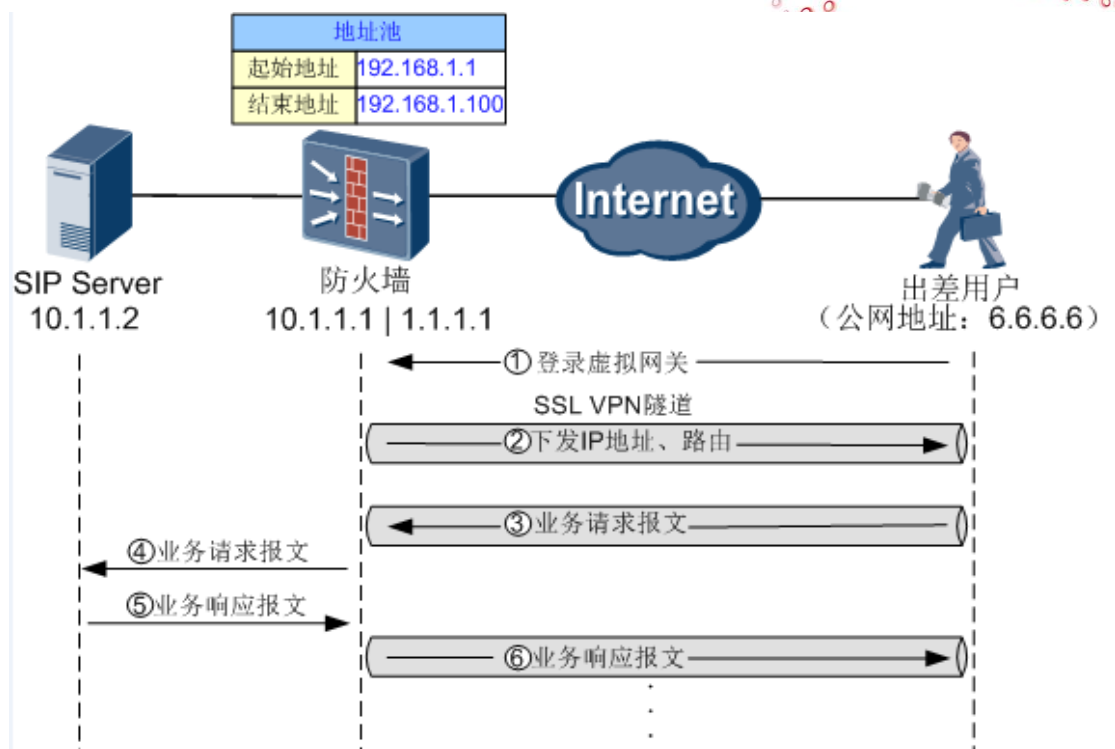


从上图可以看出，用户的业务系统种类繁多，不胜枚举。可是如果再深挖几层，我们就会发现，无论用户上层的业务系统有多少，它终究还是要依赖下层的协议为其提供通信支持，只是不同的业务系统所使用的底层的协议类型不同罢了。

Web 代理、文件共享它所支持的应用层协议是很具体的，比如 Web 代理就只能支持基于 HTTP 协议的应用；文件共享只支持 SMB、NFS 协议的应用；端口转发已经支持基于 TCP 协议的所有应用了。但有了端口转发业务也不代表 SSL VPN 就能包打天下，比如遇到基于 UDP 协议的一些应用它就鞭长莫及了（例如用户的电话会议系统使用的 SIP 协议是基于 UDP）。要使 SSL VPN 能够支持用户更多的应用，这就要求我们提供更低一层协议的支持，而网络扩展就是这么样一个功能，它一步到位直接支持到了 IP 层。所以，网络扩展业务提供给出差用户的资源类型更丰富。

本地 PC 自创虚拟网卡，出差用户巧获私网分身

出差用户使用网络扩展功能访问内网资源时，其内部交互过程如下图所示。



1. 出差用户通过IE浏览器登录虚拟网关。
2. 出差用户成功登录虚拟网关后启动网络扩展功能。

出差用户启动网络扩展功能，会触发以下几个动作：

- a. 出差用户与虚拟网关之间会建立一条SSL VPN隧道。
- b. 出差用户本地PC会自动生成一个虚拟网卡。虚拟网关从地址池中随机选择一个IP地址，分配给出差用户的虚拟网卡，该地址作为出差用户与企业内网之间通信之用。有了这个私网地址，出差用户就如同企业内网用户一样可以方便访问内网IP资源。
- c. 虚拟网关向出差用户下发到达企业内网Server的路由信息。

虚拟网关会根据网络扩展业务中的配置，向远程用户下发不同的路由信息，该路由可能会影响到出差用户的其他网络状态。具体如何影响，强叔会在后续的内容中有专门的讲解。

3. 出差用户向企业内网的Server发送业务请求报文，该报文通过SSL VPN隧道到达虚拟网关。
4. 虚拟网关收到报文后进行解封装，并将解封装后的业务请求报文发送给内网Server。
5. 内网Server响应出差用户的业务请求。
6. 响应报文到达虚拟网关后进入SSL VPN隧道。

出差用户收到业务响应报文后进行解封装，取出其中的业务响应报文。

以上就是出差用户通过网络扩展业务访问企业内网IP资源的基本过程，为了更深入的了解这个过程，强叔再结合上面的交互过程，讲一下业务请求报文进入VPN隧道以及从VPN隧道

出来以后报文的封装和解封装原理。

可靠传输模式万无一失，快速传输模式风驰电掣

网络扩展功能建立 SSL VPN 隧道的方式有两种：可靠传输模式和快速传输模式。可靠传输模式中，SSL VPN 采用 SSL 协议封装报文，并以 TCP 协议作为传输协议；快速传输模式中，SSL VPN 采用 QUIC（Quick UDP Internet Connections）协议封装报文，并以 UDP 协议作为传输协议。QUIC 也是基于 TLS/SSL 协议实现的数据加密协议，它的作用和 SSL 一样，只是经它封装的报文要基于 UDP 协议来传输。

图 1 是采用可靠性传输模式进行报文封装的过程。从图中可以看出，出差用户与企业内网(SIP Server)之间通信的源地址 (SRC: 192.168.1.1) 就是它虚拟网卡的 IP 地址。期间的报文交互经过往复的加解密之后安全到达通信双方。出差用户访问 SIP Server 时，内层报文的源端口是 5880 (随机)，目的端口是 5060，传输协议基于 UDP。外层报文的封装协议是 SSL，传输协议是 TCP。

图 1 可靠性传输模式下报文封装过程

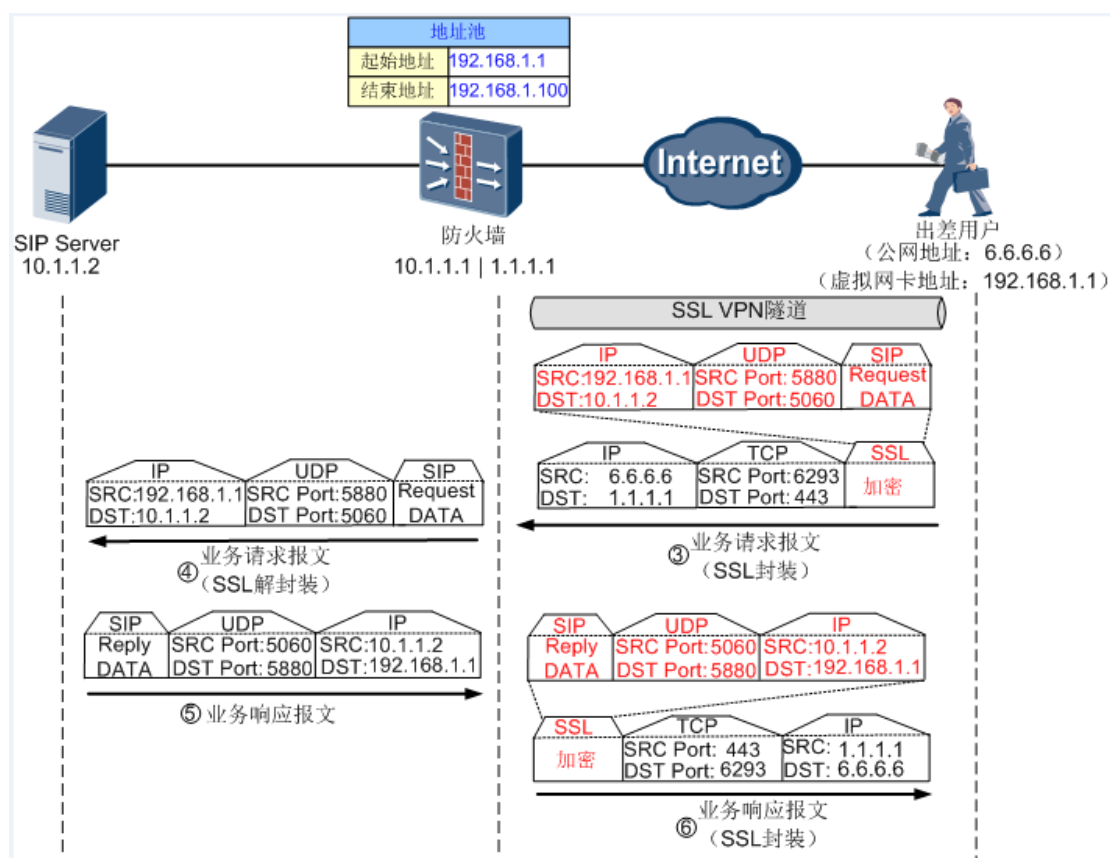
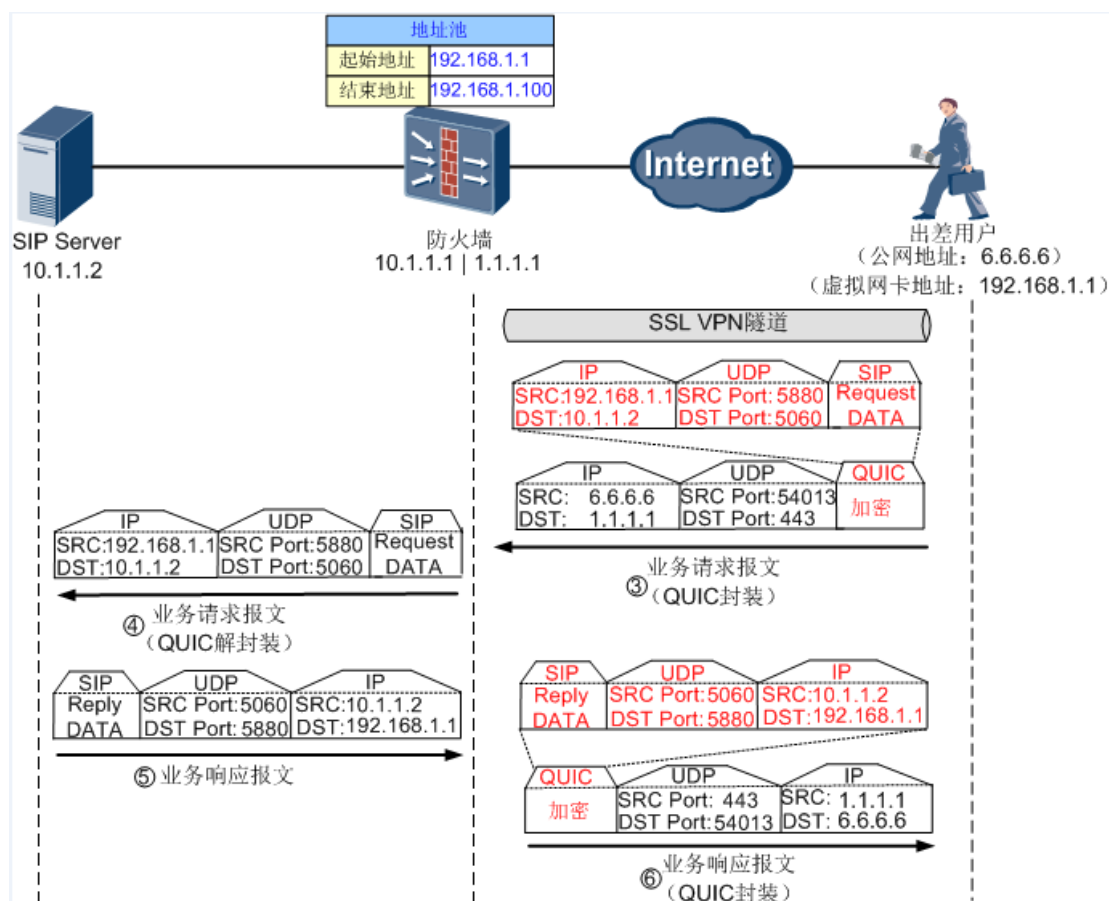


图 2 是采用快速传输模式进行报文封装的过程。该模式下报文的封装原理和可靠性模式下报

文封装原理是一样的，只是外层报文使用的封装协议由 SSL 改为了 QUIC，传输协议由 TCP 改为了 UDP。

图2 快速传输模式下报文封装过程



在网络环境不稳定的情况下推荐使用可靠性传输模式；而网络环境比较稳定的情况下，推荐使用快速封装模式，这样数据传输的效率更高。

路由表与安全域纠缠不清，内网段与地址池难解难分

光看上面这个副标题，想必小伙伴们大概也猜到了上面这几个东东之间肯定存在着一段不寻常的故事。不过小伙伴们不要多想啊，下面强叔要讲的是有关网络扩展业务中安全域间的事情。

出差用户通过网络扩展业务访问企业内网时，报文经过的安全域间有两个。

1. Untrust域到Local域。

出差用户访问企业内网 Server 前需要首先登录虚拟网关，即出差用户会和虚拟网关协商 SSL VPN 隧道，此时流量要经过 Untrust 域到 Local 域，因此配置网络扩展业务时要开放此域间策略。

2. “XXX”到Trust的域间策略，“XXX”表示源安全区域不固定。

SSL VPN 隧道建立好以后，出差用户将向 Server 发送业务请求报文。封装后的业务请求报文首先到达虚拟网关，虚拟网关会对该报文进行解封装。而解封装后的业务报文依然需要进行安全策略检查，那虚拟网关此时如何界定该业务报文的源安全域和目的安全域呢？

这个问题需要分两种情况来分析，一种是内网 Server 与虚拟网关地址池的地址在同一网段；另一种是两者地址不在同一网段。

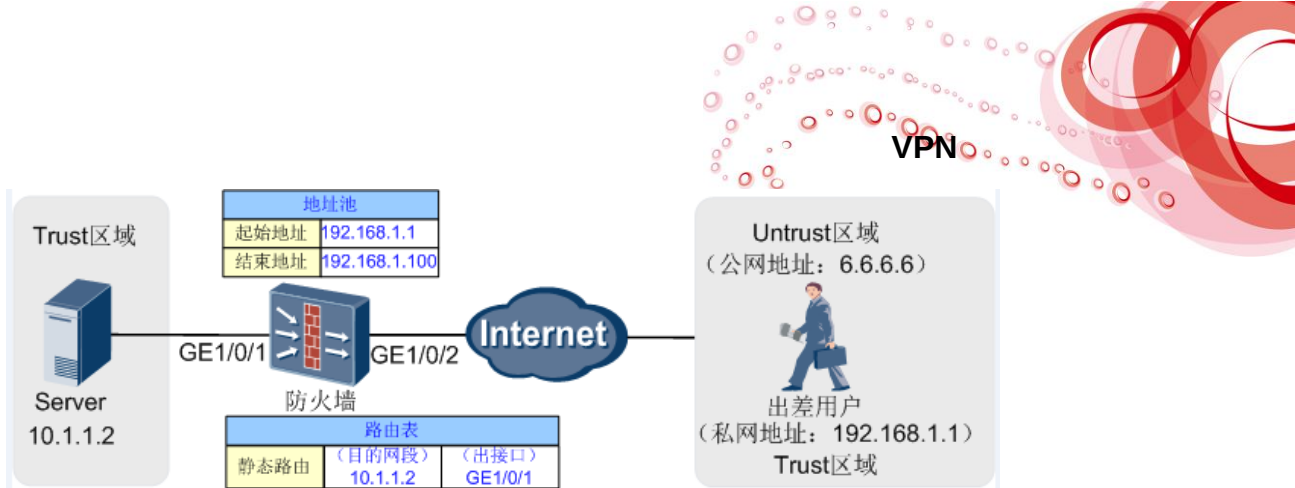
🍁 内网 Server 与虚拟网关地址池在同一网段



首先虚拟网关会以解封装后的报文(src:192.168.1.1, dst:192.168.1.200)源地址 192.168.1.1 作为目的地址查找路由表（也就是常说的**反查路由**），并以对应路由出接口所连接网络的安全域作为源安全域。我们知道，由于 Server 和地址池都在 192.168.1.0 这个网段中，因此虚拟网关上肯定存在一条到 192.168.1.0 的直连路由，出接口是 GE1/0/1。由此我们可以得出，解封装后的报文源安全域就是 GE1/0/1 接口所连网络的安全域----Trust 域。目的安全域很简单，就是虚拟网关直接以解封装后的报文目的地址 192.168.1.200 查找路由，然后找到出接口所连网络的安全域，从上图可以得知，其目的安全域也是 GE1/0/1 所连网络的安全域----Trust 域。

总结：在这种情况下，解封装后的报文所经过的安全域间就是 Trust 到 Trust，这个属于同一安全域。由于同一安全域的安全策略默认就是打开的，所以我们在配置网络扩展业务的安全策略时就不用再配置这条安全策略了。

🍁 内网 Server 与虚拟网关地址池不在同一网段



首先虚拟网关会以解封装后的报文（src:192.168.1.1，dst:10.1.1.2）源地址 192.168.1.1 作为目的地址查找路由表，并以对应路由出接口所连接网络的安全域作为源安全域。

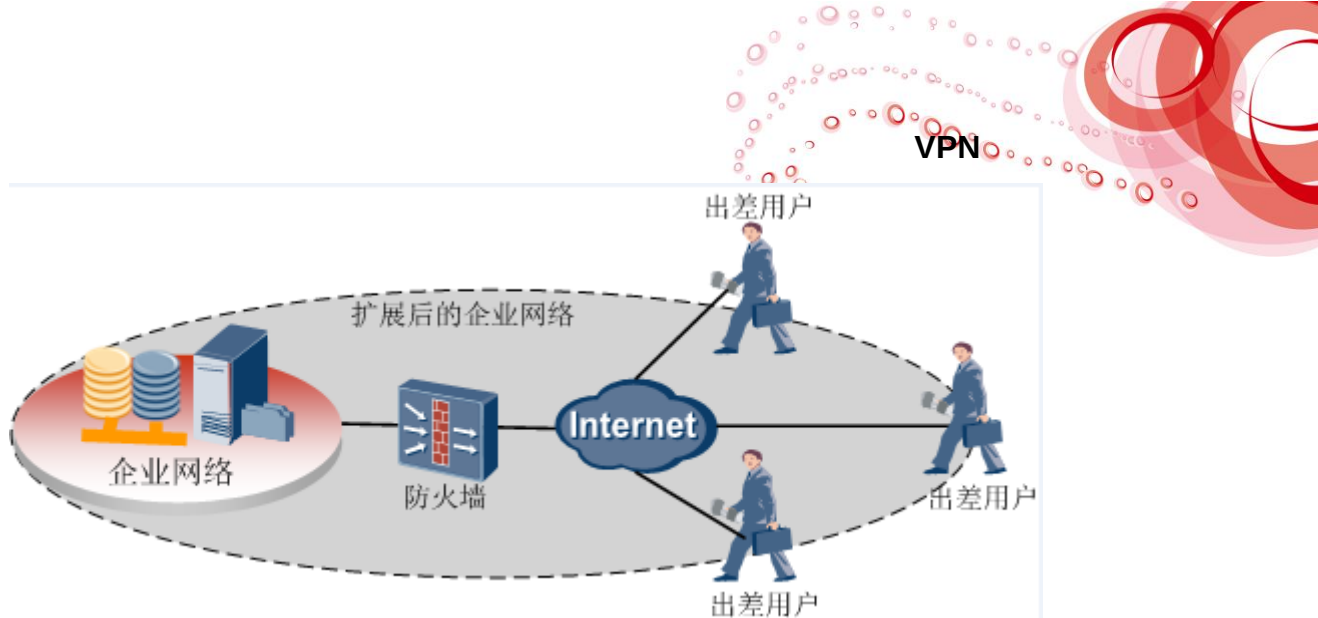
如果虚拟网关上没有到 192.168.1.0 这个网段的路由，系统就无法确定该报文的源安全域，从而导致丢包现象。为了解决这个问题，我们需要手工配置一条路由使系统能够查找到该报文的源安全域。

这条路由的目的地址就是地址池网段 192.168.1.0，出接口由管理员自行选择。也就是说此时这个报文的源安全域是什么，取决于我们配置的这条路由的出接口是哪个接口。通常我们认为这个报文是从 Internet 方向上过来的，来自于 Untrust 域。因此，我们在配置路由的时候，会把路由的出接口配置为与 Internet 相连的公网接口 GE1/0/2。目的安全域很简单，就是虚拟网关直接以解封装后的报文目的地址 10.1.1.2 查找路由，然后找到出接口所连网络的安全域，从上图可以得知，其目的安全域是 GE1/0/1 所连网络的安全域--Trust 域。因此，在这种情况下配置网络扩展业务时，要打开 Untrust 到 Trust 的域间策略。

总结：当内网 Server 与虚拟网关地址池不在同一网段时，需要配置一条目的地址是地址池网段，出接口为连接 Server 接口的路由。该路由仅用于确定安全域间关系，并不指导报文转发。网络扩展业务的使用场景以及它的实现机制在这里就讲完了，或许有的小伙伴看到这里对于这个功能为啥叫网络扩展似乎还有些困惑，强叔这里谈一下自己的理解。

对比一下网络扩展与其他三个 SSL VPN 业务的实现方式，我们不难发现，Web 代理、文件共享、端口转发这三个业务实现机制大体是相同的，即将企业网络的内部资源映射到防火墙上，然后再由防火墙呈现给出差用户。从这个角度来看，防火墙只是做了一个安全代理设备罢了，出差用户实际并没有接入到企业的内网中去。

而网络扩展有所不同，在网络扩展业务中，出差用户从防火墙中获取了一个企业内部的私网 IP 地址，并以该 IP 地址来访问企业网络的内部资源。Internet 上的用户有了企业的私网地址，用户本身就如同置身于企业网络内部一样。换个角度来看，就相当于企业网络的边界已经延伸到了出差用户那里，下图中灰色虚线边框所围成的区域就可以看做是企业网络在 Internet 上的延伸，所以这个业务叫网络扩展也就不难理解了。



很多小伙伴们看完网络扩展以后，大都会有这样的困惑，既然网络扩展这么厉害，那不管用户想访问什么类型的内网资源，直接为其开通网络扩展业务不就成了事，还要搞 Web 代理、文件共享啥的做什么？

这个问题是个关键，SSL VPN 之所以提供这么多不同层面、不同粒度的业务，就是为了控制出差用户对内网系统的访问权限，说到底就一个目的，为了安全。使用网络扩展业务以后，意味着这个出差用户就能访问企业内网所有类型的资源了，这虽然方便了用户，但也无疑会增大内网资源的管控风险。如何在满足用户需求的同时将权限控制做到恰到好处，这就需要根据用户的需求为其配置不同的业务了，从而规避上述问题。

有关网络扩展业务的配置以及各种业务之间的对比强叔会在网络扩展（下）中介绍，敬请期待。

网络扩展惊鸿出世一统乾坤（下）

两种 IP 各司其职配置简单，IP 地址如何选取大有讲究

在网络扩展的上篇中，强叔讲了那么多概念、原理的内容，一方面是为了让各位小伙伴加深对网络扩展功能的理解；另一方面，是为下面配置网络扩展做铺垫。

网络扩展业务的配置可以分为如下几个步骤：

1. 创建一个虚拟网关。
2. 在虚拟网关下创建出差用户、配置出差用户的认证方式以及角色授权。

由于上面这两个配置步骤在之前的贴子中已有介绍，强叔在此就不罗嗦了。

3. 配置网络扩展业务。



网络扩展业务只需要配置两个 IP 地址段就可以了，要配的东西少，所以很简单。但是，怎样选取这两个 IP 地址段却是大有讲究。

参数 1：可分配 IP 地址池范围

在原理部分强叔讲过，出差用户启动网络扩展功能以后，虚拟网关会给出差用户的虚拟网卡分配 IP 地址，那这个地址从哪里来？聪明的小伙伴应该已经猜到了，就是从这里配置的地址池中随机取的。

这个地址池是由网络管理员自行指定的，在指定地址池的时候需要注意该地址池网段与内网网段之间的关系。如果这个地址段与内网网段 10.1.1.2 设置在了同一个子网中，则出差用户在获得虚拟网关分配的这个地址以后，就相当于出差用户和内网服务器在一个二层交换机下连着一样，出差用户就可以直接访问服务器，不存在路由问题。假如此处的地址池和内网服务器不在同一个网段（举例中就没在同一网段），这里就需要在防火墙上配置一

条目的地址是地址池网段（192.168.1.0），出接口是与 Internet 相连的公网接口。这条路由仅用于确定安全域间关系，并不指导报文转发。详细原理强叔已经在上篇中有过介绍了，此处不再赘述。

另外，如果企业内部有专门为用户分配 IP 地址的服务器，如 DHCP 服务器、第三方认证服务器等，网络扩展中的地址池只要不与服务器分配的地址段冲突即可，各自分配各自的 IP 地址，互不影响。

参数 2：可访问内网网段列表

前面强叔说到，开启网络扩展出差用户就可以访问企业内网的所有 IP 资源，那为什么这里还有“可访问内网网段”呢？说到底还是为了控制，如果我们不配置这个参数，默认出差用户是可以访问内网的所有资源；但为了能灵活控制出差用户的访问范围，我们就增加了这个功能。

强叔在网络扩展上篇的业务交互流程中提到过：“虚拟网关会根据网络扩展业务中的配置向出差用户下发不同的路由信息，该路由会影响到出差用户的网络状态”，这句话就是针对网络扩展中“可访问内网网段”这个参数讲的，因为配或者不配这个参数，不但影响用户访问企业内网的范围，同时它也影响出差用户的其它网络状态。

- 如果网络扩展中配置“可访问内网网段”为 10.1.1.0，则虚拟网关就会向出差用户的 PC 上推送一条明细路由，目的地址是内网网段 10.1.1.0，下一跳是出差用户获取到的企业内网的私网地址 192.168.1.1。

```
C:\Users\h98006711>route print
=====
IPv4 路由表
=====
活动路由:
网络目标          网络掩码          网关          接口          跃点数
-----
0.0.0.0            0.0.0.0          10.173.62.1    10.173.63.244    10
10.1.1.0           255.255.255.0    在链路上      192.168.1.1      1
10.1.1.255         255.255.255.255 在链路上      192.168.1.1     257
=====
```

- 如果在网络扩展中不配置“可访问内网网段”这个参数，出差用户的路由又会是什么样的呢？在下图中我们可以看到，虚拟网关向出差用户推送了一条缺省路由，下一跳是出差用户获取到的企业内网的私网地址 192.168.1.1。

```
C:\Users\h98886711>route print
=====
IPv4 路由表
=====
活动路由:
网络目标          网络掩码          网关          接口          跃点数
0.0.0.0            0.0.0.0            192.168.1.1      1
10.3.1.0           255.255.255.0      在链路上         257
10.3.1.56          255.255.255.255    在链路上         257
10.3.1.255         255.255.255.255    在链路上         257
10.173.62.0        255.255.254.0      在链路上         266
```

可别小看了上述两种路由之间的差异，配置了“可访问内网网段”时，虚拟网关只是下发一些到企业内网网段的路由到出差用户，而这个路由不会影响到其他路由，也就是说出差用户想访问企业内网就访问企业内网，想访问 Internet 就访问 Internet，这个访问丝毫不受影响，是该干嘛干嘛。

如果选择了不配置这个参数，那问题就来了。通常出差用户访问 Internet 这条路由是一条缺省路由，现在虚拟网关又推送过来一条缺省路由，而且虚拟网关推送过来的这条缺省路由优先级最高（跃点数是 1），这样一来就会使得出差用户原有的缺省路由失效，即出差用户就没法访问 Internet 了。假如出差用户某一时刻必须要访问 Internet，那就只能暂时断开一下网络扩展连接，想访问内网的时候再重新启用网络扩展就行了。所以到底选择哪一种网络扩展的配置方式，这就取决于企业用户的需要了。

有了上面的几个配置，网络扩展业务就算配置完了。我们下面再来看看出差用户该如何通过网络扩展功能访问内网资源。

IE 浏览器登录简单方便，独立客户端接入亦有亮点

SSL VPN 的网络扩展功能向出差用户提供了两种访问内网的途径，一种是使用 IE 浏览器，另一种是采用独立的网络扩展客户端。

🍁 IE 浏览器

1. 出差用户在 IE 浏览器地址栏中输入虚拟网关的访问地址。
2. 出现虚拟网关登录界面后，输入用户名和密码。
3. 登录成功的出差用户可以在虚拟网关的资源页面看到“网络扩展”页签，单击网络扩展下的“启动”，出差用户就会获取到虚拟网关为其分配的企业内网 IP 地址，这样就可以直接访问企业内网资源了。

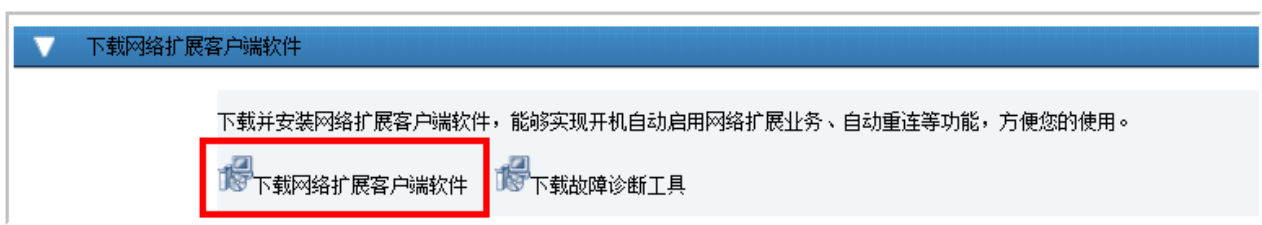


在介绍报文封装原理部分强叔提到过，建立 SSL VPN 隧道分为了可靠性传输模式和快速传输模式，而采用 IE 浏览器与虚拟网关之间建立的 SSL VPN 隧道默认使用的是快速传输模式。

🔥 独立客户端

1. 出差用户下载、安装网络扩展独立客户端。

出差用户成功登录到虚拟网关以后，然后单击界面右上角的“用户选项”，就可以看到网络扩展客户端的下载链接。安装也很简单，按照引导单击“下一步”就行了。



提示：使用独立客户端好处在于，网络扩展客户端可以在开机的时候自动启用，而且在连接断开时有自动重连功能。而使用 IE 浏览器方式时则需要每次都登录一下虚拟网关，所以比较麻烦。

2. 登录虚拟网关。

地址：虚拟网关地址

用户名、密码：管理员为出差用户分配的登录虚拟网关的用户名密码。

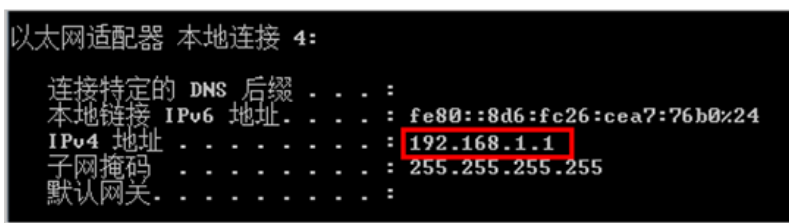
单击“登录”，出差用户就可以和内网用户一样访问内网资源了。



采用独立客户端建立 SSL VPN 隧道时，SSL VPN 的隧道建立模式可以进行配置。在登录界面中单击“选项”，然后在“隧道模式”中就可以选择是使用可靠性传输模式还是快速传输模式。隧道模式中还有个“自适应模式”，表示客户端会根据网络环境自动选择采用可靠性传输模式或是快速传输模式建立 SSL VPN 隧道。

虚拟网卡得 IP，接入成功便无疑

如果用户已经启用了网络扩展功能，那出差用户怎么判断自己的网络扩展功能是不是生效了呢？这可以看两个指标，首先需要用 ipconfig 命令看一下出差用户的 PC 有没有获取到虚拟网关分配的私网地址。按照上面的例子，如果启用网络扩展以后，出差用户的 PC 出现一个 192.168.1.0 网段内的 IP 地址。那么，恭喜您！您已经成功接入到企业的内网中了。



第二个指标就是出差用户试一试，看能不能访问企业内网的资源。

我们会经常碰到这样一些情况，就是出差用户已经获取到虚拟网关分配的 IP 地址了，但就是无法访问内网资源，这又是怎么回事呢？总体来说碰到这样的情况有两个原因：

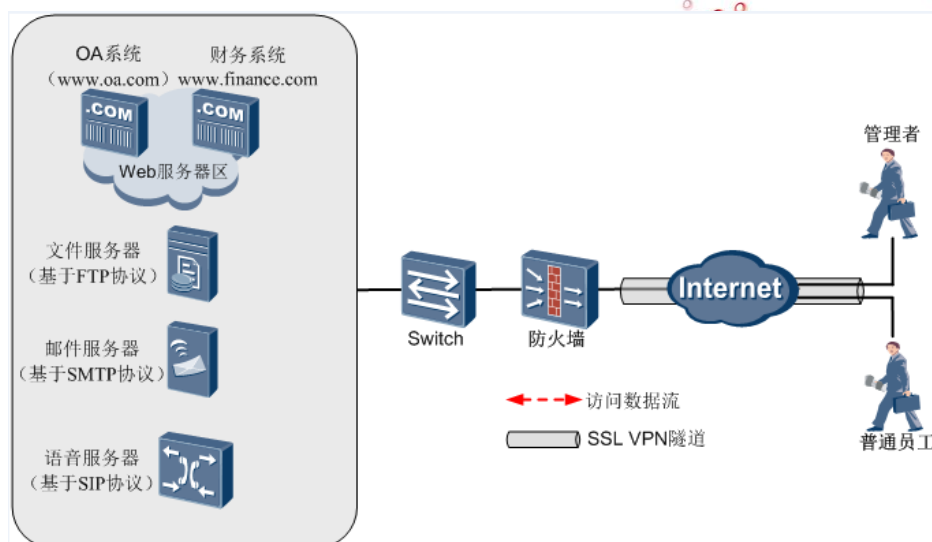
- 一是出差用户没有访问这个内网资源的业务权限（比如研发人员没有访问财务系统的权限）；
- 二是我们在配置网络扩展业务时，“可访问内网网段”中没有包含出差用户要访问的内网资源所在的网段。

这两个问题也好解决，要么出差用户向网络管理者申请业务权限；要么网络管理者在防火墙上检查一下，看是不是把内网资源都加进来了。

说到这里 SSL VPN 网络扩展自身的内容介绍就已经结束了，但考虑到 SSL VPN 这四种业务既有差异又有关联，因此有必要整体角度再对比一下四种业务的关系。

四大业务悉数登场，作用不同各有千秋

假设某企业部署了防火墙设备，为企业出差员工提供 SSL VPN 业务，其网络如下图所示。



企业出差用户访问内网的需求以及在防火墙上为出差员工开通 SSL VPN 业务的规划如下：

出差员工身份	访问需求	业务类型	角色授权
普通员工	访问 OA 系统	Web 代理	在 Web 代理业务中创建 www.oa.com 这条资源，将此资源与普通员工或普通员工所属的组进行绑定。
	使用企业的邮件系统收发邮件。	端口转发	在端口转发业务中创建一条邮件服务器的资源，将此资源与普通员工或普通员工所属的组进行绑定。
管理者	访问 OA 系统和财务系统	Web 代理	在 Web 代理业务中创建 www.oa.com (已创建)、 www.finance.com 这两条资源，将此资源与管理者或管理者所属的组进行绑定。
	访问文件服务器	文件共享	在文件共享业务中创建一条文件服务器资源，将此资源与管理者或管理者所属的组进行绑定。
	使用企业的邮件系统收发邮件。	端口转发	在端口转发业务中创建一条文件服务器资源，并将邮件服务器资源与管理者或管理者所属的组进行绑定。
	召开电话会议	网络扩展	启用网络扩展功能，并将语音服务器所在的地址配置到“可访问内网网段”中，然后将网络扩展业务与管理者或管理者所在的组进行绑定。

网络业务配置完成后，不同身份的用户在登录虚拟网关后所能看到的资源也不相同，其所见资源如下。

普通员工

普通出差员工在登录虚拟网关后，可以看到自己所能访问的资源连接，单击链接就可以访问该资源。



🔥 管理者

出差的管理者在登录虚拟网关后可以看到如下界面。



其中 Web 代理、文件共享资源都是以链接形式供用户选择。端口转发和网络扩展需要单击“启动”以后才能使用。但是出差用户如何知道自己点了启动以后都能访问企业的哪些内网资源呢？这个就需要网络管理员通过其他途径，例如网络公告之类的将企业内网的资源服务器域名、地址告知出差用户。在这一点上，Web 代理、文件共享相对有优越性，因为出差用户在使用这两种业务的时候，自己能够访问哪些资源，这在登录到虚拟网关以后是可以从资源列表中看到的。

出差用户访问企业内网的需求与防火墙上开启何种 SSL VPN 业务之间的对应关系可以归纳成两点。

🔥 出差用户访问企业内网的资源类型（Web 资源、文件资源、TCP、IP）决定了网络管理员应该选择 SSL VPN 的何种业务。

例如，出差员工只需要访问 Web 资源和邮件资源，就为该用户启用 Web 代理和端口转发两种业务；而管理者需要访问四种类型的资源，那就需要为其开启四种业务。

需要说明的是，网络扩展由于兼备了前三种业务的功能，如果说为了配置方便，也可以只为管理者开启网络扩展业务，让管理者可以访问内网所有的 IP 资源。

🍁 出差用户是否拥有对某一资源的访问权限，这是通过角色授权配置来决定的。

例如，出差员工和管理者同样都开启了 Web 代理业务，出差员工只能访问 OA 系统（www.oa.com），而管理者却同时拥有访问 OA 系统和财务系统（www.finance.com）的权限，这就是在角色授权中配置的。

附：规格列表

SSL VPN 业务对出差用户所使用的操作系统及浏览器有要求，在实际使用中，经常有小伙伴们会问到这个支持情况，此处强叔就把这个支持情况整理了一下，以便大家查阅。

网络扩展所支持的操作系统规格

SSL VPN 业务		支持的操作系统
Web 代理	Web 改写	-
	Web Link	Windows 2000 Professional SP3 及其以上（32 位） Windows Server 2000 SP3 及其以上（32 位） Windows Server 2003（32 位） Windows XP SP1 及其以上（32 位） Windows Vista（32 位/64 位） Windows 7（32 位/64 位） Windows Server 2008（32 位/64 位） 说明：64 位操作系统下，浏览器要以 32 位模式运行。
文件共享		Windows 2000 Professional SP3 及其以上（32 位） Windows Server 2000 SP3 及其以上（32 位） Windows Server 2003（32 位） Windows XP SP1 及其以上（32 位） Windows Vista（32 位/64 位） Windows 7（32 位/64 位） Windows Server 2008（32 位/64 位） 说明：64 位操作系统下，浏览器要以 32 位模式运行。
端口转发		Windows 2000 Professional SP3 及其以上 Windows XP SP1 及其以上 Windows Vista Windows Server 2000 SP3 及其以上 Windows Server 2003 Windows 7 Windows Server 2008 X86 支持的系统均为 32 位操作系统。
网络扩展	浏览器登录	Windows 2000 Professional SP3 及其以上（32 位）

		Windows Server 2000 SP3 及其以上 (32 位) Windows Server 2003 (32 位) Windows XP SP1 及其以上 (32 位) Windows Vista (32 位/64 位) Windows 7 (32 位/64 位) Windows Server 2008 (32 位/64 位) 说明: 64 位操作系统下, 浏览器要以 32 位模式运行。
	Huawei AnyOffice VPN 客 户 端登 录	Windows 2000 Professional SP3 及其以上 (32 位) Windows Server 2000 SP3 及其以上 (32 位) Windows Server 2003 (32 位) Windows XP SP1 及其以上 (32 位) Windows Vista (32 位/64 位) Windows 7 (32 位/64 位) Windows Server 2008 (32 位/64 位) Mac OS X 10.x.x 说明: 64 位操作系统下, 浏览器要以 32 位模式运行。

网络扩展所支持的浏览器规格

SSL VPN 业务		支持的浏览器及版本
Web 代理	Web 改写	Internet Explorer 6/7/8/9 Mozilla Firefox 4.0~12.0 Google Chrome 10.0~20.0 Opera 9.0~12.0 Safari 3.0~5.1.x 360 安全浏览器 3.x~5.0 Symbian Series 60 Web Browser 7.0-7.4 BlackBerry Browser 4.6.0 天天浏览器 V2.3 及以上版本 UC Browser 7.x 及以上版本
	Web Link	Internet Explorer 6/7/8/9
文件共享		Internet Explorer 6/7/8/9
端口转发		Internet Explorer 6/7/8/9
网络扩展	浏览器登录	Internet Explorer 6/7/8/9
	Huawei AnyOffice VPN 客户端登录	-

说明: “-” 表示对应业务对操作系统或浏览器不做要求。

本篇是 VPN 技术的最后一篇, 下一篇强叔将带领小伙伴们进入光怪陆离的双机世界, 敬请期待!!!

思考: 如果出差用户要访问企业内网的 Web 资源, 那在防火墙上为用户开通网络扩展功能以后, 用户能不能正常访问内网的 Web 资源, 这样的业务控制是否合理, 为什么?

🍁 防火墙报文处理流程

强叔被金庸大侠影响甚深，年少时曾手捧老先生各大部头废寝忘食，至今仍对多个经典情节念念不忘。早在十多年前，便与同道兄弟充分讨论过：金庸作品中，最厉害的招式应是独孤九剑，因令狐冲在内力全失情况下，一剑挑瞎十五人眼睛；而最厉害的内功当属扫地僧的内功，秒掉了萧峰父子和慕容父子，算得上前无古人、后无来者。最要命的是，我们无从得知扫地神僧是怎样练成的，这让强叔一直心痒不已。

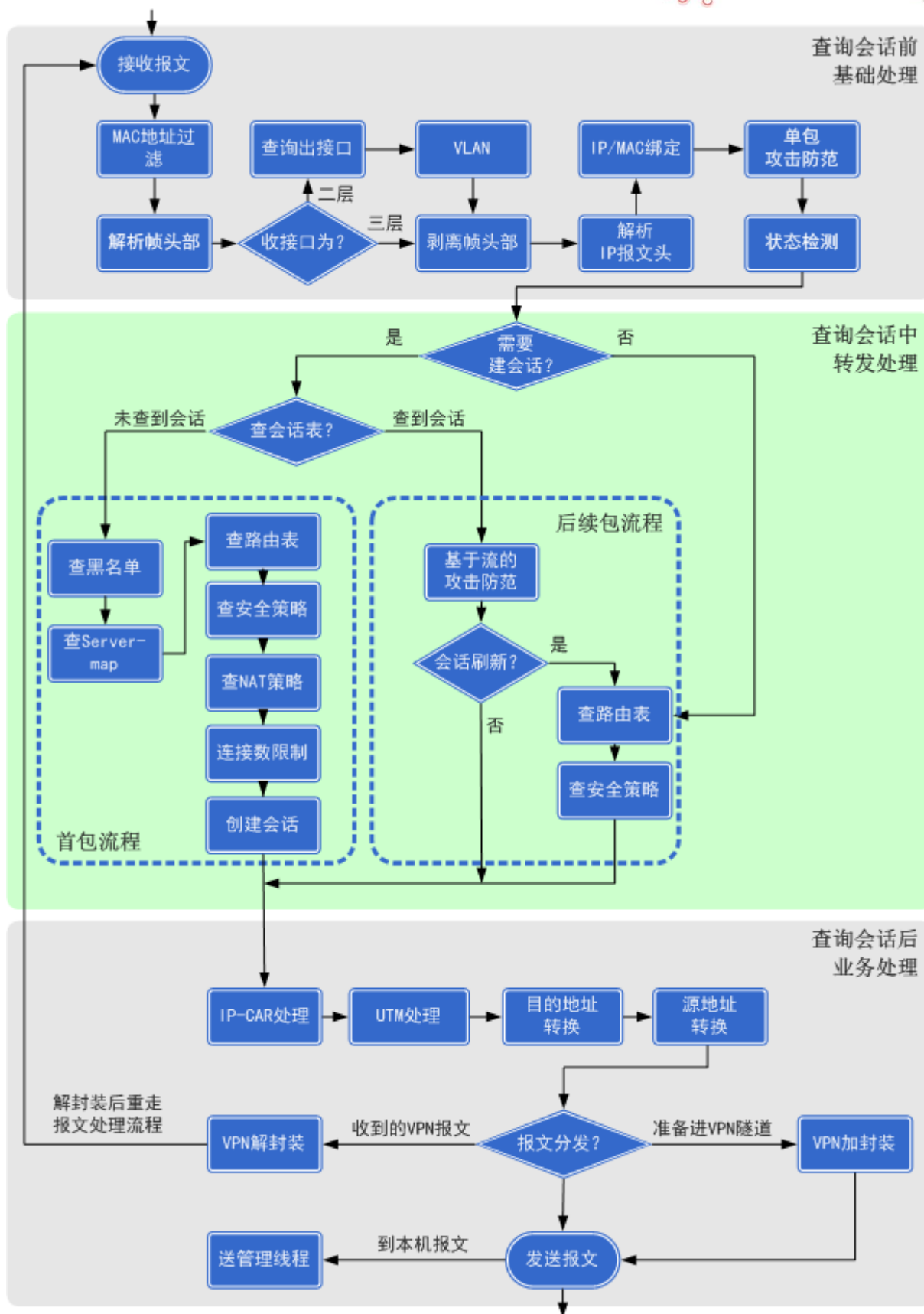
而回到强叔开讲的防火墙，强叔绝不能让各位小伙伴一直抓耳挠腮——掌握了防火墙报文处理流程，就是掌握了防火墙实现的精髓，就如练成了扫地神僧的内功。表面上看来还是原来的小伙伴，也许貌不惊人，但其实内心那是相当强大。当你能够娴熟地定位防火墙配置类故障，无论走到哪里，都会如那漆黑中的萤火虫一样，那样的鲜明，那样的出众，那样的拉风~~

在《华为防火墙产品一览》中，强叔曾介绍：华为出品低、中、高端共三个系列防火墙，对应当前的主力产品型号分别为 USG2000、USG5000 和 USG9000。〈注〉2013 年 8 月发布了下一代防火墙 USG6000 系列，强叔后续针对性详述。这三个系列产品毫无疑问都基于状态检测与会话机制实现，但是中低端防火墙与高端防火墙在架构实现上有些区别：中低端防火墙采用集中式架构，高端防火墙采用分布式架构，这使得它们在对报文处理的实现上，稍有不同。不过这个稍有不同并不影响主线，强叔的思路是单刀直入，先讲大同的报文处理流程（同样会话机制），再强调求同存异（架构差异影响）。

华为大同：全系列状态检测防火墙报文处理流程

下面先以 USG5500 系列产品为例，介绍防火墙报文处理流程。

状态检测与会话机制是华为防火墙对报文处理的关键点：即防火墙收到报文后，何时、如何创建会话、命中会话表的报文被如何转发。那么可以此关键点为界线延伸，我们将报文经防火墙处理分为**查询会话表前、中、后**三个阶段，参见如下 USG5500 报文处理全景图。友情提醒：图很复杂，对三个阶段有印象即可，重点看图下的文字拆解吧。



我们来进一步介绍下每个阶段的目的和处理过程。

1. 查询会话之前的处理过程：基础处理。

这个阶段的主要目的是解析出报文的帧头部和 IP 报文头部，并根据头部当中的信息进行一些基础的安全检测（单包攻击防范）。

2. 查询会话的处理过程：转发处理，关键是会话建立。

这个阶段到了防火墙对报文转发处理的核心。建立会话表，根据会话表转发报文，是状态检测防火墙的精髓。此部分要梳理清楚思路，耗费很多笔墨，还请小伙伴们耐心观看。

报文到此阶段，防火墙最先判断该报文是否要创建会话。

1) OSPF、IGMP 组播报文等一些协议报文是不创建会话的，那么这些报文就会去直接查路由和安全策略。

2) 而 TCP 首包、UDP 等报文都需要创建会话，那么判断该报文要创建会话后，接下来马上查询会话表中是否已经创建了该报文的会话。

🍁 对于会话表中不存在匹配任一表项的报文，防火墙判断该报文为某一流量的首包，进行首包处理流程。

🍁 对于会话表中存在匹配表项的报文，防火墙判断该报文为某一流量的后续包，进行后续包处理流程。

这也是强叔在《状态检测和会话机制》一节中提到的“为数据流的第一个报文建立会话，数据流内的后续报文直接根据会话进行转发，提高了转发效率。”

🍁 首包处理流程要点：

1) 先使该报文与黑名单匹配，若报文源地址命中黑名单，则此报文被丢弃，不再继续后续流程。

- 2) 查询该报文是否命中**正反向 Server-map 表**, 若报文中**Server-map 表**, 记录**Server-map 表**中的信息。

在《ASPF：隐形通道》一节中，强叔曾正面讲解了 Server-map 表，并指出“在防火墙转发中非常重要”，此处再回顾总结下：

关于正、反向 Server-map 表：

- 🍁 在《NAT Server 三十二字真言》一节中提到，配置 NAT Server 后会生成正、反向 Server-map 表。
- 🍁 Nat Server, any -> 1.1.1.1:9980[10.1.1.2:80] 为正向 Server-map 表项，用于用户向服务器主动发起访问。
- 🍁 Nat Server Reverse, 10.1.1.2[1.1.1.1] -> any 为反向 Server-map 表项，用于服务器主动访问公网。即配置 NAT Server 同时打通了服务器和公网之间出入两个方向的地址转换通道。

关于静、动态 Server-map 表：

- 🍁 配置 NAT Server 等功能后，直接生成静态 Server-map 表。若报文匹配静态 Server-map 表，则记录 Server-map 中转换后目的 IP 地址和端口信息，继续后续流程。
- 🍁 防火墙上还存在一个动态生成的 Server-map 表，这是由于配置 ASPF、三元组 NAT、NAT No-PAT 功能后，首包建立会话后，触发建立了一个动态表项，而后续报文若匹配动态 Server-map 表，则不再对此报文进行安全策略匹配，直接按此表信息创建新会话。****ASPF、三元组 NAT、NAT No-PAT 原理请参见强叔 1-18 篇章节****

- 3) 继续根据 2) 的记录结果，查询命中哪条路由，优先查询策略路由。若未命中策略路由，则查询正向路由表，决定报文的下一跳和出接口。

这里说明下：为什么要强调根据 2) 的记录结果呢？因为此时报文未进行实质转换，但查询路由是使用虚拟出来的、命中 Server-map 表经转换后的报文查询，这是为了最终经过防火墙各种处理完毕的报文能够正确转发。

- 4) 查询是否命中安全策略，已知报文入接口源地址、判断出报文出接口后，可以查询到该出入接口所在安全区域的安全策略配置，若报文没有匹配到安全策略或匹配到安全策略中定义为“阻断”的规则，则报文被丢弃，不再继续后续流程；若报文匹配安全策略中定义为“允许”的规则，则继续后续流程。

- 5) 查询是否命中源 NAT 策略，若报文匹配到一条源 NAT 策略，则记录 NAT 转换后的源 IP 地址和端口信息。

- 6) 顺利通过安全策略匹配检查后，终于走到这一步：根据上述记录结果，创建会话。

后续包处理流程：很简单，判断会话是否需要刷新。当为首包创建会话的各表项和策略（如路由表、安全策略）变化时，会话需要刷新。若需要则继续查询路由、查询安全策略，若不需要直接进行后续阶段处理。

总结：

- ✿ 强叔在开篇中提到防火墙与路由器的区别，对于路由器来说可说是使出浑身解数（通过路由表）将报文转发出去，而对于防火墙来说要作为守护神坚决阻挡非法报文（只有通过各种安全检查建立会话表才可能被转发）。所以防火墙丢弃一些报文是正常的处理流程，包括第一阶段的解决 IP 报文头、单包攻击，第二阶段的黑名单、安全策略，第三阶段的 UTM 处理、限流等等，是报文的主要丢弃点。
- ✿ 进行故障定位时，防火墙上是否创建了指定某条流量的会话，是定位要考虑的关键界线。若未创建会话，则考虑是报文是否到达设备、接口丢包、命中黑名单或安全策略、无路由、或 NAT 等配置出现问题；而存在会话，则向后续的安全业务处理阶段考虑。

3. 查询（或创建或刷新）会话的处理过程：进行实质各安全业务处理、及之后的报文去向。

报文在首包处理流程中经一通查询、创建会话后，就与后续包处理流程殊途同归了，即都进入第 3 阶段。

此阶段首先进行基于 IP 的限流、IPS 等 UTM 业务处理，报文顺利通过检查之后，到了实质性的地址转换：根据已创建的会话表，进行目的地址转换、源地址转换。

这个目的地址转换、源地址转换对应配置就是 NAT Server、源 NAT 策略等 NAT 功能，对应报文则是在第 2 阶段首包流程中已经经过一通查询并创建了会话表。

那么，如果想要在此阶段能够准确地对报文进行源地址转换，有两个常见的 points 需要关注：

- ✿ 一是报文在前面流程中是否命中了 Server-map 表，尤其是反向 Server-map 表，若已经命中，那么报文会根据反向 Servermap 表进行源地址转换，不会再往后匹配源 NAT 策略了。
- ✿ 一是报文在后续流程中是否计划要进行 VPN 加封装，如果在此时定义该条数据流进行了 NAT 转换，那么后续就无法进入 VPN 协商流程了。

而按计划进行目的地址转换问题不大，一般是配置 NAT Server，其报文处理优先级很高，报文走入歧途可能性就小。

最后，该转换转换，该过滤过滤，一切安全业务流程顺利通过后，报文终于到了分发十字路口：

- ✿ 如 ping、OSPF 等路由协议报文是要到防火墙本身，会被上送至管理层面处理；
- ✿ 如 VPN 报文（防火墙为隧道终点，收到 VPN 报文）就会被解封装、并在解封装后重走一

次上述 1、2、3 流程；

如准备进入 VPN 隧道的报文（防火墙为 VPN 隧道起点）就会被进行 VPN 封装等等。

求同存异：集中式与分布式防火墙差异对报文处理流程影响

防火墙大同的报文处理流程介绍完毕，我们来看一下集中式与分布式架构的区别会导致哪些差异。

对于集中式低端防火墙，来往报文会被上送至一个集中的 CPU 模块（可能由多个 CPU 组成）进行处理。

集中式防火墙一般为盒式设备，可以插接多种扩展接口卡，但设备的总机性能恒定，即取决于该设备配置的 CPU 模块处理能力。上述介绍的 [USG5500 产品报文处理流程](#) 即为其全部流程。

对于分布式高端防火墙，对报文的处理就会比较复杂，有多种情况，上述介绍的报文处理流程是其中一个最核心的子集—SPU 板业务处理流程。

分布式防火墙一般为框式设备，以 USG9000 来说，由两块标配主控板 MPU、交换网板 SFU、接口板 LPU、业务板 SPU 组成，其中 LPU 与 SPU 的槽位可混插，客户按需购买。相比集中式防火墙，分布式防火墙由各种单板组成，每种单板各司其责。

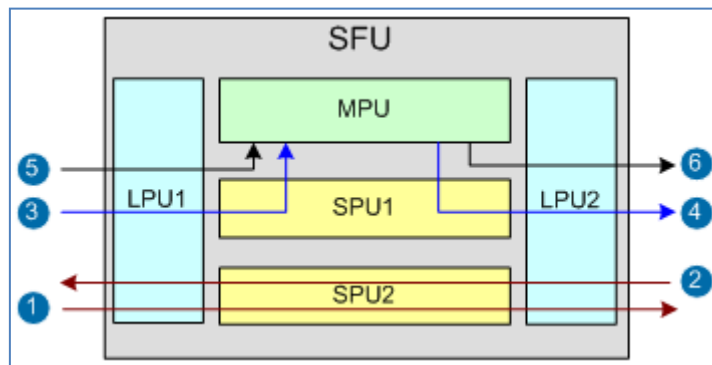
主控板 MPU：主要负责系统的控制和管理工作，包括路由计算、设备管理和维护、设备监控等。

交换网板 SFU：主要负责各板之间数据交换。

接口板 LPU：主要负责接收和发送报文，以及 QoS 处理等。

业务板 SPU：主要负责防火墙的安全业务处理，包括安全策略、NAT、攻击防范、VPN 等。设备总机性能随插接 SPU 板的数量增加而线性增加，这是分布式防火墙的特性和价值所在。

那么我们来看一下，各类报文在分布式高端防火墙上是如何被处理的：



①②表示经过防火墙访问其他网络设备的报文，由业务板专门进行安全业务处理，既对应上述介绍的报文处理流程。

③④表示通过业务板到防火墙自身的报文、或防火墙自身经业务板发出的报文。如 Ping、telnet 登录、FTP 访问防火墙的报文、以及网管监控的 SNMP 报等文。

⑤⑥表示接口板直接上送主控板的报文、或主控板直接由接口板发出的报文，不过业务板。如 OSPF 路由学习报文，VRRP Hello 报文，IP-LINK 探测报文等。

已经提到①②是防火墙业务处理的核心，也正是防火墙的安全防护价值所在。③④多用于设备管理、定位故障；⑤⑥多用于网络互联如路由学习。

还有一个更复杂的问题：当一台分布式防火墙配置多个业务板时，报文经过接口板处理后，会被送至哪个业务板呢？如上图中的①②③④报文都会经业务板处理，那么会被送至 SPU1 还是 SPU2 呢？

这时会有选择“由哪块 SPU 板进行业务处理”的动作，即 USG9000 产品文档中提到的 hash 选板。默认配置是根据报文的源+目地址经过运算选择。----由于配置了多个 SPU 板，业务可能在防火墙多个业务板分别建立会话。

例如，以 PC 访问 Web 服务器来说，假设 PC 发往 Web 服务器在业务板 SPU1 上建立了会话，而 Web 服务器的回应报文可能会由业务板 SPU2 处理。一句话就是请求报文与回应报文有可能被分配到防火墙的不同业务板处理。那么可能会产生一个问题：由于请求报文基于业务板 SPU1 建立了会话，而回应报文查不到反向会话被丢弃。实际上，USG9000 在建立业务板的会话表时，同时会进行 hash 预测，准确预测回应报文会被哪块业务板处理，在该业务板上同步建立起一条反向会话，保证回应报文能够被正确转发。当然，正反向会话间会定时同步，保证同步老化、统计报文信息等等。

下面我们以实例来说明，熟悉防火墙报文处理流程，不需要再死记配置限制，而是根据逻辑直接判断如何正确配置。

例 1，当安全策略遇上 NAT Server。

在 eNSP 模拟器上，PC1 (1.1.1.2) ----防火墙 USG5500----FTP 服务器 (192.168.1.2) 简单组网，在防火墙上配置 NAT Server，将 FTP 服务器地址转化为与 PC 同一网段的地址，配置如下：

```
[USG5500] nat server 0 global 1.1.1.5 inside 192.168.1.2
```

若希望 PC1 及其他多个 PC 能且只能访问该 FTP 服务器，应该如何配置严格的安全策略？实际是基于固定的目的地址配置安全策略问题。

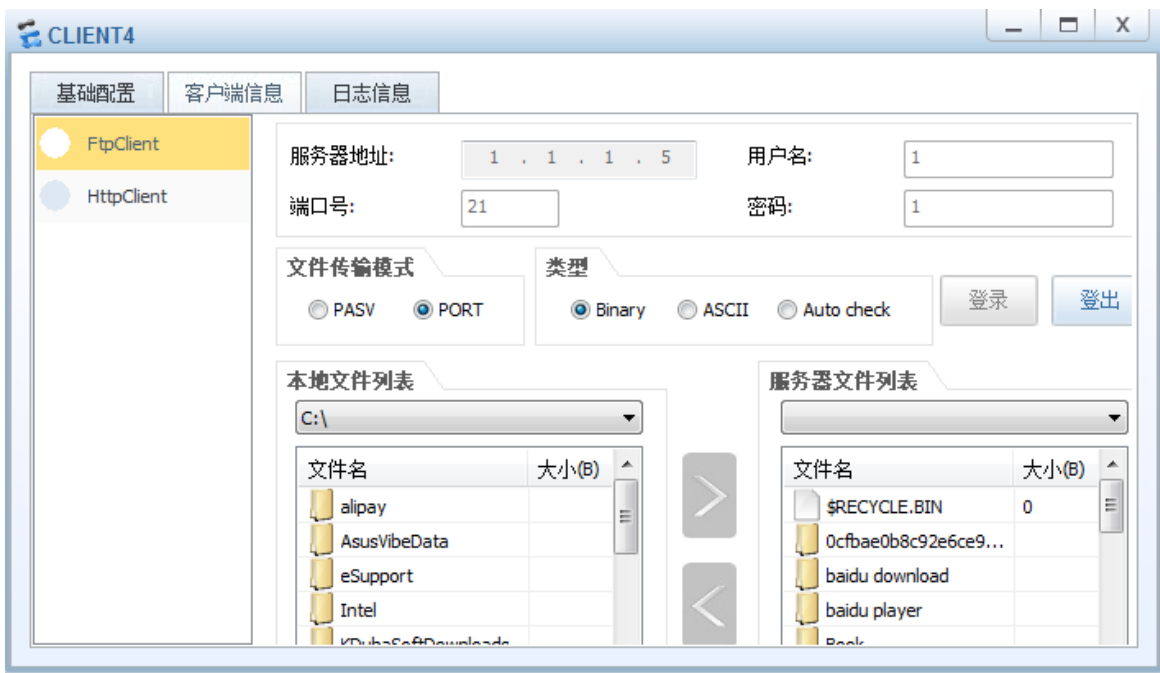
分析：根据防火墙报文处理流程，先进行 NAT Server 转换、查询 Server-Map 表，再进行安全策略处理。那么，该固定的目的地址，应配置为该 FTP 服务器的真实地址（即 inside 地址）。

```
[USG5500] dis cu | include policy
```

```
policy interzone trust untrust inbound
policy 0
policy destination 192.168.1.2 0.0.0.0
```

验证如下：

- 1) 关闭防火墙所有域间缺省包过滤，只保留上述在 trust 与 untrust 域间配置的该条安全策略。
- 2) 在 PC 上访问 FTP 服务器，可以成功看到 FTP 服务器的文件列表。



- 3) 查看防火墙 USG5500 会话表，可以看到 FTP 服务器已经做了 NAT Server，且 FTP 控制通道与数据通道均已建立，

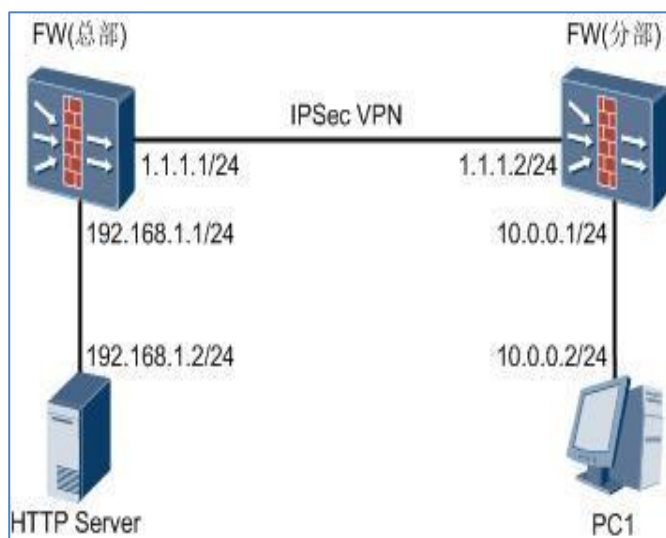
```
<USG5500>d f s t
21:37:59 2014/07/06
Current Total Sessions : 2
ftp VPN:public --> public 1.1.1.2:2061+-->1.1.1.5:21[192.168.1.2:21]
ftp-data VPN:public --> public 192.168.1.2:20[1.1.1.5:20]-->1.1.1.2:2062
```

- 4) 修改上述安全策略，将目的地址改为 FTP 服务器 NAT Server 的 global 地址。再使用 PC 访问该 FTP 服务器，发现无法成功登录；查看防火墙会话表，会话表项为 0。

结论：根据防火墙报文处理流程，先查询 Server-Map 表，根据查询的记录结果再进行安全策略处理。当需求为允许多个外网客户端访问经过 NAT Server 转换的服务器时，安全策略配置允许到达的目的地址为该服务器的内网真实地址，而非 NAT Server 的 global 地址。

例 2，当源 NAT 遇上 NAT Server。

某公司分部与总部互联组网和关键配置如下，其中 HTTP Server 无法访问分部资源，而其他通信完全正常。



```

nat-policy interzone trust untrust outbound
policy 0      //访问分部的流量不做源地址转换//
action no-nat
policy source 192.168.1.0 mask 24
policy destination 10.0.0.0 mask 24
policy 5      //总部员工上网进行源 NAT 转换//
action source-nat
policy source 192.168.1.0 mask 24
easy-ip GigabitEthernet0/0/1
#
nat server protocol tcp global 1.1.1.1 9980 inside
192.168.1.2 80
    
```

分析：防火墙上涉及 HTTP Server 的配置如下：

- ① NAT Server，将 HTTP Server 的私网 192.168.1.2：80 映射成为 1.1.1.1：9980
- ②源 NAT，策略定义 192.168.1.0 网段去往分部不进行源 NAT 转换

查看防火墙的 Server-map 表：

```

-----
Nat Server, any -> 1.1.1.1:9980[192.168.1.2:80], Zone: ---
  Protocol: tcp(Appro: unknown), Left-Time: ---:---:---, Addr-Pool: ---
  VPN: public -> public

Nat Server Reverse, 192.168.1.2[1.1.1.1] -> any, Zone: ---
  Protocol: any(Appro: ---), Left-Time: ---:---:---, Addr-Pool: ---
  VPN: public -> public
    
```

NAT Server 转换成功，生成正反向 Server-map 表。且按照反向 Server-map 表含义，当该 HTTP Server 主动访问网络时，亦进行 NAT Server 转换。根据防火墙报文处理流程，优先查询正反向 Server-map 表，命中反向 Server-map 表后，源 NAT 不再生效。这个问题的解决办法是，配置 NAT Server 时增加 no-reverse 参数，要求不生成反向 Server-map 表，这样总可以继续查询 NAT 策略，使该 HTTP Server 可以正常访问分部资源。

结论：上述报文处理流程第 3 阶段时已进行提醒，进行源 NAT 处理时，注意报文在前面流程中是否命中了 Server-map 表，尤其不要忽略反向 Server-map 表，如果已经命中，此时源 NAT 处理不会再生效。

防火墙配置类问题与报文处理流程相关的，可能还有很多，本文难以穷尽，只捡两个实际常用 NAT 功能的例子抛砖，供小伙伴参考。文档字数很多，逻辑难免有不严谨之处，请小伙伴琢磨指证。

强叔研习金庸多年，可只见一个扫地神僧往来无古今，亦只闻一曲笑傲江湖流传百年。世上没



内功心法

有什么捷径，掌握防火墙的转发精髓仍需在处理问题实践锻炼中不断琢磨，在参考材料字里行间中不断思考，本文仅提供神功练成的入门之法，强叔与小伙伴们一同继续修行吧。

数字证书浅析

在 IPsec VPN 篇章中，我们介绍了 IPsec 隧道两端设备使用证书进行身份认证的内容，在刚刚推出的 SSL VPN 开篇中，也介绍了证书认证的相关内容。作为网络世界的“身份证”，证书在身份认证的场景中已经得到了普遍应用。

大家可能已经习惯了用户名+密码的认证方式，而对证书这种认证方式还不太了解。为此，强叔带来了本篇内功心法，揭示证书的来龙去脉和使用方法，帮助大家提升内力。

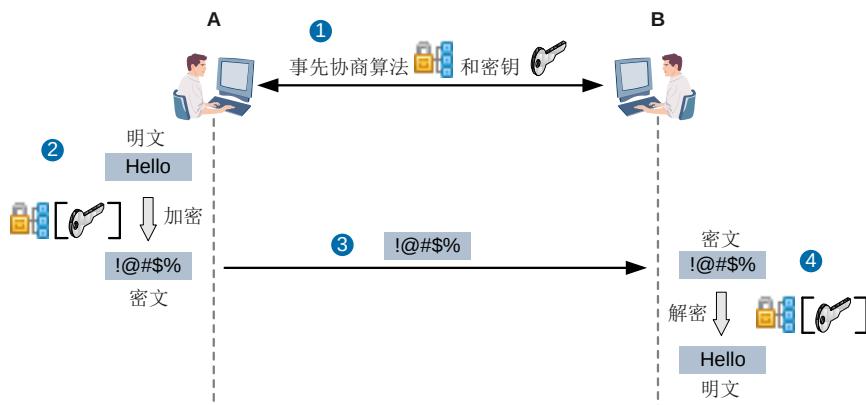
根基——公钥密码学

在介绍证书之前，我们先来了解一些密码学的基本概念，包括对称密码学和非对称密码学，为理解证书的实现原理打好基础。

首先，一提到加密，我们自然会想到通信双方使用**相同的算法和密钥去加密和解密数据**。加密和解密过程中用到的密钥是双方都知道的，即双方的“共享密钥”。这种加密方式称为**对称密码学**，也叫做单钥密码学。

如下图所示，A 向 B 发送数据，A 使用双方事先协商好的算法和密钥来加密数据，B 使用相同的算法和密钥来解密数据。反之同理，B 向 A 发送数据时，B 加密数据而 A 解密数据，双方使用的都是相同的算法和密钥。

注意：我们用“**算法[密钥]**”来表示使用该算法和密钥对数据进行加密或解密处理，下文中的类似表达方式含义相同。



对称密码学的优点是效率高，开销小，适合加密大量的数据。但对称密码学要求通信双方事先协商好密钥，这就要求在协商过程中必须做好保密，密钥只能让使用的人知道，不能泄露。另外，如果通信方数量庞大，比如 A 需要和数百个对象通信，为了安全起见，就需要在 A 上维护

数百个不同的共享密钥，这就为密钥更新和管理带来诸多不便。

为了解决对称密码学面临的问题，公钥密码学横空出世，开创了密码学的新方向。公钥密码学使用了两个不同的密钥：一个可对外界公开，称为“公钥”；一个只有所有者知道，称为“私钥”。

这一对密钥具有如下特点：

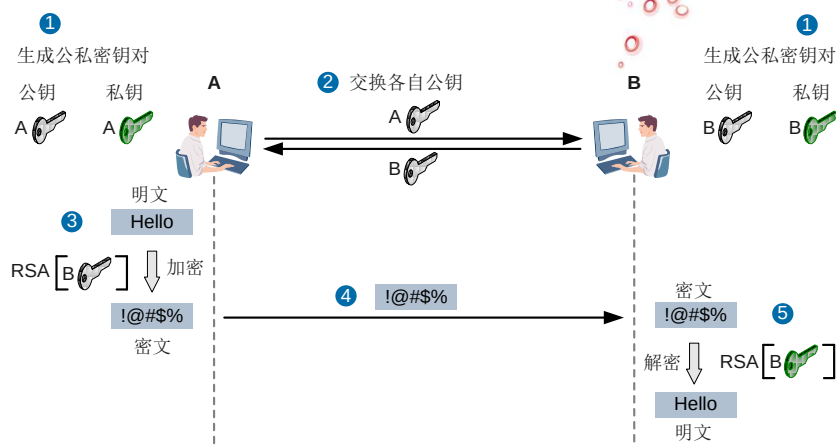
- ❖ 用公钥加密的信息只能用相应的私钥解密，反之用私钥加密的信息也只能用相应的公钥解密，即用其中任一个密钥加密的信息只能用另一个密钥进行解密。
- ❖ 要想由一个密钥推出另一个密钥，在计算上是不可能的。

由于公钥密码学使用了两个不同的密钥，所以属于非对称密码学，也称为双钥密码学。目前常用的公钥密码学算法有以下几种：

- ❖ RSA (Rivest, Shamir and Adleman)，这个由三位发明者名字命名的算法是当前最著名、应用最广泛的公钥密码学算法。RSA 可实现数据加解密、真实性验证和完整性验证。
- ❖ DSA (Digital Signature Algorithm)，中文名称叫数字签名算法，可实现签名功能。本篇对 DSA 不做过多的介绍，大家可以自行查阅相关资料。
- ❖ DH (Diffie-Hellman)，也是由发明者名字命名的一种密钥交换算法，通信双方通过一系列的数据交换，最终计算出密钥，以便用于以后的报文加密。在 IPsec 中就用到了 DH 算法，使得建立 IPsec 隧道的两端网关设备可以计算出密钥，而不必担心密钥泄露，我们在 IPsec 篇中对此进行过介绍。

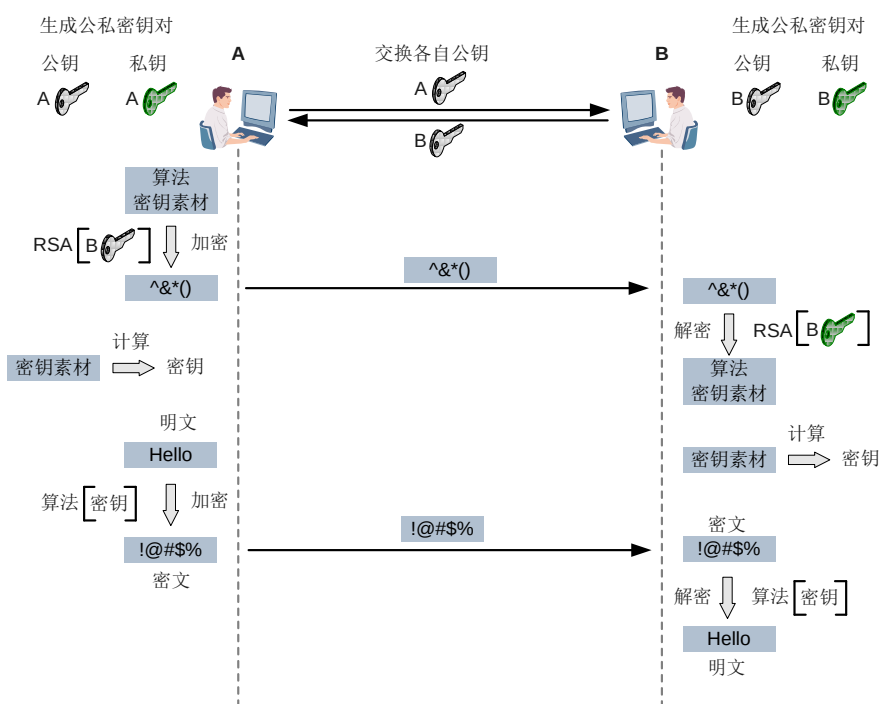
数据加解密

下面我们就以 RSA 算法为例，介绍通信双方如何实现数据加解密功能。如下图所示，A 和 B 各自生成自己的公钥和私钥，并且相互交换了双方的公钥（至于双方是通过什么途径来交换公钥，我们先留个伏笔，后面会讲到）。A 要向 B 发送数据，A 使用 B 的公钥加密数据然后发送给 B，B 收到后使用自己的私钥解密数据。因为其他人没有 B 的私钥，即使截获报文也无法解密，这就保证了数据传输的安全性。B 向 A 发送数据时，过程同理。



与对称密码学相比，公钥密码学加密数据的计算非常复杂，而且开销大、速度较慢，所以不适用于加密大量数据的场景。在实际使用中，通信双方通常会使用公钥密码学来交换密钥素材，双方最终计算出密钥，而用对称密码学来加密实际的数据，两者配合使用，保证了加密速度和安全性。

如下图所示，A 向 B 发送数据，A 使用 B 的公钥把加密算法和密钥素材加密后发给 B，B 收到后使用自己的私钥解密，这样就得到了算法和密钥素材（B 也可以将自己要采用的算法和密钥素材发给 A，即双方通过协商方式来确定算法和密钥素材），双方根据密钥素材计算出密钥。然后就可以通过对称密码学的机制，使用相同的算法和密钥对数据进行加密和解密。

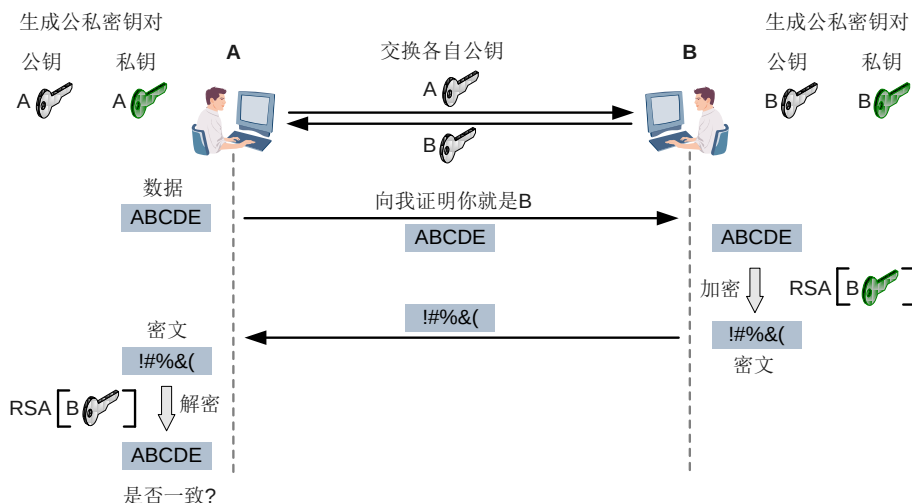


真实性验证

除了加解密，RSA 还能实现真实性验证，即身份认证功能，这也是利用了公钥密码学中由任一

个密钥加密的信息只能用另一个密钥进行解密这一原理。

如下图所示，A 要认证 B 的身份，首先 A 向 B 发送数据（例如一串字符串），B 用自己的私钥将数据加密，然后发送给 A。A 用自己所持有的 B 的公钥解密，将解密后的数据和原始数据进行对比，如果一致，说明数据的确是由 B 发过来的。因为 B 的私钥只能由 B 持有，因此 A 可以通过判断对方是否持有私钥来判断对方是否就是 B，进而实现了对 B 的身份认证，这就是使用公钥密码学来进行身份认证的理论基础。



加解密功能是数据发送方使用接收方的公钥加密，接收方使用自己的私钥解密。而身份认证功能是被认证方使用自己的私钥进行加密，认证方使用被认证方的公钥进行解密。

完整性验证

使用私钥加密公钥解密这种方式，还能实现完整性验证，即签名功能。所谓签名，就是在数据的后面再加上一段内容，可以证明数据没有被修改过，那怎么样生成签名呢？

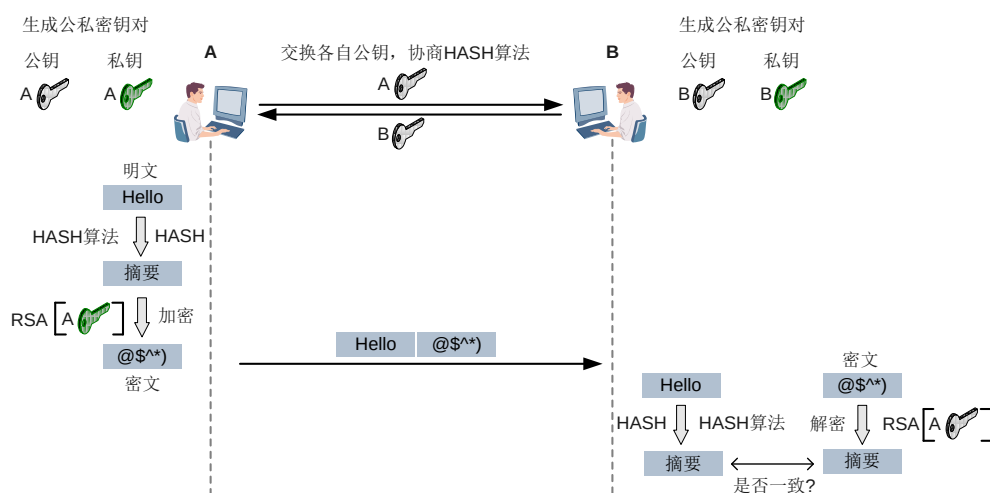
发送方可以用自己的私钥加密数据，然后连同数据一起发给接收方。但是，由于公钥密码学计算复杂、速度慢的原因，通常发送方一般不直接使用私钥加密数据，而是先将原始数据经过某种计算得出一段较短的数据，然后发送方使用自己的私钥加密这段新的数据，这和直接加密原始数据的效果是一样的，而且还兼顾了安全和处理速度。这种计算方式是通过 HASH 密码学来实现的。

HASH 密码学可以将任意长的字符串通过哈希计算出固定长度字符串，并且该计算是单向运算，无法逆推。最重要的是，原字符串任意字符的变化都会导致不同的计算结果。HASH 计算后得出的信息通常称为原字符串的摘要信息，也可以称为指纹信息。通过对比摘要信息，就可以判断数据是否被修改，所以 HASH 密码学通常用于保证数据的完整性。常见的 HASH 算法有 MD (Message Digest algorithm) 系列、SHA (Secure Hash Algorithm) 系列等，之前在 IPsec

篇中我们也都配置过这些算法。

如下图所示, A 向 B 发送数据之前, 先使用 HASH 算法将数据进行 HASH 计算, 得出摘要信息。然后使用自己的私钥将摘要信息加密, 形成签名, 最后将数据和签名一并发给 B。B 收到后, 使用相同的 HASH 算法也对数据进行 HASH 计算, 得到摘要信息; 然后 B 使用 A 的公钥对签名进行解密, 得到另一个摘要信息。B 将两个摘要信息进行对比, 如果两者一致, 就说明数据确实是从 A 发过来的, 并且没有被修改过, 这样既验证了 A 的身份又实现了数据完整性保护。

注意: 为了便于讲解, 图中 A 发给 B 的信息没有加密, 实际情况下, A 会使用通过计算得出的密钥将信息加密后再发给 B, B 使用相同的密钥解密后再进行处理。



我们花费了一些笔墨介绍公钥密码学的原理, 相信大家对公钥密码学已经有了初步的了解。必须强调一点, 公钥密码学能够保证安全的重要前提是**私钥必须得到安全妥善的保存**, 不能被泄露。接下来我们便要解决前面留下的问题, 即通信双方是通过什么途径来获取到对方的公钥。通常情况下, 通信双方在每次通信时都把自己的公钥发给对方, 这是最直接的方法。但是这种方法存在安全问题, 因为谁都可以生成公钥和私钥, 仅凭一个公钥, 我们无法判断收到的公钥到底是不是对方的。除非对方把公钥和身份信息同时发送过来, 并且有绝对可信的第三方做担保, 证明这个公钥确实是发送方的。也就是说, 通信双方就需要一个安全可信的载体来交换公钥, 这个载体就是证书。

载体——证书

证书, 也叫做数字证书, 是网络世界中的“身份证”。证书将持有者的身份信息和公钥关联到一起, 保证公钥确实是这个证书持有者的, 通过证书就可以确认持有者的身份。证书由权威的、公正的、可信任的第三方机构颁发, 我们把证书的颁发机构称为 CA (Certificate Authority), 相当于现实生活中的公安局。

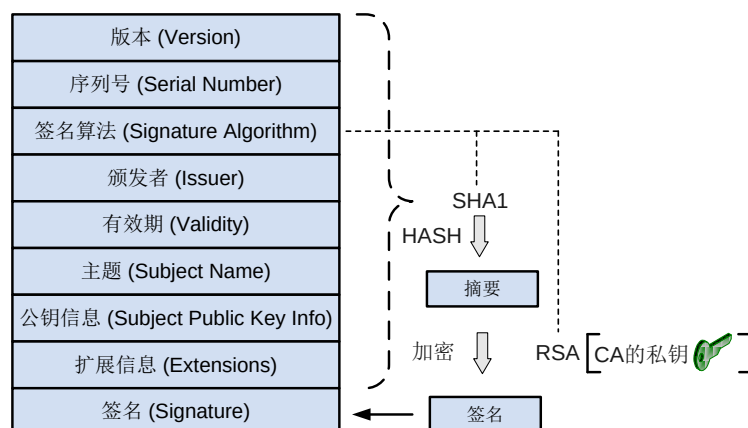
证书属性

公钥是通过证书来向外界公开分发的，证书中必然会含有持有者的公钥信息。除此之外，证书中还包括哪些信息呢？下图展示了一个遵循 X.509 v3 版本规范的证书格式，我们简要介绍一下其中的关键信息。

版本 (Version)
序列号 (Serial Number)
签名算法 (Signature Algorithm)
颁发者 (Issuer)
有效期 (Validity)
主题 (Subject Name)
公钥信息 (Subject Public Key Info)
扩展信息 (Extensions)
签名 (Signature)

- 🍁 签名算法：生成该证书的签名时所使用的 HASH 密码学算法和公钥密码学算法。
- 🍁 颁发者：谁颁发了这个证书，即 CA 的名称。
- 🍁 主题：该证书时颁发给谁的，即证书持有者的名称。
- 🍁 公钥信息：证书持有者的公钥信息。
- 🍁 签名：CA 对该证书的签名，又叫做 CA 的指纹信息。

下图展示了证书中签名的形成过程，操作都是在证书颁发之前，也就是在 CA 上来进行的。首先，CA 使用签名算法中的 HASH 密码学算法（如 SHA1）生成证书的摘要信息，然后使用签名算法中的公钥密码学算法（如 RSA），配合 CA 自己的私钥对摘要信息进行加密，最终形成签名。



证书颁发

通过前面的介绍我们了解到，证书中包含了公钥信息，而公钥和私钥必须成对出现，所以公私

密钥对的存在就变成了制作证书的前提。那么公私密钥对是如何生成的呢？不同网络设备的生成方式有所差别：

- 对于 PC 来说，系统没有提供直接生成公私密钥对的功能，所以一般都是由 CA 来帮助 PC 生成公私密钥对，在此基础上生成证书，然后 PC 将 CA 生成的公私密钥对和证书同时导入到自己的系统中。此时 CA 就兼顾了证书颁发和密钥对管理的功能。
- 对于防火墙来说，我们可以先在防火墙上生成公私密钥对，然后将公钥以及其他一些信息提供给 CA，CA 根据这些信息来生成证书，防火墙只导入生成后的证书。当然，也可以由 CA 来帮助防火墙生成公私密钥对，在此基础上生成证书，然后防火墙将 CA 生成的公私密钥对和证书同时导入，省去了在防火墙上生成公私密钥对的过程。

如果从 Verisign、Geotrust、Globalsign 等专业的证书颁发机构申请证书，操作繁琐也不灵活。其实我们可以自己搭建一个 CA，我们作为 CA 的管理员来为网络中需要证书的设备如 PC、防火墙等来生成证书。搭建 CA 的方式有很多种，常用的 Windows Server 操作系统就可以配置成 CA，为网络设备生成公私密钥对和证书，也可以根据设备提交的信息只生成证书。另外，一些开源的第三方证书管理软件如 XCA 也能够作为 CA 来为网络设备生成公私密钥对和证书。CA 生成证书后，会把证书以文件的形式颁发给使用者。常见的证书存储格式如下表所示。

DER	二进制编码，后缀名 <code>.der/.cer/.crt</code> 不包含私钥
PEM	BASE 64 编码，后缀名 <code>.pem/.cer/.crt</code> 不包含私钥
PKCS #12	PKCS 编码，后缀名 <code>.p12</code> 包含私钥

注意：公私密钥对必须成对出现才能保证公钥密钥学正常运转，所以如果 CA 同时为设备生成了公私密钥对和证书，就需要将公私密钥对（主要是其中的私钥）和证书同时颁发给设备，颁发时需要根据不同情况选择证书的存储格式。例如，颁发给 PC 时就要选择 PKCS #12 格式，将私钥和证书同时颁发；颁发给防火墙时，因为防火墙只支持 DER/PEM 格式，所以要选择 DER/PEM，同时还必须将公私密钥对以单独文件的形式颁发给防火墙。

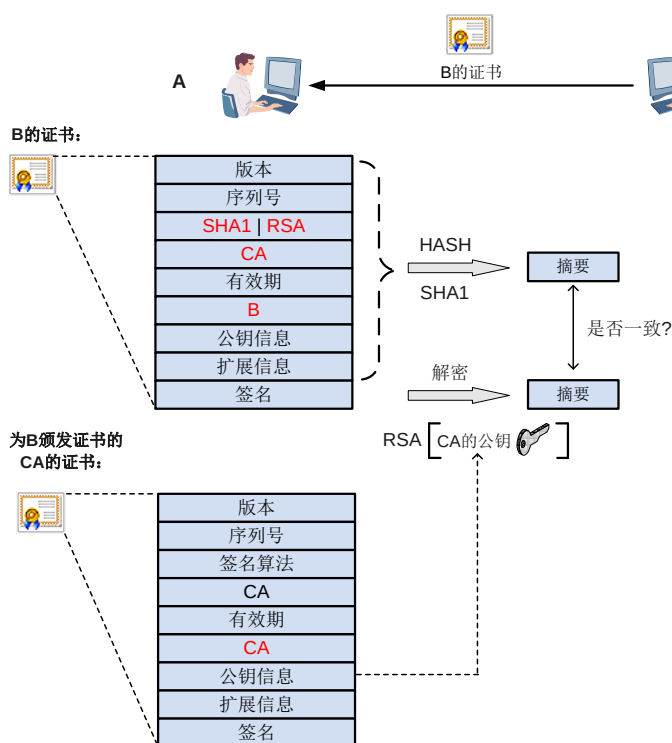
证书验证

证书颁发给使用者后，使用者就会拿着证书到处证明自己的身份。如果我们收到了一个这样的证书，怎么才能判断这个证书就是合法的，不是伪造的呢？还记得前面我们介绍过的 HASH 密码学吗，我们可以利用 HASH 密码学原理，通过证书中的签名来验证证书的真伪。

例如，A 收到了 B 发过来的证书，想要验证这个证书的真伪，此时 A 首先需要获取到为 B 颁发证书的那个 CA 的公钥，用这个公钥解密证书中的签名，得到摘要信息；然后 A 使用证书中签

名算法里面的 HASH 密码学算法对证书进行 HASH 计算，也得到一个摘要信息。A 将两个摘要信息进行对比，如果两者一致，就说明证书确实是由这个 CA 颁发的（能用 CA 的公钥解密说明该 CA 确实持有私钥），并且没有被篡改过，该证书没有问题。当然，也会同时检查证书是否在有效期内。

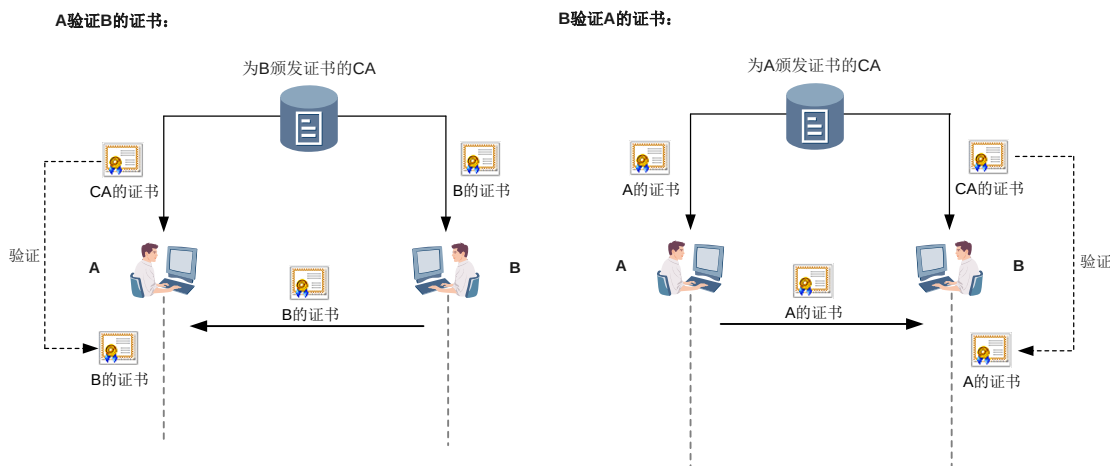
到这里大家可能会有疑问，为 A 颁发证书的那个 CA 的公钥又该如何获取呢？答案还是证书，从 CA 的证书获取。也就是说，CA 除了给别人颁发证书外，它本身也有自己的证书，证书中包含 CA 的公钥。A 使用 CA 的证书验证 B 的证书的过程如下图所示。



再进一步，如何判断为 B 颁发证书的这个 CA 的证书真伪以及是否被篡改过？答案是用 CA 自己的公钥来验证自己的签名，即用 CA 证书中的公钥解密该证书中的签名，得到摘要信息；然后使用 CA 证书中签名算法里面的 HASH 密码学算法对该证书进行 HASH 计算，得到另一个摘要信息，两者一致，说明该 CA 证书是真实的，没有被篡改过。此时有一个新的问题，假设黑客伪造了 CA 证书，证书中的签名是黑客用自己的私钥来签名，而公钥就是黑客自己的公钥，那么对该 CA 证书的验证也能通过。所以上述验证过程的前提是 CA 证书必须是在可信任的机构获取的，保证没有被伪造。

如果通信双方要互相验证对方的证书，那就要分别获取到为对方颁发证书的 CA 的证书。如下图所示，A 如果要验证 B 的证书，则 A 必须先获取为 B 颁发证书的 CA 的证书，用 CA 的证书验证 B 的证书；B 如果要验证 A 的证书，则 B 必须先获取为 A 颁发证书的 CA 的证书，用 CA 的证书验证 A 的证书。如果 A 和 B 是由同一个 CA 颁发的证书，那么两者获取到的 CA 的证书

是相同的，如果 A 和 B 是由两个不同的 CA 颁发的证书，那么两者获取到的 CA 的证书是不同的。



对于证书的使用方法，下面我们再进行一下总结：

1. 通信双方各自持有自己的证书，当一方需要向另一方证明身份时，就把自己的证书发送过去。双方不用事先保存对方的证书，只在验证时接收对方发送过来的证书即可。
2. 一方验证另一方证书的真伪时，必须事先获取到为另一方颁发证书的 CA 的证书，用这个 CA 证书来验证对方的证书。

注意：因为不同的 CA 会颁发不同证书，所以在证书的世界中，每个人可能会持有多个证书。同理，每个人也会获取到多个不同的 CA 证书。在这种情况下，一方收到另一方发过来的证书时，会根据证书中颁发者的名称，在自己的系统中查找对应的 CA 证书，找到后就用这个 CA 证书来验证。如果没有找到，那就说明事先没有获取到 CA 证书，也就无法判断对方发送过来的证书的真伪。

下面我们就来看一下证书在 IPSec VPN 和 SSL VPN 中的具体应用。

应用——IPSec VPN 和 SSL VPN

证书在 IPSec 中的应用

在 IPSec 中，我们需要为两台网关设备（防火墙）生成证书，第一种方法，我们可以先在网关设备上生成公私密钥对，然后将公钥以及网关设备的通用名称等信息提供给 CA，由 CA 根据这些信息来生成证书。网关设备申请证书时，支持在线和离线两种方式，这些内容我们在《IPSec 引入数字证书，身份认证简单便捷》一篇中都已经介绍过，这里就不再赘述。

第二种方法，我们还可以直接在 CA 上帮助网关设备生成公私密钥对和证书，然后网关设备将 CA 生成的公私密钥对和证书同时导入，这样网关设备也能用导入的证书来进行身份认证。具体

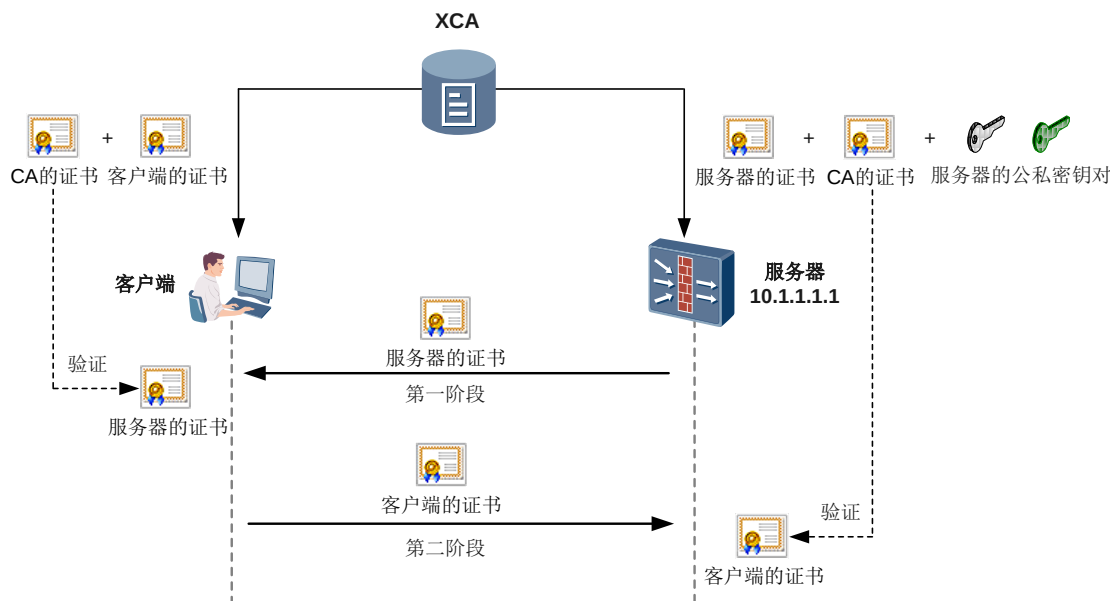
的操作方法可以参照下面介绍的为 SSL VPN 客户端和服务端生成证书的操作。

注意：如果两台网关设备是从同一个 CA 申请的证书，那么两台设备上安装的 CA 证书也是相同的；如果两台网关设备从不同的 CA 申请证书，那就需要在两台网关设备上分别安装为对方颁发证书的那个 CA 的证书。

证书在 SSL VPN 中的应用

在《远程接入 IPSec 现短板，SSL VPN 登上历史舞台》一篇中，我们分析了客户端和服务端（即防火墙设备）建立 SSL 连接的过程，其中包括了两个阶段：首先是客户端对服务器进行身份认证，然后是服务器对客户端进行身份认证。由于客户端一般都是 PC，不能直接生成公私密钥对，所以下面我们使用 XCA 来帮助客户端和服务端生成公私密钥对和证书，然后在客户端和服务端上分别导入各自的公私密钥对和证书。

下图展示了在 SSL VPN 连接建立过程中，不同类型的证书的安装和使用方法。由于客户端和服务端都是由 XCA 颁发的证书，所以两者获取到的 CA 的证书是相同的。另外，由于我们使用了 XCA 生成的证书替换了服务器内置的证书，所以还需要将 XCA 为服务器生成的公私密钥对导入到服务器中。



第一阶段：客户端验证服务器的身份

我们先来看一下第一个阶段。在这个阶段中，服务器将自己的证书发送给客户端，客户端需要用为服务器颁发证书的 CA 的证书来验证这个证书。但是，客户端在自己的系统上没有找到 CA 的证书，无法验证服务器证书的真伪，所以浏览器会提示警告框。

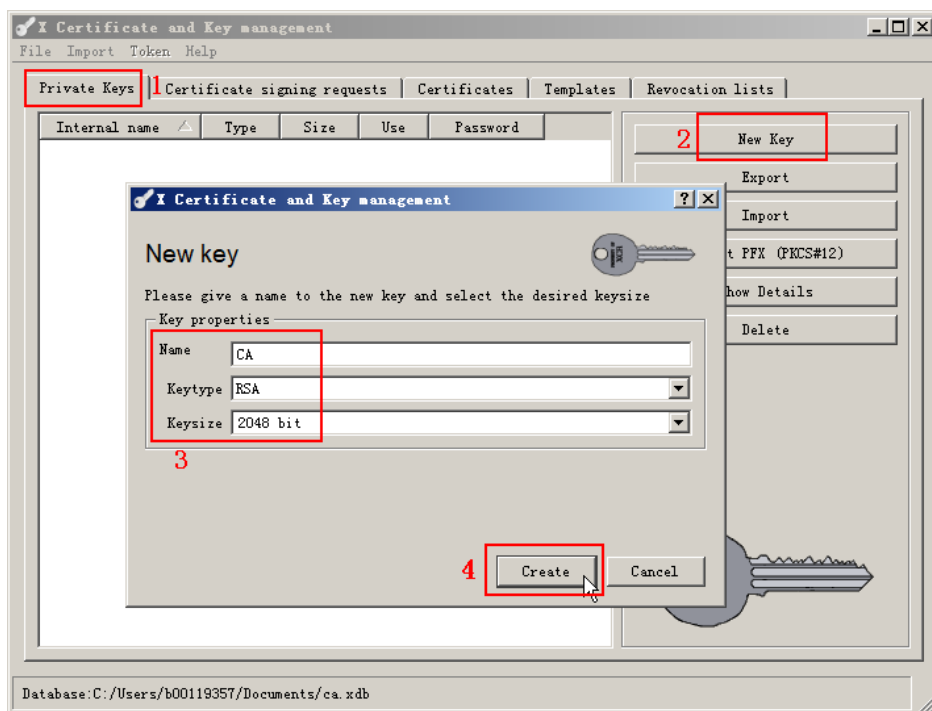
我们可以先忽略这个警告，等出现登录页面后，在页面左侧下载 CA 证书并安装到客户端的系统中，就可以验证服务器证书的真伪，后续客户端登录时就不会再出现警告框了。

可见，服务器向 CA 申请好证书后，将自己的证书连同 CA 的证书一块内置到系统中。当服务器需要被客户端验证时，就会将自己的证书发给客户端；同时，在登录页面上提供 CA 证书，供客户端下载使用。

当然，我们也可以替换掉服务器内置的证书，使用我们自己认可的 CA，如 XCA 为服务器颁发新的证书。下面介绍如何使用 XCA 软件来生成 CA 证书和服务器的证书。

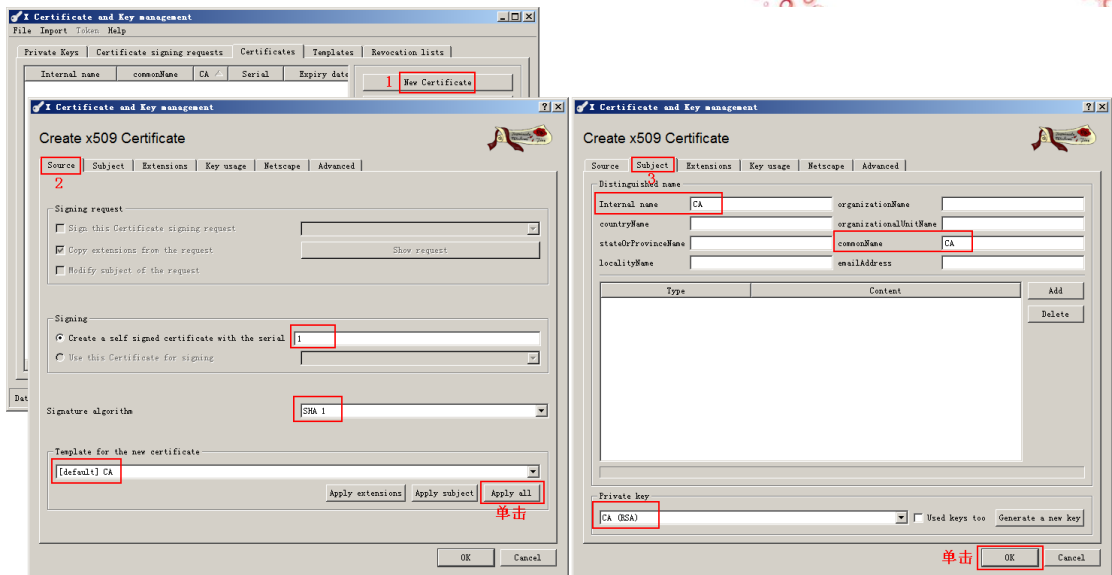
操作步骤如下：

1. 从 <http://sourceforge.net/projects/xca/> 下载并安装 XCA 软件，具体过程略。
2. 在“File”菜单中创建一个新的数据库，输入数据库的名字，然后设置该数据库的密码，使用这个数据库来保存密钥对和证书的信息。
3. 在“Private Keys”页签，生成 CA 的公私密钥对。

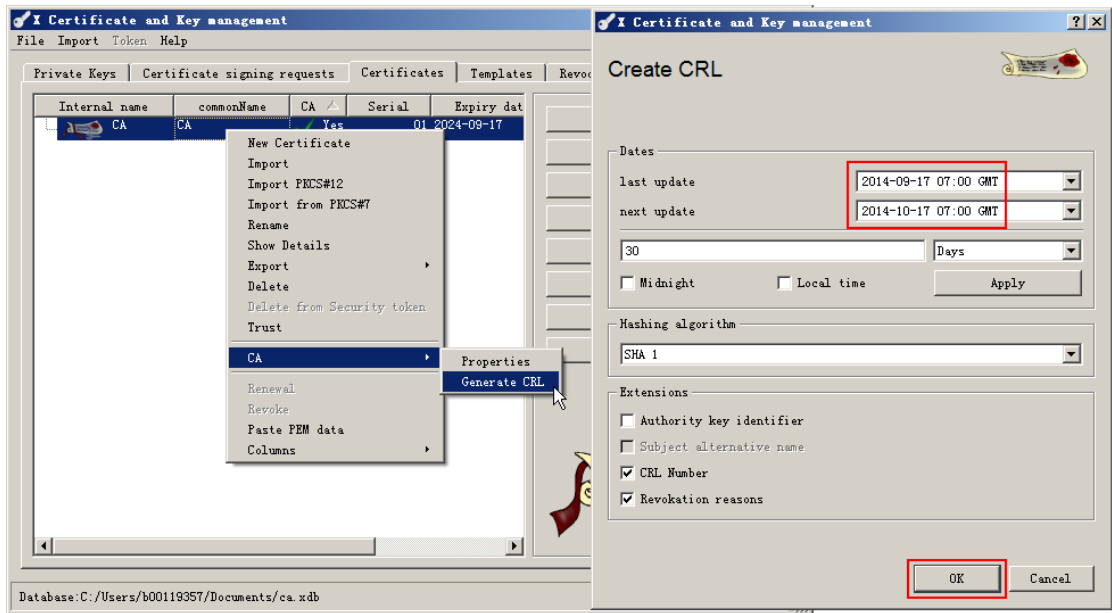


参考上述步骤，生成服务器的公私密钥对，密钥对的 Name 为 10.1.1.1，即用服务器的 IP 地址标识，其他参数不变。

4. 在“Certificate”页签，生成 CA 的证书。



然后为该 CA 证书生成 CRL，即设置该证书的有效期。

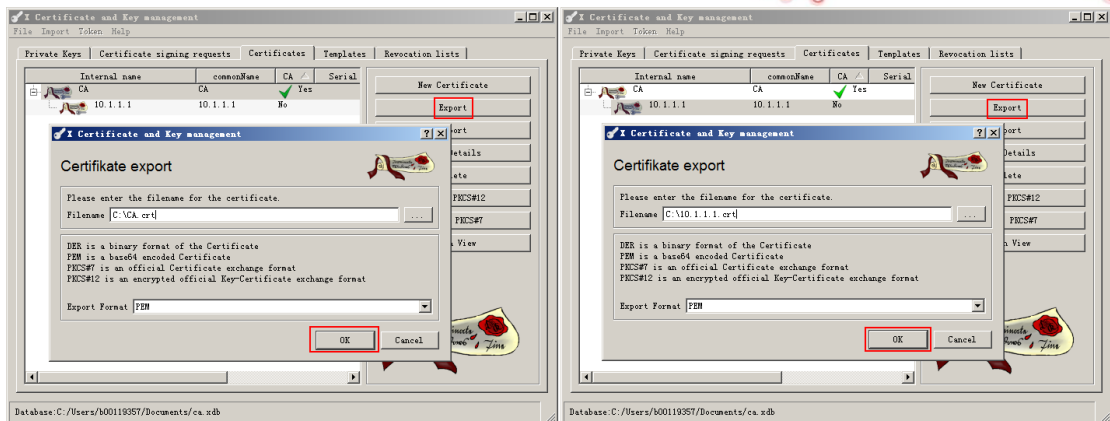


5. 参考上述步骤，生成服务器的证书，与 CA 证书生成过程中有差异的参数取值如下表所示。

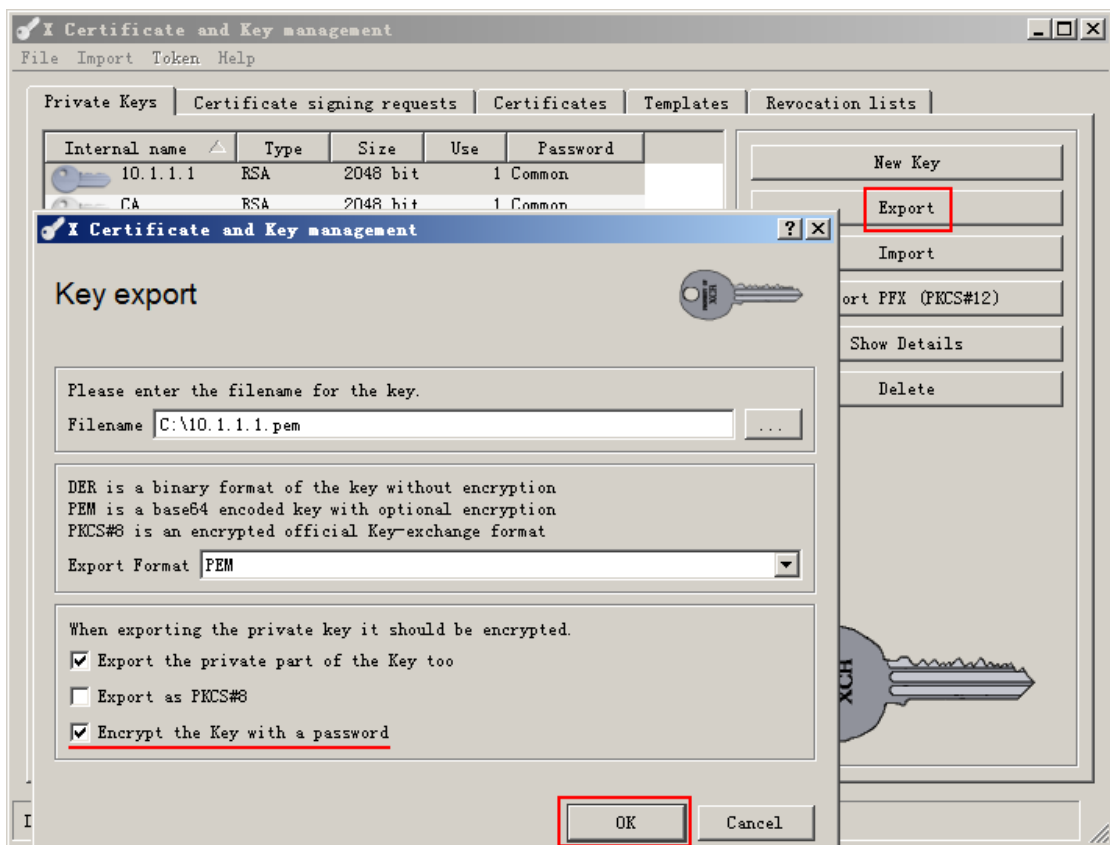
注意：服务器证书的 CN 字段取值必须与服务器 IP 地址一致的要求，否则在客户端上无法验证通过，所以我们这里将证书的名称设置为 10.1.1.1。

Source 页签	Signing	Use this Certificate for signing: CA
	Template for the new certificate	[default] HTTPS_server
Subject 页签	Internal name	10.1.1.1
	commonName	10.1.1.1
	Private key	10.1.1.1 (RSA)

6. 以 PEM 格式导出 CA 证书 CA.crt 和服务证书 10.1.1.1.crt，保存文件备用。



7. 导出服务器的公私密钥对 **10.1.1.1.pem**，保存文件备用。为了保证密钥对的安全，导出时要设置密码。

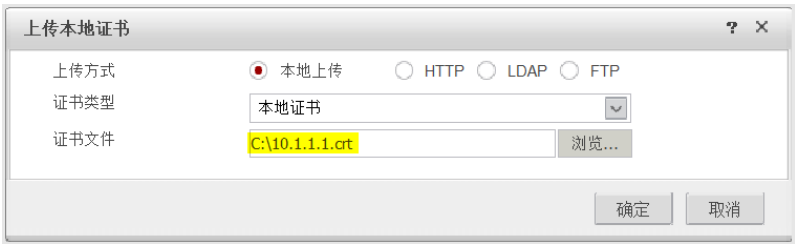


8. 将 CA 证书 **CA.crt** 发送至客户端，在客户端上双击 **CA.crt**，按照提示将该证书存储到“受信任的根证书颁发机构”中。
9. 将服务器的公私密钥对文件 **10.1.1.1.pem** 上传至服务器，然后执行如下命令导入公私密钥对，**password** 处输入导出时设置的密码。

注意：在 SSL 握手过程的第 3 次通信中，客户端会使用服务器的公钥将随机数 **pre-master-key** 加密后发给服务器，要求服务器上必须存在私钥才能解密。

```
[Server] pki import rsa-key-pair 10.1.1.1 pem 10.1.1.1.pem password huawei
```

10. 在服务器上安装自己的证书 **10.1.1.1.crt**。



11. 配置服务器使用 **10.1.1.1.crt** 来向客户端证明自己的身份。

本地证书	10.1.1.1.crt
客户端CA证书	default
证书认证方式?	证书匿名

完成上述操作后，当客户端访问服务器的登录页面时，就可以用 XCA 的 CA 证书来验证服务器发送过来的证书。由于服务器的证书也是由 XCA 颁发的，所以验证通过，浏览器不会提示警告框。

第二阶段：服务器验证客户端的身份

下面来看一下第二个阶段，即服务器使用证书方式对客户端进行身份认证的过程。在这个阶段中，客户端要向服务器发送自己的证书，服务器使用为客户端颁发证书的 CA 的证书来验证客户端的证书。下面我们就以服务器使用证书匿名认证方式认证客户端为例，介绍使用 XCA 软件为客户端生成证书的过程。

操作步骤如下（省略了与第一阶段相似的部分截图）：

- 1. 在“Private Keys”页签，生成客户端的公私密钥对，密钥对的 Name 为 Client。
- 2. 在“Certificate”页签，按照下表中的参数取值，生成客户端的证书。

Source 页签	Signing	Use this Certificate for signing: CA
	Template for the new certificate	[default] HTTPS_client
Subject 页签	Internal name	Client
	commonName	Client
	Private key	Client (RSA)

- 3. 以 PKCS #12 格式导出客户端的证书 Client.p12(因为 PKCS #12 格式的证书中包含了私钥，所以导出时需要设置密码)，保存文件备用。
- 4. 将客户端证书发送至客户端，在客户端上双击 Client.p12，按照提示安装即可，安装过程中需要输入导出该证书时设置的密码。

Firewall

本期策划：徐慧洋

**本期作者：白杰、徐慧洋、王蕾、
李苗苗、赵欣、惠博**

美术编辑：潘艺从

版权所有

©华为技术有限公司2014

保留一切权利

关于华为防火墙，你还
想了解更多吗？请点击

[【强叔侃墙】汇总贴](#)

如有任何问题，请留言
至xuhuiyang.xu@huawei.com